

Functorial Type Theory: Univalent Foundations for Mathematics

The emdash contributors

expanded development edition / version 0.8.3-dev / 2026-09-11

Expanded development edition

This is a working edition of *Functorial Type Theory: Univalent Foundations for Mathematics*. The WalkingEnd/Nat encode-decode argument remains its mathematical centre. Around it, a second spiral develops cut elimination, category theory, weighted universal constructions, directed duality, and a categorical-kernel-first formal presentation. A third follows Yoneda's probes through presheaves, sieves, sites, and sheafification into constructive algebraic geometry, taking the invertibility sieve $D_R(f)$ as prior to any representing open. A fourth returns from directed motion to its groupoidal realization: paths close selected formers, the Circle restores inverse powers, groupoidification freely realizes directed cells as paths, and a profiled Gray closure exposes a genuinely directed inter-changer. A fifth spiral uses face codes, directed join, and iterated outgoing paths to construct canonical dependent simplexes in variable dimension. A sixth returns to universal structure: adjunctions generate monads, finite products acquire a triangular cut calculus, and slice base change extends to the chain $\Sigma_u \dashv u^* \dashv \Pi_u$. A seventh connects selected universal operations to whole homology, exact windows, and retained computer-algebra results. Chapter details, notation, and cross-references may still change. The active implementation remains authoritative whenever prose and code disagree.

Book record: [10.5281/zenodo.21544186](https://doi.org/10.5281/zenodo.21544186). Code and source: github.com/hotdocx/emdash. The [bibliographic self-reference](#) identifies the book; the archived editions and current source may have different version dates.

Copyright © 2026 the emdash contributors. Except where separately identified, the book text is licensed under CC BY-SA 3.0. See [Credits and Third-Party Attribution](#) and the source files `book/CREDITS.md` and `book/LICENSE.md`.

Formal status — research boundary. The title names the intended foundational programme. It does not assert that this edition already implements a finished proof assistant, every weak omega-categorical coherence, or a universal computational univalence principle.

Preface

The emdash contributors maintain this book and its accompanying code together. The [book DOI](#) identifies the published book record; [the source repository](#) holds the ongoing mathematical and software development. A full citation appears in the [bibliography](#). This draft's own version and date are given in its opening material.

Type theory is often introduced through terms and substitution. Category theory is often introduced through objects and arrows. Functorial type theory starts from the conviction that these are not two unrelated beginnings. Substitution has action; action has coherence; and a useful formal language should retain that structure rather than recover it after the fact.

The resulting theory has two kinds of motion. Equality supplies groupoidal paths: they can be reversed, transported along, and compared through equivalence and univalence. Categories supply directed arrows: an arrow may have no inverse, and that failure is mathematical information. Functors act on both objects and higher arrows. Cat-valued families turn dependent substitution into directed reindexing. Transformations and their higher components express the coherence of those actions.

This edition is organized around a single calculation because a foundation is best learned while it is doing something. Consider an opaque category generated, in the higher-inductive sense selected by emdash, by a base object and one directed endomorphism. Its endomorphisms look like

$$\mathrm{id}, \quad \ell, \quad \ell^2, \quad \ell^3, \quad \dots$$

The calculation proves that this list is exhaustive at the level of the underlying carrier:

$$\mathrm{Hom}_W(*, *) \simeq \mathbb{N}.$$

The proof is not obtained by defining the hom to be a datatype of words. Instead it constructs a Cat-valued code, transports zero forward along a directed arrow, builds a contextual decoder, and produces a directed normalization cell from every based arrow toward its coded power. Only then does one-dimensionality turn that cell into equality. This order—directed structure first, equality at the boundary—is the small example in which the larger programme becomes visible.

The argument deliberately echoes the calculation $\Omega(S^1) \simeq \mathbb{Z}$ in the *Homotopy Type Theory* book. The echo is not an identification. A loop in an identity type is invertible, while the walking endomorphism is not. The circle needs positive and negative powers; the directed object has only natural powers. The circle's code uses univalence to turn successor on the integers into a path in a universe; the directed code sends its generator directly to a successor functor, which need not be an equivalence. What fails to transfer is as instructive as what does.

The title's phrase "univalent foundations" therefore describes a layer, not a device forced into every proof. Emdash contains checked equality, equivalence, groupoid-univalence, restricted truncated-universe univalence, and equality-valued omega-equivalence interfaces. The WalkingEnd calculation also shows why a univalent foundation for directed mathematics must permit actions that are not equivalences.

The exposition follows a spiral. The prologue states the central theorem with minimal prerequisites. Chapters 1–7 then develop the judgments, categories, families, logic, equivalence, induction, directed higher induction, and categorical height needed to understand the proof. Chapter 8 returns to the calculation in full.

The later chapters move outward from that proof in a second spiral. Chapter 9 organizes functorial computation as a calculus of cuts. Chapters 10–15 develop categories, functors, adjunctions, Yoneda, duality, structure identity, and saturation. Chapters 16–17 treat weighted limits and colimits before returning to directed geometry through join.

A third spiral begins in Chapters 18–24. It turns Yoneda's field of probes toward local-to-global geometry: presheaves organize changing views, higher sieves retain categories of witnesses, and ordinary sieves record stable local questions. Sites select which sieves cover, while matching families state descent as one restriction-of-Hom problem. Direct cover completion then freely adjoins coherent solutions and assembles the Cat-valued sheaf reflector from its whole Hom universal property. Commutative algebra then supplies set-carrier rings, finite unit-ideal certificates, free extension interfaces, and localizations characterized by contractible factor spaces. The recurring geometric example is the sieve $D_U(s)$ of every probe along which a section becomes invertible. A localization may represent this question on affine points, but the sieve is meaningful before representability is known. The affine functor of points then turns selected localizations into basic charts, multiplication into pointwise intersection, and finite unit-ideal certificates into the generated big Zariski topology, while keeping structure-sheaf and localization-locality assumptions explicit. Starting from one supplied global ringed object, a covering sieve, and two constructively generating affine realizations, the spiral then reaches a binary site-relative scheme presentation. Whole slice restrictions and a selected actual chart intersection inherit their maps from the single global structure presheaf; atlas-first gluing and comparison with classical or functorial qcqs schemes remain visible boundaries. On that actual intersection, polynomial and localization universality construct the Laurent coordinate changes of two supplied affine-line charts. The spiral ends with an assumption-explicit projective-line presentation: the overlap calculation is checked, while construction of the global object, graded $\mathbf{Proj}$, general projective space, and non-affineness remain visible boundaries.

A fourth spiral begins in Chapters 25–28. It returns the directed theory to the groupoidal layer without identifying the two. Paths in products split and reassemble homwise, and dependent transport through those coordinates agrees coherently with primitive equality induction. The Circle then restores the inverse powers deliberately absent from WalkingEnd: successor-localized integers support an internal encode–decode proof of its based loop space. WalkingEnd and the two-ended WalkingArrow next become finite tests of free inversion before the same mapping property is stated for an arbitrary source category. The final chapter turns back toward direction. Whole internal laxity yields a computational strict-functor profile, one selected right Gray closure, and a nonidentity interchanger on the walking square. These are substantial checked slices, not a claim that every categorical former is groupoidally closed, that source-functorial groupoidification has already been packaged as an adjunction, or that a full Crans–Gray biclosed monoidal structure has been constructed.

A fifth spiral begins in [Chapter 29](#). It treats the retained higher action as recursive geometry. Injective face codes form an internal semi-simplex category, directed joins form the ordinal shapes $\Delta[n]$, and iterated outgoing-path categories present a simplex as a base cell with a dependent cell above it. One structural successor computes a canonical ordinal dependent simplex in variable dimension; selected dimensions zero through four, every nonempty face observation, and one further action are checked. Degeneracies, a whole category of dependent simplexes, and its mapping-category equivalence with $\text{Functor}(\Delta[n], C)$ remain the next boundary rather than being inferred from the object-level recursion.

A sixth spiral begins in [Chapter 30](#), after Chapter 12 has made the monad generated by an adjunction computationally explicit. It returns to the structural cuts of Chapter 9 with new checked owners: a selected whole binary-product functor, a terminal object, and their thin cartesian package. The same calculus then becomes indexed. Postcomposition supplies Σ_u , chosen pullbacks supply u^* , and chosen dependent products supply Π_u , with

$$\Sigma_u \dashv u^* \dashv \Pi_u.$$

The chapter derives pullback objects, projections, and their directed square from slice base change and a counit rather than from a stored cone record. It stops before Beck-Chevalley, Frobenius, pushout duality, derived slice exponentials, or a final convention-sensitive locally cartesian closed package.

A seventh spiral, [Chapter 31](#), puts universal factors to work in homological algebra. Addition turns selected products into biproducts; kernels and cokernels provide factors; Abelian normality and local covers make the snake construction possible without an elementwise choice of preimages. Coherent kernel/cokernel adjunctions then make homology a whole functor on native zero diagrams. The chapter proves the selected six-term snake theorem and all three interiors of a homology window, supplies a whole connecting transformation and finite iteration, and follows a nonsplit polynomial example through categorical execution and explicit proof-CAS interpretation. The final symbolic zero-endpoint attachment stays deferred, separate from the native endpoint computation and the checked window/iterator interfaces.

[Appendix G](#) then states how the mathematical surface, checked categorical kernel, bounded TypeScript elaborator through explicit Core, and external models fit together, with the `Lambdapi` kernel remaining the mathematical authority.

The book is evidence-aware without being a source-code catalogue. Checked claims name their evidence in compact notes. Free mathematical development is welcome, but it is named as such and states what an emdash implementation would require. This lets the prose reach beyond the current library without blurring the line between a plausible design and a theorem already accepted by the kernel.

Formal status — mathematical development. This preface states the expository and research programme. Each theorem-like claim in the chapters carries its own status and, when checked, a machine-verified evidence link.

How To Read This Book

The shortest route is the [prologue](#), followed by the compact prerequisite review and proof in [Chapter 8](#). The foundational route reads Chapters [1–7](#) first and then returns to the same calculation with every interface available. The second spiral begins with the cut calculus in [Chapter 9](#), develops ordinary and native category theory through [Chapter 15](#), and culminates in weighted universals, duality, and join in Chapters [16–17](#). The local-to-global spiral begins with presheaves and sieves in [Chapter 18](#), then selects covers and formulates descent in [Chapter 19](#), and constructs Cat-valued sheafification by direct cover completion in [Chapter 20](#). [Chapter 21](#) adds the representation-free commutative algebra that turns invertibility into computational affine charts, and [Chapter 22](#) constructs the functor-of-points bridge from $D(f)$ and localization to the generated big Zariski site. [Chapter 23](#) begins with a supplied global ringed object, recognizes two constructively generating regions as affine, imposes topology-local ring behaviour on the actual slice, and derives a selected chart intersection from the global structure presheaf. [Chapter 24](#) constructs Laurent coordinate changes on that literal overlap, packages the resulting supplied projective-line capability, and separates it from the still-unconstructed graded `Proj` route. The fourth spiral begins with paths, structured transport, and groupoidal closure in [Chapter 25](#). [Chapter 26](#) carries out the Circle/Integer encode–decode theorem; [Chapter 27](#) passes from the WalkingEnd and WalkingArrow tests to category-indexed free inversion; and [Chapter 28](#) recovers a directed interchanger from whole laxity inside one profiled Gray closure. [Chapter 29](#) then turns dependent hom, outgoing paths, face codes, and join into a variable-dimensional semisimplicial construction. [Chapter 30](#) returns to universal structure: it follows products and terminality into slices, chosen pullbacks, and the dependent chain $\Sigma_u \dashv u^* \dashv \Pi_u$. The [contents](#) and [glossary/index](#) provide stable anchor-based navigation.

Five reading paths make the dependencies explicit:

Reader	Main path	Consult when needed
type theorist	Prologue; Chapters 1, 3–8, 10, 12, 15, 25–27, 29, and 30	Chapters 2, 9, and 28 for directed action and laxity; Appendix G for the formal presentation
category theorist	Prologue; Chapters 2, 5, and 8–30	Chapters 1, 3, 4, and 7 for equality, propositions, univalence, and height
algebraic geometer	Chapters 13, 16, 18–24, and 30	Chapters 2, 3, 5, 6, and 12 for the directed, logical, inductive, universal, and adjoint foundations
implementer	Chapters 1, 2, 6, 8, 9, and 25–30; Appendices A, B, E, F, and G	the theorem chapters whose evidence route is being inspected
external reviewer	Chapters 2.6, 8, and 25–30; then the integrated reviewer, live or local	Appendices A, B, F, and G for notation, evidence, status, and architecture

These are paths through one dependency graph, not separate foundations. In particular, the category-theory route still uses equality-local reasoning, and the type-theory route still needs directed functor action.

For the executable-review path, open the `integrated reviewer` or run `./scripts/pnpmw run reviewer:dev` from the repository root. The wholly client-side workbench offers editable examples across the four binder modes. It lets the reader inspect explicit Core, inferred and expected classifiers, structural lowering, computation, and source-located failures; the same page runs the three-part research report and opens this book. Its text notation is a bounded executable subset. The mathematical notation used throughout the book is intentionally broader and should not be read as a complete parser grammar.

Composition is written in categorical order:

$$g \circ f : x \longrightarrow z$$

means “first f , then g .” Functor action is written $F[x]$ on objects and $F[f]$ on arrows. The path category $\mathbf{Path}(A)$ retains equality-local, hence groupoidal, structure inside the directed calculus. Symbols such as W , $*$, ℓ , **Code**, **encode**, and **power** are mathematical abbreviations; the notation appendix maps them to active Lambdapi names.

Evidence Status

Every theorem-like assertion has one of four evidence statuses:

- **Checked.** An active declaration and a regression or reviewer example establish the stated interface.
- **Formal consequence.** The assertion follows from named checked interfaces, but the library does not yet package the result under the stated name.
- **Mathematical development.** The theory is developed in ordinary mathematics with explicit prerequisites and a plausible future emdash owner.
- **Research boundary.** A construction is conjectural, underspecified, or blocked on named infrastructure.

A typical note looks like this:

Formal status — checked. Evidence `WE-HOM-NAT-CARRIER`.

The evidence identifier resolves through `book/evidence.json` to declarations and executable checks. It is traceability metadata, not a replacement for the proof in the prose.

The active Lambdapi sources outrank the book. The current implementation and safe-development procedure are described in the repository’s current-status report; the canonical-syntax report owns the mathematical notation, which is broader than the reviewed executable text subset. Dated reports preserve design history but do not silently revive retired interfaces.

Adapted passages and licenses are recorded in `book/references/third-party-sources.json`. Hadzihasanovic, Kolomatskaia--Shulman, and Herbelin--Ramachandra are comparative references only: Chapters 28--29 use fresh prose and do not claim that their checked constructions are the full tensors or semisimplicial presentations of those sources.

Contents

Front matter

- [Preface](#)
- [How To Read This Book](#)

Main text

- [Prologue: The Natural Number Hidden In A Directed Loop](#)
- [1. Judgments, Universes, And Computation](#)
- [2. Categories, Functors, And Directed Families](#)
- [3. Logic, Propositions, And Sets](#)
- [4. Equivalence And Univalence](#)
- [5. Induction, Arrow Induction, And Universal Properties](#)
- [6. Directed Higher Inductive Types](#)
- [7. Truncation And Categorical Height](#)
- [8. Synthetic Directed Homotopy Theory](#)
- [9. Transforms And The Calculus Of Cuts](#)
- [10. Categories, Precategories, And Categorical Identity](#)
- [11. Functors, Transforms, And Functor Categories](#)
- [12. Adjunctions And Equivalences](#)
- [13. Yoneda, Representability, And Profunctors](#)
- [14. Strictness, Dagger Structure, And Duality](#)
- [15. Structure Identity And Saturation](#)
- [16. Weighted Universal Constructions](#)
- [17. Weighted Colimits, Duality, And Join](#)
- [18. Presheaves And Sieves](#)
- [19. Sites, Covers, And Descent](#)
- [20. Sheafification By Cover Completion](#)
- [21. Commutative Algebra By Universal Property](#)
- [22. Affine Geometry And The Sieve \$D\(f\)\$](#)
- [23. Schemes From Covering Charts](#)
- [24. The Projective Line And The Boundary Of Construction](#)
- [25. Paths And The Groupoidal Shadow](#)
- [26. The Circle And The Integer Line](#)
- [27. Free Inversion And Groupoidification](#)
- [28. Laxity, Interchange, And The Gray Direction](#)

- [29. Simplexes From Dependent Homs](#)
- [30. Cartesian Structure And Dependent Products](#)
- [31. Additive, Abelian, And Homological Computation](#)

Appendices

- [Appendix A. Notation](#)
- [Appendix B. Emdash Evidence](#)
- [Appendix C. From The Circle To The Walking Endomorphism—And Back](#)
- [Appendix D. Glossary And Concept Index](#)
- [Appendix E. Computation And Normalization](#)
- [Appendix F. Implementation Status And Research Directions](#)
- [Appendix G. Formal Presentation Of Functorial Type Theory](#)

Back matter

- [Bibliography](#)
- [Credits And Third-Party Attribution](#)
- [License For The Book](#)

Prologue: The Natural Number Hidden In A Directed Loop

Imagine a world with one distinguished place and one permitted step that returns to it. Call the place `*` and the step `ell`. We may stand still, take the step once, take it twice, and continue:

$$\text{id}_*, \quad \ell, \quad \ell \circ \ell, \quad \ell \circ \ell \circ \ell, \quad \dots$$

If this is the *free* directed situation, every journey from `*` back to `*` should have a unique length. That sentence sounds obvious only because it quietly identifies two descriptions:

1. the syntactic description of a journey as a finite word in one generator;
2. the universal object characterized by how maps out of it behave.

Functorial type theory keeps them apart. The walking endomorphism `W` is presented opaquely by a category, a base, a directed loop, explicit one-dimensionality, and a contextual elimination principle. Its object type, hom-categories, identities, and composition do not reduce to a word datatype.

Formal status — checked. Evidence `WE-SIGNATURE` and `WE-ONE-DIMENSIONAL`. One-dimensionality says that each hom-category is discrete; it neither makes `W` a discrete category nor makes `ell` invertible.

There is also a concrete category `BNat`. It has one object, natural numbers as endomorphisms, zero as identity, and addition as composition. The recursor for `W` gives a functor

$$W \longrightarrow \mathcal{BNat}$$

sending `*` to the sole object and `ell` to one. This is the expected arithmetic model. It is not a definition of `W`, and a functor into a model cannot by itself prove that two arrows of `W` with the same image are equal.

Formal status — checked. Evidence `WE-BNAT-MODEL`. `BNat` is separate model evidence. A reverse functor and a categorical equivalence have not been packaged.

The central theorem supplies the missing exhaustiveness statement by a different route.

Theorem preview. There is an equivalence of underlying carriers

$$\text{Hom}_W(*, *) \simeq \mathbb{N}.$$

Formal status — checked. Evidence `WE-HOM-NAT-CARRIER`. The current package is a `TypeEquiv`, together with a native equality-valued facade. A monoid or hom-category equivalence is not being asserted here.

Measuring An Abstract Arrow

How can an opaque arrow be assigned a number? We construct a Cat-valued family

$$\mathbf{Code} : W \longrightarrow \mathbf{Cat}$$

whose fibre at $*$ is the path category of natural numbers. The generator acts by the successor functor. For any endpoint x and any based arrow $p : * \rightarrow x$, define

$$\mathbf{encode}_x(p) = \mathbf{Code}[p](0).$$

This is directed transport in its plainest form: a functor acts on an object. When p begins with the generator, its code begins with a successor. There is no predecessor action because none was requested by the signature.

To go back, natural numbers produce powers:

$$\ell^0 = \text{id}_*, \quad \ell^{n+1} = \ell \circ \ell^n.$$

At first this decoder exists only over the base. The decisive move is to generalize it over every endpoint. The based representable family

$$\mathbf{Rep}_*(x) = \mathbf{Hom}_W(*, x)$$

acts on a base arrow by postcomposition. A displayed transformation

$$\mathbf{decode}^d : \mathbf{Code} \Longrightarrow \mathbf{Rep}_*$$

must therefore reconcile successor in $\mathbf{Code}$ with postcomposition by $\mathbf{ell}$ in the representable. Its coherence winds forward through the natural levels:

$$\ell \circ \ell^n \longrightarrow \ell^{n+1}.$$

This family of higher arrows is the **directed spiral**.

Normalize Before Comparing

Applying the displayed decoder's arrow action to $p : * \rightarrow x$ at zero produces

$$\nu_p : p \longrightarrow \mathbf{decode}_x(\mathbf{encode}_x(p))$$

inside the hom-category $\mathbf{Hom}_W(*, x)$. This is not initially an equation. It is a directed 2-cell whose orientation records a normalization process.

Formal status — checked. Evidence `WE-NORMALIZATION-CELL`. The normalization cell is obtained from the contextual displayed eliminator; it is not postulated as a word-reduction axiom.

Only now is one-dimensionality used. The hom-category is discrete, so a cell between parallel based arrows determines equality:

$$p = \text{decode}_x(\text{encode}_x(p)).$$

At the base, `decode` computes to `power`. Reversing the displayed equality gives

$$\ell^{\text{encode}(p)} = p.$$

The other composite is arithmetic. The generator-prefix equation and natural number induction give

$$\text{encode}(\ell^n) = n.$$

These two equations package the carrier equivalence.

Formal status — checked. Evidence `WE-NORMALIZATION-PATH`, `WE-POWER-ENCODE`, and `WE-ENCODE-POWER`. Direction is retained until the first of these equalities is extracted.

Why The Answer Is Not The Integers

For the circle, a loop is an equality path and therefore has an inverse. Its powers are indexed by integers. Here `ell` is a directed arrow. Nothing in the category laws manufactures a reverse arrow, and the calculation detects that absence.

Encoding sends `ell` to one and the identity to zero. Consequently the generator is not the identity. If an arrow `r` were a right inverse, then encoding $\ell \circ r = \text{id}$ would force a successor to equal zero. Hence no right inverse exists, and in particular `ell` carries no native omega-equivalence evidence.

Formal status — checked. Evidence `WE-LOOP-NOT-IDENTITY`, `WE-LOOP-NO-RIGHT-INVERSE`, and `WE-LOOP-NONINVERTIBLE`.

The missing negative integers are therefore not an incomplete case of the proof. They are the numerical trace of directionality.

The Road To The Proof

The next seven chapters unpack the interfaces used above.

- [Chapter 1](#) separates definitional computation, proof-time comparison, and equality evidence.
- [Chapter 2](#) introduces iterated hom-categories, functors, transfors, and directed Cat-valued families.
- [Chapter 3](#) develops constructive logic, propositions, and sets.
- [Chapter 4](#) distinguishes equality, ordinary equivalence, categorical isomorphism, and univalence.
- [Chapter 5](#) moves from ordinary induction to arrow induction and contextual universal properties.
- [Chapter 6](#) states the selected directed higher-inductive signature and its contextual eliminator.
- [Chapter 7](#) explains categorical height and why a directed cell in a discrete hom-category yields equality.

- Chapter 8 returns to every step of the construction, including the spiral, the two inverse laws, and the noninvertibility consequences.

The second spiral asks what the same computational discipline contributes to category theory. Chapter 9 isolates its cut calculus; Chapters 10–15 develop categorical identity, functor categories, adjunctions, Yoneda, duality, and saturation; and Chapters 16–17 organize limits, colimits, and join by representability and opposite duality. Appendix G collects the formal rule schemas and states the metatheoretic boundary.

The third spiral asks how local questions become geometry. Chapters 18–20 pass from presheaves and sieves through sites and descent to direct Cat-valued sheafification. Chapters 21–24 develop universal-property algebra and affine localization, place the invertibility sieve before any representing open, and assemble site-relative scheme and supplied projective-line presentations without concealing their hypotheses.

The fourth spiral returns to the relation between paths and arrows. Chapter 25 realizes selected directed action inside path categories and compares product transport with equality induction. Chapter 26 restores the inverse powers missing from the central WalkingEnd theorem by proving the Circle/Integer encode–decode equivalence. Chapter 27 upgrades that comparison to free inversion, first for one- and two-endpoint walking shapes and then for an arbitrary source category. Chapter 28 keeps the target directed again: whole laxity, a computational strict profile, and one right Gray closure produce a nonidentity walking-square interchanger with higher action still available.

The fifth spiral asks what that still-available action constructs when it is iterated. Chapter 29 combines a computing category of injective faces with the native recursion $S_{n+1} = \text{PathOut}_{S_n}(s_n)$. It constructs a canonical ordinal dependent simplex in variable dimension, maps it into arbitrary targets, and retains both face action and another hom action without postulating a new coherence record at every dimension.

The sixth spiral returns to universal structure with that lesson in hand. Chapter 30 starts from whole selected products and terminality, then moves through slices to pullback base change and the chain $\Sigma_u \dashv u^* \dashv \Pi_u$. Products, pullbacks, and dependent products are presented by their introduction/elimination cuts and whole higher action, not by disconnected objectwise universal properties.

The larger aim is not merely to calculate one hom. It is to show how a type theory can let groupoidal equality and noninvertible arrows coexist, interact, and compute—without quietly turning one into the other.

1. Judgments, Universes, And Computation

A foundation does not begin by placing mathematical objects in a previously given container. It begins by saying which expressions may be formed, which expressions inhabit which classifiers, and which calculations count without further proof. The elementary judgments have the familiar shape

$$A \text{ classifier}, \quad a : A, \quad a \equiv b : A.$$

The last display uses $\equiv$ only as the glyph for *judgmental* or definitional equality. It is not an inhabitant of an identity type. This distinction will matter throughout the book: computation may make two expressions the same input to every later rule, while a path is mathematical data that can itself be transported, inverted, and compared.

Emdash represents its small type-like classifiers by a universe called `Grpd`. A classifier `A:Grpd` decodes to the ambient type of its elements, written mathematically simply as `a:A` and represented in the kernel by `a:tau(A)`. This two-level presentation is an implementation device. Unless a decoding boundary is at issue, we suppress `tau` and reason in ordinary type-theoretic notation.

[Appendix G](#) gives the full formal reading of this convention. It separates external contexts and typing judgments from internal classifiers, then asks of each major construction for its formation, introduction, elimination, computation, uniqueness, and functorial action. The last two clauses will often be independent: a beta rule can be checked even when a general uniqueness theorem remains open, and a pointwise construction is incomplete until its arrow and higher-cell action is known.

1.1 Contexts And Families

A judgment rarely stands alone. It is made in a context:

$$x : A, \quad y : B(x), \quad z : C(x, y) \vdash t : D(x, y, z).$$

Each declaration may depend on those before it. Substitution replaces a variable by a term of the right classifier and acts through every expression that follows. In ordinary dependent type theory this action is often left implicit in notation. Functorial type theory will progressively expose it: first as path action, then as functor action on directed arrows, and finally as coherent action on higher cells.

An ordinary function classifier is written

$$A \longrightarrow B,$$

and a dependent function classifier as

$$\prod_{x:A} B(x).$$

An element `f` of the second classifier assigns `f(x) : B(x)`. Lambda abstraction introduces such an element, and application to an argument computes by beta reduction. The nondependent function classifier is the constant-family case.

A dependent pair classifier is

$$\sum_{x:A} B(x).$$

An element is a pair `(x, u)` with `u : B(x)`. Equality of dependent pairs is not merely a pair of unrelated equalities: it consists of a base path together with a fibre path lying over that base path. Emdash exposes this observation explicitly. For dependent functions it exposes pointwise path observation and a converse function-extensionality constructor. Thus the basic Sigma and Pi layers already carry the path structure later used in equivalence and truncation arguments.

Formal status — checked. Evidence `TT-SIGMA-PI-PATHS` covers dependent-pair projections and path views together with the checked `happly / funext` equivalence for dependent functions. This is an implemented equality interface, not a claim that every future categorical section category is merely a pointwise Pi type.

1.2 Three Ways Expressions Agree

The word “equal” hides three operationally different relations in the current development.

First, a **runtime rewrite** chooses a normal form. For example, applying the Nat eliminator to zero returns its zero branch, while applying it to a successor returns the successor branch with the recursive result. Such a rule is part of computation: later expressions literally see the reduced term.

Second, a **proof-time comparison** helps elaboration recognize two typed presentations without orienting either as the other's runtime normal form. This is useful when a mathematical construction has two stable categorical views—for example, a displayed-family facade and the corresponding ordinary functor-category view. Proof-time comparison is intentionally narrower than an equation available to the mathematician.

Third, an **identity term**

$$p : x =_A y$$

is internal evidence. It may be an argument to a function, the index of a dependent family, or the object of a higher identity type. Its existence does not by itself make `x` and `y` judgmentally interchangeable.

These layers cooperate, but they should not be conflated. A rewrite is chosen because a form is intended to compute. A proof-time comparison is chosen because two rigid presentations should elaborate together. A path is chosen because equality is part of the mathematics. Much of emdash's engineering is the discipline of putting a law at the right one of these layers.

As a small example, Nat addition is defined by recursion in its left input:

$$0 + n \equiv n, \quad \text{succ}(m) + n \equiv \text{succ}(m + n).$$

Associativity is an identity term proved by Nat induction; it is not installed as a global reassociation rewrite. The distinction keeps computation predictable while leaving the algebra available propositionally.

Formal status — mathematical development. The three-layer terminology is the book's explanation of the active kernel discipline. Exact rewrite and unification ownership remains an implementation matter governed by the current SOP; the mathematical chapters rely only on the checked interfaces cited in their evidence notes.

1.3 Elementary Inductive Classifiers

The initial object language contains classifiers for Empty, Unit, Bool, and Nat. Their constructors have the expected readings:

Empty	no constructor
Unit	tt
Bool	false, true
$\mathbb{N}$	0, succ(n).

An inductive classifier is characterized computationally by how dependent data are defined from its constructors. For Nat, a motive $P : \text{Nat} \rightarrow \text{Grpd}$, a base value $z : P(0)$, and a step

$$s : \prod_{n:\mathbb{N}} P(n) \longrightarrow P(n + 1)$$

determine

$$\text{ind}_{\mathbb{N}}(P, z, s, n) : P(n)$$

with the zero and successor computations. Recursion is the constant-motive special case; induction retains the dependence on the number being analyzed. The power function in Chapter 8 is exactly such a Nat elimination, with a hom carrier as its motive.

The equality classifiers of visible Unit, Bool, and Nat constructors also have bounded observational computations. In particular, zero cannot equal a visible successor, and equality of two visible successors reduces to equality of their predecessors. These classifier computations do not erase a generic reflexivity proof into the unique constructor of Unit, nor do they establish a global normalization or canonicity theorem for every open Nat term.

Formal status — checked. Evidence `TT-ELEMENTARY-INDUCTION` covers the decoded elementary classifiers and their dependent eliminators. The currently selected reusable Nat library additionally supplies addition, associativity, successor action, and sethood. A retired experimental bin-

ary coproduct is not part of this edition's active elementary API.

1.4 Identity And Path Induction

For $x, y : A$, the classifier $x=y$ records identity paths. Reflexivity gives

$$\text{refl}_x : x = x.$$

The fundamental elimination rule says that a property of paths can be proved by treating the reflexive path. In the right-based form used by emdash, fix $y : A$, take a motive

$$P : \prod_{x:A} (x = y) \longrightarrow \mathbf{Grpd},$$

and provide $u : P(y, \text{refl}_y)$. Then every $p : x=y$ receives an element of $P(x, p)$, and the construction computes to u on literal reflexivity.

Path symmetry, transitivity, and action follow from this eliminator:

$$\begin{aligned} p^{-1} &: y = x, \\ q \cdot p &: x = z, \\ \text{ap}_f(p) &: f(x) = f(y). \end{aligned}$$

For a dependent function $f : \text{prod}_-(x:A) B(x)$, ordinary ap is not well typed because its two outputs lie in different fibres. Dependent path action instead gives a path over p , written

$$\text{apd}_f(p) : f(x) =_p^B f(y).$$

This is the first appearance of a recurring idea: motion in the base changes the classifier in which the endpoint lives. In the equality-local fragment, the motion is invertible and is handled by path induction. In a directed family over a category, Chapter 2 will replace it with an explicit functor between fibres.

Formal status — checked. Evidence `TT-EQUALITY-INDUCTION` covers reflexivity, right-based dependent equality induction, ap , and apd . Emdash does not assume equality reflection: possessing a term $p : x=y$ does not turn an arbitrary expression containing x into the same runtime term containing y .

1.5 Propositions As Classifiers

Under the broad propositions-as-types reading, to prove a statement is to construct an inhabitant of its classifier. Conjunction is represented by a pair, implication by a function, universal quantification by a dependent function, and existential data by a dependent pair. Negation of A is the function classifier

$$\neg A := A \longrightarrow \mathbf{Empty}.$$

This reading is constructive: an inhabitant of an existential Sigma type contains a witness, and an inhabitant of an implication is an operation on evidence. It does not make every classifier proof-irrelevant. A Boolean, for example, carries the information of which constructor was chosen. Chapter 3 will isolate the narrower classifiers whose inhabitants are all equal and will call those *proposition-valued*.

No classical principle is needed for the WalkingEnd calculation. The negative results there are constructive functions: an alleged equality or inverse is sent to the empty classifier by inspecting its Nat code.

1.6 From Terms To Actions

The material so far is groupoidal. Identity paths are reversible, and an ordinary function acts on them through `ap`. That is enough for homotopy type theory's slogan that functions behave functorially on paths. Functorial type theory retains this layer but adds a second one: a function between object carriers is not automatically the whole meaning of a directed functor. A directed functor must act on arrows, on arrows between arrows, and so on.

The transition is therefore not from “types” to an unrelated category theory. It is from implicit action generated by identity induction to explicit action over possibly noninvertible arrows. The next chapter makes that action part of the primary syntax.

2. Categories, Functors, And Directed Families

An ordinary type family varies with a term and transports along equality. A directed family varies with an object and acts along arrows that need not be equalities. This small change reorganizes the entire dependent calculus. The action is no longer reconstructed only from path induction: it is presented as a functor and is therefore available at every cell dimension.

The central example already suggests the shape. `Code` is not merely an assignment of a category to the base object of `W`. It must send the generator to the successor functor. The based representable is not merely the collection of hom carriers. It must act on an arrow by postcomposition. The decoder is not merely a family of functions. It must compare these two actions coherently.

For classical category-theoretic background, see [Mac Lane](#). For broader context on directed type theory, see [GPT 5.6 Codex](#). The present calculus makes its own computational and formal choices.

2.1 Categories With Iterated Homs

For a category `A`, write

$$\text{Obj}(A)$$

for its classifier of objects. For $x, y : \text{Obj}(A)$, `HomA(x, y)` does not take `HomA(x, y)` to be only a set or a type of arrows. It is another category. Its objects are 1-arrows $f : x \rightarrow y$; its arrows are 2-cells between such arrows; iterating `Hom` reveals 3-cells and beyond.

Schematically,

$$\begin{aligned} f &: x \longrightarrow y, \\ \alpha &: f \Longrightarrow g, \\ \Gamma &: \alpha \Rrightarrow \beta, \\ &\vdots \end{aligned}$$

are read by repeatedly taking the object classifier of a hom-category. This is the strict/lax omega-oriented *shape* of the implementation. It should not be mistaken for a completed semantics of every weak omega-category: the available composition, action, and coherence interfaces are explicit and grow as consumers require them.

Every object has an identity arrow, and composable arrows have a composite

$$x \xrightarrow{f} y \xrightarrow{g} z \quad \rightsquigarrow \quad g \circ f : x \longrightarrow z.$$

We always read $g \circ f$ as first f , then g . Identity, composition, and their higher actions are global operations of the category calculus. Some laws are selected as runtime reductions; others are exposed as typed propositional or proof-time comparisons. The mathematical reading is the familiar category law, while the implementation distinguishes which direction is a useful normal form.

Formal status — checked. Evidence `CAT-ITERATED-HOMS` covers the category, object, and iterated-hom interface. The phrase “omega-oriented” describes this unbounded hom iteration; it does not claim a finished theory of arbitrary weak coherence.

2.2 The Equality-Local Category

Every groupoid classifier A determines a path category

$$\text{Path}(A).$$

Its objects are elements of A , and its hom from x to y is the path category of the identity classifier $x=y$. Its identity is reflexivity and its composition is path concatenation. Since paths have symmetry, this is the groupoidal fragment of the categorical universe.

An ordinary function $f:A \rightarrow B$ lifts to an iterable functor

$$\text{Path}(f) : \text{Path}(A) \longrightarrow \text{Path}(B).$$

On objects it is f ; on a path it is ap_f ; on higher paths it continues by the generic functor calculus. This is a precise bridge between the type-theoretic material of Chapter 1 and the directed material here.

It is important that the bridge is an inclusion of a special case, not a collapse. A directed category C may contain an arrow $x \rightarrow y$ without any path $x=y$. Conversely, an object path can be mapped to an arrow through the equality-local core inclusion. Later equivalence interfaces state when that inclusion captures all arrows.

Formal status — checked. Evidence `CAT-PATH-CATEGORY` covers the recursive path category and the functor induced by an ordinary function. No rule identifies a general directed hom-category with an equality type.

2.3 Functors Act At Every Dimension

A functor $F:A \rightarrow B$ has an object action

$$x \longmapsto F[x]$$

and, for every pair x, y , a hom functor

$$F_1[x, y] : \text{Hom}_A(x, y) \longrightarrow \text{Hom}_B(F[x], F[y]).$$

Evaluating the hom functor at an arrow f gives the familiar $F[f]$. Because the result before evaluation is itself a functor, the same construction already acts on 2-cells between f and g , then on higher cells. The notation suppresses this projection ladder, but the structure does not stop at the first arrow.

Functoriality has the expected equations

$$F[\text{id}_x] = \text{id}_{F[x]}, \quad F[g \circ f] = F[g] \circ F[f].$$

In the active calculus these are owned globally. A named construction that is already a functor does not receive a private copy of the identity and composition laws. This is more than code reuse: it expresses the foundational thesis that substitution-like operations should be internalized until their action and coherence come from the generic functor structure.

Formal status — checked. Evidence `CAT-FUNCTOR-CALCULUS` covers object action, full hom action, capped arrow action, and the generic identity and composition surfaces.

2.4 Transforms And Naturality

Given functors $F, G: A \rightarrow B$, the next hom-category is written

$$\text{Transf}(F, G).$$

An object $\eta: F \Rightarrow G$ has a component at every object,

$$\eta[x] : F[x] \longrightarrow G[x].$$

Its action over an arrow $f: x \rightarrow y$ expresses naturality. In a 1-categorical shadow, this is the familiar square comparing

$$G[f] \circ \eta[x] \quad \text{and} \quad \eta[y] \circ F[f].$$

In the iterated-hom presentation, naturality is not an external proposition attached after the components are chosen. It is part of the higher action of the transform object. Further homs between transforms provide higher transforms and their components.

Vertical composition is computed componentwise, and identity transforms have identity components. Whiskering and functor action provide horizontal interaction. Chapter 8.2 uses precisely this infrastructure when the two compositions on 2-endomorphisms meet in the Eckmann–Hilton calculation.

Formal status — checked. Evidence `CAT-TRANSFOR-CALCULUS` covers transforms, their point components, and the next naturality projection. The book uses “transform” for the dimension-uniform notion and “transformation” when discussing its ordinary categorical shadow.

2.5 The Directed Universe And Its Families

The category of categories has

$$\text{Obj}(\text{Cat}) = \text{Cat}, \quad \text{Hom}_{\text{Cat}}(A, B) = \text{Functor}(A, B).$$

Thus an arrow in the categorical universe is a functor. A Cat -valued directed family over K is consequently a functor

$$E : K \longrightarrow \text{Cat}.$$

We write its fibre over k as $E[k]$. A base arrow $p : k \rightarrow l$ induces the reindexing functor

$$E[p] : E[k] \longrightarrow E[l].$$

This is covariant directed transport. It can carry an object $u : E[k]$ forward to $E[p](u)$ even when p has no inverse. Ordinary path transport is recovered when K is a path category, but directed transport is strictly more general.

A morphism between families $E, D : K \rightarrow \text{Cat}$ is a displayed or fibrewise functor

$$\Phi : E \Longrightarrow D.$$

It has a functor $\Phi[k] : E[k] \rightarrow D[k]$ in each fibre. Over a base arrow $p : k \rightarrow l$, its coherent comparison has the directed form

$$D[p](\Phi[k](u)) \longrightarrow \Phi[l](E[p](u)).$$

The arrow points from “map, then transport in D ” toward “transport in E , then map.” It need not be an identity and need not be invertible. This is the first elementary appearance of laxity in the dependent calculus. Strict/cartesian constructions may make the comparison compute to an identity, but that is additional structure or focused computation, not the default meaning of a family morphism.

Formal status — checked. Evidence `CAT-DIRECTED-FAMILIES` covers Cat -valued families, fibres, arrow reindexing, fibre functors, and the canonical displayed comparison cell. A general directed family is not required to send its base arrows to equivalences.

2.6 Totals And Sections

A family can be read in two complementary ways. Its **total category** gathers all fibre objects together:

$$\sum_{k:K} E[k].$$

An object is a pair (k, u) with $u : E[k]$. An arrow

$$(k, u) \longrightarrow (l, v)$$

contains a base arrow $p : k \rightarrow l$ and a fibre arrow

$$\alpha : E[p](u) \rightarrow v$$

in $E[l]$. The canonical transport arrow is the special case

$$(p, \text{id}_{E[p](u)}) : (k, u) \rightarrow (l, E[p](u)).$$

This description makes directed dependence concrete. Transport is an actual arrow in the total category, and its direction is inherited from the base.

The **section category** is the dependent product

$$\prod_{k:K} E[k].$$

A section s assigns an object $s[k] : E[k]$ and, for every base arrow, a coherent comparison

$$s[p] : E[p](s[k]) \rightarrow s[l].$$

An arrow between sections is not an arbitrary pointwise family of fibre arrows. Its components must be natural over the base. This distinction disappears over a trivial base but is essential over a directed one.

For a constant family with fibre A , the total is the product category $K \times A$, while the section category is the functor category $\text{Functor}(K, A)$. These are useful checks on the definitions: a section of a constant family is precisely a functorial choice varying over K .

Formal status — checked. Evidence `CAT-SIGMA-PI` covers the Sigma total, its canonical transport arrow, the Pi section facade, and section evaluation. Some broader Sigma/Pi adjunctions and generic section-total packaging remain future work; they are not prerequisites for Chapter 8.

2.6.1 Fibrewise Contexts

Totals make genuine dependence visible, but not every pair of declarations over a base depends on one another. Let $B, C : K \vdash \mathbf{Cat}$ be two families over the same category. Their fibrewise product is the transparent family

$$P(B, C)[k] := B[k] \times C[k], \quad P(B, C)[p] := B[p] \times C[p].$$

The second formula uses the same base arrow p in both components. It is the action required by a context of independent siblings

$$k : K, \quad b : B[k], \quad c : C[k].$$

This differs from a genuinely dependent context

$$k : K, \quad a : A[k], \quad b : B[k, a].$$

In the first context, b and c may be projected, paired, exchanged, or contracted while the common prefix k is fixed. In the second, exchanging a and b without transporting the classifier of b is generally ill-typed.

The fixed-base structural maps are

$$\begin{aligned} \text{projL}_d(B, C) &: P(B, C) \Longrightarrow B, \\ \text{projR}_d(B, C) &: P(B, C) \Longrightarrow C, \\ \text{pair}_d(\Phi, \Psi) &: E \Longrightarrow P(B, C), \end{aligned}$$

where $\Phi : E \Longrightarrow B$ and $\Psi : E \Longrightarrow C$. They satisfy the displayed product cuts

$$\text{projL}_d \circ \text{pair}_d(\Phi, \Psi) \rightsquigarrow \Phi, \quad \text{projR}_d \circ \text{pair}_d(\Phi, \Psi) \rightsquigarrow \Psi.$$

Their fibre functors, base-arrow actions, and selected internalized higher cells compute componentwise. Exchange and contraction are consequently derived maps:

$$\begin{aligned} \text{swap}_d(B, C) &:= \text{pair}_d(\text{projR}_d, \text{projL}_d), \\ \text{diag}_d(B) &:= \text{pair}_d(\text{id}_d, \text{id}_d). \end{aligned}$$

The family $P(B, C)$ is built from ordinary categorical product functoriality; it is not a new primitive family former. Reindexing a grouped context is presented canonically by reindexing its two families and rebuilding their fibrewise product.

Formal status — checked. Evidence `CAT-FIBREWISE-CONTEXT` covers the transparent fibrewise family, its displayed projections and pairing, componentwise object, arrow, and selected higher action, the two product cuts, and the derived swap and diagonal. It does not license exchange across a genuine dependency edge.

2.6.2 Base Change And Evaluation

A functor $F : A \vdash K$ can change the base of a family $D : K \vdash \mathbf{Cat}$. The pulled-back family F^*D has fibre $(F^*D)[a] = D[F[a]]$. Its total category carries a canonical functor

$$\text{Tot}(F, D) : \sum_{a:A} (F^*D)[a] \longrightarrow \sum_{k:K} D[k]$$

with computations

$$(a, u) \longmapsto (F[a], u), \quad (p, \alpha) \longmapsto (F[p], \alpha).$$

The base component is acted on by F , while the fibre component is retained. This is the asymmetric Grothendieck totalization of family reindexing. It is not a claim that arbitrary functors between total categories admit a computational pullback.

The same fibrewise structure supports coherent evaluation. Fix an ordinary category A and a family $B : K \vdash \mathbf{Cat}$, and write

$$S(A, B)[k] := \mathbf{Functor}(A, B[k]).$$

Then

$$\mathbf{Eval}_d(B) : P(S(A, B), \mathbf{Const}_K(A)) \Longrightarrow B$$

projects in each fibre to ordinary functor evaluation:

$$\mathbf{Eval}_d(B)[k] = \mathbf{Eval}(A, B[k]).$$

Weakening is supplied by the displayed terminal map

$$\mathbf{Terminal}_d(E) : E \Longrightarrow \mathbf{Const}_K(1).$$

Composing it with a constant section yields a fixed argument. Pairing that argument with a varying functor and applying $\mathbf{Eval}_d$ therefore interprets application without asking the user to supply a separate naturality square. Base-arrow and higher action are inherited from the internal functorial calculus.

Formal status — checked. Evidence `CAT-BASE-CHANGE-TOTALIZATION` covers the object and arrow action of asymmetric pullback totalization. Evidence `CAT-DISPLAYED-EVALUATION` covers constant-domain displayed evaluation, terminal weakening, and their retained generic base and higher action. Arbitrary mixed-domain evaluation remains outside this result.

2.7 Representables And Variance

Fix $x : K$. The covariant fixed-source representable family is

$$\mathbf{Rep}_x[k] := \mathbf{Hom}_K(x, k).$$

For $p : k \rightarrow l$, it acts by postcomposition:

$$\mathbf{Rep}_x[p](q) = p \circ q.$$

This is exactly the action used in the WalkingEnd decoder: a based arrow $q : * \rightarrow k$ is carried along p to $p \circ q : * \rightarrow l$.

The source variable is contravariant. Given $r : x \rightarrow y$, precomposition defines a family morphism

$$\mathbf{Rep}_y \Longrightarrow \mathbf{Rep}_x, \quad q \longmapsto q \circ r.$$

Keeping these two variances distinct is not pedantry. Postcomposition changes the target of an arrow; precomposition changes its source. Emdash gives them separate runtime owners and compares alternate presentations only where the types justify it.

Formal status — checked. Evidence `CAT-REPRESENTABLE` covers the fixed-source representable and its contravariant source action. Generic functoriality owns its action in the varying target.

2.8 The Interfaces Needed By The Main Proof

The Chapter 8 calculation uses only a compact part of this chapter:

1. `Code:W -> Cat` is a directed family, so every based arrow `p:* -> x` acts by a functor `Code[p]`.
2. The target family is the representable `Rep_*`, whose base-arrow action is postcomposition.
3. Natural powers form a functor from `Path(Nat)` to the based hom-category, not merely a function on objects.
4. A contextual decoder is a family morphism from `Code` to `Rep_*`; its base-arrow comparison produces the normalization cell.
5. The ordinary and higher functor laws supply identity, composite, and cell action without WalkingEnd-specific copies.

The proof therefore depends on category theory at exactly the point where a carrier-only argument would lose direction. The next chapters add the logical, equivalence, induction, and height interfaces needed to turn that directed construction into a numerical classification.

3. Logic, Propositions, And Sets

The propositions-as-types reading begins with a useful ambiguity. Any classifier may be read as a proposition: to prove it is to construct an inhabitant. But not every classifier behaves like a truth value. An inhabitant of `Bool` tells us which Boolean was chosen; a natural number contains still more data. Logic becomes precise only after we distinguish *having evidence* from *having no relevant choice of evidence*.

This distinction is particularly important in a directed foundation. A classifier may carry higher identity information even when the surrounding category also carries noninvertible arrows. Proposition and set evidence control the groupoidal identity layer; they do not erase directed structure.

3.1 Propositions As Types, Broadly Read

The basic logical readings are constructive:

statement	classifier
P and Q	$P \times Q$
P implies Q	$P \longrightarrow Q$
for every $x : A$, $P(x)$	$\prod_{x:A} P(x)$
there is $x : A$ with $P(x)$	$\sum_{x:A} P(x)$
falsehood	<code>Empty</code> .

Under this reading, a proof of an existential statement contains a witness and its evidence. A proof of an implication is an operation that transforms evidence. Negation is

$$\neg P := P \longrightarrow \text{Empty}.$$

This is enough to state the negative theorems of Chapter 8. To prove that the walking loop is not the identity, for instance, one assumes an equality and constructs an inhabitant of `Empty` by mapping it to an impossible `Nat` equality.

What this reading does not supply is automatic proof irrelevance. If `P` has two inhabitants, the bare fact that both prove a statement says nothing yet about whether they are equal. That is a separate property of `P`.

3.2 Contractibility And Proposition-Valuedness

A classifier `A` is **contractible** when it has a chosen centre and a path from that centre to every point:

$$\text{isContr}(A) := \sum_{a_0:A} \prod_{a:A} (a_0 = a).$$

Contractibility contains both existence and uniqueness, with uniqueness understood homotopically. The centre is data, but the total classifier of contractibility evidence is itself proposition-valued: any two such packages are equal.

A classifier is **proposition-valued** when its identity classifiers are contractible. In the familiar shorthand, any two of its inhabitants are equal, and there is no further choice among the paths witnessing that fact. It may be inhabited, like `Unit`, or uninhabited, like `Empty`. If it is inhabited, it is contractible.

The adjective “proposition-valued” is deliberate. We still use every classifier as a proposition in the broad Curry–Howard reading. The additional property says that inhabitation carries no mathematical information beyond truth.

Formal status — checked. Evidence `LOGIC-TRUNCATION-PREDICATE` covers `IsContr` and the recursive truncation predicate whose `-1` and `0` specializations are proposition- and set-valuedness.

3.3 Sets As A Homotopical Property

A classifier `A` is **set-valued** when every identity classifier `x=y` is proposition-valued. Equivalently, any two parallel paths in `A` are equal. This is the type-theoretic notion of a set: it concerns the height of identity information, not membership in a global cumulative set universe.

The hierarchy begins

<code>-2</code>	:	contractible classifiers,
<code>-1</code>	:	proposition-valued classifiers,
<code>0</code>	:	set-valued classifiers,
<code>1</code>	:	groupoid-valued classifiers,
	:	

and each successor level asks that every identity classifier lie one level lower. Chapter 7 develops this recursion uniformly. For now, sethood provides the exact uniqueness principle needed for arithmetic codes.

`Unit` is proposition-valued because it is contractible. `Empty` is proposition-valued by elimination: from an alleged first inhabitant one may derive the required path data. `Nat` is set-valued by nested induction on its visible constructors. Equality of zero with a successor reduces to `Empty`; equality of two successors reduces to predecessor equality; the remaining cases reduce to the proposition evidence for `Unit` or `Empty`.

Formal status — checked. Evidence `LOGIC-NAT-SETHOOD` supplies explicit terms witnessing that `Unit` and `Empty` are proposition-valued and `Nat` is set-valued. `Nat` sethood is not inferred merely from a rewrite table; it is an inhabitant of the internal `IsSetGrpd(Nat)` classifier.

3.4 Evidence Is Not Erasure

It is tempting to read “being a property” as permission for the kernel to erase every proof to one constant. That is not the policy here. Emdash retains evidence terms and proves that the classifier containing them is proposition-valued.

For every level `n` and classifier `A`, the classifier

`isTruncn(A)`

of evidence that `A` lies at level `n` is proposition-valued. Hence two truncation witnesses are equal, but their constructors and projections remain available to computation. This matters for packaged universes: a package can retain both a carrier and its truncation evidence while still having equality controlled by the carrier.

The same style recurs for equality-valued equivalence evidence. Rather than declaring inverse-law proofs irrelevant by fiat, the theory proves that their classifier is a proposition. Retained evidence supports computation and transport; proposition-valuedness supports uniqueness and truncation.

Formal status — checked. Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP` establishes property-valuedness at every recursive truncation level. It introduces no proof-erasure rewrite.

3.5 Constructive Reasoning

The foundational rules used in this book are constructive. To prove `P or Q` one must select a side in whatever sum-like interface is in scope; to prove an existential one must provide a witness; to refute a claim one must map its evidence to `Empty`. The law of excluded middle and double negation elimination are not silently used as global inference rules.

This does not forbid classical mathematics. A classical principle can be studied as additional structure, restricted to a suitable proposition-valued universe, or derived in a context where decidability data are available. The point is bookkeeping: a proof should reveal when such data enter.

The `WalkingEnd` theorem needs none. Its code is computed by functor action, its inverse laws by contextual elimination and `Nat` induction, and its noninvertibility by successor/zero discrimination. The argument is therefore valid in the constructive core presented here.

Formal status — mathematical development. This section states the logical discipline of the book. It does not claim a complete internal library of all connectives, propositional truncations, choice principles, or classical axioms.

3.6 Two Uses Of Sethood In The Main Calculation

Sethood enters Chapter 8 in two distinct ways.

First, Nat sethood controls equality in the code fibre $\text{Path}(\text{Nat})$. It ensures that higher coherence between arithmetic equalities is unique at the needed level. This lets the power construction lift equality of naturals without introducing uncontrolled higher data.

Second, the based hom carrier is proved set-valued after the calculation. One proof comes directly from the one-dimensional signature of $\mathbb{W}$; another transports Nat sethood backward across the carrier equivalence. Agreement of the conclusions does not collapse the proofs: each exposes a different reason for sethood.

This is typical of univalent foundations. A property may be reached through dimension, through equivalence, or through a direct induction. Since the classifier of evidence is proposition-valued, the resulting witnesses agree, yet their constructions remain mathematically informative.

4. Equivalence And Univalence

Univalence is often summarized by the phrase “equivalent things may be identified.” In a directed setting, that sentence must be typed with care. Equivalent *classifiers*, isomorphic objects of a 1-category, recursively invertible arrows, and arbitrary functors are different notions. The theory becomes clearer, not more cumbersome, when each receives its own interface.

The WalkingEnd example is the test. Its generator is a perfectly good arrow, and `Code` sends it to a perfectly good functor, but neither is an equivalence. At the same time, the final comparison between its endomorphism carrier and `Nat` is a type equivalence. Univalent foundations must support both statements without forcing one into the shape of the other.

4.1 Maps With Contractible Fibres

For a function `f : A -> B` and `b : B`, its homotopy fibre is

$$\text{fib}_f(b) := \sum_{a:A} (f(a) = b).$$

The function is an equivalence when every such fibre is contractible. A `TypeEquiv(A,B)` packages `f` with this evidence. The centre of the fibre over `b` selects an inverse value `f-1(b)`, and its path component gives

$$f(f^{-1}(b)) = b.$$

Contractibility of the fibre also yields the other law

$$f^{-1}(f(a)) = a.$$

This definition has two advantages. It treats being an equivalence as a property of a fixed map, and it supplies a canonical selected inverse from the contractible-fibre evidence. Conversely, explicit inverse data with both homotopies can be adjusted and converted into contractible-fibre evidence. That is the route used by the WalkingEnd carrier comparison: `encode` and `power` are built first, their two round trips are proved, and then the equivalence package is formed.

Formal status — checked. Evidence `EQUIV-TYPE` covers homotopy fibres, the equivalence-map predicate, `TypeEquiv`, its selected inverse, and both inverse paths.

4.2 Algebra Of Type Equivalences

Equivalences have reflexivity, symmetry, and composition:

$$\begin{aligned} \text{id}_A &: A \simeq A, \\ e^{-1} &: B \simeq A, \\ e_{BC} \circ e_{AB} &: A \simeq C. \end{aligned}$$

Composition is written in categorical order; the forward map of the last line sends `a` first through `e_AB` and then through `e_BC`. The inverse laws are constructed from explicit quasi-inverse data and the path algebra of Chapter 1.

Not every mathematically true equation between equivalence packages is chosen as a runtime eta rule. What computes is the public interface needed by consumers: the selected forward map of reflexivity, symmetry, and composition, together with designated projections of inverse data. Remaining coherence is propositional. This is the same separation between computation and evidence that we saw for Nat associativity.

Formal status — checked. Evidence `EQUIV-TYPE-ALGEBRA` covers reflexivity, symmetry, composition, and their selected forward-map computation.

4.3 Univalence In The Groupoid Universe

Every universe path `p : A=B` induces a type equivalence by transporting elements:

$$\text{idtoequiv}(p) : A \simeq B.$$

Univalence supplies the converse in a coherent way. In the groupoid universe, `emdash` has an operational decoder

$$\text{ua}(e) : A = B$$

and named round trips

$$\text{ua}(\text{idtoequiv}(p)) = p, \quad \text{idtoequiv}(\text{ua}(e)) = e.$$

Transport along the decoded path agrees propositionally with the forward map of `e`. The qualification “propositionally” is important. A single operational decoder owns the comparison; the kernel does not turn every universe transport expression into arbitrary equivalence application by a broad rewrite.

This gives the groupoid/type layer its univalent reading: identity in the universe and equivalence of decoded classifiers carry the same information through the named interface. It does not say that a noninvertible arrow in an arbitrary category is an identity.

Formal status — checked. Evidence `UNIV-GROUPOID` covers the decoder-oriented groupoid-univalence equivalence, both round trips, and the selected transport agreement.

4.4 Universes With Retained Truncation Evidence

For each truncation level `n`, the theory also packages a classifier together with evidence that it is `n`-truncated:

$$\mathbf{TruncU}_n := \sum_{A:\mathbf{Grpd}} \mathbf{isTrunc}_n(A).$$

The evidence is retained, not erased. Nevertheless, because truncation evidence is proposition-valued, equality of packages is controlled by equality of their carriers. Combining that fact with groupoid univalence gives the restricted comparison

$$(A, h_A) = (B, h_B) \simeq A \simeq B.$$

The familiar universes of propositions, sets, and groupoids are the `-1`, `0`, and `1` cases of this package. This is a concrete example of structure identity: once a field is proposition-valued, identifying the carrier supplies the dependent identification of that field.

Formal status — checked. Evidence `UNIV-TRUNCATED` covers the carrier-equivalence view of paths between packaged truncated classifiers and its named encode/decode laws. It is a restricted univalence theorem, not a truncation reflector.

4.5 Three Categorical Notions Of Invertibility

At the categorical layer we distinguish three interfaces.

An ordinary isomorphism between objects `x, y : C` consists of a forward arrow, one inverse arrow, and equality-valued left and right inverse laws:

$$f : x \rightarrow y, \quad g : y \rightarrow x, \quad g \circ f = \text{id}_x, \quad f \circ g = \text{id}_y.$$

For a fixed arrow `f`, the native classifier `OmegaEquivAlong(f)` carries recursively equality-valued inverse evidence. Its selected left and right inverse arrows may be presented separately, with their laws living in the next hom-categories. Finally, `OmegaEquiv(C, x, y)` packages a chosen forward arrow together with such evidence.

The distinction allows the interface to remain iterable. Inverse laws at one dimension are equality data in a hom-category, where the same machinery can act again. An ordinary isomorphism has a checked one-way lift into the native omega-equivalence package, but the notions are not collapsed definitionally.

Formal status — checked. Evidence `EQUIV-OMEGA` covers the fixed-arrow and packaged equality-valued interfaces and the transparent package built from an object path. Evidence `EQUIV-ORDINARY-ISO-LIFT` covers the one-way lift from ordinary isomorphism evidence.

For an object path `p : x = y`, path induction constructs a forward arrow, an inverse, and both cancellation laws. Thus equality can be *reified* as a computational equivalence package. A bare classifier-level cast may permit an identity term to be used at the stable equivalence facade, but it does not magically give that term the projections of a package; observable computation uses the explicit construction.

The converse direction has a more delicate status. The current native surface supports selected equality/equivalence casts and rigid universe boundaries, but a full general equivalence between arbitrary object equality and ordinary 1-categorical isomorphism evidence is not a compatibility prerequisite and is not claimed here.

Formal status — research boundary. Evidence `UNIV-FULL-OBJECT-ISO` records the intentionally absent two-sided ordinary-isomorphism/object-equality theorem. Its absence does not weaken the checked path-to-equivalence construction used elsewhere in the book.

4.6 Equivalence Evidence Is A Property

For a fixed arrow `f`, two complete choices of equality-valued inverse evidence are equal. The proof does not erase the choices. It shows that the left- and right-inverse fibres are contractible, using ordinary associativity, unit laws, and the supplied inverse equations, and then transfers that contractibility to the native record.

Consequently,

$$\text{OmegaEquivAlong}(f)$$

is proposition-valued for every category and fixed arrow, without assuming that the category has finite height or locally set-valued homs. This is the categorical analogue of the fact that “is an equivalence” should be a property of a map even though an explicit quasi-inverse package may contain choices.

Formal status — checked. Evidence `EQUIV-EVIDENCE-PROP` is the dimension-independent property theorem for the native fixed-arrow evidence.

4.7 Hom Action And Groupoidal Sources

Equivalence evidence must itself act functorially if the foundation is to be usable at higher dimension. The one-way derived hom-action layer sends fixed-arrow omega-equivalence evidence through the next hom action of an ordinary functor. In particular, if a category is coherently groupoidal, its core inclusion is an omega-equivalence and each directed arrow can be related to an object path through the selected homwise inverse.

For a directed family `D : C -> Cat`, coherent groupoidality of `C` then implies that transport along a base arrow is an equivalence:

$$D[f] : D[x] \simeq D[y].$$

This recovers the familiar groupoidal behavior of path-indexed families as a special case. It also states the boundary sharply: without groupoidality, `D[f]` remains only a functor.

Formal status — checked. Evidence `EQUIV-HOM-ACTION` covers the one-way next-hom action and its specialization to equivalence of fibre transport over a coherently groupoidal source.

4.8 Why Successor Needs No Univalence In Chapter 8

The circle's universal cover sends an invertible loop to successor on the integers. To define it as a universe-valued family, one first recognizes successor as an equivalence and then uses univalence to obtain a universe path.

The WalkingEnd code has a different type:

Code : $W \longrightarrow \text{Cat}$.

Its generator is a directed arrow in W , so its image need only be a functor. Natural-number successor qualifies directly even though it misses zero and has no inverse. Forcing univalence into this step would change the mathematics by demanding reversibility where the signature deliberately has none.

The carrier equivalence at the end of the proof is nevertheless genuinely a type equivalence, and the surrounding universes retain their checked univalence interfaces. The lesson is not that univalence disappears. It is that univalence governs equivalence and identity, while directed functoriality also governs noninvertible action.

Chapter 26 returns to the other side of this contrast with the now-active Circle construction. Integer successor is first packaged as an equivalence; univalence then turns it into the universe loop used by the universal cover. The WalkingEnd and Circle codes therefore share an encode–decode rhythm while retaining different reasons why their monodromy is well typed.

5. Induction, Arrow Induction, And Universal Properties

Induction is often introduced as a collection of unrelated proof rules: one for natural numbers, another for equality, and still others for higher inductive types. A more durable view is that an inductively presented object comes with a way to extend compatible data from its generators. Recursion extends nondependent data; induction extends data in a family; contextual elimination lets both the source and target vary. The increasing amount of context is not bureaucratic overhead. It is what records naturality.

In a functorial setting, this extension principle should itself preserve arrows and higher cells. That demand turns a familiar logical rule into a categorical construction. This chapter begins with ordinary induction, then develops the outgoing-arrow category and the arrow-induction interface used throughout *emdash*. The result is a bridge between equality induction and the contextual eliminator of the walking endomorphism.

5.1 Data On Generators

For the natural numbers, dependent induction has the familiar shape. Given a family $P(n)$, a base element

$$z : P(0),$$

and a step operation

$$s_n : P(n) \longrightarrow P(n + 1),$$

there is a section

$$\text{ind}_{\mathbb{N}}(z, s) : \prod_{n:\mathbb{N}} P(n)$$

whose observations at zero and successor compute to the supplied data. If P is constant, the same interface is recursion. Thus recursion is not a second principle: it is dependent elimination with no meaningful dependency.

The same pattern appears for a category presented by an object and an endomorphism. To map it into a category $\mathcal{C}$, one supplies an object of $\mathcal{C}$ and an endomorphism of that object. To define a dependent section, one supplies an object in the fibre over the generating object and a lift over the generating arrow. To compare two varying families, one supplies a functor between their base fibres and a coherent cell over the generator. Chapters 6 and 8 will instantiate precisely these three levels.

This perspective separates two questions:

1. what data an eliminator accepts; and

2. how its observations compute at the constructors.

Both matter. Existence without computation cannot drive a normalization proof; computation without a coherent extension does not define an action on the whole object.

In the rule schema of [Appendix G.4](#), generator data are the introduction clauses, the induced section or functor is elimination, and the constructor observations are computation. Uniqueness or initiality is a separate universal clause, while coherent action on arrows and higher cells is the specifically functorial clause. We will report all five independently rather than treating the word “induction” as evidence for whichever clauses have not yet been supplied.

5.2 Equality Induction As The Local Case

For a type A , right-based equality induction says that, with the right endpoint y fixed, a family

$$C(x, p) \quad (x : A, p : x = y)$$

is determined by its value at (y, refl_y) . In one conventional orientation,

$$\frac{d : C(y, \text{refl}_y)}{J(d, x, p) : C(x, p)}.$$

The reflexive observation computes to d . Ordinary action on paths and dependent action on paths are specializations of this principle. They explain why a function respects equality and how a section over a family produces a dependent path.

Formal status — checked. Evidence `TT-EQUALITY-INDUCTION`. The equality-local layer provides reflexivity, right-based dependent induction, ordinary path action, and dependent path action.

Equality induction is already functorial when it is read inside the path category $\text{Path}(A)$: objects are elements of A , arrows are equalities, and every arrow is invertible. But this is a special, groupoidal base. A directed category contains arrows that need not have inverses, so we want an induction interface whose statement does not presuppose that a given arrow is an equality.

5.3 The Category Of Outgoing Arrows

Fix a category Z and an object x . The category of arrows leaving x is the total category of the represented family

$$\text{PathOut}_Z(x) := \sum_{y: Z} \text{Hom}_Z(x, y).$$

Its objects are pairs (y, p) with $p : x \rightarrow y$. It has a distinguished reflexive object

$$\text{reflout}_x := (x, \text{id}_x).$$

For every outgoing arrow $p : x \rightarrow y$, functorial action of the representable family supplies a canonical total arrow

$$\rho_{x,y,p} : (x, \text{id}_x) \longrightarrow (y, p) \quad \text{in } \text{PathOut}_Z(x).$$

Its base component is p ; its fibre endpoint reduces by the unit law $p \circ \text{id}_x = p$. The direction of ρ is significant. It starts at the reflexive arrow and reaches p . Nothing in the construction produces a reverse arrow, and no invertibility premise is used.

Formal status — checked. Evidence IND-PATHOUT . PathOut_cat is the Sigma total of the fixed-source representable, pathout_refl_obj is its distinguished object, and $\text{pathout_refl_arrow}$ is the canonical Sigma transport arrow ρ .

The name “PathOut” emphasizes an analogy, not an identification. When $Z = \text{Path}(A)$, its objects really are equality paths out of x . For a general Z , they are directed arrows. The same categorical shape therefore hosts both ordinary path induction and a genuinely directed extension principle.

The source object also varies contravariantly. An arrow $r : x \rightarrow y$ induces a functor

$$\text{PathOut}_Z(y) \longrightarrow \text{PathOut}_Z(x), \quad (z, q) \longmapsto (z, q \circ r).$$

Precomposition changes which object the arrows leave, while postcomposition changes where they arrive. Keeping these variances separate will prevent a common mistake in Chapter 8: the based representable there varies covariantly in its endpoint, not contravariantly in its source.

5.4 Fixed-Source Arrow Induction

Let

$$E : \text{PathOut}_Z(x) \longrightarrow \text{Cat}$$

be a directed family. An element

$$u \in E[\text{reflout}_x]$$

can be transported along every canonical arrow ρ . This produces a section

$$\text{path_ind}(x, E, u) : \prod_{q : \text{PathOut}_Z(x)} E[q],$$

whose readable value is

$$\text{path_ind}(x, E, u)(y, p) = E[\rho_{x,y,p}](u).$$

This formula is arrow induction: data at the identity arrow extends along every outgoing arrow. Because E is Cat-valued, the result is a section with coherent action, rather than merely a function that chooses one object in each fibre.

Formal status — checked. Evidence `IND-ARROW`. The fixed-source section is `path_ind_sec`; `PathInd_func` packages its action in the motive and base datum, and `PathInd_transfd` internalizes the theorem while the source object varies.

There are three layers here, and it helps not to conflate them:

- `path_ind_sec` is the readable theorem at a fixed source;
- `PathInd_func` says that this theorem acts functorially on motives and their data;
- `PathInd_transfd` records coherence as the source object moves.

The third layer uses the contravariant source action of `PathOut` and the corresponding pullback of motives. Its naturality is internal to the displayed transformation. One need not append an external family of hand-written commuting squares.

5.5 Composition As A Diagnostic

An abstract eliminator earns its keep by recovering an operation we already understand. Fix `x` and give `(y, p)` the represented motive

$$E(y, p) := (\text{Rep}_Z(y) \longrightarrow \text{Rep}_Z(x)).$$

At the reflexive arrow, the identity transformation supplies the base datum. Arrow induction then extends it to every $p : x \rightarrow y$. Evaluating the result at $q : y \rightarrow z$ gives

$$q \circ p : x \longrightarrow z.$$

Thus ordinary categorical composition appears as transport from the identity case. The resulting operation is still packaged functorially, so its action on higher cells remains available to later constructions.

Formal status — checked. Evidence `IND-COMPOSITION`. `CompMotive_catd` is the represented motive, `path_comp_sec` is the induced section, and `path_comp_func` exposes composition as the object action of a functor.

This benchmark is more informative than a proof that some carrier-level function exists. It checks the direction of `rho`, the variance of the representable, and the agreement of the generic Sigma and Pi constructions. If any of those are reversed, the result has the wrong endpoints before one even asks for associativity.

From mathematical notation to executable evidence

There is now a small reviewer-facing notation for following this construction through the TypeScript implementation:

```

PathOut(Z, x)
rho(Z, x, y, p)
Ind(Z, x, E, u)
compose(Z, x, y, z, p, q)

```

These are four views of the development above: the outgoing-arrow category, its canonical arrow from the reflexive object, the induced section, and the value of the composition functor at q . They are not a second foundation or a new declaration language. Parsing one of them merely records which construction is being discussed, with its variables and source location.

This distinction matters in a browser. The browser can display that the corresponding construction was qualified at the pinned semantic checkpoints, but it says explicitly that it has not rerun the semantic check. An explicit Node invocation such as `./scripts/emdash pathout check composition-normal-form` instead assembles the existing theory profile, checks the resulting explicit Core term with the TypeScript checker, and compares the composition term with its reviewed normal form. Only that result is labelled a fresh TypeScript semantic check; the first assembly in a process may take several minutes.

The division of labour mirrors the mathematics. A small sealed profile owns the primitive path-induction constants and their exact computation rules; the `PathOut`, induction, and composition constructions remain transparent library material built from them. TypeScript/emdash is the production checker for this presentation, while `Lambdapi` remains a separately bounded conformance oracle. The notation therefore makes the evidence easier to inspect without changing what counts as evidence.

5.6 Returning To Literal Equality

Set $Z = \text{Path}(A)$ and let D be a directed family over that path category. There are now two ways to move u along an equality $p : x = y$:

1. use the functorial action of D on the arrow p ; or
2. use primitive equality induction with a function-valued motive.

They need not be definitionally the same expression. The active calculus proves that they are propositionally equal:

$$\text{transport}_D^{\text{structured}}(p, u) = \text{transport}_D^J(p, u).$$

Formal	status	—	checked.	Evidence	IND-PATH-COMPARISON
<code>path_cat_structured_transport_agrees_ind_eqr</code> compares the existing directed-family action with primitive right-based equality induction. It does not introduce a second equality eliminator or turn the comparison into a global rewrite.					

This is the precise relation between the equality-local and directed views. Functorial transport extends beyond equality paths; on a literal path category, it agrees with `J` by a theorem. The groupoidal case is therefore recovered without making all directed arrows groupoidal.

5.7 Contextual Elimination

Fixed-source arrow induction varies a motive over outgoing arrows. A contextual eliminator goes one step further: it compares two families over an inductively presented base. Schematically, suppose $R, D : K \rightarrow \mathbf{Cat}$. At each base object it should provide a functor

$$T_k : R[k] \longrightarrow D[k],$$

and for each base arrow $f : k \rightarrow k'$ a directed comparison

$$D[f] \circ T_k \Longrightarrow T_{k'} \circ R[f].$$

This is exactly the data of a displayed functor in the lax direction selected by the calculus. If R is terminal, a displayed functor becomes a section of D . If both R and D are constant, it becomes an ordinary functor out of K . Section induction and recursion are therefore special cases of contextual elimination.

For an inductively presented K , the desired eliminator should build the whole displayed functor from compatible data at the constructors. The `WalkingEnd` interface of Chapter 6 realizes this pattern for one point and one directed loop. Chapter 8 then uses the genuinely contextual form: its source is the code family and its target is a representable family, so neither side can be discarded as mere bookkeeping.

5.8 Universal Properties: What Is And Is Not Packaged

Induction principles often admit a universal-property formulation. For an ordinary inductive type, one expects the canonical algebra to be initial, or homotopy-initial when maps carry higher equality. For a directed higher inductive category, one similarly expects a category of algebras, structured functors, and coherent transfors in which the presented object is suitably initial.

That formulation is mathematically attractive because it gathers recursion, induction, uniqueness, and higher coherence into one statement. It is also more demanding than writing a selected eliminator. One must define the algebra category at every relevant cell dimension, identify its notion of equivalence, and prove contractibility or an equivalent universal mapping property.

Formal status — research boundary. Evidence `IND-GENERAL-INITIALITY`. The current calculus packages `Nat` and equality induction, generic `PathOut` arrow induction, and the selected `WalkingEnd` eliminator. It does not yet prove a general equivalence between such interfaces and homotopy-initial categorical algebras.

We will therefore use “walking” in its presentation-theoretic sense and use the eliminator that is actually checked. Full functor-category initiality is a strengthening target, not an implicit premise of the computation.

5.9 The Interface Needed For Encode–Decode

The next two chapters isolate the additional ingredients required by the walking-endomorphism calculation:

- Chapter 6 supplies a contextual eliminator whose generator datum is a coherent directed transformation;
- Chapter 7 supplies one-dimensionality, allowing a directed 2-cell between parallel based arrows to be read as equality.

The roles are deliberately ordered. Elimination constructs directed data. Truncation may later show that some of that data is unique or equality-like. Reversing the order would erase the very directionality the example is meant to expose.

6. Directed Higher Inductive Types

An ordinary inductive type is specified by point constructors. A higher inductive type may also be specified by path constructors and constructors between paths. In a directed foundation, the corresponding presentation may contain objects, directed arrows, and higher directed cells. The change is small in notation and large in meaning: an arrow constructor does not carry an inverse unless inverse data are separately supplied.

This chapter develops the one directed higher-inductive interface that the current emdash calculus actually implements. It is enough to support the central calculation of Chapter 8 and to reveal what a reusable directed-HIT schema would have to provide. We will not promote this single example into a general theorem about all cell presentations.

6.1 Constructors With Direction

At a schematic level, a directed higher-inductive category may have:

- **object constructors**, such as $b : \text{Obj}(W)$;
- **arrow constructors**, such as $\ell : b \rightarrow b$;
- **cell constructors**, identifying or comparing composites of arrows;
- **dimension conditions**, controlling which higher cells remain nontrivial.

The eliminator must mirror these constructors. A nondependent recursor into a category $\mathcal{C}$ asks for an interpretation of every object, arrow, and cell constructor in $\mathcal{C}$. A dependent eliminator into a family $\mathcal{D}$ asks for lifts in the appropriate fibres. A contextual eliminator between two families asks for fibre functors together with comparison cells. In each case, constructor computations say that the induced map observes the supplied data at the constructors.

For a groupoidal path constructor, one may freely use inverse paths in later reasoning. For a directed arrow constructor, that move is unavailable. If a presentation contains only

$$\ell : b \longrightarrow b,$$

then it generates forward composites $\text{id}_b, \ell, \ell \circ \ell, \dots$; no negative power has been introduced. This is the fundamental distinction between the walking endomorphism and the circle.

The formal ledger for such a signature is therefore longer than a constructor list. It records formation, introductions and their boundaries, the chosen eliminator, constructor beta rules, higher action and coherence, dimension data, and the current uniqueness status. The general template and the exact WalkingEnd instance are collected in [Appendix G.6](#); this chapter develops the same clauses in mathematical order.

6.2 The Selected Walking Signature

Write $\mathcal{W}$ for the category `WalkingEnd`, with constructors

$$* : \text{Obj}(\mathcal{W}), \quad \ell : \text{Hom}_{\mathcal{W}}(*, *).$$

The category is opaque. Its object classifier, hom-categories, identities, and composition are not defined by reducing them to a `Nat` model. Alongside the constructors, the signature retains evidence that $\mathcal{W}$ is one-dimensional: every hom-category is discrete.

Formal status — checked. Evidence `WE-SIGNATURE`. `WalkingEnd_cat`, `walking_base`, and `walking_loop` are opaque declarations. `walking_end_is_one_cat` is explicit signature evidence; it does not unfold the category into a concrete presentation.

One-dimensionality is a height condition, not an invertibility condition. It says that parallel 2-cells in each hom are equality-like and that there are no nontrivial higher directed layers there. It does not say that every 1-cell of $\mathcal{W}$ has a reverse. Chapter 7 will make the distinction precise.

Why keep $\mathcal{W}$ opaque? Because the theorem in Chapter 8 is then a consequence of its constructors, eliminator, and dimension evidence. If its based hom were defined to be `Nat`, the calculation would merely restate a definition. Opacity turns the comparison with `Nat` into a normalization theorem.

6.3 The Contextual Algebra

Let

$$R, D : \mathcal{W} \longrightarrow \text{Cat}$$

be directed `Cat`-valued families. At the base, suppose we are given a functor

$$u : R[*] \longrightarrow D[*].$$

There are two ways to move an element of `R[*]` once around the generator and then apply `u`:

$$D[\ell] \circ u, \quad u \circ R[\ell].$$

The selected lax orientation asks for a transformation

$$\sigma : D[\ell] \circ u \Longrightarrow u \circ R[\ell].$$

These data form the contextual algebra of the walking endomorphism. Its eliminator produces a displayed functor

$$\text{ind}^d(R, D, u, \sigma) : R \Longrightarrow D$$

over the whole of $\mathcal{W}$.

Formal status — checked. Evidence `WE-CONTEXTUAL-ELIMINATOR`. The owner is `walking_end_ind_funcd`. Its base component computes to `u`, and its displayed action on the literal generator computes to the component of `sigma` at the selected source object.

The direction of `sigma` deserves attention. It is not an equality asserting strict commutation, nor is it automatically reversible. It is a directed cell comparing the two composites. This is the right amount of coherence for the displayed-functor interface used by `emdash`: composition with base action commutes laxly in the selected direction, and higher naturality continues through the generic transfer calculus.

The word “contextual” means that the eliminator retains `R`, the context from which inputs are drawn. Removing `R` too early would reduce `u` to a single object and `sigma` to a single lift. That specialization is useful, but it cannot construct Chapter 8’s decoder from one nonconstant family to another.

6.4 Sections And Recursors Are Special Cases

Take `R` to be the constant terminal family. A functor $R[*] \rightarrow D[*]$ is determined by an object `d` of the base fibre. The generator coherence reduces to an arrow

$$\bar{\ell} : D[\ell](d) \longrightarrow d.$$

Contextual elimination then yields a dependent section

$$\text{ind}(D, d, \bar{\ell}) : \prod_{x:W} D[x].$$

Take `D` constant as well, at a category `C`. Its transport is the identity, so the data become an object `c : Obj(C)` and an endomorphism $f : c \rightarrow c$. The resulting section is an ordinary functor

$$\text{rec}(C, c, f) : W \longrightarrow C$$

with the expected observations

$$\text{rec}(*) = c, \quad \text{rec}[\ell] = f.$$

Formal status — checked. Evidence `DHIT-DERIVED-ELIMINATORS`. `walking_end_ind_sec` and `walking_end_rec_func` are transparent specializations of the contextual eliminator, with checked base and loop observations.

This dependency is architecturally important. There is one semantic elimination principle, not three unrelated black boxes. The derived views retain the generic functor and transfer action, which is why a recursor-built object such as `Code` can later be acted on at higher cells.

6.5 What “Computes” Means

A constructor computation can be visible in several ways:

1. a runtime reduction may expose the supplied constructor datum;
2. a typed equality may state that two stable observations agree;
3. a proof-time comparison may join expressions without selecting either as the runtime normal form.

For the walking eliminator, the contextual base projection and the displayed generator cell are the constructor-specific runtime owners. Narrow projections expose the same data through the derived section and recursor views. Transparent equality theorems record those observations at their readable types.

This distinction prevents a tempting but dangerous simplification. One should not add a new constructor-specific rewrite merely because a theorem is mathematically true. Generic functoriality already owns preservation of identities, composition, and ordinary naturality. The HIT-specific rules own only what observing the literal constructors must reveal.

The book will normally write a constructor equation with $\mathrel{=}$. A formal status note determines whether that equation is runtime computation, propositional equality, or a mathematical development. This keeps the main line readable without blurring the operational contract.

6.6 From A Pointwise Step To A Coherent Spiral

Chapter 8 requires more than the ordinary recursor. Its code family and based representable family are both nonconstant:

$$\mathsf{Code}, \mathsf{Rep}_* : W \longrightarrow \mathsf{Cat}.$$

At the base, natural-number powers give a carrier function

$$n \longmapsto \ell^n.$$

Because the source is the path category of Nat , equality action lifts this function to a functor

$$\mathsf{power} : \mathsf{Path}(\mathbb{N}) \longrightarrow \mathsf{Hom}_W(*, *).$$

The contextual algebra still needs a transformation

$$\mathsf{Rep}_*[\ell] \circ \mathsf{power} \Longrightarrow \mathsf{power} \circ \mathsf{Code}[\ell].$$

Pointwise, the endpoints are $\ell \circ \ell^n$ and ℓ^{n+1} . Nat recursion makes them propositionally equal, but a family of equalities is not by itself the required transformation into a directed hom-category. The higher action and endpoint comparisons must be assembled coherently.

Emdash does this through a restricted equality-local core inclusion and a `PathLift` construction. Informally, `PathLift` turns a function whose source carries equality paths into a functor with directed target. The non-strict spiral inserts the necessary comparison before the lifted step; ordinary

path functoriality discharges the other side. Its readable component has the forward direction

$$\ell \circ \ell^n \longrightarrow \ell^{n+1}.$$

Formal status — checked. Evidence `WE-SPIRAL`. The selected `walking_power_spiral` is the explicit restricted-core-inclusion construction, and `walking_power_spiral_cell` exposes the directed component shown above.

The spiral is a small but representative lesson in functorial type theory. Carrier equality supplies useful input, yet the consumer asks for a coherent cell with functorial higher action. The bridge between the two must be an explicit construction; it cannot be hidden by treating every directed hom as an identity type.

6.7 The Derived Decoder Interface

Substituting

$$R = \text{Code}, \quad D = \text{Rep}_*, \quad u = \text{power}, \quad \sigma = \text{spiral}$$

into contextual elimination yields

$$\text{decode}^d : \text{Code} \Longrightarrow \text{Rep}_*.$$

Its fibre component at `x` is a functor from codes over `x` to based arrows ending at `x`; at the base it computes to the power functor. Naturality of this single displayed functor is what later produces a directed normalization cell for every opaque based arrow.

This is stronger than defining only a function `Nat -> Hom_W(*,*)`. The latter supplies candidate normal forms but contains no explanation of why an arbitrary arrow reaches its candidate. The contextual eliminator supplies exactly that missing action.

6.8 Toward A General Directed-HIT Schema

The walking interface suggests a reusable architecture. A general schema would need at least:

- a language of object, arrow, and higher-cell boundaries;
- generated recursion, dependent elimination, and contextual elimination;
- functorial action at every exposed cell level;
- a policy distinguishing computational constructor rules from propositional coherences;
- subject-reduction and overlap checks for the generated rules;
- algebra and morphism categories suitable for universal-property theorems;
- optional dimension or truncation constructors whose consequences are separately controlled.

Pushouts, quotients, general directed intervals, and cell complexes would test different parts of such a design. None follows merely by changing the name of the walking generator. The selected groupoidal interval used later in Chapters 26–27 is a separate two-point HIT, while its directed

WalkingArrow source is derived from join; neither supplies a generic signature compiler.

Formal status — research boundary. Evidence `DHIT-GENERAL-SCHEMA`. The active code implements the selected opaque WalkingEnd signature and its eliminator interfaces. It does not yet implement a generic signature compiler or arbitrary directed higher-inductive categories.

The selected example is nevertheless substantial enough to guide that future work. It forces object action, arrow action, a genuine lax coherence cell, higher functorial lifting, constructor computation, and an interaction with finite categorical height. The next chapter develops the last item before we use all of them together.

7. Truncation And Categorical Height

An induction principle tells us how to construct data. A truncation principle tells us how much distinction that data can retain. These ideas meet in the walking-endomorphism proof: contextual elimination first constructs a directed 2-cell, and one-dimensionality then shows that the parallel 1-arrows it connects are equal.

There are two related height measures in `emdash`. Recursive groupoidal truncation measures the identity types of a classifier. Recursive categorical dimension measures the hom-categories of a directed category. They agree in important boundary cases, but they are not the same definition. Keeping them separate lets us use equality locally without declaring all directed arrows invertible.

7.1 Recursive Truncation

The truncation levels begin at `-2` and continue by successor. For a classifier `A`, define

$$\begin{aligned}\text{isTrunc}_{-2}(A) &:= \text{isContr}(A), \\ \text{isTrunc}_{n+1}(A) &:= \prod_{x,y:A} \text{isTrunc}_n(x = y).\end{aligned}$$

Thus:

- a `(-2)`-truncated classifier is contractible;
- a `(-1)`-truncated classifier is a proposition;
- a `0`-truncated classifier is a set;
- a `1`-truncated classifier has set-valued identity types;
- each further level permits one more layer of identity structure.

The recursive equation is the important part. To prove that `A` is `(n+1)`-truncated, one proves that every identity classifier `x=y` is `n`-truncated. Truncation arguments can therefore descend through identity types until they reach contractibility.

Formal status — checked. Evidence `LOGIC-TRUNCATION-PREDICATE`. `IsTruncGrpd` implements this recursion, with `IsPropGrpd` and `IsSetGrpd` as the proposition and set abbreviations.

This is a property of an existing classifier. Supplying `h:isTrunc_n(A)` does not replace `A` by a quotient, erase its points, or create a new type. It certifies that higher identity distinctions already collapse at the stated level.

7.2 Truncation Evidence Is Itself A Property

There may be many expressions witnessing that `A` is `n`-truncated. Mathematically, the choice should not matter. The evidence classifier itself is proposition-valued:

$$\text{isProp}(\text{isTrunc}_n(A)).$$

Consequently any two truncation witnesses are equal. Packages may retain the witness for later use without turning it into observable structure.

Formal status — checked. Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP`. The theorem is uniform in the truncation level and the classifier.

There is a useful distinction between **retaining** evidence and **erasing** it. Emdash generally retains evidence in Sigma-like packages so a consumer can project it. Proposition-valuedness then proves that two choices are equal when equality is needed. This is constructive proof irrelevance, not an instruction to remove the field from the data representation.

7.3 Closure Principles

The recursive definition supports the standard structural operations.

First, truncation is monotone:

$$\text{isTrunc}_n(A) \longrightarrow \text{isTrunc}_{n+1}(A).$$

A type with no distinctions above level `n` also has none above the weaker level `n+1`.

Second, dependent functions preserve a pointwise truncation bound. If every `B(x)` is `n`-truncated, then

$$\prod_{x:A} B(x)$$

is `n`-truncated. At successor levels the proof uses the equivalence between equality of functions and pointwise equality.

Third, dependent sums preserve a common bound. If `A` and every `B(x)` are `n`-truncated, then

$$\sum_{x:A} B(x)$$

is `n`-truncated. The successor proof analyzes equality of dependent pairs as a base path together with a path over it.

Formal status — checked. Evidence `TRUNC-CLOSURE`. The active operations are `is_trunc_grpd_succ`, `is_trunc_pi`, and `is_trunc_sigma`, each with recursive base and successor behavior.

Truncation is also invariant under equivalence and closed under retracts. For the latter, suppose `Y` is a retract of `X`: there are maps

$$Y \xrightarrow{s} X \xrightarrow{r} Y$$

with $r \circ s$ equal to the identity on Y . Any truncation bound on X descends to Y .

Formal status — checked. Evidence `TRUNC-RETRACT`. `is_trunc_retract` works uniformly at every recursive truncation level from explicit retraction data.

These closure results are not merely a catalogue. Pi closure makes proposition evidence stable under universal quantification; Sigma closure controls total spaces and evidence-retaining universes; retract closure lets a normalization or equivalence argument transfer height to a less explicit carrier.

7.4 Universes Of Truncated Classifiers

For each level n , the package

$$\text{TruncGrpdU}(n) := \sum_{A:\text{Grpd}} \text{isTrunc}_n(A)$$

retains a classifier and its truncation evidence. Since the evidence is a proposition, equality of packages is governed by the carriers rather than by an arbitrary choice of proof.

The selected truncated-universe univalence theorem identifies package equality with carrier equivalence:

$$(X = Y) \simeq \text{TypeEquiv}(\text{carrier}(X), \text{carrier}(Y)).$$

Both directions and their round trips are named. This is a useful, precise univalent universe: it ranges over classifiers already equipped with one fixed truncation bound.

Formal status — checked. Evidence `UNIV-TRUNCATED`. The package is `TruncGrpdU`; `trunc_grpd_univalence_type_equiv` supplies the carrier equivalence between package identity and `TypeEquiv`.

The adjective “restricted” matters. This theorem does not by itself identify objects in an arbitrary directed category with ordinary categorical isomorphisms. Nor does it construct a truncation of an arbitrary input. It is a univalence theorem for an evidence-retaining subuniverse of the groupoidal classifier universe.

7.5 Finite Directed Dimension

Truncation follows identity types. Directed dimension follows hom-categories. The nonnegative dimension codes are generated by

$$0_{\text{cat}} \quad \text{and} \quad \text{succ}_{\text{cat}}(n).$$

Their classifier is recursive:

$$\begin{aligned}\text{isNCat}(0, C) &:= \text{isDiscreteCat}(C), \\ \text{isNCat}(n+1, C) &:= \prod_{x,y:\text{Obj}(C)} \text{isNCat}(n, \text{Hom}_C(x, y)).\end{aligned}$$

A zero-dimensional category is discrete. A one-dimensional category may have nonidentity directed 1-arrows, but each of its hom-categories is discrete. A two-dimensional category may have nontrivial 2-cells, while the next homs are discrete, and so on.

There is a corresponding object-truncation level:

$$\begin{aligned}\text{catLevel}(0) &= 0, \\ \text{catLevel}(n+1) &= \text{catLevel}(n) + 1.\end{aligned}$$

If C has categorical dimension n , then its object classifier is truncated at $\text{catLevel}(n)$. In particular, objects of a discrete category form a set, and objects of a one-dimensional category form a 1-truncated classifier.

Formal status — checked. Evidence `CAT-DIMENSION`. `IsNCat` owns the homwise recursion, `cat_dim_trunc_level` computes the corresponding groupoidal level, and `ncat_obj_trunc` proves the object-truncation consequence.

This bridge does not collapse the two notions. `IsNCat` constrains all iterated directed homs; object truncation records only equality structure on the object classifier. Two categories can have equally truncated object classifiers while differing radically in their directed arrows.

7.6 One-Dimensionality Of The Walking Endomorphism

The `WalkingEnd` signature contains

$$\text{isNCat}(1, W).$$

Unfolding the successor clause gives, for every $x, y : \text{Obj}(W)$,

$$\text{isDiscreteCat}(\text{Hom}_W(x, y)).$$

In particular the based hom

$$H_x := \text{Hom}_W(*, x)$$

is discrete. Given two based arrows $p, q : * \rightarrow x$, a directed 2-cell

$$\alpha : p \longrightarrow q \quad \text{in } H_x$$

can therefore be converted to an equality $p=q$ of objects of the hom-category.

Formal status — checked. Evidence `WE-ONE-DIMENSIONAL` . `walking_end_hom_discrete` specializes the dimension witness, and `walking_end_based_cell_to_path` converts a based 2-cell to equality.

This operation is local to the next hom. It does **not** produce an arrow $q \rightarrow p$, an inverse for `p` , or an inverse for `ell` in `W` . Equality between the *objects of a discrete hom-category* and invertibility of those objects as *arrows of the ambient category* are different statements.

7.7 The Exact Height Step In Encode–Decode

Chapter 8 constructs, for every based arrow $p : * \rightarrow x$, a directed normalization cell

$$\nu_p : p \longrightarrow \text{decode}_x(\text{encode}_x(p))$$

inside `H_x` . Only then does hom-discreteness give

$$p = \text{decode}_x(\text{encode}_x(p)).$$

This is the sole step in the hard inverse where one-dimensionality is used. It converts already-constructed directed information into equality; it does not help construct the information.

The other inverse,

$$\text{encode}_*(\ell^n) = n,$$

is proved independently by Nat induction using the zero and successor computations. It does not require hom-discreteness. Nat sethood plays two nearby but distinct roles: it makes the concrete `BNat` hom-category discrete, and it can be transported backward along the final carrier equivalence to give a second proof that the based-endomorphism carrier is a set. Neither role should be substituted for the directed normalization step.

This separation is a model for later proofs:

1. build a cell using functorial or contextual action;
2. invoke a dimension hypothesis at the exact hom level where the cell lives;
3. extract equality only if the target theorem needs it.

The intermediate cell can carry an orientation or support further composition even when its equality shadow cannot.

7.8 From Properties To A Classified Reflector

A truncation operation assigns to every classifier A a new classifier $\|A\|_n$, together with a universal map and elimination into n -truncated targets. The active groupoidal layer now realizes this idea in a classified form. The category `NType`(n) contains retained n -truncated classifiers, and

$$\mathbf{Trunc}_n(A) : \mathbf{NType}(n)$$

is the primary result. Decoding its carrier gives the ambient classifier $\|A\|_n$. The point constructor computes, while dependent elimination is restricted to motives that themselves land in $\mathbf{NType}(n)$. Recursion derives a whole map action; identity, composition, and a retained path action follow from that same eliminator rather than from an unrelated registry of truncation laws.

This reflector extends rather than replaces the predicates and closure theorems developed earlier in the chapter. The predicates state when a given classifier is already truncated. The reflector constructs a classified truncated target. Finite categorical height remains separate: `WalkingEnd`'s one-dimensionality is signature evidence and is not obtained by truncating an arbitrary directed category after the fact.

Formal status — checked. Evidence `TRUNC-REFLECTOR`. The active construction is a computational groupoidal truncation reflector with classified motives. A general directed categorical truncation, arbitrary quotient/HIT schema, and comparison with every classical hub-and-spoke presentation remain future work.

We now have every prerequisite for the main proof: equality-local action, functors and directed families, contextual elimination, equivalence packages, recursive truncation, and homwise categorical height. The next chapter puts them together without identifying direction with invertibility; Chapter 26 later returns to the reflector through the connectedness of the Circle.

8. Synthetic Directed Homotopy Theory

Homotopy type theory studies spaces by calculating identity types and loop spaces internally. A directed foundation admits another kind of calculation: we may calculate a hom whose arrows are not assumed invertible. The first example is the walking endomorphism.

Write $\mathcal{W}$ for `WalkingEnd`, $*$ for its base, and $\ell : * \rightarrow *$ for its generating arrow. Composition is in categorical order, so $g \circ f$ means first `f` and then `g`. For any object x of $\mathcal{W}$, abbreviate the based hom-category by

$$H_x := \text{Hom}_{\mathcal{W}}(*, x).$$

An object of H_x is a directed arrow $* \rightarrow x$. An arrow of H_x is a directed 2-cell between two such arrows. The signature states that $\mathcal{W}$ is one-dimensional, which means precisely that every H_x is discrete. We will exploit this discreteness only after a directed 2-cell has been constructed.

The proof uses six interfaces:

- `Path(A)`, the equality-local path category of a type `A`;
- functor action `F[f]` on arrows and its higher action on cells;
- a directed `Cat`-valued family $E : K \rightarrow \mathbf{Cat}$ and its fibre `E[k]`;
- a displayed functor between two such families;
- natural-number induction;
- a carrier equivalence `TypeEquiv(A,B)`, consisting of maps and inverse laws.

Chapters 1–7 develop these notions. This chapter uses them as a compact working language and returns to implementation names only in formal-status notes.

The proof also follows the formal rule ledger of [Appendix G.4](#). The `WalkingEnd` constructors and contextual eliminator supply formation, introduction, elimination, and beta computation; the code family and spiral supply the missing action and coherence; one-dimensionality is used only afterward. Full initiality is the uniqueness clause and remains separate from the checked encode–decode calculation.

8.1 The Based Endomorphisms Of The Walking Endomorphism

The target statement is:

Theorem 8.1 (the walking-endomorphism calculation). There is an equivalence of underlying carriers

$$\text{Obj}(H_*) = \text{Hom}_{\mathcal{W}}(*, *) \simeq \mathbb{N}.$$

Formal status — checked. Evidence `WE-HOM-NAT-CARRIER`. The active package is `walking_hom_nat_type_equiv`, with an additional equality-valued omega-equivalence facade at the groupoid/type level.

The qualification “underlying carriers” matters. The theorem does not definitionally replace `H_*` by a concrete Nat category. Nor does the current package include a monoid isomorphism, a reverse functor from the concrete model `BNat`, an equivalence of hom-categories, or the full initiality of `W`. We will identify exactly which stronger statements follow on paper and which still require formal infrastructure.

8.1.1 Getting Started

The obvious endomorphisms are the natural powers of the generator:

$$\begin{aligned}\ell^0 &:= \text{id}_*, \\ \ell^{n+1} &:= \ell \circ \ell^n.\end{aligned}$$

The recursion prefixes one copy of `ell` at each successor. Thus ℓ^2 is $\ell \circ \ell$ and no inverse power is present. The definition is an ordinary Nat eliminator whose motive is the object carrier of `H_*`.

Formal status — checked. Evidence `WE-POWER`. The object map is `walking_power`, and `walking_power_func` : `Path(N)` $\rightarrow H_*$ supplies its equality-local higher action.

To prove that these powers exhaust the endomorphisms, we seek a measurement

$$\text{encode}_* : \text{Obj}(H_*) \longrightarrow \mathbb{N}$$

such that

$$\text{encode}_*(\ell^n) = n \quad \text{and} \quad \ell^{\text{encode}_*(p)} = p.$$

The first equation is approachable by Nat induction. The second quantifies over an arbitrary opaque arrow $p : * \rightarrow *$. We cannot inspect `p` as a word because no word datatype is installed as the hom of `W`.

There is a second, subtler obstruction. An induction principle for a based path normally becomes useful only after the other endpoint has been generalized. Fixing both endpoints too early hides the varying family on which induction acts. The same phenomenon appears here in directed form: the calculation must be stated not only for `H_*` but for every based hom `H_x`.

We therefore look for a family of codes `Code[x]` and maps

$$\begin{aligned}\text{encode}_x &: \text{Obj}(H_x) \longrightarrow \text{Obj}(\text{Code}[x]), \\ \text{decode}_x &: \text{Obj}(\text{Code}[x]) \longrightarrow \text{Obj}(H_x),\end{aligned}$$

natural in the directed endpoint `x`. The base instance will then be the desired Nat comparison.

The encoder is the easy half once `Code` is known: start at zero and let the arrow act,

$$\text{encode}_x(p) := \text{Code}[p](0).$$

Formal status — checked. Evidence `WE-ENCODE`. `walking_encode` is defined for every endpoint and every based arrow, not only for endomorphisms at the base.

At the base, the computations we want are

$$\text{encode}_*(\text{id}_*) = 0$$

and

$$\text{encode}_*(\ell \circ p) = \text{succ}(\text{encode}_*(p)).$$

The first is the identity action of a functor. The second will follow from the generator computation of `Code` together with generic functoriality on a composite. No `WalkingEnd`-specific composition rewrite is needed.

Formal status — checked. Evidence `WE-ENCODE-PREFIX`. The prefix equation is propositional; its owner specializes ordinary functor action on composition and the literal generator computation.

8.1.2 The Free-Monoid Model

Before constructing `Code`, it helps to exhibit the arithmetic object we expect `W` to resemble. Define a category `BNat` by

$$\text{Obj}(\text{BNat}) = \mathbf{1}, \quad \text{Hom}_{\text{BNat}}(\bullet, \bullet) = \text{Path}(\mathbb{N}).$$

Its identity is zero. With our composition convention,

$$m \circ n := m + n,$$

where addition recurses in its left input:

$$0 + n = n, \quad (m + 1) + n = (m + n) + 1.$$

The underlying objects of the sole hom-category are therefore the natural numbers. Its higher arrows are equality paths between naturals. Since `Nat` is a set, that hom-category is discrete, so `BNat` satisfies the same one-dimensionality contract as `W`.

The generator in `BNat` is `1`. Applying the ordinary recursor of `W` produces

$$J : W \longrightarrow \text{BNat}, \quad J(*) = \bullet, \quad J[\ell] = 1.$$

Functoriality forces `J` to send a displayed composite of generators to the corresponding sum. Thus `BNat` demonstrates that the walking signature has the expected one-object Nat-monoid interpretation.

Formal status — checked. Evidence `WE-BNAT-MODEL`. The identity and recursive composition of `BNat`, their propositional agreement with Nat addition, its one-dimensionality, and `walking_bnat_model_func` are checked.

This model does **not** settle Theorem 8.1. A functor `J` gives a number to every endomorphism, but it need not reflect equality: two unknown arrows might map to the same number. Nor does a map *out* of an opaque inductive object supply a map back. The model is valuable precisely because it remains separate:

- it checks that the signature can be interpreted without collapsing zero and one;
- it fixes the intended orientation of identity, composition, and generator;
- it predicts the normal forms;
- it does not place those normal forms into the definition of `W`.

This is the categorical analogue of testing a presentation in a familiar model before proving its universal consequences. The exhaustiveness proof must come from the eliminator and dimension evidence of `W` itself.

Formal status — research boundary. Evidence `WE-FULL-CATEGORICAL-COMPARISON` records what is absent: a reverse `BNat` $\rightarrow$ `W` functor, a packaged categorical equivalence, and full functor-category initiality require reusable monoid-action-to-functor and functor-extensionality infrastructure.

8.1.3 The Directed Cover In Functorial Type Theory

We now define the family that measures arrows. The ordinary recursor for `W` says that a functor out of the walking object is determined by a target object and one endomorphism of that object. Take the target category to be `Cat`, choose `Path(Nat)` as the object, and choose the successor functor as its endomorphism. The result is

$$\text{Code} : W \longrightarrow \text{Cat}$$

with constructor computations

$$\text{Code}[*] = \text{Path}(\mathbb{N}), \quad \text{Code}[\ell] = \text{Succ}.$$

Formal status — checked. Evidence `WE-CODE`. Both the base fibre and literal generator action are checked recursor observations.

There is an important contrast with the circle. A universe-valued family over the circle must send its loop to an equality of types. Univalence can produce such an equality from successor on the integers because integer successor is an equivalence. Natural-number successor is not an equivalence: it

misses zero. The directed recursor asks only for an endofunctor, so it accepts successor directly. Univalence remains part of the surrounding foundation, but it is neither needed nor appropriate to turn this action into a reversible path.

At the base fibre, picture

$$0 \longrightarrow 1 \longrightarrow 2 \longrightarrow 3 \longrightarrow \dots$$

as levels of a helix over the directed loop. One traversal moves upward by one level. There is a boundary at zero and no downward motion. The picture is a guide to the action of the family; it is not a claim that a topological covering space or a contractible total category has been constructed.

For any based arrow $p : * \rightarrow x$, functor action gives

$$\mathbf{Code}[p] : \mathbf{Code}[*] \longrightarrow \mathbf{Code}[x].$$

Evaluating at zero defines $\mathbf{encode_x}(p)$. At $x=*$ the result is a natural number. For a generator-prefixed arrow, strict functoriality factors the action:

$$\begin{aligned} \mathbf{encode}_*(\ell \circ p) &= \mathbf{Code}[\ell \circ p](0) \\ &= \mathbf{Code}[\ell](\mathbf{Code}[p](0)) \\ &= \mathbf{succ}(\mathbf{encode}_*(p)). \end{aligned}$$

The target of the decoder is the based representable family

$$\mathbf{Rep}_* : W \longrightarrow \mathbf{Cat}, \quad \mathbf{Rep}_*[x] = H_x.$$

For $f : x \rightarrow y$, its action is postcomposition:

$$\mathbf{Rep}_*[f](q) = f \circ q.$$

Thus a decoder varying over x should be a displayed functor

$$\mathbf{decode}^d : \mathbf{Code} \Longrightarrow \mathbf{Rep}_*.$$

At the base, its object map ought to send n to $\mathbf{ell}^n$. But an object map alone is too little. It must act on equality paths between naturals, and it must be coherent with the base generator.

The pointwise power function lifts to a functor

$$\mathbf{power} : \mathbf{Path}(\mathbb{N}) \longrightarrow H_*.$$

The lift uses equality action and inclusion into the directed hom-category. This retains the higher action needed by the contextual eliminator rather than truncating power to a bare function.

The generator coherence has the form

$$\mathbf{Rep}_*[\ell] \circ \mathbf{power} \Longrightarrow \mathbf{power} \circ \mathbf{Code}[\ell].$$

At a natural number `n`, its readable component is

$$\sigma_n : \ell \circ \ell^n \longrightarrow \ell^{n+1}$$

inside `H_*`. The endpoints express the same recursive power equation, but the contextual eliminator requires a coherent transformation, not merely a family of object equalities. Emdash constructs it by lifting the equality between the two step functions through the restricted equality-local core and adding the endpoint adjustments demanded by the ambient directed hom. This is the **spiral**.

Formal status — checked. Evidence `WE-SPIRAL`. The selected spiral is the explicit-core-inclusion two-factor construction; its readable component has the direction shown above.

The contextual elimination principle for `W` may be read as follows. Given directed families $R, D : W \rightarrow \mathbf{Cat}$, a base functor

$$u : R[*] \longrightarrow D[*],$$

and a transformation

$$D[\ell] \circ u \Longrightarrow u \circ R[\ell],$$

it produces a displayed functor $R \Rightarrow D$. Substituting `R=Code`, `D=Rep_*`, `u=power`, and `sigma` equal to the spiral yields the desired contextual decoder.

Formal status — checked. Evidence `WE-CONTEXTUAL-ELIMINATOR` and `WE-CONTEXTUAL-DECODER`. The displayed decoder is `walking_directed_decode_funcd`, and its base fibre computes to the power functor.

This construction is why the generalization over all endpoints is not a stylistic flourish. The decoder is coherent because it is one displayed functor over the whole opaque object, not a collection of unrelated functions defined only at the base.

8.1.4 The Encode-Decode Proof

We now follow the dependency order of the construction. The order matters: compressing the argument into two carrier functions would hide its directed content.

Step 1: powers with higher action

Nat induction defines `power(n)=ell^n` on objects. Equality action lifts this function to

$$\text{power} : \text{Path}(\mathbb{N}) \longrightarrow H_*.$$

At zero and successor,

$$\text{power}(0) = \text{id}_*, \quad \text{power}(n + 1) = \ell \circ \text{power}(n).$$

The functorial lift is essential: the next step needs to transport equality of naturals to a 2-cell between powers.

Step 2: the spiral

Postcomposition by `ell` after power and power after successor are two functors from `Path(Nat)` to `H_*`. The recursive power equation gives equality of their underlying object functions. The equality-local lift, restricted core inclusion, and directed endpoint adjustments turn this into

$$\sigma : \text{Rep}_*[\ell] \circ \text{power} \Longrightarrow \text{power} \circ \text{Code}[\ell].$$

Its component `sigma_n` points from generator-prefix composition toward the successor power. This is the coherence algebra consumed by the HIT eliminator.

Step 3: the contextual decoder

Contextual elimination now supplies

$$\text{decode}^d : \text{Code} \Longrightarrow \text{Rep}_*.$$

Projecting to a fibre gives, for every `x`,

$$\text{decode}^d[x] : \text{Code}[x] \longrightarrow H_x.$$

Write `decode_x(c)` for its object action. At the base this functor is judgmentally the power functor, so

$$\text{decode}_*(n) = \ell^n.$$

Step 4: the directed normalization cell

Let $p : * \rightarrow x$ be arbitrary. A displayed functor does more than provide fibrewise maps: it compares transport in its source and target families along every base arrow. Apply this comparison to `p` and to zero in `Code[*]`.

On the source side, the representable action gives

$$\text{Rep}_*[p](\text{power}(0)) = p \circ \text{id}_* = p.$$

On the target side, `Code` action gives `Code[p](0)=encode_x(p)`, then the fibre decoder gives `decode_x(encode_x(p))`. The displayed comparison is therefore a directed 2-cell

$$\nu_p : p \longrightarrow \text{decode}_x(\text{encode}_x(p))$$

in `H_x`.

Formal status — checked. Evidence `WE-NORMALIZATION-CELL`. The term is the displayed hom-action of the contextual decoder at `p` and zero. Its source reduces through representable postcomposition, `power(0)`, and the right unit.

This is the conceptual climax of the proof. It says that an unknown arrow can move, by a directed higher cell, toward the canonical power selected by its code. We have not yet said that the two arrows are equal.

Step 5: equality from categorical height

The one-dimensionality witness for `W` makes `H_x` discrete. In a discrete category, a hom between two objects can be converted to equality of those objects. Applying this operation to `nu_p` gives

$$\bar{\nu}_p : p = \text{decode}_x(\text{encode}_x(p)).$$

Formal status — checked. Evidence `WE-NORMALIZATION-PATH`. The implementation explicitly constructs `walking_directed_normalization_cell` before applying hom-discreteness in `walking_directed_normalization_path`.

This final conversion forgets the orientation of normalization, but it does not make `p` or `ell` invertible. It uses the absence of nontrivial 2-dimensional variation in a hom-category; it says nothing about inverses for its objects as 1-arrows of `W`.

Step 6: the hard inverse

Specialize to `x=*`. Because the base decoder computes to power, `bar(nu)_p` becomes

$$p = \ell^{\text{encode}_*(p)}.$$

The inverse law for the desired equivalence is conventionally oriented the other way, so we take symmetry:

$$\ell^{\text{encode}_*(p)} = p.$$

Formal status — checked. Evidence `WE-POWER-ENCODE`. This is the difficult carrier inverse, and it is derived from directed normalization rather than from induction on an exposed word.

In the circle calculation, the analogous fixed-loop problem is repaired by generalizing the endpoint and using path induction. Here the endpoint is also generalized, but the eliminator supplies a directed displayed action. The normalization cell is the directed replacement for the equality that path induction would have produced immediately in a groupoidal setting.

Step 7: the easy inverse

For `n:Nat`, prove

$$\text{encode}_*(\ell^n) = n$$

by Nat induction.

At zero, power is the identity and a functor sends identity to identity, so acting on zero returns zero. At a successor,

$$\begin{aligned} \text{encode}_*(\ell^{n+1}) &= \text{encode}_*(\ell \circ \ell^n) \\ &= \text{succ}(\text{encode}_*(\ell^n)) \\ &= \text{succ}(n). \end{aligned}$$

The middle equality is the generator-prefix encoding formula; the last is the induction hypothesis acted on by successor. No negative case is required.

Formal status — checked. Evidence `WE-ENCODE-POWER`. The theorem `walking_encode_power_roundtrip` is native Nat induction over the checked prefix equation.

Step 8: package the equivalence

The forward function is `encode_*` and the inverse is `power`. The previous two steps provide both quasi-inverse laws, so they determine

$$\text{Hom}_W(*, *) \simeq \mathbb{N}.$$

The encoder is also packaged as a functor

$$H_* \longrightarrow \text{Path}(\mathbb{N}),$$

obtained by taking the hom-action of `Code` and evaluating at zero. This functor acts on 2-cells between endomorphisms. Its ordinary functor laws should not be confused with preservation of the *horizontal* monoid composition of the endomorphisms themselves.

Formal status — checked. Evidence `WE-STRUCTURED-ENCODER` and `WE-HOM-NAT-CARRIER`. The structured encoder and carrier equivalence are active; a structured reverse functor and a monoid package are not.

The proof can now be summarized without erasing its architecture:

$$\begin{array}{ccc} p & & \\ \downarrow \nu_p & \Longrightarrow_{\text{discreteness}} & p = \ell^{\text{encode}(p)}, \\ \text{decode}(\text{encode}(p)) & & \end{array}$$

followed by the independent Nat-inductive calculation `encode(ell^n)=n`.

8.1.5 Consequences And The Missing Negative Integers

The equivalence has immediate structural consequences, but the most illuminating ones concern what the walking generator cannot do.

The based hom is a set

There are two checked proofs that the underlying carrier `Hom_W(*,*)` is a set.

1. **By dimension.** One-dimensionality makes `H_*` discrete, and discreteness includes sethood of its object carrier.
2. **By comparison.** Natural numbers form a set, and truncation is invariant under the carrier equivalence.

Formal status — checked. Evidence `WE-HOM-SETHOOD`. The two proofs are separately named, so the dimensional and equivalence-based explanations remain visible.

The generator is not the identity

The prefix computation gives

$$\text{encode}_*(\ell) = 1, \quad \text{encode}_*(\text{id}_*) = 0.$$

If `ell=id_*`, functorial action of `encode` on that equality would yield `1=0`, whose Nat equality classifier is empty.

Formal status — checked. Evidence `WE-LOOP-NOT-IDENTITY`.

The generator has no right inverse

Suppose $r : * \rightarrow *$ and

$$\ell \circ r = \text{id}_*.$$

Encoding the left side and using the prefix formula gives `succ(encode(r))`; encoding the right side gives zero. Again a successor cannot equal zero. Therefore no such `r` exists.

Formal status — checked. Evidence `WE-LOOP-NO-RIGHT-INVERSE`. The statement is specifically the absence of a right inverse in the displayed composition orientation; no stronger cancellation theorem is being silently imported.

Native omega-equivalence evidence for an arrow contains, among its data, a right inverse and its equality law. The preceding result therefore rules out such evidence for `ell`.

Formal status — checked. Evidence `WE-LOOP-NONINVERTIBLE`.

Composition and addition

The carrier theorem strongly suggests the monoid formula

$$\text{encode}_*(q \circ p) = \text{encode}_*(q) + \text{encode}_*(p).$$

Its orientation agrees with `BNat` : `q` is the outer arrow and its code is the left input of addition. A paper proof follows from the checked interfaces. First use `Nat` induction and associativity to show

$$\ell^{m+n} = \ell^m \circ \ell^n.$$

Then replace `q` and `p` by their normalized powers and use `encode(power(k))=k`. What is absent is not the mathematical argument but its selected library package and its interaction with a future reverse functor.

Formal status — formal consequence. Evidence `WE-COMPOSITION-ADDITION`. The current kernel does not expose a named monoid-isomorphism object for this statement.

Why naturals replace integers

For the circle, every loop is an equality path and hence can be reversed. Powers extend from naturals to integers, the code action has both successor and predecessor, and the result is group-valued.

For `W`, the generator is a directed arrow with no supplied inverse. The code action moves

$$0 \mapsto 1 \mapsto 2 \mapsto \dots$$

and cannot move below zero. Natural numbers record the free monoid generated by forward motion. The negative integers are missing for the same reason the right inverse is missing: direction has not been group-completed.

The comparison with the Circle now proceeds by exactly such an explicit free-inversion construction. A whole functor

$$W \longrightarrow \text{Path}(S^1)$$

sends the base and directed generator to the Circle base and loop. It sends every natural power to the corresponding nonnegative Circle power, and the Circle encoder reads that image as the canonical inclusion $\mathbb{N} \rightarrow \mathbb{Z}$. More strongly, restriction along this functor is a whole mapping equivalence against every groupoidal target. The theorem does not make ℓ invertible inside `W`; it characterizes the separate groupoidal object obtained by freely allowing inverse motion.

Formal status — checked. Evidence `WE-GROUP-COMPLETION`. The concrete `WalkingEnd`–Circle restriction/extension theorem is active and iterable. Chapter 27 places it beside category-indexed groupoidification. A reverse `BNat` functor and a packaged monoid isomorphism for the original carrier theorem remain separate questions.

The calculation has reached its intended boundary. It proves that an opaque directed generator has exactly the expected natural powers, and it proves noninvertibility rather than assuming it. The later universal mapping theorem strengthens the surrounding comparison, but it is not a hidden premise of the Nat encode–decode proof.

8.2 Higher Groupoidal Shadows

The surrounding calculus is directed, but equality-local and groupoidal phenomena remain inside it. The first neighboring example is the Eckmann–Hilton argument.

Let $\mathcal{B}$ be a category and x an object. The 2-endomorphisms of the identity 1-arrow form the carrier

$$2\text{End}_{\mathcal{B}}(x) := \text{Hom}_{\text{Hom}_{\mathcal{B}}(x,x)}(\text{id}_x, \text{id}_x).$$

There are apparently two ways to combine elements. Vertical composition is ordinary composition in the hom-category $\text{Hom}_{\mathcal{B}}(x, x)$. Horizontal composition is obtained by whiskering/postcomposition at the identity 1-arrow. They share the identity 2-cell as a unit, and the ordinary functoriality of whiskering supplies interchange. The classical Eckmann–Hilton calculation then makes the operations agree and commute:

$$\beta \cdot \alpha = \alpha \cdot \beta.$$

Formal status — checked. Evidence `EH-COMMUTATIVITY`. The active term `EH_comm` derives commutativity from the two compositions, shared units, and interchange in the iterated-hom representation.

This result does not undo the directed character of $\mathcal{W}$. It lives one dimension higher at the identity arrow, where the relevant comparison is equality-local. The coexistence is the point: a directed theory can contain groupoidal shadows at controlled boundaries without declaring all of its arrows reversible.

9. Transforms And The Calculus Of Cuts

Composition is indispensable, but a syntax made only of nested composites quickly forgets why a particular reassociation matters. Functorial type theory therefore gives important cuts a name and a computational owner. A functor acts on an arrow; postcomposition and precomposition act on represented homs; a transform acts off the diagonal; and a universal construction eliminates a map through the object it represents. Each operation controls its own reassociation.

This point of view is inspired by the categorical proof theory of Došen's Cut Elimination in Categories. We use that work only as a conceptual reference. The presentation below is newly written for emdash, whose iterated homs and directed families require a different formal architecture.

The gain is not merely prettier notation. A selected cut can compute while retaining the functor or transform that acts on the next hom. An unrestricted associativity rewrite would erase that information, create competing normal forms, or loop by repeatedly changing brackets. Controlled cut elimination instead answers three questions at once: what is being eliminated, where its normal form lives, and which higher action survives.

9.1 Four Levels Of Cuts

We shall use four levels throughout the rest of the book.

1. An **arrow cut** composes one chosen arrow on the left or right. Its owners are the lower-star postcomposition and upper-star precomposition actions.
2. A **family cut** composes next to an arrow that varies naturally. Its owner is the off-diagonal action of a transform.
3. A **structural cut** eliminates data introduced by a product, dependent total, curry, or related type former. Its owner is the corresponding projection or eliminator.
4. A **universal cut** factors through a chosen representing object. Its owner is an adjunction, representability comparison, or another explicit universal-property interface.

These levels are not four unrelated collections of rewrite rules. They form a progression. Arrow cuts explain local composition; family cuts explain naturality; structural cuts explain computation for categorical type formers; and universal cuts explain why constructions such as adjoints, Yoneda maps, and weighted limits compute.

We also keep four equality modes distinct. A runtime reduction selects a normal form. A proof-time comparison helps `Lambdapi` elaborate two intended presentations without making either one compute to the other. A propositional equality is an internal witness. A mathematical equality in free-form theory states the intended theorem while naming the interface still needed to check it. The symbol $\rightsquigarrow$ below is reserved for an actual selected runtime reduction; an ordinary equality sign does not silently make that claim.

In the terminology of [Appendix G.4](#), each named cut is an elimination followed by a computation rule. Its formation and introduction data determine which composite is well typed; its full functor or transfor owner supplies higher action; and any eta or uniqueness principle is stated separately. This is why a pointwise naturality equation cannot replace `tapp1_func`, and why a universal comparison needs both beta and eta rather than one attractive factorization formula.

9.2 Arrow Cuts

Composition is written $g \circ f$: first f , then g . The two star actions record which side of a represented hom is moving.

If $g : w \rightarrow x$ and $u : x \rightarrow y$, then

$$u_*(g) := u \circ g : w \rightarrow y$$

is **postcomposition** by u . If $u : x \rightarrow y$ and $h : y \rightarrow z$, then

$$u^*(h) := h \circ u : x \rightarrow z$$

is **precomposition** by u . Lower star is covariant in the moving target; upper star is contravariant in the moving source. They are different operations, not typographic variants.

The implemented forms are slightly more general. For $K : A \rightarrow B$ and $p : x \rightarrow y$ in A , postcomposition uses $K[p]$ on a hom ending at Kx , while precomposition uses $K[p]$ on a hom beginning at Ky . Retaining K is what lets the action iterate on higher cells.

9.2.1 Example 1: Postcomposition Accumulates

Let

$$g : w \rightarrow x, \quad u : x \rightarrow y, \quad v : y \rightarrow z.$$

Two consecutive lower-star cuts reduce to one:

$$v_*(u_*(g)) \rightsquigarrow (v \circ u)_*(g).$$

Both sides are arrows $w \rightarrow z$. The selected normal form retains a single postcomposition action whose moving arrow is $v \circ u$; it does not expand to a raw threefold composite. The generic owner is `hom_postcomp_fapp0`, and the displayed equality mode is runtime reduction. Before capping at g , `hom_postcomp_func` remains a functor between hom-categories, so its action on 2-cells between possible values of g is still available.

The functor-indexed version says the same thing. If $p : x \rightarrow y$ and $q : y \rightarrow z$ in A , then consecutive action by $K[p]$ and $K[q]$ accumulates under the single arrow $q \circ p$. Ordinary functoriality belongs to the generic `fapp*` calculus; no constructor receives a private composition law.

9.2.2 Example 2: Precomposition Reverses The Action Order

Let

$$u : w \rightarrow x, \quad v : x \rightarrow y, \quad h : y \rightarrow z.$$

The corresponding upper-star reduction is

$$u^*(v^*(h)) \rightsquigarrow (v \circ u)^*(h).$$

Both sides are arrows $w \rightarrow z$. The selected normal form is one precomposition action along $v \circ u$. The generic owner is `hom_precomp_along_fapp0`, and this is again runtime reduction. Its uncapped form `hom_precomp_along_func` remains a functor on the represented hom, with the next hom-action intact.

The order is worth reading twice. Starting with h , we first precompose by v and then by u , but the accumulated base arrow is $v \circ u$. Thus

$$u^* \circ v^* = (v \circ u)^*.$$

That reversal is contravariance, not a special exception to associativity.

Formal status — checked. Evidence `CAT-HOM-CUTS`. The full and capped lower-star and upper-star actions have identity, consecutive-action, and adjacent raw-cut computations. Their ordinary-composition readings remain proof-time comparisons where selecting a second runtime normal form would be harmful.

9.2.3 Why There Is No Global Associativity Rewrite

The ordinary associator is available as a proof-time comparison and as propositional evidence. It is not installed as a pair of unrestricted runtime rules. Such rules would either orient every composite toward a normal form that ignores semantic owners or permit both bracketings and loop.

Star accumulation is narrower. It reassociates exactly when one side is a represented-hom action, and its result retains that action as a stable head. The same policy will govern the next three levels: eliminate the cut at the construction that understands it.

9.3 Family Cuts

Let $F, G : A \rightarrow B$ and let $\eta : F \Rightarrow G$. A point component $\eta_x : Fx \rightarrow Gx$ is only the diagonal of a more useful operation. For every $x, y : A$ there is an off-diagonal functor

$$\eta_{x,y} : \text{Hom}_A(x, y) \longrightarrow \text{Hom}_B(Fx, Gy).$$

For $f : x \rightarrow y$, write its value as $\eta[f] : Fx \rightarrow Gy$. Setting $f = \text{id}_x$ recovers η_x ; retaining the whole functor retains its action on 2-cells between arrows f .

Formal status — checked. Evidence `TRANSF-POINT-OFFDIAGONAL`. `tapp0_fapp0` observes the point component, while `tapp1_func` and `tapp1_fapp0` expose the iterable and capped off-diagonal actions.

9.3.1 Example 3: The Two Naturality Cuts

Take arrows

$$h : w \rightarrow x, \quad f : x \rightarrow y, \quad g : y \rightarrow z.$$

There are two neighboring cuts, one on each side of the varying arrow:

$$\begin{aligned} G[g] \circ \eta[f] &\rightsquigarrow \eta[g \circ f], \\ \eta[f] \circ F[h] &\rightsquigarrow \eta[f \circ h]. \end{aligned}$$

The first source and target are $Fx \rightarrow Gz$; the second are $Fw \rightarrow Gy$. In each case the selected normal form is one off-diagonal action on the composite source arrow. The generic owner is `tapp1_fapp0`, and both displayed equalities are runtime reductions. At the uncapped level `tapp1_func` remains a functor between hom-categories, so a 2-cell between f and f' is carried to a 2-cell between $\eta[f]$ and $\eta[f']$ after the cut has normalized.

Formal status — checked. Evidence `TRANSF-STRICT-NATURALITY`. Both full-functor and capped-arrow forms are owned by the generic `tapp*` calculus. Constructor-specific copies of ordinary naturality are neither needed nor desired.

The familiar naturality square is the identity-boundary instance. For $f : x \rightarrow y$, the expressions $G[f] \circ \eta_x$ and $\eta_y \circ F[f]$ both normalize through the common interior $\eta[f]$. Naturality is therefore not an equality proof added after defining a family of point components. It is computation exposed by the family action itself.

Identity and vertical composition follow the same architecture. The identity transfor and a vertical composite live in the transformation category, so their component projections use the generic identity and composition calculus. Higher transfor arise by iterating the next hom rather than by inventing a separate coherence language at every dimension.

9.4 Structural Cuts

A structural cut applies an eliminator to data whose introduction form is known. Product projections are the simplest case, but they already reveal an important distinction between the general categorical theory and the currently checked category-of-categories instance.

9.4.1 Example 4: The Product/Projection Benchmark In A General Category

Let K be a category equipped with chosen binary products. Take objects

$$A_0, A_1, B_0, B_1, C : \text{Obj}(K)$$

and arrows

$$h : A_0 \rightarrow A_1, \quad k : B_0 \rightarrow B_1, \quad g : A_1 \rightarrow C.$$

Write $h \times k : A_0 \times B_0 \rightarrow A_1 \times B_1$ for the induced product arrow, and distinguish the two projections by

$$\pi_1^1 : A_1 \times B_1 \rightarrow A_1, \quad \pi_1^0 : A_0 \times B_0 \rightarrow A_0.$$

Upper-star precomposition in K gives

$$(\pi_1^1)^*(g) : A_1 \times B_1 \rightarrow C.$$

The Došen-style product cut is the equation

$$(\pi_1^1)^*(g) \circ (h \times k) = (\pi_1^0)^*(g \circ h),$$

with both sides in $\text{Hom}_K(A_0 \times B_0, C)$. Its intended computational orientation is left to right. The calculation combines two controlled steps:

$$\pi_1^1 \circ (h \times k) = h \circ \pi_1^0$$

by product projection, followed by upper-star accumulation. The arrow k disappears because the first projection observes only the first component.

The source is a composite out of $A_0 \times B_0$ and the target is a single upper-star cut with the same source and codomain. The selected normal form is $(\pi_1^0)^*(g \circ h)$. The active chosen-product interface now supplies the required whole product functor, whole projections, stable antecedential projection operations, and a derived product map. Its direct Došen naturality, beta, distribution, and eta rules retain higher action rather than stopping at the displayed 1-arrow equation.

The generic action $P[h, k]$ is proof-time comparable with the triangular map formed from the two antecedential projections. Thus the readable equation above is supported by one selected product calculus, while runtime keeps its owner-aligned smaller cuts visible.

Formal status — checked. Evidence `TRIANGULAR-BINARY-PRODUCTS`. For a supplied `BinaryProducts(K,P)` structure, the whole product, projection, and pairing owners compute the general beta/distribution/eta calculus and retain higher action. Existence of such a structure is selected data; no claim is made that every category has binary products.

9.4.2 The Checked Cat-Specialized Legs

Set $K = \text{Cat}$. Then A_i, B_i, C are categories and h, k, g are functors. The active product-valued-functor representation exposes projection by a Sigma observation. Its checked structural reduction is

$$\mathbf{fst}(h \times k) \rightsquigarrow h \circ \pi_1^0.$$

The checked, owner-aligned upper-star cut is

$$(\pi_1^0)^*(h^*(g)) \rightsquigarrow (h \circ \pi_1^0)^*(g),$$

and $h^*(g)$ is proof-time comparable with the readable composite $g \circ h$. Both owner-aligned sides retain the upper-star precomposition head and therefore its higher action on transfor between possible functors g .

The literal **Cat** instance of the general equation is not currently one runtime reduction. A focused typed audit found both sides well formed, checked the two owner-aligned legs, and found that neither runtime conversion nor typed reflexivity joins the raw composite $\pi_1^1 \circ (h \times k)$ directly to the selected observation $\mathbf{fst}(h \times k)$. Packaging that narrow projection/composition comparison would suffice. Installing a broad product eta rewrite would be a much stronger and unsafe response.

Formal status — formal consequence. Evidence `CUT-PRODUCT-PROJECTION`. The **Cat**-specialized equation follows from controlled associativity, upper-star composition, and the checked product projection. The diagnostic suite checks the owner-aligned reduction. The literal raw-composite bridge is not packaged, so the textbook display is not labeled a checked kernel reduction.

9.4.3 Example 5: Product Beta Is Elimination After Introduction

In a category K with selected `BinaryProducts(K,P)`, arrows $p : X \rightarrow A$ and $q : X \rightarrow B$ have a pairing $\langle p, q \rangle : X \rightarrow A \times B$. The characteristic structural cuts are

$$\pi_1 \circ \langle p, q \rangle = p, \quad \pi_2 \circ \langle p, q \rangle = q.$$

The source and target of the first equation lie in $\mathbf{Hom}_K(X, A)$, and those of the second lie in $\mathbf{Hom}_K(X, B)$. Their selected normal forms are p and q . More generally, postcomposing either projection with h gives the checked triangular beta rules from (30.7), and pairing distributes through precomposition. The whole pairing functor retains arrows between possible legs.

Formal status — checked. Evidence `TRIANGULAR-BINARY-PRODUCTS`. The general selected product interface computes projection after pairing, pairing distribution, and eta. Chapter 30 develops this whole interface and its terminal and indexed continuations.

The active category-of-categories specialization is nevertheless concrete. For arrows $p : a \rightarrow a'$ in A and $q : b \rightarrow b'$ in B , the pair (p, q) is an arrow $(a, b) \rightarrow (a', b')$ in the product category. The projection functors compute:

$$\pi_1[p, q] \rightsquigarrow p, \quad \pi_2[p, q] \rightsquigarrow q.$$

Here the selected normal forms are the component arrows themselves. The specialized owners are the capped hom-actions of `Product_projL_func` and `Product_projR_func`, and the equality mode is runtime reduction. Before capping, each full hom-action remains a projection functor from a product of hom-categories, so higher component cells remain available.

Formal status — checked. Evidence `CAT-PRODUCT-CALCULUS`. Product categories, product maps, projection functors, and their object and hom projections have focused componentwise computations. This is evidence for the desired general architecture, not an assertion that arbitrary object-level products in every K are already implemented.

9.4.4 Fibred Structural Cuts

The same introduction/elimination pattern appears in a dependent categorical context. Let $B, C : K \vdash \mathbf{Cat}$ be independent families over one base, and let $P(B, C)$ be their fibrewise product. For displayed functors

$$\Phi : E \Longrightarrow B, \quad \Psi : E \Longrightarrow C,$$

pairing introduces a map $\mathbf{pair}_d(\Phi, \Psi) : E \Longrightarrow P(B, C)$, while the two displayed projections eliminate it. Their structural cuts are whole displayed-functor reductions:

$$\mathbf{projL}_d \circ \mathbf{pair}_d(\Phi, \Psi) \rightsquigarrow \Phi, \quad \mathbf{projR}_d \circ \mathbf{pair}_d(\Phi, \Psi) \rightsquigarrow \Psi.$$

These equations say more than pointwise product beta. At an object $k : K$, pairing is the ordinary product pairing in the fibre. Over a base arrow $p : k \rightarrow l$, its action is the pair of the two displayed actions over the same p . Its canonical internalized cell at a fibre object u is likewise componentwise:

$$\mathbf{cell}(\mathbf{pair}_d(\Phi, \Psi), p, u) = (\mathbf{cell}(\Phi, p, u), \mathbf{cell}(\Psi, p, u)).$$

Thus the elimination cuts remain valid while object action, base-arrow action, and the selected next-cell observation stay internally functorial. This is the structural calculus of independent siblings $k : K, b : B[k], c : C[k]$; it does not exchange a variable with another whose classifier depends on it.

Formal status — checked. Evidence `CAT-FIBREWISE-CONTEXT` covers the fixed-base displayed projections and pairing, both whole projection-after-pairing reductions, and their componentwise fibre, base-arrow, internalized-cell, and selected higher observations. The arbitrary- K chosen-object-product interface of Examples 4 and 5 remains a separate mathematical development.

9.5 Universal Cuts

A universal property turns a family of maps into a chosen object together with inverse ways of introducing and eliminating a factorization. Its computation laws are higher-level cut elimination. Adjunction triangles are the first example; representability, co-Yoneda, and weighted limits continue the same line.

9.5.1 Example 6: The Two Adjunction Triangles

Let $F : R \rightarrow L$ be left adjoint to $G : L \rightarrow R$, with unit $\eta : \text{id}_R \Rightarrow GF$ and counit $\varepsilon : FG \Rightarrow \text{id}_L$.

For $g : X \rightarrow X'$ in R and $f : FX' \rightarrow Y$ in L , the left triangle cut is

$$\varepsilon[f] \circ F[\eta[g]] \rightsquigarrow f \circ F[g].$$

Both sides have source FX and target Y . The selected normal form removes the adjacent unit-counit detour while preserving the ordinary functor action $F[g]$.

Dually, for $f : X \rightarrow GY'$ in R and $g : Y' \rightarrow Y$ in L , the right triangle cut is

$$G[\varepsilon[g]] \circ \eta[f] \rightsquigarrow G[g] \circ f.$$

Both sides have source X and target GY . The selected normal form again removes the universal detour and retains the functorial image of the boundary arrow.

The owners are not arbitrary transformations with the same types. They are the stable unit and counit observations of the indexed `Adjunction` witness, and both equations are runtime reductions. Their components use the off-diagonal `tapp1` action, so action on higher cells in f and g remains part of the surrounding functorial calculus.

Formal status — checked. Evidence `ADJ-TRIANGLE-CUTS`. `unit_adj_transf` and `counit_adj_transf` are the selected observations that trigger the two reductions. Independently named unit-shaped and counit-shaped transformers do not acquire these computations by type alone.

9.5.2 Example 7: The Shaped Co-Yoneda Beta Cut

For a profunctor $P : A \rightsquigarrow B$, the right and left unit maps have the forms

$$P \otimes_B U_B \Longrightarrow P, \quad U_A \otimes_A P \Longrightarrow P.$$

If p is a shaped element and id is the matching identity-shaped hom element, their component cuts reduce as

$$\varepsilon_P^R(p \otimes \text{id}) \rightsquigarrow p, \quad \varepsilon_P^L(\text{id} \otimes p) \rightsquigarrow p.$$

The source is a shaped element of the corresponding tensor and the target is the original shaped element of P . The normal form is p ; the owners are the two `Prof_coyoneda_*` transformations; and the equality mode is runtime reduction on the selected shaped cells. Naturality-fusion remains available for a profunctor map $P \rightarrow P'$, so this beta law is not merely a capped set-level equation.

Formal status — checked. Evidence `PROF-COYONEDA` . Chapter 13 develops the representable and profunctor context required to read this calculation as the computational core of Yoneda.

9.5.3 Example 8: Weighted-Limit Beta And Eta

A computational weighted-limit witness is a comparison between the weighted cone profunctor and the representable hom profunctor at the proposed limit. After reindexing along a probe $M : I \rightarrow B$, it supplies operations

$$\text{push}(r) : R \Longrightarrow \text{Hom}(M, L), \quad \text{pull}(s) : R \Longrightarrow \text{Cone}_W(M, F).$$

Their universal cuts reduce in both directions:

$$\text{pull}(\text{push}(r)) \rightsquigarrow r, \quad \text{push}(\text{pull}(s)) \rightsquigarrow s.$$

The source and target are profunctor maps of the displayed kinds, and the normal forms are the original maps r and s . The generic owners are `prof_comparison_push` and `prof_comparison_pull`; the weighted names merely specialize their types. The equality mode is runtime beta/eta reduction. The comparison can still be reindexed, symmetrized, and composed, so the higher universal structure is retained instead of being collapsed to a chosen set-level bijection.

Formal status — checked. Evidence `PROF-COMPARISON-BETA-ETA` . Chapter 16 turns this general comparison calculus into the weighted-limit interface.

9.5.4 Example 9: A Right Adjoint Preserves The Weighted Limit

Suppose $L_a : A \rightarrow B$ is left adjoint to $R_a : B \rightarrow A$, and a comparison certifies $L : J' \rightarrow B$ as the W -weighted limit of $F : J \rightarrow B$. Three universal cuts compose: move a cone across the adjunction, use the given representation, and move the representing hom back. The output is a comparison certifying $R_a L$ as the weighted limit of $R_a F$.

The source is the supplied weighted-limit comparison together with the adjunction; the target is the transported comparison in A . The selected normal form is the composite of the two adjunction-mate comparisons and the reindexed limit comparison. Its owner is `right_adjoint_preserves_weighted_limit_cov_comp`. This is a checked construction, not a rewrite asserting that arbitrary limit syntax commutes with every right adjoint. The resulting comparison retains its push/pull beta-eta action on every probe and profunctor map.

Formal status — checked. Evidence `WEIGHTED-LIMIT-PRESERVATION` . The theorem is developed in Chapter 16 after weights, cones, and mate comparison have been introduced.

9.5.5 Example 10: The Weighted-Colimit Dual

A W -weighted colimit in B is represented by the corresponding weighted limit in B^{op} . Applying the preceding theorem to the opposite adjunction gives the dual statement: a left adjoint preserves the selected weighted colimit.

The source is a weighted-colimit comparison and an adjunction; the target is the transported comparison after the left adjoint. The selected normal form is the opposite of the right-adjoint preservation composite, with double opposites and reversed composition reduced by the generic duality owners. The construction is checked under `left_adjoint_preserves_weighted_colimit_con`. Its output is still a full comparison with push/pull behavior after passing to opposites, not merely an equality of chosen objects.

Formal status — checked. Evidence `WEIGHTED-COLIMIT-PRESERVATION`. Chapter 17 develops the variance and opposite-category calculation in full.

9.6 What Cut Elimination Preserves

The ten examples have a common shape:

introduction or action + matching elimination $\longrightarrow$ one semantic owner.

Their normal forms are deliberately not all raw composites. Lower star keeps postcomposition visible; upper star keeps precomposition visible; `tapp1` keeps the varying arrow family visible; product projection keeps a structural component visible; and a universal comparison keeps its inverse operations visible. This is what makes the calculus iterable.

In particular, a capped equation is not the whole meaning of a construction. The full `hom_postcomp_func`, `hom_precomp_along_func`, and `tapp1_func` operations act on the next hom. Product projection has a full hom-functor. Profunctor comparison can be reindexed and composed. A good reduction removes the cut without discarding the object that higher-dimensional consumers need.

9.7 Directed Family Transfers And Lax Comparison

Strict cut elimination should not be confused with strictness of every directed comparison. Let $E, D : K \rightarrow \mathbf{Cat}$ be directed families and let

$$\Phi : E \Longrightarrow D$$

be a natural family morphism. At each $k : K$ it has a fibre functor $\Phi_k : E[k] \rightarrow D[k]$. A transfer $\epsilon : \Phi \Rightarrow \Psi$ has a fibrewise transfer and point components at objects of each fibre.

Formal status — checked. Evidence `TRANSFD-FIBRE-COMPONENTS`. The stable displayed presentations are `Transfd_cat` and `Transfd ; Fibre_transf` and `Fibre_transf_app` expose their fibre and object projections.

When the base moves along $p : x \rightarrow y$, the two relevant transports do not have to be equal. Starting with $u : E[x]$, the internal displayed hom action gives a directed cell

$$\chi_{p,u}^{\Phi} : D[p](\Phi_x u) \longrightarrow \Phi_y(E[p]u)$$

in $D[y]$. This is the displayed laxity cell. It has a direction and need not be invertible. The generic functor and transfor cuts around it compute strictly, but the comparison itself remains mathematical data.

Formal status — checked. Evidence `FUNCTORD-DISPLAYED-LAXITY`. The endpoint functors are `functord_transport_lhs_func` and `functord_transport_rhs_func`; the active component is `fdappl_int_cell`.

For a fibre arrow $\alpha : E[p](u) \rightarrow v$, the capped displayed action has the readable form

$$\Phi_y[\alpha] \circ \chi_{p,u}^\Phi : D[p](\Phi_x u) \longrightarrow \Phi_y v.$$

The direct internal-hom projection is the runtime owner. Expanding it into this composite on every use would discard the stable higher action and give the comparison a second owner.

9.8 Total Categories And The WalkingEnd Decoder

The family morphism induces a functor on total categories,

$$\Sigma\Phi : \sum_{k:K} E[k] \longrightarrow \sum_{k:K} D[k].$$

An arrow in the source has the form $(p, \alpha) : (x, u) \rightarrow (y, v)$, where $\alpha : E[p](u) \rightarrow v$. Its image is

$$(p, \Phi^d[p, u, v, \alpha]).$$

Thus the base arrow is retained and the fibre arrow is exactly the capped displayed hom action. The laxity cell is the special case in which $v = E[p](u)$ and α is the identity.

Formal status — checked. Evidence `FUNCTORD-SIGMA-ACTION`. `sigma_map_func` owns the total functor, whose arrow projection uses the capped internal displayed action rather than reconstructing a comparison composite.

For the contextual WalkingEnd decoder

$$\text{decode}^d : \text{Code} \Longrightarrow \text{Rep}_*,$$

the generator comparison is the spiral. Evaluated at a base arrow $p : * \rightarrow x$ and zero, its endpoints reduce to

$$p \quad \text{and} \quad \text{decode}_x(\text{encode}_x(p)).$$

The generic laxity component is therefore the directed normalization cell of Chapter 8. Replacing it by a commuting equality at the outset would erase the content of normalization. Equality is extracted only later, using discreteness of the target hom-category.

9.9 The Packaging Boundary

The calculus now has a continuous line from local arrow composition to weighted universal properties. Its discipline is equally important in the negative direction.

- The arbitrary- K product benchmark needs a general chosen-product interface. Its **Cat** instance still awaits one narrow packaged projection/composition comparison; neither gap justifies broad product eta.
- A displayed family morphism has a whole internal laxity transformation and its component projection ladder. Ordinary post/left and pre/right surfaces are transparent specializations of that owner rather than duplicate naturality squares.
- General ends, coends, and arbitrary Kan extensions require universal interfaces stronger than the selected profunctor operations.
- Runtime conversion, proof-time comparison, internal equality, and equivalence remain different judgments.

Formal status — checked. Evidence `FUNCTORD-WHOLE-LAXITY`. The whole displayed owner, both ordinary variance surfaces, their capped cells, and the functor-compositor specialization are active. Their next actions remain in the generic hom calculus. This does not claim a complete weak omega-category coherence theorem or remove the prototype's historical strict endpoint cuts.

Cut elimination is therefore not a feature catalogue. It is the organizing principle by which functorial type theory decides what should compute, what should merely compare, and what higher structure must remain visible after a calculation is complete.

10. Categories, Precategories, And Categorical Identity

Chapter 2 introduced the categorical language needed for the WalkingEnd calculation. We now return to the word *category* with a different question: when should equality of objects agree with categorical sameness? The answer depends on which categorical layer is under discussion. This second pass is a spiral over Chapters 2, 4, and 7, not a replacement for their definitions.

The distinction is foundational. Ordinary univalent category theory begins with set-valued homs and asks for an identity-to-isomorphism map to be an equivalence. Emdash begins with iterated hom-categories, where arrows may have directed higher arrows between them. The ordinary theory is therefore an important specialization of functorial type theory, but it cannot serve as the definition of the ambient native category.

10.1 A Translation Table, Not A Collapse

The following table fixes the translation used throughout the remainder of the book.

Layer	Characteristic data	Equality or height condition	Role here
Native <code>Cat</code>	objects and iterated <code>Hom_cat</code> classifiers	no set-valued-hom premise in the definition	ambient directed higher language
Equality-local <code>Path(A)</code>	elements of <i>A</i> and identity evidence between them	every arrow is induced by equality	groupoidal specialization used for ordinary type theory
Finite <code>IsNCat</code> evidence, including <code>OneCat</code>	a native category plus a recursive hom-height witness	one-dimensionality makes each next hom discrete	bridge to ordinary category-shaped examples
HoTT precategory	a type of objects and set-valued homs	equality between parallel arrows is proposition-valued	ordinary 1-categorical specialization
HoTT category	a HoTT precategory for which object identity agrees with isomorphism	<code>idtoiso</code> is an equivalence	univalent 1-category notion, not the definition of native <code>Cat</code>
HoTT strict category	a HoTT precategory whose object type is a set	object identity is proposition-valued, without assuming <code>idtoiso</code> is an equivalence	a qualified truncation notion, unrelated to runtime strictness

Three warnings belong beside the table.

First, *set-valued hom* means a truncation condition, not that a hom must be presented by a classical set outside type theory. Second, *strict category* in the HoTT sense refers to the truncation level of the object type; it says nothing about which emdash expressions reduce at runtime. Third, a HoTT strict category need not be a univalent category: object identity may be proposition-valued while nontrivial isomorphisms remain. Finally, a `OneCat` witness controls the height of a native category but does not by itself identify object equality with isomorphism.

Formal status — mathematical development. The table is a translation discipline, not an implemented equivalence of packages. A generic construction identifying native finite-height categories with HoTT precategories would have to choose the ordinary hom-set presentation and prove compatibility with native composition and object equality.

10.2 The Ordinary Precategory Specialization

An ordinary HoTT precategory $\mathcal{A}$ consists of:

- a type $\text{Obj}(\mathcal{A})$ of objects;
- for each a, b , a set $\text{Hom}_{\mathcal{A}}(a, b)$;
- identity arrows id_a ;
- composition $g \circ f$, typed so that $f : a \rightarrow b$ and $g : b \rightarrow c$;
- left-unit, right-unit, and associativity equalities.

Because each hom is a set, equality between parallel arrows is a proposition. The category laws therefore do not require an additional tower of coherence data. This is precisely the simplifying hypothesis that ceases to be available in a genuinely higher native category: there, the equality or cell between two composites may itself have nontrivial higher structure.

The emdash specialization replaces an external record of hom sets by a native category C equipped with one-dimensional evidence. For every pair x, y , the witness

$$\text{IsNCat}(1, C)$$

makes the next hom-category $\text{Hom}_C(x, y)$ discrete. Its objects are the ordinary arrows; the discreteness evidence controls their higher equality. The same witness also implies the corresponding truncation bound on $\text{Obj}(C)$, but it still says nothing about whether every ordinary isomorphism comes from object identity.

Formal status — checked. Evidence `CAT-ITERATED-HOMS` and `CAT-DIMENSION`. Native `Cat` has category-valued homs, `IsNCat` recurses through those homs, and finite-dimensional evidence yields the predicted truncation evidence for the object classifier.

This gives a reliable rule of reading: use ordinary precategory reasoning when the hom-height assumptions make it valid, but keep the native iterated owner visible whenever a construction must act on the next cell.

10.3 Isomorphism And Identity-To-Isomorphism

For objects a, b of an ordinary precategory, an isomorphism $a \cong b$ consists of arrows

$$f : a \longrightarrow b, \quad g : b \longrightarrow a$$

with $g \circ f = \text{id}_a$ and $f \circ g = \text{id}_b$. Since the homs are sets, an inverse to a fixed f is unique when it exists. Consequently the type of isomorphism evidence between two fixed objects is itself a set.

Object identity always produces an isomorphism. Given $p : a = b$, identity induction defines

$$\text{idtoiso}_{\mathcal{A}}(p) : a \cong b$$

by sending reflexivity to the identity isomorphism. This map exists before assuming univalence. The additional assertion that $\mathcal{A}$ is a *category* in the HoTT sense is that

$$\text{idtoiso}_{\mathcal{A}} : (a = b) \longrightarrow (a \cong b)$$

is an equivalence for every a, b .

Formal status — mathematical development. Evidence `UCAT-IDENTITY-ISOMORPHISM`. This is the ordinary univalent 1-categorical theorem adapted from the HoTT category spine. It is not a theorem about every native `Cat`.

Univalence changes how one reasons about structure. A property or construction invariant under isomorphism can be transported along object identity because isomorphism and identity carry the same information. It also implies that the object type of an ordinary category is a 1-type: its identity types are equivalent to isomorphism types, and those are sets.

The effect on arrows is concrete. If $p : a = a'$, $q : b = b'$, and $f : a \rightarrow b$, then transporting f across both endpoint identities agrees with

$$\text{idtoiso}(q) \circ f \circ \text{idtoiso}(p)^{-1} : a' \longrightarrow b'.$$

Thus endpoint transport is a lower-star cut at the target and an upper-star cut at the source. This ordinary formula anticipates the represented-hom actions used throughout the native calculus.

The implication should not be reversed carelessly. Having a 1-type of objects does not make a precategory univalent, just as having discrete homs does not force isomorphic objects to be equal.

10.4 Five Nearby Notions Of Sameness

The native theory places several useful notions next to one another. Their proximity is a feature, but it is not an identification.

Notion	Typical expression	What it compares
object identity	$p : x = y$ in $\text{Obj}(C)$	two objects in the underlying equality layer
path-generated arrow	$\text{path_to_hom}(p) : x \rightarrow y$	the directed arrow induced by object identity
ordinary isomorphism	$\text{IsoEvidence}_C(x, y)$	inverse arrows with equality-valued cancellation

carrier equivalence	$A \simeq B$ via <code>TypeEquiv</code>	two decoded groupoidal carriers
native omega-equivalence	<code>OmegaEquiv_C(x, y)</code>	a selected arrow with recursively usable inverse evidence

There are checked maps between some adjacent entries. Object identity yields a path-generated arrow and a transparent native omega-equivalence package. Ordinary isomorphism evidence also yields native omega-equivalence evidence. A `TypeEquiv`, however, compares carriers rather than two objects of an arbitrary category, and it must not be substituted for categorical equivalence without an explicit construction.

Formal status — checked. Evidence `EQUIV-OMEGA` and `EQUIV-TYPE`. The active interfaces expose object-path and carrier-equivalence constructions separately; neither declaration asserts a blanket identification of all five rows.

This ladder explains why the book qualifies the word *equivalence*. A theorem about carrier equivalence, ordinary categorical equivalence, or recursive omega-equivalence has different inputs and different higher consequences.

10.5 Equality-Local Categories

For a groupoidal classifier A , the native category

$$\text{Path}(A)$$

has the elements of A as objects and identity evidence $x = y$ as the hom from x to y . Reflexivity supplies identities, and ordinary functions act functorially on paths. This is the native home of the familiar principle that equality behaves like a groupoid.

Formal status — checked. Evidence `CAT-PATH-CATEGORY`. `Path_cat`, `Path_cat_func`, and `path_map_func` retain the equality-local category and its iterable functor action.

The ordinary and native readings coincide only under the appropriate height condition. If A is a 1-type, each identity type $x = y$ is a set, so $\text{Path}(A)$ can be read as an ordinary HoTT precategory. For a general A , its identity types may have higher identity types, and the native iterated-hom presentation retains them rather than truncating them away.

This example also separates groupoidality from directed categorical univalence. Every arrow in $\text{Path}(A)$ comes from identity by construction. An arbitrary native category may contain noninvertible arrows, and its identity layer cannot be recovered merely by declaring those arrows to be paths.

10.6 Preorders, Posets, And Skeletal Reasoning

An ordinary preorder can be regarded as a precategory whose homs are mere propositions: there is at most one arrow from a to b . Isomorphism then means $a \leq b$ and $b \leq a$. The preorder is a univalent category exactly when this mutual reachability agrees with identity—the type-theoretic form of

antisymmetry. Thus posets are the proposition-valued-hom examples of the general identity-to-isomorphism principle.

This example is useful because it exposes a frequent confusion. A skeletal presentation says that isomorphic objects are literally or propositionally the same in a chosen presentation. Univalence says that the canonical map from identity to isomorphism is an equivalence, so the identification is stable under dependent reasoning. The latter is an internal principle, not merely a convention for selecting representatives.

The category of small sets is the other basic example. Its arrows are functions, and its identity-to-isomorphism map is the restriction of the identity-to-equivalence map for types. Universe univalence therefore makes it a univalent category. Categories of groups, rings, and similar structures follow when equivalence of carriers preserving the structure is shown to agree with identity of the structured object. Chapter 15 develops that structure-identity pattern.

Formal status — mathematical development. The preorder/poset example is ordinary univalent 1-category theory, as are the category-of-sets and structured-object examples. The active code has proposition and set truncation evidence plus selected universe-univalence results, but no generic packaged preorder or structured-category construction is claimed here.

10.7 The Checked Ordinary-Isomorphism Lift

The active calculus supports one direction of the native comparison with no finite-height premise. Given ordinary isomorphism evidence $i : x \cong y$, its forward arrow and inverse laws define

$$\text{iso_evidence_omega_equiv}(i) : \text{OmegaEquiv}_C(x, y).$$

The construction keeps the forward arrow selected by i and packages its inverse evidence in the recursive native facade. It is a structural lift, not a decoder identifying isomorphism evidence with object identity.

Formal status — checked. Evidence `EQUIV-ORDINARY-ISO-LIFT`. The owners are `iso_evidence_omega_along` and `iso_evidence_omega_equiv`; diagnostics check their projections and reflexive behavior.

This is the chapter's central checked theorem. It says that ordinary inverse data is strong enough to enter the native equivalence calculus. It does not say that native equivalence, ordinary isomorphism, and object identity are interchangeable in every category.

10.8 The Native Univalence Boundary

A full categorical identity theorem for the ambient directed theory would need to answer at least four questions.

1. Which equivalence classifier is compared with object identity?
2. Does the comparison retain a selected directed arrow?

3. How does it act on every next hom-category?
4. Under what height, saturation, or groupoidality hypotheses are the two directions inverse?

The active code has selected univalence theorems for groupoidal and truncated universes, object-path-to-equivalence constructions, finite-height truncation, and the one-way ordinary-isomorphism lift. It does not package a general equivalence

$$(x = y) \simeq \text{IsoEvidence}_C(x, y)$$

for arbitrary native C , nor a general saturation theorem that freely adds such identities.

Formal status — research boundary. Evidence `UNIV-FULL-OBJECT-ISO`. A future owner must compare object identity and the selected categorical equivalence notion while remaining coherent with iterated hom action. Chapter 15 returns to this problem through the structure identity principle and Rezk-style saturation.

The title *univalent foundations* should therefore be read layer by layer. Univalence is the disciplined alignment of identity with the appropriate equivalence, not a license to erase the distinctions that make the directed higher theory computational.

11. Functors, Transforms, And Functor Categories

Chapter 9 studied transforms as a calculus of cuts. We now make their categorical organization explicit. Functors are objects of a functor category, transforms are arrows between those objects, and higher transforms are obtained by iterating the same hom construction.

This is another spiral rather than a repetition. Ordinary 1-category theory starts from object functions, hom functions, and pointwise naturality squares. The native calculus packages all of these as iterable categorical action. A point component is the identity-boundary case of an off-diagonal action, and a naturality square is the visible boundary of a computation.

11.1 Functors Act At Every Retained Dimension

For ordinary precategories $\mathcal{A}$ and $\mathcal{B}$, a functor $F : \mathcal{A} \rightarrow \mathcal{B}$ consists of an object function and functions

$$F_{a,b} : \text{Hom}_{\mathcal{A}}(a, b) \longrightarrow \text{Hom}_{\mathcal{B}}(Fa, Fb)$$

that preserve identities and composition. In the set-valued-hom specialization, these are ordinary functions between hom sets.

A native emdash functor retains more structure. Its action on a pair of objects is itself a functor

$$F_{x,y} : \text{Hom}_{\mathcal{A}}(x, y) \longrightarrow \text{Hom}_{\mathcal{B}}(Fx, Fy).$$

Evaluating that functor at an arrow $f : x \rightarrow y$ gives $F[f]$. Keeping the full hom functor visible also keeps its action on 2-cells between arrows, and then on higher cells by iteration.

Formal status — checked. Evidence `CAT-FUNCTOR-CALCULUS`. `fapp0` is object action, `fapp1_func` is the full next-hom action, and `fapp1_fapp0` is its value at one arrow. Identity and composition reductions belong to these generic owners.

The preservation law is oriented as cut elimination:

$$F[g] \circ F[f] \rightsquigarrow F[g \circ f].$$

This is not a theorem copied onto each constructor. It is computation of the global functor-action interface. A specialized construction should expose its own semantic projections, while ordinary functoriality remains owned here.

11.2 From Natural Transformations To Transforms

For ordinary functors $F, G : \mathcal{A} \rightarrow \mathcal{B}$, a natural transformation $\eta : F \Rightarrow G$ is usually presented by arrows

$$\eta_x : Fx \longrightarrow Gx$$

such that every $f : x \rightarrow y$ satisfies

$$G[f] \circ \eta_x = \eta_y \circ F[f].$$

The native transfor retains the common interior of this square. For every pair x, y it supplies an off-diagonal functor

$$\eta_{x,y} : \text{Hom}_A(x, y) \longrightarrow \text{Hom}_B(Fx, Gy).$$

We write $\eta[f] : Fx \rightarrow Gy$ for its value at f . The point component is recovered at the identity:

$$\eta[\text{id}_x] \rightsquigarrow \eta_x.$$

Formal status — checked. Evidence `TRANSF-POINT-OFFDIAGONAL`. The point projection is `tapp0_fapp0`; `tapp1_func` and `tapp1_fapp0` expose the full and capped off-diagonal actions.

This presentation does not add exotic data to an ordinary natural transformation. In the one-dimensional specialization, naturality determines the off-diagonal value in either familiar way:

$$\eta[f] = G[f] \circ \eta_x = \eta_y \circ F[f].$$

In the native higher setting, however, retaining $\eta_{x,y}$ as a functor also retains its action on cells between possible f 's. Point components alone would hide that action and force it to be reconstructed later.

11.3 Naturality Is A Pair Of Family Cuts

Take composable arrows

$$h : w \rightarrow x, \quad f : x \rightarrow y, \quad g : y \rightarrow z.$$

The two strict naturality computations are

$$\begin{aligned} G[g] \circ \eta[f] &\rightsquigarrow \eta[g \circ f], \\ \eta[f] \circ F[h] &\rightsquigarrow \eta[f \circ h]. \end{aligned}$$

Setting f to an identity makes the usual naturality square reappear. Both boundary composites normalize through the same off-diagonal interior $\eta[f]$. Thus naturality is not merely a proposition verified after a family of components has been assembled; it is the way the family action absorbs neighboring cuts.

Formal status — checked. Evidence `TRANSF-STRICT-NATURALITY`. Both capped equations and their uncapped hom-functor forms are runtime reductions of the global `tapp1*` calculus. The full forms retain action on the next cells.

This is the chapter's central checked theorem. It explains why the calculus uses a transfor rather than a bare dependent function of point components: the transfor is the computational natural family.

11.4 The Functor Category

For native categories A and B , the category

$$[A, B] := \text{Functor_cat}(A, B)$$

has functors $A \rightarrow B$ as objects. Its hom-category between F and G is

$$\text{Transf_cat}(F, G).$$

An identity arrow in $[A, B]$ is the identity transfor. Composition in $[A, B]$ is vertical composition of transfors. Iterating the hom of $\text{Transf_cat}(F, G)$ yields modifications and higher transfors without changing the ambient notion of category.

Formal status — checked. Evidence `CAT-TRANSFOR-CALCULUS`. `Functor_cat` and `Transf_cat` are active native categories, and `Hom_cat(Functor_cat(A,B), F, G)` reduces to the corresponding transfor category.

In the ordinary set-valued-hom specialization, the same construction gives a precategory of functors and natural transformations. Equality between natural transformations is pointwise because the codomain homs are sets. A natural transformation is a natural isomorphism exactly when each component is an isomorphism.

The native formulation is intentionally stronger at the interface. It does not force all higher transfor structure to be proposition-valued merely because the first components resemble ordinary natural transformations.

11.5 Whiskering And Horizontal Composition

Functor composition acts on transfors in two one-sided ways. If $\eta : F \Rightarrow G$ and $K : B \rightarrow C$, post-whiskering gives

$$K\eta : KF \Rightarrow KG.$$

If $H : X \rightarrow A$, pre-whiskering gives

$$\eta H : FH \Rightarrow GH.$$

These are the transfor-level actions of the same postcomposition and precomposition functors studied in Chapter 9. They are not independent definitions of naturality.

Now take

$$\alpha : F \Rightarrow G : A \rightarrow B, \quad \beta : H \Rightarrow K : B \rightarrow C.$$

Their horizontal composite can be read at an object a in either of the ordinary forms

$$\beta_{Ga} \circ H[\alpha_a], \quad K[\alpha_a] \circ \beta_{Fa}.$$

Naturality of β identifies the two. The native calculus packages the pair (α, β) under the product-composition action, so its off-diagonal value first transports the A -arrow through α and then through β . This gives a single iterable owner rather than two competing component formulas.

Formal status — checked. Evidence `TRANSF-HORIZONTAL-CALCULUS`. The generic owner is `comp_prod_fapp1_fapp0`; diagnostics check its point, full off-diagonal, and capped off-diagonal projections. The ordinary two-formula equality is its 1-categorical reading.

11.6 Interchange And Controlled Coherence

Vertical and horizontal composition satisfy interchange. Schematically, for a composable four-cell grid,

$$(\beta_2 \circ \beta_1) * (\alpha_2 \circ \alpha_1) = (\beta_2 * \alpha_2) \circ (\beta_1 * \alpha_1).$$

In an ordinary functor precategory this is an equality of natural transformations, proved component-wise using associativity and naturality. In the native calculus, the corresponding computation is organized by the generic product-composition action and the off-diagonal vertical-composite folds. A representable four-cell instance is exposed as propositional interchange evidence.

The equality mode matters. Functor composition has associativity and unit comparisons, and ordinary category theory packages their familiar pentagon and triangle coherences. Emdash does not install unrestricted reassociation in both directions as runtime computation. Instead:

- a semantic owner absorbs a neighboring cut when it has a selected normal form;
- proof-time comparison is used when two presentations should elaborate together but neither should replace the other globally;
- propositional equality records a theorem without changing runtime normal forms;
- higher transfors retain coherence data that is not truncated away.

This is the categorical version of controlling associativity. Parentheses may be suppressed in mathematical prose when the intended composite is unambiguous, but the formal presentation must still select an owner and an equality mode.

11.7 The Ordinary Functor-Category Theorem

Let $\mathcal{A}$ be an ordinary precategory and $\mathcal{B}$ an ordinary univalent category. Then the functor precategory $[\mathcal{A}, \mathcal{B}]$ is a univalent category. The reason is pointwise but not merely syntactic:

1. an identity $F = G$ gives a natural isomorphism by identity induction;
2. a natural isomorphism gives pointwise isomorphisms $Fx \cong Gx$;
3. univalence of $\mathcal{B}$ turns those into pointwise identities;
4. function extensionality assembles equality of object functions;
5. the functor laws and naturality data are propositions at this height, so the assembled equality determines the whole functor.

The same analysis shows that identity between ordinary functors agrees with natural isomorphism.

Formal status — mathematical development. Evidence `UCAT-FUNCTOR-CATEGORY`. This is the HoTT 1-categorical theorem under set-valued-hom and univalent-codomain hypotheses. The active native `Functor_cat` supplies the categorical object, but this general identity-to-natural-isomorphism equivalence is not a checked emdash theorem.

11.8 The Native Functor-Category Boundary

A native analogue cannot be obtained by deleting the word *set* from the ordinary proof. One must choose the relevant sameness of functors—object identity, ordinary isomorphism in `Functor_cat`, pointwise native omega-equivalence, or a higher adjoint equivalence—and then prove that the comparison respects:

- point components;
- off-diagonal arrow action;
- cells between source arrows;
- vertical and horizontal composition;
- every further retained hom level.

Pointwise object formulas are therefore necessary but insufficient. The active code has the functor category, the full transfor calculus, and selected object-path and ordinary-isomorphism lifts. It does not yet combine them into a general univalence theorem for native functor categories.

Formal status — research boundary. The missing owner is a native category-univalence package stable under `Functor_cat` and coherent with `tapp1_func` at every retained dimension. Chapter 15 discusses the saturation problem that such a theorem would have to solve.

The computational lesson survives independently of that boundary. Functors and transfor already form a native higher category, and their naturality is already internal computation. Univalence would identify the right notion of sameness in this existing structure.

12. Adjunctions And Equivalences

An adjunction is the first universal construction in which the cut calculus becomes a theorem about a chosen relationship between two functors. It can be presented by a unit and counit, by a natural equivalence of homs, or by representability. These presentations explain one another, but they should not be collapsed before their hypotheses and equality modes have been stated.

Equivalence requires the same care. Carrier equivalence, isomorphism of two objects, equivalence of ordinary categories, and native omega-equivalence answer different questions. This chapter builds the adjunction interface first, derives its selected monad computation, and then uses the same representability perspective to organize those notions.

12.1 Indexed Adjunction Data

Let

$$F : A \longrightarrow B, \quad G : B \longrightarrow A.$$

An adjunction $F \dashv G$ has a unit and counit

$$\eta : \text{id}_A \Rightarrow GF, \quad \varepsilon : FG \Rightarrow \text{id}_B,$$

subject to the two triangle identities. In ordinary notation these say

$$(\varepsilon F) \circ (F\eta) = \text{id}_F, \quad (G\varepsilon) \circ (\eta G) = \text{id}_G.$$

The active `Adjunction` classifier keeps F and G as indices. Its `unit_adj_transf` and `counit_adj_transf` observations are stable computational heads. This matters: triangle computation is attached to the selected adjunction witness, not to every pair of transformations having the same displayed types.

The direct-TypeScript authoring layer can package already declared F , G , η , and ε —or a counit and whole natural hom transpose—as an indexed witness with proof-time agreements. It expands into ordinary logical-framework declarations: no new adjunction notion and no runtime alias. [Appendix G.5](#) places this convenience at its precise trust boundary.

12.2 The Triangle Cuts

The checked equations are stronger than the diagonal component formulas. Take

$$g : X \rightarrow X', \quad f : FX' \rightarrow Y.$$

The left triangle cut is

$$\varepsilon[f] \circ F[\eta[g]] \rightsquigarrow f \circ F[g].$$

Both sides are arrows $FX \rightarrow Y$. The unit moves g into the GF boundary, the functor F acts on that off-diagonal component, and the counit removes the resulting FG detour.

Dually, for

$$f : X \rightarrow GY', \quad g : Y' \rightarrow Y,$$

the right triangle cut is

$$G[\varepsilon[g]] \circ \eta[f] \rightsquigarrow G[g] \circ f.$$

Both sides are arrows $X \rightarrow GY$. Setting f and g to suitable identities recovers the familiar point-wise triangle identities. Retaining arbitrary f and g exhibits the naturality and higher action consumed by the cut.

Formal status — checked. Evidence `ADJ-TRIANGLE-CUTS`. The indexed owner is `Adjunction ; unit_adj_transf` and `counit_adj_transf` expose the rigid observations that trigger both runtime reductions.

These are the chapter's central checked computations. They illustrate the general policy from Chapter 9: a universal detour reduces only at the universal construction that owns it.

12.3 Transposing Arrows

The ordinary hom formulation of the same adjunction is a natural equivalence

$$\Phi_{a,b} : \text{Hom}_B(Fa, b) \simeq \text{Hom}_A(a, Gb).$$

Starting from the unit and counit, its two directions are

$$\begin{aligned} \Phi_{a,b}(u) &= G[u] \circ \eta_a, \\ \Phi_{a,b}^{-1}(v) &= \varepsilon_b \circ F[v]. \end{aligned}$$

The triangle identities cancel the introduced unit-counit pairs, while naturality makes the construction contravariant in a and covariant in b . This is why transposition is more than a family of bijections: it is a comparison of represented hom functors.

In the active interface, let $M : I \rightarrow A$ and $H : K \rightarrow B$ be arbitrary probes. The reindexed comparison has the profunctor form

$$\text{Hom}_B(FM, H) \simeq \text{Hom}_A(M, GH).$$

The endpoints I and K remain variable, so naturality in both arguments is part of the comparison. Maps into either side can be pushed across the adjunction and pulled back, with beta and eta computation supplied by the generic comparison owner.

Formal status — checked. Evidence `ADJ-HOM-PROF-COMPARISON`. `Adjunction_hom_prof_comparison` is the binary representable comparison, and `Adjunction_hom_prof_comparison_along` reindexes it along arbitrary probes. Its push/pull beta and eta laws are inherited from `ProfComparison`; no second adjunction-specific cancellation calculus is added.

The component formulas above are the mathematical reading of this package. The stable runtime owner is the profunctor comparison, not a global rewrite that expands every mate into a unit/counit composite.

12.4 The Monad Generated By An Adjunction

An adjunction also generates an endofunctor with algebraic multiplication. For $F : A \rightarrow B$ left adjoint to $G : B \rightarrow A$, put

$$T = GF : A \longrightarrow A.$$

The adjunction unit is already a transformation

$$\eta : \text{id}_A \Rightarrow T.$$

The counit supplies multiplication by applying G on the left and F on the right:

$$\mu = G\varepsilon F : T^2 \Rightarrow T. \tag{12.1}$$

Thus (T, η, μ) is the monad selected by the adjunction. Equation (12.1) is not an opaque multiplication law: the whole multiplication observation computes to the transparent $G\varepsilon F$ construction.

The useful triangular operation, following the cut-elimination viewpoint of Došen, is extension. For an arrow

$$f : X \longrightarrow TY,$$

define

$$f^* : TX \longrightarrow TY, \quad f^* = \mu_Y \circ T[f]. \tag{12.2}$$

Extension is a whole functor between the relevant Hom categories, not merely a function on arrows. It therefore retains action on higher arrows between possible f 's.

Let $\eta^c(f)$ denote the off-diagonal unit action on $f : X \rightarrow Y$. The two ambient Došen cuts are

$$\begin{aligned} g^* \circ \eta^c(f) &\rightsquigarrow g \circ f, \\ g^* \circ f^* &\rightsquigarrow (g^* \circ f)^*. \end{aligned} \tag{12.3}$$

The first removes a unit followed by elimination. The second combines two successive extensions into one smaller extension cut. Their orientation is part of the computational presentation: the right-hand side retains one triangular owner rather than expanding both extensions into multiplication and functor action.

The standard multiplication component also compiles into this language:

$$\mu_X \rightsquigarrow (\text{id}_{TX})^*. \quad (12.4)$$

This monad calculus lives in the ambient category $\mathcal{A}$. An explicit Kleisli category would package objects and Kleisli composition as a separate construction. It is not needed for the reductions (12.3), and the active interface does not introduce one merely to rename ordinary composition with extension.

For the adjunction-generated instance, the canonical monad unit computes at runtime. The adjunction unit is also a proof-time usability endpoint for that same declaration. Multiplication computes to $G\varepsilon F$, while its component-first extension normal form and whole-first semantic component are joined by a non-opaque equality path. These are two presentations of one selected multiplication, not parallel monad structures.

Formal status — checked. Evidence `MONAD-TRIANGULAR-COMPUTATION`. A selected monad has whole unit, multiplication, and extension owners; the two ambient cuts (12.3) and the multiplication-component bridge compute. `adjunction_monad` constructs the selected instance with runtime unit and transparent $G\varepsilon F$ multiplication semantics. Free-monad syntax, a global commuting decision procedure, and an explicit Kleisli category are not claimed.

12.5 Adjoints As Representability

Fix $F : A \rightarrow B$ and an object $b : B$. The contravariant hom functor

$$a \longmapsto \text{Hom}_B(Fa, b)$$

is represented by an object Gb exactly when there is a natural equivalence

$$\text{Hom}_B(F-, b) \simeq \text{Hom}_A(-, Gb).$$

If such representing objects are chosen coherently as b varies, they form a functor $G : B \rightarrow A$, and the representations assemble into $F \dashv G$. Conversely, an adjunction supplies these representations by its hom comparison.

This characterization explains the direction of the terminology. A *right adjoint* assigns representing objects to the functors $\text{Hom}_B(F-, b)$; a *left adjoint* is the functor whose outgoing homs are being represented.

Formal status — mathematical development. Evidence `UCAT-ADJOINT-REPRESENTABILITY`. The equivalence between ordinary adjunction data and coherent pointwise representability belongs to the univalent 1-category development. The active code checks the forward adjunction-to-profunctor comparison and has chosen representation packages, but it does not reconstruct a general adjunction from arbitrary local representations.

Chapter 13 will supply the Yoneda theorem that makes representing objects unique in the correct sense. Chapter 16 will then define weighted limits by the same pattern.

12.6 Uniqueness Of Adjoints

Suppose $F : A \rightarrow B$ has two right adjoints G and G' . Their hom representations give, for every b ,

$$\mathrm{Hom}_A(-, Gb) \simeq \mathrm{Hom}_B(F-, b) \simeq \mathrm{Hom}_A(-, G'b).$$

Yoneda turns this into a canonical isomorphism $Gb \cong G'b$, natural in b . If A is an ordinary univalent category, those isomorphisms determine identity of the right-adjoint functors. Hence the type asserting that F has a right adjoint is a mere proposition under the appropriate univalence hypothesis.

Formal status — mathematical development. Evidence `UCAT-ADJOINT-UNIQUENESS`. This is the ordinary HoTT theorem. A native version must choose among identity, natural isomorphism, adjoint equivalence, and omega-equivalence while retaining higher coherence.

Uniqueness therefore does not mean that all chosen units and counits are judgmentally the same. It means that the space of choices has the claimed truncation once categorical identity has been aligned with the relevant equivalence.

12.7 A Ladder Of Equivalence Notions

The word *equivalence* will be qualified according to the following table.

Notion	Data or property	What it controls
carrier equivalence	a <code>TypeEquiv</code> between decoded classifiers	elements and identity structure of two carriers
ordinary object isomorphism	inverse arrows $x \rightleftarrows y$ inside one category	categorical sameness of two objects at the 1-cell level
isomorphism of precategories	a functor that is fully faithful and whose object map is a carrier equivalence	strict invertibility of the whole ordinary presentation
categorical equivalence	a functor with a quasi-inverse promoted to coherent adjoint-equivalence data	sameness up to natural isomorphism
weak equivalence	fully faithful and merely essentially surjective	property-level ordinary categorical equivalence criterion

native omega-equivalence	a selected arrow with recursively usable inverse evidence	equivalence inside an arbitrary native iterated category
--------------------------	---	--

These rows interact only through explicit theorems. A carrier equivalence between object types need not preserve homs. An ordinary isomorphism compares objects in one category, not two entire categories. A categorical equivalence uses functors and natural isomorphisms. Native omega-equivalence can be applied to objects of `Cat_cat`, but its recursive equality-valued interface is not definitionally the HoTT package of fully faithful and essentially surjective data.

12.8 Full Faithfulness And Essential Surjectivity

For an ordinary functor $F : \mathcal{A} \rightarrow \mathcal{B}$:

- F is **fully faithful** if every hom map from $\text{Hom}_{\mathcal{A}}(a, a')$ to $\text{Hom}_{\mathcal{B}}(Fa, Fa')$ is an equivalence;
- F is **split essentially surjective** if each $b : \mathcal{B}$ comes with a chosen $a : \mathcal{A}$ and a chosen isomorphism $Fa \cong b$;
- F is **essentially surjective** if the existence of such a and such an isomorphism is merely asserted.

For ordinary precategories, an adjoint equivalence yields full faithfulness and split essential surjectivity, and those chosen data reconstruct an adjoint equivalence. Replacing split existence by mere existence gives the weaker property traditionally called a weak equivalence.

When both sides are univalent categories, full faithfulness makes the type of possible preimages of an object a proposition. Essential surjectivity can then be upgraded to a coherent choice, so weak equivalence and categorical equivalence agree. The same univalence also turns an equivalence into an isomorphism of the underlying precategory presentations.

Formal status — mathematical development. Evidence `UCAT-EQUIVALENCE-CRITERIA`. These are the ordinary HoTT 1-categorical equivalence theorems. No native fully-faithful or essentially-surjective package with coherent higher action is claimed active.

The distinction between split and mere essential surjectivity is constructive, not bureaucratic. An adjoint equivalence needs data that can be applied; a mere existence statement deliberately hides its witness. Univalence supplies the uniqueness needed to recover that data in the ordinary categorical case.

12.9 Adjointification

Sometimes one begins with a functor F , a proposed inverse G , and natural isomorphisms

$$GF \cong \text{id}_A, \quad FG \cong \text{id}_B$$

whose chosen unit and counit do not yet satisfy the triangle equations. In ordinary category theory, one of the two isomorphisms can be adjusted so that the triangles hold. This **adjointification** turns equivalence data into an adjoint equivalence.

The triangles are therefore coherent normalization data, not an arbitrary extra burden. They make mate transposition inverse in a controlled way and permit universal cuts to reduce without choosing a fresh proof at every use.

Formal status — mathematical development. Ordinary adjointification is part of the 1-categorical theory. The active `Adjunction` relation starts with a selected triangle-computing witness; it does not expose a generic constructor that adjusts arbitrary quasi-inverse transfors.

12.10 Identity Of Ordinary Categories

There is one further univalent step. For ordinary precategories, identity of the complete precategory structures corresponds to isomorphism of precategories: a fully faithful functor whose object map is a carrier equivalence. When the precategories are univalent categories, categorical equivalence and such isomorphism agree. Consequently identity of ordinary categories corresponds to equivalence of categories.

Schematically,

$$(\mathcal{A} = \mathcal{B}) \simeq (\mathcal{A} \cong \mathcal{B}) \simeq (\mathcal{A} \simeq_{\text{cat}} \mathcal{B}),$$

with the second comparison restricted to univalent categories. Since the type of functors has the expected 1-categorical truncation, the type of ordinary categories is a 2-type.

Formal status — mathematical development. Evidence `UCAT-CATEGORY-IDENTITY`. This is the ordinary HoTT result, not a checked identity-to-equivalence theorem for the native universe `Cat_cat`. Chapter 15 places it in the broader structure-identity and saturation programme.

12.11 What The Active Equivalence Layer Proves

At the object level, the active code packages carrier equivalence and native omega-equivalence separately. It also lifts ordinary isomorphism evidence to native omega-equivalence evidence.

Formal status — checked. Evidence `EQUIV-ORDINARY-ISO-LIFT`. The lift is one-way and retains the ordinary forward arrow. It does not prove the full-faithful/essentially-surjective characterization of functors.

At the functor level, an adjunction has checked triangle cuts and a checked binary representable comparison. These facts are sufficient for the later weighted-limit preservation theorem. They are not yet a complete native theory of categorical equivalence.

There is also an important mapping equivalence whose source-functorial packaging is not yet an adjunction. For every category C and groupoid G , restriction along the groupoidification unit gives

$$\text{Hom}_{\text{Grpd}}(\text{Groupoidify}(C), G) \simeq \text{Functor}(C, \text{Path}(G)).$$

The forward and inverse operations are whole functors and satisfy whole beta/eta comparisons. This is exactly the target-side mapping statement one expects from a left adjoint, but the current source assignment has not yet been given functor action on $C \rightarrow D$.

Formal status — checked. Evidence `GENERIC-GROUPOIDIFICATION-MAPPING`. Chapter 27 constructs the unit, recursor, and concrete tests. Calling the current package an adjunction would be premature until `Groupoidify_func` and its source-functor laws are checked.

12.12 The Native Higher Boundary

A higher fully-faithful functor should compare whole hom-categories, then the homs between their arrows, and so on. A higher essential-surjectivity condition should specify which equivalence witnesses inhabit its fibres and whether their evidence is propositional. Turning those conditions into a native adjoint equivalence would require:

- iterable hom-equivalence packages;
- a coherent object-level essential-surjectivity interface;
- natural unit and counit transforers with full off-diagonal action;
- triangle coherence at every retained level;
- a chosen relationship with object identity or saturation.

Formal status — research boundary. The missing result is a native fully-faithful-plus-essentially-surjective characterization compatible with `OmegaEquiv`, `Functor_cat`, and higher transforers. Ordinary HoTT equivalence theorems are used only in their stated 1-categorical specialization.

The next chapter begins where the ordinary and native stories already meet: representable homs. Yoneda explains why their maps are controlled by elements; the checked co-Yoneda cut then gives that explanation a directed profunctorial computation.

13. Yoneda, Representability, And Profunctors

Representability turns the arrows out of, or into, one object into a universal coordinate system. Yoneda says that a natural map out of such a coordinate system is determined by its value at an identity. Adjunction says that one hom coordinate system is represented by another object. Weighted limits will repeat the same argument for a more elaborate functor.

The ordinary Yoneda lemma is therefore the conceptual center of this part of the book. We first develop its univalent 1-categorical form, then identify the native representable families already used by arrow induction, and finally pass to Cat-valued profunctors. The chapter's central *checked* computation is a shaped co-Yoneda cut: tensor an element with the appropriate identity-shaped hom, apply the co-Yoneda map, and recover the original element.

The full ordinary Yoneda lemma below is mathematical development adapted from the HoTT Book. The active artifact checks native representables and the shaped Cat-valued co-Yoneda beta/fusion equations. It does not yet package a general Cat-valued Yoneda equivalence or a semantic coend. Kelly supplies enriched background for these patterns, while Bénabou is a classical reference for their bicategorical organization.

13.1 Representables And Variance

Let $\mathcal{A}$ be an ordinary precategory. A presheaf is a functor

$$P : \mathcal{A}^{\text{op}} \longrightarrow \mathbf{Set}.$$

For $a : \mathcal{A}$, the contravariant representable presheaf is

$$y(a) := \mathbf{Hom}_{\mathcal{A}}(-, a).$$

An arrow $u : a \rightarrow b$ induces a natural transformation $y(u) : y(a) \Rightarrow y(b)$ by postcomposition:

$$y(u)_x(f) = u \circ f = u_*(f).$$

This defines the Yoneda embedding

$$y : \mathcal{A} \longrightarrow [\mathcal{A}^{\text{op}}, \mathbf{Set}].$$

The active formal development also lifts this picture from sets to Cat-valued presheaves. Its Yoneda presheaf evaluates to the represented hom, and restriction reuses the existing pullback of directed families. Gathering all arrows into a produces the restriction-oriented total

$$\mathbf{Into}_{\mathcal{A}}^-(a) := \sum_{x : \mathcal{A}^{\text{op}}} \mathbf{Hom}_{\mathcal{A}}(x, a)$$

whose opposite is the conventional slice $\mathcal{A}/a$. A Cat-valued family on this total is a higher sieve; requiring its values to be subterminal gives an ordinary sieve. [Chapter 18](#) begins the local-to-global spiral by developing these constructions as mathematics, separating witness-bearing higher sieves from proposition-valued membership, and explaining why sieve pullback is the natural language of a changing probe.

This bridge does not strengthen the Yoneda theorem claimed in the present chapter. A general Cat-valued Yoneda equivalence and full-faithfulness theorem remain separate from the checked representable action and shaped co-Yoneda calculation below.

There is a covariant mirror

$$y^a := \text{Hom}_{\mathcal{A}}(a, -) : \mathcal{A} \longrightarrow \mathbf{Set}.$$

It is contravariant in the represented object a : an arrow $u : a \rightarrow b$ induces precomposition

$$u^* : \text{Hom}_{\mathcal{A}}(b, -) \Longrightarrow \text{Hom}_{\mathcal{A}}(a, -).$$

The lower-star and upper-star actions from Chapter 9 are therefore the two variance legs of representability.

13.2 The Ordinary Yoneda Equivalence

For a presheaf $P : \mathcal{A}^{\text{op}} \rightarrow \mathbf{Set}$, the Yoneda equivalence is

$$\text{Nat}(y(a), P) \simeq P(a).$$

Its forward map evaluates a natural transformation at the identity:

$$\text{encode}(\alpha) := \alpha_a(\text{id}_a).$$

For $p : P(a)$, the reverse map defines a natural transformation by

$$\text{decode}(p)_x(f) := P[f](p), \quad f : x \rightarrow a.$$

The first composite is immediate from preservation of identity:

$$\text{encode}(\text{decode}(p)) = P[\text{id}_a](p) = p.$$

For the other composite, naturality of α at $f : x \rightarrow a$ gives

$$P[f](\alpha_a(\text{id}_a)) = \alpha_x(f).$$

Thus $\text{decode}(\text{encode}(\alpha)) = \alpha$ by extensionality. The proof has the same shape as Chapter 8: choose a code by evaluation, decode it by functorial transport, and calculate both composites. Here the distinguished constructor is the identity arrow of a representable.

The covariant form is equally important. For $Q : \mathcal{A} \rightarrow \mathbf{Set}$,

$$\mathrm{Nat}(\mathrm{Hom}_{\mathcal{A}}(a, -), Q) \simeq Q(a),$$

and the inverse sends $q : Q(a)$ to the family $f : a \rightarrow x \mapsto Q[f](q)$.

Formal status — mathematical development. Evidence `UCAT-YONEDA`. These are the ordinary set-valued Yoneda equivalences. They are stated under the HoTT precategory assumptions and are not labeled as the active full Cat-valued theorem.

13.3 Full Faithfulness And Uniqueness Of Representation

Apply Yoneda with $P = y(b)$. One obtains

$$\mathrm{Nat}(y(a), y(b)) \simeq \mathrm{Hom}_{\mathcal{A}}(a, b).$$

Under this equivalence, an arrow $u : a \rightarrow b$ corresponds to postcomposition by u . Hence the Yoneda embedding is fully faithful: it recovers every hom from the natural maps between representables.

More generally, a representation of P consists of an object a and a natural isomorphism $y(a) \cong P$. If a and b both represent P , full faithfulness produces a canonical isomorphism $a \cong b$. When $\mathcal{A}$ is a univalent category, this isomorphism corresponds to object identity, and representability becomes a property rather than an uncontrolled choice of presentation.

Formal status — mathematical development. This full-faithfulness and uniqueness result is part of evidence `UCAT-YONEDA`. The conclusion uses ordinary category univalence exactly where an isomorphism of representing objects is turned into identity.

13.4 Native Representable Families

For a native category Z and an object x , `emdash` has the Cat-valued fixed-source representable

$$\mathrm{Rep}_Z(x)[y] := \mathrm{Hom}_Z(x, y).$$

Its target action is postcomposition. Its dependence on the represented source is contravariant: for $p : x \rightarrow y$, the transport

$$\mathrm{Rep}_Z(y) \longrightarrow \mathrm{Rep}_Z(x)$$

acts by

$$q \longmapsto q \circ p = p^*(q).$$

The full operation is a functor between directed families, so it retains action on cells between possible q 's.

Formal status — checked. Evidence `CAT-REPRESENTABLE` . `Rep_catd` owns the fixed-source family and `Rep_transport_func` owns its upper-star source action.

The outgoing-arrow category from Chapter 5 is the total category

$$\text{PathOut}_Z(x) = \sum_{y:Z} \text{Hom}_Z(x, y).$$

Its distinguished object (x, id_x) and its canonical arrow to every (y, p) are built from the representable family. Arrow induction extends data from that reflexive outgoing arrow.

This is Yoneda-shaped, but it is not literally the ordinary Yoneda lemma. Yoneda classifies natural transformations from a representable; the selected arrow-induction theorem eliminates a dependent motive over the total category of outgoing arrows. Both begin at an identity arrow because identities are the universal points of representables.

13.5 From One Endpoint To Two

Opposites and products make the two variances explicit. In the ordinary 1-categorical setting, currying and uncurrying compare functors

$$\mathcal{A} \times \mathcal{B} \longrightarrow \mathcal{C}$$

with functors

$$\mathcal{A} \longrightarrow [\mathcal{B}, \mathcal{C}].$$

Applying this organization to composition shows that `hom` is functorial in both endpoints, contravariantly in its source and covariantly in its target.

The two representable variances combine in the `hom` bifunctor

$$\text{Hom}_X(-, -) : X^{\text{op}} \times X \longrightarrow \text{Cat}.$$

Reindexing its two endpoints along functors $F : A \rightarrow X$ and $G : B \rightarrow X$ gives

$$\text{Hom}_X(F-, G-) : A^{\text{op}} \times B \longrightarrow \text{Cat}.$$

This two-variable view is the bridge from Yoneda to profunctors. It makes contravariance and covariance simultaneous, lets endpoint functors vary, and turns `hom` transposition for an adjunction into a comparison of profunctors.

The price of the richer codomain is that ordinary elementwise proofs no longer automatically supply all higher naturality. The active development therefore selects shaped elements and explicit comparison maps before claiming a general coend-based composition.

13.6 Cat-Valued Profunctors

For categories A and B , a Cat-valued profunctor from A to B is a functorial family

$$P : A^{\text{op}} \times B \longrightarrow \mathbf{Cat}.$$

We write $P : A \text{ prof } B$ informally and write its fibre at (a, b) as $P(a, b)$. The opposite on A records the variance:

- an arrow $p : a' \rightarrow a$ acts contravariantly in the first endpoint;
- an arrow $q : b \rightarrow b'$ acts covariantly in the second endpoint.

Profunctors with fixed endpoints form a category. A vertical map

$$r : P \Longrightarrow Q$$

is a natural family morphism over $A^{\text{op}} \times B$. Its identity, vertical composition, components, and higher action are inherited from the directed-family and transfer calculus of Chapters 2 and 9.

Formal status — checked. Evidence `PROF-CATEGORY`. `Prof_base(A,B)=Aop times B`; `Prof_cat` is the fixed-endpoint category and `ProfMap` its vertical-map classifier.

This definition introduces no new primitive notion of naturality. A profunctor is a familiar Cat-valued functor on a product base, viewed through notation that makes its two variances visible.

13.7 The Unit Hom Profunctor

Every category X has a canonical profunctor

$$U_X : X^{\text{op}} \times X \longrightarrow \mathbf{Cat}, \quad U_X(x, y) := \text{Hom}_X(x, y).$$

On an arrow pair

$$p : x' \rightarrow x, \quad q : y \rightarrow y',$$

it acts by cutting on both sides:

$$h \longmapsto q \circ h \circ p.$$

The opposite variance is not decorative: it is exactly what makes the first composition type-correct. Identity and composite action follow from the generic hom bifunctor.

More generally, given functors

$$F : A \longrightarrow X, \quad G : B \longrightarrow X,$$

reindex the unit to obtain the binary representable

$$\mathrm{Hom}_X(F-, G-) : A \rightsquigarrow B$$

with fibres

$$\mathrm{Hom}_X(Fa, Gb)$$

and action

$$h \longmapsto G(q) \circ h \circ F(p).$$

Formal status — checked. Evidence `PROF-REPRESENTABLE`. `Unit_prof` owns the hom pro-functor, and `Hom_prof_along(F,G)` is its endpoint-reindexed presentation with the two-sided `Hom_fapp0` action.

Two specializations are useful. The **companion** of $F : A \rightarrow B$ is represented covariantly by `F`; the **conjoint** is represented contravariantly by `F`. These are not assumed to be inverse profunctors. They are the two variance choices obtained from the same ambient hom.

13.8 Reindexing Endpoints

Let `P : A prof B`, $F : A' \rightarrow A$, and $G : B' \rightarrow B$. Endpoint reindexing is

$$(F, G)^* P : A' \rightsquigarrow B', \quad ((F, G)^* P)(a', b') := P(Fa', Gb').$$

Semantically this is pullback along

$$F^{\mathrm{op}} \times G : (A')^{\mathrm{op}} \times B' \longrightarrow A^{\mathrm{op}} \times B.$$

The operation acts on vertical maps, is neutral at identity endpoint functors, and accumulates nested reindexings by composing both endpoints. For representables, it gives the expected equation

$$(F', G')^* \mathrm{Hom}_X(F-, G-) = \mathrm{Hom}_X((F \circ F')-, (G \circ G')-).$$

Formal status — checked. Evidence `PROF-REINDEXING`. `Prof_reindex` is the stable object owner and `Prof_reindex_func` provides its vertical functorial action.

The separation between a profunctor and its reindexed view is analogous to the separation between `WalkingEnd` and `BNat`. A readable model or pullback presentation does not become a definitional replacement for the object it explains.

13.9 Representability As A Chosen Comparison

A profunctor $P : B \text{ prof } J$ is right-represented by a functor $L : J \rightarrow B$ when it is isomorphic, in the fixed-endpoint profunctor category, to the conjoint of L :

$$P \cong \text{Hom}_B(-, L-).$$

A representation package retains both the chosen functor and its isomorphism evidence:

$$\sum_{L:J \rightarrow B} \text{IsoEvidence}(P, \text{Hom}_B(-, L-)).$$

Formal status — checked. Evidence `PROF-REPRESENTATION-PACKAGE` . The classifiers are `IsRepresentedBy_iso` and `Representation_iso` ; the current comparison is ordinary isomorphism evidence in `Prof_cat` .

This is a useful universal-property interface even before uniqueness of the representing object is packaged. A weighted limit, for example, can be presented as a chosen representation of a cone profunctor. The computational strength of that use depends on how much of the comparison is exposed by the consumer.

13.10 Cells With Moving Endpoints

Vertical maps keep endpoints fixed. Equipment-style cells allow endpoint functors to move as well. Given

$$R' : A' \rightsquigarrow B', \quad R : A \rightsquigarrow B, \quad F : A' \rightarrow A, \quad G : B' \rightarrow B,$$

a cell over F, G is a natural family morphism

$$c : R' \Longrightarrow (F, G)^* R.$$

If the source profunctor is the unit U_I , such a cell is a shaped element of R with endpoint functors $I \rightarrow A$ and $I \rightarrow B$. The narrow application operation sends a shaped element of R' through c to the corresponding shaped element of R .

Formal status — checked. Evidence `PROF-SHAPED-CELLS` . `Prof_transf_cat` is the endpoint-changing cell category, `Prof_hom` is its unit-source specialization, and `Prof_cell_apply` is the selected application operation.

The operation is intentionally narrower than arbitrary horizontal composition of equipment cells. It supplies the consumer needed by the co-Yoneda calculation without pretending that all bicategorical coherence is already present.

13.11 Tensor As A Selected Composite

For

$$P : A \rightsquigarrow B, \quad Q : B \rightsquigarrow X,$$

write

$$P \otimes_B Q : A \rightsquigarrow X$$

for their selected profunctor tensor. In ordinary enriched category theory one expects a coend-like formula

$$(P \otimes_B Q)(a, x) \simeq \int^{b:B} P(a, b) \times Q(b, x).$$

This formula is the intended mathematical reading, not a definition unfolded by the active kernel. `Prof_tensor` is opaque. What is exposed is the behavior required by current consumers:

- reindexing in the outer endpoints distributes through the tensor;
- the middle category stays fixed;
- vertical maps in both factors induce a vertical map of tensors;
- compatible shaped elements compose to a shaped tensor element.

Formal status — checked. Evidence `PROF-TENSOR`. The fixed-middle object is `Prof_tensor`; `Prof_tensor_func` owns fixed-endpoint bifunctionality and `Prof_tensor_hom_hom` composes shaped elements.

Opacity is mathematically honest here. A genuine coend construction would need a quotient or universal colimit with its own eliminator and computation rules. The selected tensor can support tested cut-elimination interfaces without serving as evidence that this general construction has already been built.

13.12 The Shaped Co-Yoneda Cut

The unit profunctor should behave as a unit for tensor. The active interface supplies natural maps in both orientations:

$$\begin{aligned} \varepsilon_P^R : P \otimes_B U_B &\Longrightarrow P, \\ \varepsilon_P^L : U_A \otimes_A P &\Longrightarrow P. \end{aligned}$$

They are components of transformations natural in `P`, rather than a collection of unrelated vertical maps.

Now let `p` be a shaped element of `P` with middle shape $M : I \rightarrow B$. The unit profunctor has a canonical shaped element given by the identity transformation on `M`. Tensor the two elements and apply the right co-Yoneda map. The cut eliminates:

$$\varepsilon_P^R(p \otimes \text{id}_M) = p.$$

Dually, for an A -shaped endpoint F ,

$$\varepsilon_P^L(\text{id}_F \otimes p) = p.$$

Formal status — checked. Evidence `PROF-COYONEDA`. `Prof_coyoneda_cov_transf` and `Prof_coyoneda_con_transf` are the two natural transformations; their component maps have the shaped-element beta equations above.

These equations are the chapter's central theorem. They express Yoneda-style evaluation as computation: insert an identity-shaped hom, form the composite, and the co-Yoneda counit removes the cut.

The transformations also fuse with an arbitrary vertical map $r : P \rightarrow P'$. Applying the naturality component of co-Yoneda to $p \otimes \text{id}$ reduces to applying `r` directly to `p`:

$$\varepsilon_{P'}((r \otimes \text{id})(p \otimes \text{id})) = r(p),$$

with the analogous left-unit equation. This is not a separate law attached to every `r`; it is the off-diagonal action of the co-Yoneda transfor from Chapter 9.

13.13 Adjunctions As Representable Hom Comparisons

Chapter 12 characterized a right adjoint $G : B \rightarrow A$ to $F : A \rightarrow B$ by representations

$$\text{Hom}_B(F-, b) \simeq \text{Hom}_A(-, Gb).$$

Yoneda explains both the representing object and its uniqueness. The profunctor formulation adds simultaneous naturality in the source probe and in b . For arbitrary $M : I \rightarrow A$ and $H : K \rightarrow B$, the active comparison is

$$\text{Hom}_B(FM, H) \simeq \text{Hom}_A(M, GH).$$

Formal status — checked. Evidence `ADJ-HOM-PROF-COMPARISON`. The adjunction supplies this reindexable `ProfComparison`; its push and pull operations have the generic comparison beta/eta laws.

Conversely, an ordinary pointwise family of such representations reconstructs a right adjoint only after the representing objects and comparisons have been made coherent in b . The active representation package records a chosen representing functor and ordinary isomorphism evidence in the profunctor category, but no general reverse constructor from that package to `Adjunction` is asserted.

Formal status — mathematical development. The ordinary reverse representability theorem is part of `UCAT-ADJOINT-REPRESENTABILITY`. Evidence `PROF-REPRESENTATION-PACKAGE`, checked in Section 13.9, supplies the native chosen representation interface on which a future reverse construction could be based.

This is the bridge to weighted universals. A weighted cone construction will produce another profunctor; a weighted limit is the functor that represents it. The same comparison beta/eta laws will eliminate the corresponding universal cuts.

13.14 The Full Cat-Valued Yoneda Boundary

The computations capture an essential Yoneda idea: represented hom data can be inserted as an identity-shaped cut and then eliminated. The existing right-representable embedding, endpoint reindexing, shaped cells, and co-Yoneda maps give a credible computational core for a broader theorem.

A full Cat-valued Yoneda theorem would package more. One expects a functor from `A` into an appropriate presheaf or profunctor category, a precise mapping-category equivalence, naturality in the represented object and the presheaf, and a proof that the embedding is fully faithful at every retained cell level. Those data have not been gathered into one active theorem.

Formal status — research boundary. Evidence `YONEDA-FULLY-FAITHFUL`. The intended future owner should be built from the existing representable functor, shaped-cell, and co-Yoneda interfaces and should expose a reusable equivalence rather than add isolated projection rules.

The checked beta equations remain valuable without that package. They are local computational theorems, not merely motivational analogies.

13.15 The Coend And Bicategorical Boundary

To promote the tensor to a general profunctor composition, one would need:

- a coend or coinsertion construction for Cat-valued data;
- its dinatural elimination and computation principles;
- associativity and left/right unit comparisons;
- coherent higher cells for those comparisons;
- horizontal composition of arbitrary endpoint-changing cells;
- compatibility with opposites, reindexing, and internal homs.

Only selected parts of this list are active. Fixed-endpoint tensor action, outer reindexing, shaped composition, and both co-Yoneda counits have checked owners. They do not automatically assemble into a bicategory.

Formal status — research boundary. Evidence `PROF-GENERAL-COEND`. A future implementation should give the opaque tensor a universal semantics or relate it to a separately constructed coend; it should not infer general coherence from the current endpoint beta rules alone.

This staged boundary mirrors the rest of the book. We expose enough functorial structure to compute a central example, retain the higher action that example consumes, and state the missing universal package explicitly.

14. Strictness, Dagger Structure, And Duality

The word *strict* enters category theory by several doors. A HoTT strict category has a set of objects. An emdash naturality cut is strict when it selects a runtime normal form. A rewrite is strict in yet another sense when it is judgmental computation rather than internal equality. None of these conditions implies either of the others.

The neighboring notion of a dagger illustrates why this vocabulary matters. A dagger is a *chosen* reversal internal to one category, whereas the opposite construction reverses any category from the outside. The distinction is the same one that has guided the book throughout: a generic operation, a selected structure, and a computational presentation have different owners.

This chapter adapts the strict- and dagger-category discussions of the [HoTT Book](#) to the ordinary 1-categorical specialization, then states the additional coherence a native directed version would need. Its central checked theorem is opposite duality. The dagger theory is mathematical development with an explicit implementation boundary.

14.1 A Qualified Strictness Vocabulary

The following terms remain separate throughout the book.

Qualified notion	Meaning	Does not imply
HoTT strict category	an ordinary precategory whose object type is a set	category univalence
HoTT category	an ordinary precategory for which <code>idtoiso</code> is an equivalence	that the object type is a set
gaunt category	a HoTT category that is also strict	runtime strictness
native <code>ISNCat(n, C)</code>	recursive finite height of the hom-categories	object identity agrees with isomorphism
strict naturality cut	a selected <code>tapp1</code> composite reduces to one off-diagonal action	all coherence is judgmental
computational strict-functor property	<code>IsStrictFunctor(F)</code> compares the existing compositor with its equality-induced arrow; <code>StrictFunctor(A, B)</code> pairs F with that evidence	every ambient functor is strict or every coherence is judgmental
runtime strictness	an oriented kernel reduction chooses a normal form	object truncation or invertibility
dagger category	identity agrees with <i>unitary</i> isomorphism	identity agrees with every isomorphism

In particular, the HoTT phrase *strict category* begins with a **precategory**, not with a univalent category. Chapter 10's translation table uses this definition. A strict precategory may still have nontrivial automorphisms that cannot come from its proposition-valued object identity.

The proof-carrying row is the profile used in Chapter 28. It does not decode a second functor grammar. `IsStrictFunctor` constrains the compositor already extracted from an ambient functor, and the exact `StrictFunctor` package pairs that functor with its evidence while leaving the ambient transformation and higher-hom calculus shared with lax maps. This is a local property/evidence specialization, not a blanket compositor-to-identity rewrite or evidence that every historical global strict endpoint cut has already migrated.

Formal status — checked. Evidence `GRAY-WALKING-INTERCHANGER` includes the selected strict-object/lax-arrow profile and its nonidentity walking-square interchanger. The full Crans–Gray monoidal structure remains outside the checked boundary.

14.2 Strict Categories In Ordinary Univalent Foundations

Let $\mathcal{A}$ be an ordinary precategory. It is *strict* when $\text{Obj}(\mathcal{A})$ is a set:

$$\prod_{x,y:\text{Obj}(\mathcal{A})} \text{isProp}(x = y).$$

This condition says that two proofs of equality between objects agree. It does not say that every isomorphism comes from equality. For example, the one-object category associated to a nontrivial group is strict: its object type is the unit type. It is not a HoTT category, since its nonidentity group elements are automorphisms while the unique object has only the reflexive identity path.

A poset on a set-valued carrier gives the contrasting example. Its homs are propositions and antisymmetry identifies mutual reachability with object identity, so it is both strict and univalent. More generally, a HoTT category is strict exactly when it is *gaunt*: its isomorphisms contain no additional object sameness beyond proposition-valued identity.

Strict categories therefore support a stricter package-level notion of sameness than categorical equivalence. Equality of their presentations agrees with isomorphism of the corresponding precategory structures, whereas an equivalence can still change a presentation by choosing merely equivalent objects. This can be useful, but it is not the default notion of sameness in univalent category theory.

Formal status — mathematical development. Evidence `UCAT-STRICT-CATEGORY`. The definition and the gaunt comparison belong to ordinary univalent 1-category theory. They are not a definition of native `Cat` or a theorem about arbitrary finite-dimensional native categories.

14.3 Three Native Notions That Strict Categories Do Not Control

Finite directed height is the first separate notion. A witness

$$\text{IsNCat}(n, C)$$

recurses through native hom-categories. At dimension one it says that every hom-category is discrete and entails that the object classifier is a 1-type. HoTT strictness instead asks directly that the object classifier be a set. Neither condition supplies a native identity-to-isomorphism theorem.

The second notion is strict naturality. For an ordinary transfor $\eta : F \Rightarrow G$, the generic off-diagonal action has the two reductions

$$\begin{aligned} G[g] \circ \eta[f] &\rightsquigarrow \eta[g \circ f], \\ \eta[f] \circ F[h] &\rightsquigarrow \eta[f \circ h]. \end{aligned}$$

The phrase *strict transfor* in this book describes this selected two-sided cut behavior; it is not a new classifier of categories. A displayed lax comparison can instead retain a directed naturality cell without forcing it to equality.

The third notion is runtime strictness itself. The arrow $t \rightsquigarrow u$ records a chosen normal form in the `Lambdapi` theory. An internal equality $t = u$, a proof-time comparison, and a free-form mathematical equation have different force. A category can have proposition-valued object identity while none of its interesting operations compute, or support rich directed higher cells while selected naturality cuts do compute.

Formal status — checked. Evidence `CAT-DIMENSION` and `TRANSF-STRICT-NATURALITY`. The active theory separately checks recursive dimension/object truncation and the two ordinary `tapp1` naturality reductions. No checked theorem identifies these interfaces.

14.4 Opposite Duality Computes

For every native category C , the opposite category has the same objects and reversed homs:

$$\mathrm{Hom}_{C^{\mathrm{op}}}(x, y) = \mathrm{Hom}_C(y, x).$$

Identity arrows are unchanged, while the factors of composition reverse. The active operation is involutive not only on categories but through the iterable functor and transfor layers:

$$\begin{aligned} (C^{\mathrm{op}})^{\mathrm{op}} &\rightsquigarrow C, \\ (F^{\mathrm{op}})^{\mathrm{op}} &\rightsquigarrow F, \\ (\alpha^{\mathrm{op}})^{\mathrm{op}} &\rightsquigarrow \alpha. \end{aligned}$$

Opposite reverses vertical composition of transfors. It also turns an adjunction

$$F \dashv G$$

into

$$G^{\mathrm{op}} \dashv F^{\mathrm{op}}.$$

The new unit is the opposite of the old counit, and the new counit is the opposite of the old unit. Applying opposite twice reduces to the original adjunction package. This is a computational duality, not a silent convention that suppresses variance.

Formal status — checked. Evidence `OP-DUALITY`. The principal owners are `Op_cat`, `Op_func`, `Op_transf`, and `Op_adjunction`. Their involution and variance-reversal rules are the checked basis for the weighted-colimit duality in Chapter 17.

14.5 Dagger Structure Is Chosen Self-Duality

An ordinary $\dagger$ -precategory is a precategory $\mathcal{A}$ equipped with an operation

$$(-)^\dagger : \text{Hom}_{\mathcal{A}}(x, y) \longrightarrow \text{Hom}_{\mathcal{A}}(y, x)$$

satisfying

$$\begin{aligned} (\text{id}_x)^\dagger &= \text{id}_x, \\ (g \circ f)^\dagger &= f^\dagger \circ g^\dagger, \\ (f^\dagger)^\dagger &= f. \end{aligned}$$

Equivalently, it is an identity-on-objects functor-like map

$$D : \mathcal{A}^{\text{op}} \longrightarrow \mathcal{A}$$

with a chosen involution law. The word *chosen* is essential. The opposite construction gives $\mathcal{A}^{\text{op}}$ for every $\mathcal{A}$; it does not give a functor from that opposite back to $\mathcal{A}$, much less one that fixes objects and squares to the identity.

14.6 Unitary Arrows And Dagger Univalence

An arrow $f : x \rightarrow y$ is *unitary* when its dagger is its two-sided inverse:

$$f^\dagger \circ f = \text{id}_x, \quad f \circ f^\dagger = \text{id}_y.$$

Every unitary arrow is an isomorphism, but an isomorphism need not be unitary. Object identity always produces a unitary isomorphism by identity induction, so there is a canonical map

$$\text{idtoUnitary}_{\mathcal{A}} : (x = y) \longrightarrow (x \cong_{\dagger} y).$$

A $\dagger$ -category is a $\dagger$ -precategory for which this map is an equivalence. Its selected notion of object sameness is unitary isomorphism, not arbitrary isomorphism.

Two examples separate the notions. In a groupoid, define $f^\dagger = f^{-1}$; then every arrow is unitary. For finite-dimensional inner product spaces with arbitrary linear maps, the dagger is the adjoint linear map. The unitary isomorphisms are the isometries, while many invertible linear maps are not unit-

ary. Thus this $\dagger$ -category need not be a HoTT category under ordinary isomorphism.

Formal status — mathematical development. Evidence `UCAT-DAGGER-CATEGORY`. This is the ordinary set-valued-hom theory: dagger laws are equalities, unitarity is a property, and dagger univalence compares object identity with unitary isomorphism.

14.7 Opposite, Duality, And Dagger Compared

The three reversal notions have different input data.

Construction	Data supplied	Result
opposite	any category C	a category C^{op} with arrows reversed
categorical duality	a chosen equivalence $C^{\text{op}} \simeq D$	a comparison between two categories
dagger	a chosen identity-on-objects involutive map $C^{\text{op}} \rightarrow C$	a unitary notion internal to C

A native dagger would have to act at every retained hom dimension. At minimum it would require:

1. a functor $D : C^{\text{op}} \rightarrow C$;
2. identity-on-objects data, with an explicit decision about whether it computes or is witnessed coherently;
3. an involution comparison between $D \circ D^{\text{op}}$ and id_C ;
4. compatibility of that comparison with off-diagonal and higher action;
5. a unitary-arrow classifier whose evidence is stable under identity, composition, and the next hom action;
6. a qualified identity-to-unitary-equivalence interface.

The ordinary equations above are a plausible strict specialization of this design, not a license to erase the higher coherence. In particular, `Op_cat` supplies only the first half of the ambient reversal and cannot serve as a native dagger by itself.

Formal status — research boundary. Evidence `NATIVE-DAGGER-INTERFACE`. No dagger/unitary owner is active. Side task `FTTX-S12` remains a specification target, and this chapter does not add a prose-only kernel name or infer dagger structure from opposite duality.

14.8 Duality As A Proof Method

Opposite duality becomes useful when a theorem has already exposed its variance. A right adjoint preserves weighted limits because a representable comparison can be transported through its hom adjunction. Passing to opposites exchanges:

right adjoint $\longleftrightarrow$ left adjoint,
 weighted limit $\longleftrightarrow$ weighted colimit,
 unit $\longleftrightarrow$ counit,
 upper-star source action $\longleftrightarrow$ lower-star target action.

Chapter 17 will use these checked opposite owners to derive the colimit preservation theorem rather than repeat the limit proof with reversed arrows. A dagger could internalize a particular self-dual instance of such an argument, but the general theorem needs only opposite duality. This is why the book includes dagger structure for conceptual completeness without making it the foundation of categorical duality.

15. Structure Identity And Saturation

Univalence is most useful when it propagates from bare carriers to the structures mathematicians put on them. The *structure identity principle* says that structure-preserving equivalence is the identity of structured objects. *Saturation* asks the complementary question: if a categorical presentation does not yet have the desired identity-to-equivalence property, can it be completed universally into one that does?

The ordinary 1-categorical answers are the structure identity theorem and Rezk completion. They also reveal the architecture a directed version would need. Evidence must be a property when it is intended to be inessential; transport of structure must act coherently; the selected equivalence notion must be stated; and a completion is characterized by its mapping property, not merely by a newly constructed carrier.

This chapter adapts Sections 9.8 and 9.9 of the [HoTT Book](#). The ordinary theorems are mathematical development. Emdash already checks several local ingredients, but a general native structure identity principle and Rezk completion remain research boundaries.

15.1 Truncation And Saturation Answer Different Questions

The following distinctions are essential.

Condition	What it controls	What it does not supply
object truncation	the height of $x = y$	a comparison with categorical equivalence
finite <code>IsNCat</code> evidence	recursive height of hom-categories	category univalence
ordinary category saturation	$(x = y) \simeq (x \cong y)$	proposition-valued object identity
dagger saturation	$(x = y) \simeq (x \cong_{\dagger} y)$	identity with every ordinary isomorphism
prospective native saturation	identity agrees with one specified higher equivalence classifier	a canonical choice of that classifier

A strict category can therefore be unsaturated, while a saturated category can have a non-set-valued object type. Likewise, the finite-height theorem used for `WalkingEnd` bounds identity types but does not turn an arbitrary ordinary isomorphism into object identity.

15.2 A Notion Of Structure Over A Carrier Category

Let $\mathcal{X}$ be an ordinary precategory. A notion of structure (P, H) over $\mathcal{X}$ consists of:

1. a type $P(x)$ of structures on each object x ;
2. for $f : x \rightarrow y$, $\alpha : P(x)$, and $\beta : P(y)$, a proposition $H_{\alpha, \beta}(f)$ saying that f preserves the structures;

3. evidence that identity arrows preserve structure;
4. evidence that composites of structure-preserving arrows preserve structure.

For structures $\alpha, \beta : P(x)$ on the same carrier, define

$$\alpha \leq_x \beta \equiv H_{\alpha, \beta}(\text{id}_x).$$

Identity and composition make this a preorder. The notion of structure is called *standard* when this preorder is antisymmetric in every fibre. In particular, each $P(x)$ is then a set.

The associated precategory $\mathbf{Str}_{P, H}(\mathcal{X})$ has objects

$$(x, \alpha) : \sum_{x : \text{Obj}(\mathcal{X})} P(x)$$

and arrows

$$(x, \alpha) \longrightarrow (y, \beta) \quad := \quad \sum_{f : x \rightarrow y} H_{\alpha, \beta}(f).$$

Because H is proposition-valued, it adds a preservation condition without adding competing arrow data. Identities and composition come from $\mathcal{X}$ and the two closure laws.

15.3 The Ordinary Structure Identity Theorem

Structure identity principle. If $\mathcal{X}$ is an ordinary univalent category and (P, H) is a standard notion of structure over it, then $\mathbf{Str}_{P, H}(\mathcal{X})$ is an ordinary univalent category.

The proof isolates the two uses of uniqueness. An identity

$$(x, \alpha) = (y, \beta)$$

is a carrier identity $p : x = y$ together with

$$\text{transport}_P(p, \alpha) = \beta.$$

Since the structure fibres are sets, the second clause is a proposition. An isomorphism of structured objects is a carrier isomorphism $f : x \cong y$ together with preservation by f and by f^{-1} ; those clauses are also propositions.

Univalence of $\mathcal{X}$ converts f into a carrier identity p . By identity induction, it remains to consider $p \equiv \text{refl}_x$. The two preservation clauses then say

$$\alpha \leq_x \beta \quad \text{and} \quad \beta \leq_x \alpha.$$

Antisymmetry gives $\alpha = \beta$. This constructs the inverse to `idtoiso` for structured objects; proposition-valuedness supplies the remaining coherence.

Formal status — mathematical development. Evidence `UCAT-STRUCTURE-IDENTITY`. The theorem is the ordinary HoTT structure identity principle. It depends on set-valued homs, proposition-valued preservation, base-category univalence, and fibrewise antisymmetry.

15.4 Examples And The Role Of Standardness

Functor structure is a representative example. Begin with object functions $F_0 : \text{Obj}(\mathcal{A}) \rightarrow \text{Obj}(\mathcal{B})$. The additional structure assigns arrow actions preserving identities and composition. A pointwise family of arrows is a homomorphism exactly when it is natural. If $\mathcal{B}$ is univalent, the structure identity theorem recovers the functor-category result from Chapter 11: natural isomorphism agrees with identity of functors.

Ordinary algebraic and relational structures fit the same scheme. A signature specifies operations and relations on a carrier set, while H says that a function preserves them. Function extensionality and proposition extensionality make the structure fibres sets; mutual preservation by the identity function forces equality of structures. Thus isomorphic groups, rings, ordered sets, and similar standard structures become identical in their univalent categories.

Standardness is not cosmetic. If two distinct structures on the same carrier admit identity-carrier homomorphisms in both directions, structured isomorphism contains less discriminating information than structure identity. The theorem correctly refuses to identify them until antisymmetry or an appropriate higher replacement has been supplied.

15.5 Checked Native Footholds

The active theory contains four ingredients that a native structure identity theorem should reuse.

First, truncation evidence is proposition-valued at every implemented level. Adding an `IsTruncGrpd` field to a carrier package therefore does not add a second independent notion of identity between witnesses.

Second, for a fixed arrow, native equality-valued omega-equivalence evidence is proposition-valued. The chosen arrow remains data, but two proofs that the same arrow is an omega-equivalence agree.

Third, the packaged universes of truncated classifiers have a local structure-identity theorem: package identity is equivalent to `TypeEquiv` of the retained carriers. This is the closest checked example of univalence propagating through an evidence field.

Fourth, ordinary isomorphism evidence maps one way into native omega-equivalence evidence, and finite `IsNCat` evidence yields the expected object-truncation bound. These maps organize nearby notions without asserting the missing reverse object-identity comparison.

Formal status — checked. Evidence `LOGIC-TRUNCATION-EVIDENCE-PROP`, `EQUIV-EVIDENCE-PROP`, `UNIV-TRUNCATED`, `EQUIV-ORDINARY-ISO-LIFT`, and `CAT-DIMENSION`. These are local evidence-property, package-univalence, comparison, and truncation theorems. Their conjunction is not a generic structure identity principle.

15.6 A Plausible Native Structure-Identity Interface

The ordinary (P, H) schema suppresses the higher cells of preservation evidence. A directed native version should expose them. One plausible architecture begins with:

- a native carrier category K ;
- a directed family $S : K \rightarrow \mathbf{Cat}$ of structures;
- for each base arrow $f : x \rightarrow y$ and structures $\alpha : S(x)$, $\beta : S(y)$, a category $H_f(\alpha, \beta)$ of structure-preserving lifts;
- identity, composition, and higher action for those lifts;
- a selected classifier `StructuredEquiv` $((x, \alpha), (y, \beta))$.

The total category of structures is Sigma-shaped, but a general $H_f(\alpha, \beta)$ may carry more information than the canonical transport arrow of a bare directed family. A *standardness* condition must say exactly when that extra information is property-like and when it is genuinely higher structure.

The prospective comparison is

$$\text{idtoStructuredEquiv} : ((x, \alpha) = (y, \beta)) \longrightarrow \text{StructuredEquiv}((x, \alpha), (y, \beta)).$$

A native SIP would prove this to be an equivalence under qualified base univalence and standardness hypotheses, while retaining its off-diagonal and next-hom action. The target might use ordinary isomorphism, adjoint equivalence, or native omega-equivalence; the theorem cannot be stated honestly until that choice is part of the signature.

Formal status — research boundary. Evidence `NATIVE-STRUCTURE-IDENTITY`. The active `Catd`, `Sigma_cat`, `Hom_catd`, evidence-property, and equivalence interfaces are plausible ingredients, but there is no generic structure signature, structured-equivalence classifier, or identity theorem. Side task `FTTX-S9` records this prospective owner.

15.7 Weak Equivalences And The Universal Property Of Completion

Return to ordinary precategories. A functor

$$I : \mathcal{A} \longrightarrow \hat{\mathcal{A}}$$

is a *weak equivalence* when it is fully faithful and essentially surjective, where essential surjectivity is merely inhabited rather than split by a chosen inverse on objects. A Rezk completion of $\mathcal{A}$ consists of such an I with $\hat{\mathcal{A}}$ an ordinary univalent category.

The construction is characterized by what saturated targets see. For every ordinary univalent category $\mathcal{C}$, precomposition induces

$$I^* : \mathcal{C}^{\hat{\mathcal{A}}} \longrightarrow \mathcal{C}^{\mathcal{A}}, \quad G \longmapsto G \circ I,$$

and I^* is an isomorphism of ordinary precategories. Equivalently, every functor $\mathcal{A} \rightarrow \mathcal{C}$ extends essentially uniquely across I , and every natural transformation between extensions is determined by its restriction.

The word *universal* belongs here, not merely in the statement that $\hat{\mathcal{A}}$ is saturated. Any two completions with this mapping property are uniquely equivalent in the appropriate functor category.

15.8 Why Saturated Targets See Weak Equivalences As Equivalences

The proof of the mapping property is a lesson in constructive uniqueness. If I is essentially surjective, a natural transformation out of $\hat{\mathcal{A}}$ is determined by its components on the image of I ; this makes precomposition faithful. Fullness of I , together with essential surjectivity, reconstructs the missing components and proves naturality, so precomposition is fully faithful.

To extend a functor on objects, one knows only that each object of $\hat{\mathcal{A}}$ is isomorphic to something in the image. Choosing an arbitrary representative would require choice. Instead one describes the candidate image and its comparison data as a contractible type. The crucial step uses univalence of $\mathcal{C}$: uniqueness up to unique isomorphism becomes uniqueness by identity, which is enough to define a function. This proves essential surjectivity of I^* and hence the mapping property.

In short, saturated targets turn weak equivalences into equivalences of functor categories.

The converse detects saturation. An ordinary precategory $\mathcal{C}$ is univalent exactly when every weak equivalence $H : \mathcal{A} \rightarrow \mathcal{B}$ makes

$$H^* : \mathcal{C}^{\mathcal{B}} \longrightarrow \mathcal{C}^{\mathcal{A}}$$

an isomorphism. For the reverse implication, apply the assumption to a Rezk completion $I : \mathcal{C} \rightarrow \hat{\mathcal{C}}$. Precomposition then constructs an inverse to I , making $\mathcal{C}$ isomorphic to the univalent category $\hat{\mathcal{C}}$ and hence univalent itself.

This characterization also explains why a weak equivalence need not be an isomorphism between unsaturated presentations. It becomes invertible to every target precisely when object isomorphism in that target can be absorbed as identity.

15.9 The Yoneda-Image Completion

The first construction uses Chapter 13. Let

$$\mathbf{PSh}(\mathcal{A}) := \mathbf{Set}^{\mathcal{A}^{\text{op}}}.$$

Define $\hat{\mathcal{A}}$ to be the full subcategory whose objects are presheaves P for which there merely exists an $a : \mathcal{A}$ and an isomorphism

$$y(a) \cong P.$$

The ambient presheaf category is univalent, and the condition of being merely representable is a proposition, so the full subcategory is univalent. The Yoneda embedding

$$y : \mathcal{A} \longrightarrow \hat{\mathcal{A}}$$

is fully faithful by Yoneda and essentially surjective by the definition of the image. It is therefore a Rezk completion.

This proof is short because representability has already packaged the necessary universal coordinates. Its cost is universe size: the presheaf category may live in a larger universe than $\mathcal{A}$.

15.10 The Higher-Inductive Completion

A second construction stays closer to the original universe by freely adding the missing object identities. Its object type $\text{Obj}(\hat{\mathcal{A}})$ is generated by:

- $i(a)$ for every object $a : \mathcal{A}$;
- a path $j(e) : i(a) = i(b)$ for every isomorphism $e : a \cong b$;
- coherences $j(\text{id}_a) = \text{refl}_{i(a)}$ and $j(g \circ f) = j(f) \cdot j(g)$;
- 1-truncation, so parallel 2-paths agree.

The hom family is then defined by double induction on the new object type, starting with

$$\text{Hom}_{\hat{\mathcal{A}}}(i(a), i(b)) := \text{Hom}_{\mathcal{A}}(a, b).$$

Transport along $j(e)$ is postcomposition or precomposition by e and its inverse. The identity and composition laws of $\mathcal{A}$ discharge the HIT coherences, after which identities and composition on the new hom family are defined by induction.

To show saturation, use encode-decode:

$$\begin{aligned} \text{encode}_{x,y} : (x = y) &\longrightarrow (x \cong y), & \text{encode} &= \text{idtoiso}, \\ \text{decode}_{x,y} : (x \cong y) &\longrightarrow (x = y), & \text{decode}(e) &= j(e) \text{ on generators.} \end{aligned}$$

Induction over the HIT and its paths proves both composites. The identity and composition constructors are precisely the cases needed for the code family of isomorphisms to respect reflexivity and path concatenation. Finally, $I(a) := i(a)$ is fully faithful by construction and essentially surjective by HIT induction.

Formal status — mathematical development. Evidence `UCAT-REZK-COMPLETION`. The ordinary theorem states that every precategory has a univalent Rezk completion, constructed either as the Yoneda image or by the 1-truncated HIT above, and that weak equivalences into univalent targets have the stated functor-category mapping property.

15.11 The Encode-Decode Analogy With WalkingEnd

The HIT proof deliberately echoes Chapter 8, but the two constructions solve different problems.

Feature	WalkingEnd	Rezk-completion HIT
generators	one object and one directed loop	old objects and paths for old isomorphisms
code	natural powers of the loop	isomorphisms between completed objects
decode	contextual directed normalization	a path constructor $j(e)$
invertibility	the loop is checked noninvertible	every added generator is a path and hence invertible
purpose	compute the free directed endomorphism monoid	saturate object identity

Both proofs define a code family, construct encode and decode, and calculate two composites by the relevant eliminator. That is an analogy of proof architecture. WalkingEnd is **not** a Rezk completion: its defining loop is not an isomorphism, its hom corresponds to natural numbers, and its universal problem is directed generation rather than saturation.

Formal status — checked comparison. Evidence `WE-LOOP-NONINVERTIBLE` verifies the decisive negative fact on the WalkingEnd side. The Rezk construction in the other column remains the ordinary mathematical development above.

15.12 Specification Of A Native Rezk Completion

A native completion cannot be obtained by replacing the word “isomorphism” with “equivalence” and leaving the rest implicit. A prospective interface must specify:

1. a classifier $\mathbf{Eqv}_C(x, y)$ of the selected categorical sameness;
2. a coherent map $(x = y) \rightarrow \mathbf{Eqv}_C(x, y)$;
3. the saturation predicate asserting that this map is an equivalence;
4. a category $\mathbf{Rezk}(C)$ and unit functor $\eta_C : C \rightarrow \mathbf{Rezk}(C)$;

5. local full faithfulness at every retained hom dimension and essential surjectivity relative to **Eqv**;
6. for every saturated D , an equivalence between the iterable functor categories out of $\mathbf{Rezk}(C)$ and out of C ;
7. naturality of that mapping equivalence in D , together with its transformation and higher-cell action.

Ordinary isomorphism, adjoint equivalence, and native omega-equivalence are different candidates for **Eqv**. Chapter 10 supplies maps among some of them, but no general theorem chooses one as object identity in every native category. The completion's higher universal property must therefore be designed together with its saturation predicate.

Formal status — research boundary. Evidence `NATIVE-REZK-COMPLETION`. No native saturation predicate, completion object, unit weak equivalence, or iterable mapping property is active. Constructing a carrier without these laws would not discharge `FTTX-S9`.

15.13 Identity Principles And Universal Constructions

The two halves of the chapter meet in a useful division of labor. A structure identity theorem proves that a well-behaved construction *preserves* saturation: structured objects over a saturated carrier form another saturated category. Rezk completion *creates* saturation from a presentation that lacks it. Yoneda supplies one completion because representables turn objects into universal coordinates; the HIT supplies another because encode-decode computes the freely generated identity structure.

Chapters 16 and 17 now return to universal constructions internal to a fixed categorical world. Their limits, colimits, and joins do not require a native Rezk completion to be stated, but the distinction between presentation and saturated identity will remain visible whenever uniqueness is upgraded from equivalence to equality.

16. Weighted Universal Constructions

A limit is often introduced by drawing a cone and asking for a universal vertex. That picture is indispensable, but it hides the operation that makes the vertex universal: for every probe object, the category of cones must be represented by a hom. Weighted limits expose this operation directly.

The resulting account is not a catalogue of products, equalizers, ends, and Kan extensions. It is one theorem with several specializations:

$$\text{weight and diagram} \longrightarrow \text{cone classifier} \longrightarrow \text{representing comparison}.$$

The same comparison calculus that eliminated cuts in Chapter 9 then supplies the universal beta and eta laws. Adjunction mates transport the comparison, and therefore every right adjoint preserves every selected weighted limit for which the comparison has been supplied.

Our mathematical conventions follow the Cat-enriched viewpoint of Kelly. The active artifact deliberately implements a narrower computational interface: tensor and its two residuals are symbolic objects with checked vertical beta/eta operations, not constructions from general ends and coends. This distinction lets the theorem compute without claiming semantic infrastructure that has not yet been built.

16.1 Weights Are Parameterized Shapes

Let

$$F : J \longrightarrow B$$

be a diagram. A parameterized covariant weight has the form

$$W : J' \rightsquigarrow J, \quad W : (J')^{\text{op}} \times J \longrightarrow \mathbf{Cat}.$$

For each $j' : J'$, the partial functor $W(j', -)$ is a Cat-valued weight on J . The first endpoint is contravariant, so an arrow $j'_0 \rightarrow j'_1$ acts from the weight at j'_1 to the weight at j'_0 . This is exactly the variance needed for the resulting representing objects to assemble into a functor

$$L : J' \longrightarrow B.$$

The familiar unparameterized definition is the special case $J' = \mathbf{1}$. Retaining J' is important: choosing one limit object for each parameter is not enough. The comparison below must also make those choices functorial in the parameter and iterable at the next hom level.

In the ordinary set-enriched case, a weight is a functor $W : J \rightarrow \mathbf{Set}$. The set $W(j)$ describes how many copies, positions, or generalized inputs the object Fj contributes. In the Cat-valued case those positions themselves have arrows, so a weighted cone carries coherence along both J and the internal categories $W(j', j)$.

16.2 The Profunctor Of Weighted Cones

For a probe object $b : B$ and a parameter $j' : J'$, the expected category of weighted cones is

$$\text{Cone}_W(b, F; j') \simeq [J, \text{Cat}](W(j', -), \text{Hom}_B(b, F-)).$$

An object of this category is a natural family of arrows from b into the diagram, indexed by the weight. Its arrows are the corresponding higher transformations. Varying b acts by precomposition, while varying j' acts through the contravariant endpoint of W . Thus the cone construction has the profunctorial type

$$\text{Cone}_W(F) : B \rightsquigarrow J'.$$

The active definition does not unfold the functor-category expression above. It uses the covariant profunctor implication:

$$\text{Cone}_W(F) := \text{ProfImpley}_{\text{cov}}(\text{Hom}_B(-, F-), W).$$

The literal owner is `WeightedCone_prof`, defined through `Prof_imply_cov`. This is a useful separation. The mathematical formula says what weighted cones mean in a semantic model with the necessary ends; the implication is the stable computational owner used by the present theory.

16.3 Tensor And The Two Residuals

Suppose

$$P : A \rightsquigarrow B, \quad Q : B \rightsquigarrow X, \quad O : A \rightsquigarrow X.$$

Tensor makes the middle endpoint into a cut:

$$P \otimes_B Q : A \rightsquigarrow X.$$

There are two ways to solve a mapping problem against this composite. Holding Q fixed gives a right residual with type $A \rightsquigarrow B$; holding P fixed gives a left residual with type $B \rightsquigarrow X$. The active interfaces are characterized by inverse operations

$$\begin{aligned} \text{ProfMap}(P, \text{ProfImpley}_{\text{cov}}(O, Q)) &\simeq \text{ProfMap}(P \otimes_B Q, O), \\ \text{ProfMap}(Q, \text{ProfImpley}_{\text{con}}(P, O)) &\simeq \text{ProfMap}(P \otimes_B Q, O). \end{aligned}$$

Evaluation introduces the tensor cut; lambda abstraction removes it. In both orientations the selected composites reduce:

$$\begin{aligned} \lambda(\text{eval}(t)) &\rightsquigarrow t, \\ \text{eval}(\lambda(u)) &\rightsquigarrow u. \end{aligned}$$

These are universal-property beta and eta laws in the same sense as the upper-star and `tapp1` reductions of Chapter 9. They choose a stable owner for associativity rather than globally reassociating arbitrary composites.

Formal status — checked. Evidence `PROF-CLOSED-CALCULUS`. `Prof_imply_cov` and `Prof_imply_con` are the two opaque residual objects. `Prof_eval_cov_map`, `Prof_lambda_cov_map`, `Prof_eval_con_map`, and `Prof_lambda_con_map` expose the checked fixed-endpoint beta/eta calculus. This status does not assert an end formula for either residual.

The weighted cone is now forced rather than guessed. For every $P : B \rightsquigarrow J'$,

$$\text{ProfMap}(P, \text{Cone}_W(F)) \simeq \text{ProfMap}(P \otimes_{J'} W, \text{Hom}_B(-, F-)).$$

A map on the right is precisely a P -shaped family of W -weighted cone data. The residual packages all such maps into one profunctor. In this form, a weight is not an annotation on a limit symbol: it is the profunctor cut that the cone classifier abstracts.

The tensor itself remains an opaque fixed-middle composite. Its current functorial action and shaped-element constructor are checked, but a semantic coend, associator, unitors as equivalences, and the coherence of a full profunctor bicategory are not consequences of those interfaces.

Formal status — checked interface and research boundary. Evidence `PROF-TENSOR` covers the selected tensor object, outer-endpoint reindexing, vertical bifunctoriality, and shaped tensor elements. Evidence `PROF-GENERAL-COEND` records what is missing. The displayed residual mapping laws are checked only through the fixed-endpoint eval/lambda operations above.

16.4 A Weighted Limit Is A Representation

A W -weighted limit of F is a functor

$$L : J' \longrightarrow B$$

together with a representation of the cone profunctor:

$$\Phi : \text{Cone}_W(F) \simeq \text{Hom}_B(-, L-).$$

At a pair (b, j') , this says

$$\text{Cone}_W(b, F; j') \simeq \text{Hom}_B(b, Lj').$$

The direction of the hom is worth checking. A limit receives a cone *from* the probe b , so maps from b to the limit classify cones from b . A colimit will reverse this direction in Chapter 17.

There are two active representation classifiers. The ordinary classifier `IsWeightedLimit_cov_iso` asks for isomorphism evidence in the profunctor category. The computational classifier `IsWeightedLimit_cov_comp` asks for a `ProfComparison`. The latter retains selected forward and inverse maps whose beta and eta laws compute on every incoming profunctor map.

Formal status — checked. Evidence `WEIGHTED-LIMIT-REPRESENTABILITY`. `WeightedCone_prof` constructs the cone profunctor, while `IsWeightedLimit_cov_iso` and `IsWeightedLimit_cov_comp` give the ordinary and computational representation interfaces. The checked claim is the interface and its reductions, not existence of a weighted limit for every diagram.

This is the universal-property row of the rule schema in [Appendix G.4](#). Formation fixes F , W , and the proposed representing functor L ; introduction supplies a comparison certificate; push and pull are eliminations; their inverse reductions are beta and eta; and reindexing in the probe is the action clause. Existence and univalent uniqueness are additional principles, not hidden fields of the checked classifier.

This formulation also separates *being* a limit from *choosing* one. A theorem may accept a comparison for a particular F , W , and L without postulating a global limit operator. In a univalent specialization, representability can later make the choice unique in the appropriate sense; the preservation theorem below does not need that stronger uniqueness package.

16.5 Universal Introduction And Elimination

Let

$$M : I \longrightarrow B$$

be a shaped family of probe objects, and let

$$R : I \rightsquigarrow J'$$

index a family of test data. Reindexing Φ along M gives inverse operations

$$\begin{aligned} \text{push} & : \text{ProfMap}(R, \text{Cone}_W(M, F)) \longrightarrow \text{ProfMap}(R, \text{Hom}_B(M, L)), \\ \text{pull} & : \text{ProfMap}(R, \text{Hom}_B(M, L)) \longrightarrow \text{ProfMap}(R, \text{Cone}_W(M, F)). \end{aligned}$$

Here

$$\text{Cone}_W(M, F) := \text{ProfImpley}_{\text{cov}}(\text{Hom}_B(M-, F-), W).$$

The operation `push` eliminates a supplied cone through the universal comparison and obtains its mediating map into L . The operation `pull` introduces a cone by composing a map into L with the universal cone. Their cuts reduce in both directions:

$$\begin{aligned}\text{pull}(\text{push}(r)) &\rightsquigarrow r, \\ \text{push}(\text{pull}(s)) &\rightsquigarrow s.\end{aligned}$$

Formal status — checked. Evidence `PROF-COMPARISON-BETA-ETA`. The weighted operations `weighted_limit_cov_push` and `weighted_limit_cov_pull` are typed specializations of the generic `prof_comparison_push` and `prof_comparison_pull` owners. The beta/eta rules belong to the generic comparison, so no new cancellation rule is attached to each kind of limit.

The role of R is easy to underestimate. Taking a single element would test only one cone. Allowing an arbitrary incoming profunctor map says that the universal operation is stable under families, end-point action, and the next categorical layer retained by the fixed-endpoint profunctor category. This is the functorial type-theoretic replacement for an unstructured bijection of sets of cones.

16.6 Conical Limits As The Terminal-Weight Specialization

Set $J' = \mathbf{1}$ and take the terminal weight

$$\mathbf{1}_J : \mathbf{1} \rightsquigarrow J.$$

Its only fibre is the terminal category. In the usual semantic interpretation,

$$\begin{aligned}\text{Cone}_{\mathbf{1}_J}(b, F; *) &\simeq [J, \mathbf{Cat}](\mathbf{1}, \text{Hom}_B(b, F-)) \\ &\simeq \text{Cone}(b, F).\end{aligned}$$

A functor $\ell : \mathbf{1} \rightarrow B$ selects a vertex. The weighted representation becomes the familiar conical universal property

$$\text{Cone}(b, F) \simeq \text{Hom}_B(b, \ell).$$

Products, terminal objects, pullbacks, and equalizers arise by choosing their usual indexing categories J . The weighted formulation does not require a new universal-property mechanism for each of them.

There are now also direct computational interfaces for selected binary products, a terminal object, and pullback base change. They are not defined by the opaque weighted comparison. The direct interfaces choose the runtime pairing, terminal-arrow, or slice-adjunction calculus. For binary products, an assumption-explicit adapter accepts a strict weighted comparison and records paths relating its projections to the triangular ones. It does not construct that stronger comparison automatically from product beta and eta.

Chapter 30 follows the direct route through $\Sigma_u \dashv u^* \dashv \Pi_u$. The second adjunction and its dependent product are additional selected structure; they do not follow merely by substituting a terminal weight here.

At the active-interface level, the substitution is exact:

$$\text{IsWeightedLimit}(F, \text{TerminalProf}(\mathbf{1}, J), \ell)$$

is a well-formed instance of `IsWeightedLimit_cov_comp`. This variance fact is permanent regression evidence. What is not checked is the semantic identification of the opaque profunctor implication with the displayed functor category of ordinary cones.

Formal status — formal consequence. Evidence `WEIGHTED-LIMIT-SPECIALIZATIONS`. `Terminal_prof`, `IsWeightedLimit_cov_comp`, and the focused classifier diagnostics establish the terminal-weight instance and its preservation corollary. Calling its fibres the usual cone categories is mathematical development contingent on the end semantics described below. Evidence `TRIANGULAR-BINARY-PRODUCTS`, `TERMINAL-OBJECT-COMPUTATION`, and `PULLBACK-SLICE-BASE-CHANGE` separately support the direct computational interfaces; they do not turn the terminal-weight comparison into an automatic witness.

16.7 Right Kan Extensions As Conjoint-Weighted Limits

Let

$$K : J \longrightarrow J'$$

and retain the diagram $F : J \rightarrow B$. The conjoint of K is the profunctor

$$K^* : J' \rightsquigarrow J, \quad K^*(j', j) = \text{Hom}_{J'}(j', Kj).$$

It has exactly the variance of a limit weight. Define a selected right Kan-extension comparison along K to be the weighted-limit comparison

$$\text{IsWeightedLimit}(F, K^*, \text{Ran}_K F).$$

When the residual has its expected end semantics, the fibrewise formula is

$$\text{Hom}_B(b, (\text{Ran}_K F)(j')) \simeq \text{Nat}(\text{Hom}_{J'}(j', K-), \text{Hom}_B(b, F-)).$$

This is the standard pointwise right Kan-extension formula. It says that maps into the value at j' are cones whose shape is the representable weight out of j' .

The use of a conjoint is not a mnemonic guess. `Conjoint_prof K` has type $J' \rightsquigarrow J$, so the expression

$$\text{IsWeightedLimit}(F, \text{Conjoint}(K), R)$$

is a well-formed active classifier for every $R : J' \rightarrow B$. The focused variance audit checks precisely this substitution.

The following two claims must nevertheless remain separate:

1. **Interface specialization:** conjoint weight yields an instance of the selected weighted comparison. This is a formal consequence of the active types.
2. **Semantic identification:** that instance agrees with the ordinary natural-transformation or end definition of pointwise right Kan extension. This requires a semantic end owner and coherence for the relevant Cat-valued transformation category.

Formal status — mathematical development. Evidence `WEIGHTED-END-KAN-SEMANTICS`. The standard conical and pointwise Kan-extension formulas are mathematically part of the weighted theory. `Prof_imply_cov` currently packages their computational residual but does not unfold to a general end.

16.8 Adjunction Mates Transport Representables

Let

$$S : A \longrightarrow B, \quad R : B \longrightarrow A, \quad S \dashv R.$$

For arbitrary shaped functors $M : I \rightarrow A$ and $D : J \rightarrow B$, the adjunction supplies the representable comparison

$$\mathrm{Hom}_B(SM, D) \simeq \mathrm{Hom}_A(M, RD).$$

It is natural in both endpoints and therefore lives as a `ProfComparison`, not merely as a family of unrelated pointwise equivalences. Passing this comparison through the fixed-weight implication transports whole cone profunctors.

Formal status — checked. Evidence `ADJ-HOM-PROF-COMPARISON`. `Adjunction_hom_prof_comparison_along` is the reindexed mate comparison. Its inverse directions and their cancellation are inherited from the generic profunctor-comparison calculus.

In classical category theory, a mate is often introduced by a formula using the unit and counit. Computationally, the mate is better understood as a change of representable coordinates. The triangle reductions of Chapter 12 are exactly the beta/eta laws ensuring that moving across the adjunction and back removes the cut.

16.9 The Preservation Theorem

Assume that

$$\ell : J' \longrightarrow B$$

is supplied with a computational comparison certifying it as the W -weighted limit of $D : J \rightarrow B$. We prove that

$$R\ell : J' \longrightarrow A$$

is the W -weighted limit of $RD : J \rightarrow A$.

Test the proposed representation at an arbitrary $M : I \rightarrow A$. There are three comparisons:

$$\begin{aligned} \text{Cone}_W(M, RD) &\simeq \text{Cone}_W(SM, D) && \text{by the inverse mate under the residual,} \\ &\simeq \text{Hom}_B(SM, \ell) && \text{by the supplied limit comparison,} \\ &\simeq \text{Hom}_A(M, R\ell) && \text{by the mate at the candidate limit.} \end{aligned}$$

The middle comparison is the original representation reindexed along S . The first and third are the same hom adjunction used in opposite directions and at different diagrams. Composing them yields

$$\text{Cone}_W(RD) \simeq \text{Hom}_A(-, R\ell-),$$

which is the required weighted-limit comparison.

The implementation mirrors this proof without inserting a special rewrite that says “right adjoints preserve limits.” Its three factors are:

1. `right_adjoint_weighted_limit_comp_step1`, the inverse mate mapped through the fixed-weight implication;
2. `right_adjoint_weighted_limit_comp_step2`, the supplied comparison reindexed along the left adjoint;
3. `right_adjoint_weighted_limit_comp_step3`, the mate at ℓ .

`right_adjoint_preserves_weighted_limit_cov_comp` composes these certificates. Since the result is again a `ProfComparison`, all universal push/pull beta and eta behavior survives.

Theorem 16.1 — Right adjoints preserve selected weighted limits. Given a computational W -weighted-limit comparison for D and an adjunction $S \dashv R$, the active construction returns a computational W -weighted-limit comparison for RD represented by $R\ell$.

Formal status — checked. Evidence `WEIGHTED-LIMIT-PRESERVATION`. The theorem is conditional on a supplied comparison. It neither asserts that every weighted limit exists nor that a right adjoint creates one.

This proof is the promised continuation of cut elimination. The first mate changes coordinates, the given universal comparison eliminates the central cone cut, and the final mate changes coordinates back. Associativity is controlled by composition of comparison certificates; it is not delegated to a global reassociation rule.

16.10 Ordinary-Limit And Kan-Extension Corollaries

The theorem is uniform in W , so the two audited specializations immediately give interface-level corollaries.

For the terminal weight:

If ℓ carries the selected conical-limit comparison for D , then $R\ell$ carries the corresponding comparison for RD .

For the conjoint of $K : J \rightarrow J'$:

If $\text{Ran}_K D$ carries the selected right-Kan comparison, then $R \text{Ran}_K D$ carries the corresponding comparison for $\text{Ran}_K(RD)$.

These are not new preservation algorithms. They are the same `right_adjoint_preserves_weighted_limit_cov_comp` term with $W = \text{TerminalProf}(\mathbf{1}, J)$ or $W = \text{Conjoint}(K)$. The formal consequence is therefore stronger than a slogan and weaker than the unimplemented semantic end theorem: the classifier, transported comparison, and beta/eta interface are active; the identification with every conventional presentation of limits or Kan extensions is not.

16.11 Ends, Coends, And Dependent Universal Constructions

A semantic completion of this chapter would construct the residual by an end, for example

$$\text{Cone}_W(b, F; j') \simeq \int_{j:J} [W(j', j), \text{Hom}_B(b, Fj)],$$

and tensor by a coend

$$(P \otimes_B Q)(a, x) \simeq \int^{b:B} P(a, b) \times Q(b, x).$$

Such constructions must provide more than object formulas. They need introduction and elimination principles, action on base arrows and transfor, beta/eta or comparison laws, associativity and unit coherence, and a validation route through the shaped co-Yoneda theorem. Until those owners exist, the formulas are semantic specifications for the opaque interfaces.

Dependent category theory asks for a further chain. For a base functor $f : X \rightarrow Y$, one expects suitable change-of-base functors and adjunctions

$$\Sigma_f \dashv f^* \dashv \Pi_f,$$

with Beck--Chevalley and higher naturality conditions. The current Sigma and Pi categories of directed families are important footholds, but no general dependent adjunction package connects them to arbitrary base change. A pointwise object formula would be insufficient: the construction must retain base-arrow action, off-diagonal transfor action, and iteration into the next hom.

Formal status — research boundary. Evidence `DEPENDENT-ADJUNCTIONS`. General end/coend owners, pointwise Kan packages, and the dependent adjunction chain remain separate formal projects. The active weighted comparison is the design constraint those projects should refine, not a claim that they are already present.

Chapter 17 now applies the safest of all extensions: opposite duality. It turns the checked limit theorem into a colimit theorem, after which the join shows how terminal-weight cross-arrow data can itself be internalized as a directed categorical shape.

17. Weighted Colimits, Duality, And Join

The preceding chapter classified cones by homs *into* a representing object. Colimits classify cocones by homs *out of* one. Every variance reverses: companions replace conjoints, left adjoints replace right adjoints, and the universal arrows point away from the colimit.

Opposite duality makes this reversal a proof method rather than an invitation to duplicate the theory. The active weighted-colimit classifier is defined through the weighted-limit classifier in opposite categories. Its preservation theorem is therefore the right-adjoint theorem of Chapter 16 applied once to the opposite adjunction.

The second half of the chapter gives this duality a directed geometry. The join $A \star B$ contains a left part, a right part, and an internally natural family of arrows directed from left to right. Its recursor has the same input shape as the collage of a terminal profunctor. That observation connects joins to weighted cone data while leaving the stronger collage semantics explicitly open.

17.1 Weighted Cocones

Let

$$F : J \longrightarrow B$$

be a diagram and let

$$W : J \rightsquigarrow J'$$

be a contravariant weight, meaning a $\mathbf{Cat}$ -valued profunctor

$$W : J^{\mathrm{op}} \times J' \longrightarrow \mathbf{Cat}.$$

For fixed $j' : J'$, the weight $W(-, j')$ is contravariant in J . At a candidate target $b : B$, its expected cocone category is

$$\mathrm{Cocone}_W(F; j', b) \simeq [J^{\mathrm{op}}, \mathbf{Cat}](W(-, j'), \mathrm{Hom}_B(F-, b)).$$

A W -weighted colimit is a functor

$$C : J' \longrightarrow B$$

with a representation

$$\mathrm{Cocone}_W(F; j', b) \simeq \mathrm{Hom}_B(Cj', b)$$

natural in j' and b . Compare this with the limit equation

$$\mathrm{Cone}_W(b, F; j') \simeq \mathrm{Hom}_B(b, Lj').$$

The same words “represented by” occur in both formulas, but the representable hom has changed side.

17.2 One Universal Owner Through Opposite Categories

For

$$F : J \rightarrow B, \quad W : J \rightsquigarrow J', \quad C : J' \rightarrow B,$$

the active definition is

$$\text{IsWeightedColimit}_B(F, W, C) := \text{IsWeightedLimit}_{B^{\text{op}}}(F^{\text{op}}, W^{\text{op}}, C^{\text{op}}).$$

Reversing the profunctor endpoints gives

$$W^{\text{op}} : (J')^{\text{op}} \rightsquigarrow J^{\text{op}},$$

which is exactly the variance expected by the limit classifier in the opposite categories. The representing equation there is

$$\text{Hom}_{B^{\text{op}}}(b, Cj') = \text{Hom}_B(Cj', b),$$

so it recovers the desired cocone orientation.

The owner `WeightedColimit_con` is transparent to `IsWeightedLimit_cov_comp` after applying `Op_func` and `Op_prof`. The conversion operations between a limit witness and the corresponding opposite colimit witness are identity-like wrappers after double-opposite and product-swap computation. Colimit beta/eta therefore comes from the same profunctor comparison used for limits.

Formal status — checked. Evidence `OP-DUALITY`. The involutive category, functor, transfor, profunctor, and adjunction operations justify the variance reversal. No independent colimit cancellation calculus is introduced.

17.3 Conical Colimits And Left Kan Extensions

The terminal-weight specialization now uses the opposite orientation:

$$\mathbf{1}^J : J \rightsquigarrow \mathbf{1}.$$

For a vertex $c : \mathbf{1} \rightarrow B$, the active classifier

$$\text{IsWeightedColimit}(F, \text{TerminalProf}(J, \mathbf{1}), c)$$

is well formed. Under the standard semantic interpretation its fibres become ordinary cocone categories, and the representation reads

$$\text{Cocone}(F, b) \simeq \text{Hom}_B(c, b).$$

Thus initial objects, coproducts, pushouts, and coequalizers are terminal weight specializations on their usual indexing categories.

For a functor

$$K : J \longrightarrow J',$$

the companion has the required colimit variance:

$$K_* : J \rightsquigarrow J', \quad K_*(j, j') = \text{Hom}_{J'}(Kj, j').$$

A selected left Kan-extension comparison is therefore

$$\text{IsWeightedColimit}(F, K_*, \text{Lan}_K F).$$

With the expected coend or cocone semantics, it gives the pointwise formula

$$\text{Hom}_B((\text{Lan}_K F)(j'), b) \simeq \text{Nat}(\text{Hom}_{J'}(K-, j'), \text{Hom}_B(F-, b)).$$

The focused variance audit checks both active substitutions: `Terminal_prof J Terminal_cat` and `Companion_prof K` inhabit the weight slot of `WeightedColimit_con`. As in Chapter 16, this does not by itself identify the opaque opposite residual with a semantic category of cocones.

Formal status — formal consequence. Evidence `WEIGHTED-COLIMIT-SPECIALIZATIONS`. The terminal-weight and companion classifiers, together with their preservation instances, follow from the active types and the opposite definition. Their standard conical and pointwise-left-Kan interpretations remain mathematical development under `WEIGHTED-END-KAN-SEMANTICS`.

The companion/conjoint distinction is now visible in one table:

Construction along $K : J \rightarrow J'$	Weight	Fibre
right Kan extension	conjoint $K^* : J' \rightsquigarrow J$	$\text{Hom}_{J'}(j', Kj)$
left Kan extension	companion $K_* : J \rightsquigarrow J'$	$\text{Hom}_{J'}(Kj, j')$

Suppressing the profunctor endpoints would make these formulas look deceptively interchangeable. The endpoints are the type-level variance audit.

17.4 Left Adjoints Preserve Weighted Colimits

Let

$$S : A \longrightarrow B, \quad R : B \longrightarrow A, \quad S \dashv R,$$

and suppose

$$C : J' \longrightarrow A$$

carries a comparison certifying it as the W -weighted colimit of $F : J \rightarrow A$. Passing to opposites turns the data into:

1. a weighted-limit comparison for C^{op} and F^{op} in A^{op} ;
2. the opposite adjunction $R^{\text{op}} \dashv S^{\text{op}}$;
3. a right adjoint $S^{\text{op}} : A^{\text{op}} \rightarrow B^{\text{op}}$.

Theorem 16.1 therefore supplies a weighted-limit comparison represented by

$$S^{\text{op}} C^{\text{op}} = (SC)^{\text{op}}.$$

Turning the result back around gives a W -weighted-colimit comparison for SF represented by SC .

At the level of cocones, the three-comparison proof is the familiar chain

$$\begin{aligned} \text{Cocone}_W(SF; b) &\simeq \text{Cocone}_W(F; Rb), \\ &\simeq \text{Hom}_A(C, Rb), \\ &\simeq \text{Hom}_B(SC, b). \end{aligned}$$

The first and last steps are adjunction mates, and the middle step is the given colimit representation. The implementation obtains this chain by calling the right-adjoint theorem on `Op_adjunction`; it does not repeat the three steps under new primitive names.

Theorem 17.1 — Left adjoints preserve selected weighted colimits. Given a computational W -weighted-colimit comparison for F and an adjunction $S \dashv R$, the active construction returns a computational comparison for SF represented by SC .

Formal status — checked. Evidence `WEIGHTED-COLIMIT-PRESERVATION`. The owner `left_adjoint_preserves_weighted_colimit_con` applies `right_adjoint_preserves_weighted_limit_cov_comp` to the opposite adjunction. The theorem is conditional on the supplied colimit comparison.

Consequently, the terminal and companion specializations yield the familiar interface-level corollaries: left adjoints preserve selected conical colimits and selected left Kan extensions. No new proof is hidden behind either phrase.

17.5 A Variance Ledger

The dual theorem can be remembered without reversing formulas in one's head:

Feature	Weighted limit	Weighted colimit
weight	$J' \rightsquigarrow J$	$J \rightsquigarrow J'$
universal data	arrows $b \rightarrow Fj$	arrows $Fj \rightarrow b$

represented hom	$\text{Hom}_B(b, Lj')$	$\text{Hom}_B(Cj', b)$
Kan weight	conjoint	companion
preserving adjoint	right	left
proof	direct three-comparison chain	the same chain in opposites

This ledger is more than notation. It identifies which endpoint acts by upper-star precomposition and which acts by lower-star postcomposition. The opposite construction exchanges those actions while preserving their computational owners.

17.6 The Directed Join Signature

For native categories A and B , the primitive join is a category

$$A \star B$$

with inclusion functors

$$\iota_A : A \longrightarrow A \star B, \quad \iota_B : B \longrightarrow A \star B.$$

Its characteristic constructor is not an equality between the two parts. It is an internally natural family of arrows from the left part to the right part:

$$\chi : \mathbf{1}_{A,B} \Longrightarrow \text{Hom}_{A \star B}(\iota_A -, \iota_B -),$$

where

$$\mathbf{1}_{A,B} : A \rightsquigarrow B$$

is the terminal profunctor. At objects, χ supplies

$$\chi_{a,b} : \iota_A(a) \longrightarrow \iota_B(b).$$

Because χ is a profunctor cell, naturality in both a and b is part of one internal datum. It is not an externally quantified family followed by a separately asserted equation. For shaped functors $a : I \rightarrow A$ and $b : I \rightarrow B$, `join_cross_hom` evaluates the same cell to the corresponding shaped cross arrow and retains its higher action.

There is no reverse constructor $\iota_B(b) \rightarrow \iota_A(a)$. The join is directed even when A and B individually happen to be groupoidal.

17.7 Recursion And Its Three Beta Observations

To define a functor

$$H : A \star B \longrightarrow E,$$

the active nondependent recursor accepts:

1. a functor $F : A \rightarrow E$;
2. a functor $G : B \rightarrow E$;
3. an internally natural cross cell

$$\gamma : \mathbf{1}_{A,B} \Longrightarrow \text{Hom}_E(F-, G-).$$

It returns `join_elim_func F G gamma`. The two restrictions compute:

$$\begin{aligned} H \circ \iota_A &\rightsquigarrow F, \\ H \circ \iota_B &\rightsquigarrow G. \end{aligned}$$

The selected observation of the image of the universal cross cell also computes:

$$H(\chi) \rightsquigarrow \gamma.$$

The literal third owner is `join_elim_cross_transf`. It records the cross-cell beta rule without adding a broad equation for arbitrary functor application to primitive join syntax.

Formal status — checked. Evidence `JOIN-RECURSOR`. The active owners are `Join_cat`, `join_fst_func`, `join_snd_func`, `join_cross_transf`, `join_cross_hom`, `join_elim_func`, and `join_elim_cross_transf`. The checked interface is a nondependent recursor with its three observations; it is not yet a uniqueness theorem for all functors out of the join.

The recursor repeats the book’s computational pattern. The inclusions and cross cell introduce the join; restriction and the selected cross observation eliminate it; matching introduction/elimination cuts reduce to the supplied data.

17.8 Ordinary Cones And Cocones As Join Diagrams

The join makes the orientation of conical universal data concrete. A functor

$$\mathbf{1} \star J \longrightarrow B$$

can be constructed from:

- an object $b : B$;
- a diagram $F : J \rightarrow B$;
- a natural family of cross arrows $b \rightarrow Fj$.

That is precisely the data of a cone from b to F . Reversing the two parts, a functor

$$J \star \mathbf{1} \longrightarrow B$$

can be constructed from F , an object c , and arrows $Fj \rightarrow c$: the data of a cocone.

This is an interface-level construction supplied by the recursor. It does not yet assert an equivalence between a full mapping category out of the join and a category of cones. Such an equivalence would require an eta or uniqueness principle for join maps and coherent action on transformations between them.

The connection to weights is now visible. A conical cone uses the terminal weight, and the join internalizes a terminal profunctor as its family of left-to-right cross arrows. Replacing that terminal profunctor by an arbitrary $P : A \rightsquigarrow B$ leads to the notion of a collage.

17.9 Join As A Directed Higher-Inductive Pattern

The join has the shape of a directed higher-inductive specification:

- two strata of objects, introduced through A and B ;
- a family of directed arrow constructors from every left object to every right object;
- a recursor into an arbitrary target;
- beta observations on the two strata and the cross family.

This comparison is architectural, not a claim that `Join_cat` was generated by a general directed-HIT compiler. The primitive join is one selected signature. Its cross constructor is already internally natural, which is the category-level analogue of giving all boundary coherence with the constructor.

WalkingEnd and join illustrate two different levels of the same programme. WalkingEnd has an opaque point and endomorphism together with a contextual dependent eliminator strong enough for encode-decode. Join has two category-shaped introductions and a nondependent recursor, but no dependent eliminator or computation of all its hom-categories. Thus join supplies a promising stress test for a future directed-HIT schema without borrowing properties proved only for WalkingEnd.

17.10 The Collage Comparison

For a profunctor

$$P : A \rightsquigarrow B,$$

its expected collage $\text{Coll}(P)$ contains A and B as two parts and uses $P(a, b)$ as the category of arrows from the left object a to the right object b . Its universal mapping property has the schematic form

$$\text{Fun}(\text{Coll}(P), E) \simeq \sum_{F:A \rightarrow E} \sum_{G:B \rightarrow E} \text{ProfCell}(P, \text{Hom}_E(F-, G-)).$$

Taking $P = \mathbf{1}_{A,B}$ gives exactly the *input shape* of the active join recursor. This is strong design evidence for reading $A \star B$ as the prospective collage of the terminal profunctor.

It is not yet a proof of that reading. A checked collage theorem would need:

1. an object decomposition into the left and right strata;
2. left-left and right-right hom comparisons recovering A and B ;
3. a left-right hom comparison recovering P ;
4. a right-left hom description, normally initial in the free collage;
5. composition and higher action compatible with those four regions;
6. full faithfulness of the inclusions;
7. an equivalence of mapping categories, including uniqueness and higher transformations;
8. a dependent eliminator with beta behavior on the cross cells.

The active primitive supplies the inclusions, one terminal cross cell, and the forward recursor only. It exposes none of the required hom decompositions. Likewise, one expects an opposite comparison

$$(A \star B)^{\text{op}} \simeq B^{\text{op}} \star A^{\text{op}},$$

but no such comparison is currently selected.

Formal status — research boundary.

Evidence `JOIN-COLLAGE-BOUNDARY` . Side task `FTTX-S13` owns the proposed collage semantics and dependent elimination. The current chapter claims only that the checked recursor has the terminal-collage input shape.

17.11 From Universal Cuts To Directed Geometry

Weighted limits and colimits began with a mapping problem: a profunctor of cones or cocones is represented by a hom. Join begins with the same data from the other side. Instead of representing a terminal family of cross arrows inside a target, it freely presents a domain from which such data can be interpreted by recursion.

The two constructions therefore complement one another:

weighted representation	directed join recursion
classifies maps into or out of a vertex	internalizes a family of cross arrows
beta/eta from a profunctor comparison	beta from introduction/recursion
arbitrary weight	currently the terminal profunctor
right/left adjoint preservation	prospective collage and dependent elimination

This is the globally coherent role of the active universal-construction interfaces. Tensor, residuals, weighted representation, duality, and join are not isolated features. Each controls a different kind of cut, and each leaves enough categorical action visible for a future elaboration layer to compile surface mathematics into the same computational core.

18. Presheaves And Sieves

An object rarely reveals its geometry all at once. We learn about it by probing it from other objects, by changing the probe, and by asking which properties survive that change. For an object U of a category $\mathcal{K}$, the probes are simply the arrows

$$p : V \longrightarrow U.$$

The domain V is a stage of observation. A further arrow $q : W \rightarrow V$ refines the observation, and the composite $p \circ q : W \rightarrow U$ is the same probe viewed at the finer stage. This elementary picture contains the beginning of presheaf semantics, the definition of a sieve, and eventually the local-to-global language of schemes.

The decisive shift is from asking for *the* region where a property holds to recording *every probe along which it holds*. The latter collection is automatically adapted to change of stage. It is a sieve. An open subobject may represent that sieve, but representation is an additional theorem, not part of the initial definition.

This chapter develops that distinction in three steps. A presheaf gives a coherent field of views. A higher sieve may retain a category of witnesses at each probe. An ordinary sieve remembers only a proposition: whether the probe belongs. The recurring example is the invertibility sieve $D_U(s)$ of a section s over U .

18.1 A Coherent Field Of Views

Let $\mathcal{K}$ be a category. A set-valued presheaf is a functor

$$X : \mathcal{K}^{\text{op}} \longrightarrow \mathbf{Set}.$$

Thus every object U has a set $X(U)$ of observations, and every arrow $p : V \rightarrow U$ has a restriction map

$$p^* = X(p) : X(U) \longrightarrow X(V).$$

Restriction along an identity does nothing, and restriction along a composite is successive restriction:

$$(\text{id}_U)^* = \text{id}_{X(U)}, \quad (p \circ q)^* = q^* \circ p^*.$$

These equations say more than “data vary with U .” They say exactly how one view becomes another. A section at a large stage can be inspected at every smaller stage, and two routes to the same refined view agree.

Two familiar examples give the variance its geometric meaning. On a topological space, one may assign to every open U the continuous real-valued functions on U . An inclusion $V \subseteq U$ restricts a function on U to one on V . On the opposite of a category of commutative rings, the functor of points of a ring R assigns to a test ring S the maps $R \rightarrow S$. A map of test rings changes the point by composition. In both cases, the presheaf is not merely a table indexed by objects. Its restriction maps are the mathematical content that lets observations move.

The examples also warn against reading “smaller stage” as literal spatial inclusion. A probe can be an open subset, an étale map, a ring map viewed in the opposite category, or some other morphism selected by the geometry. The categorical definition uses only arrows. Spatial language becomes justified when a later representation theorem supplies it.

Functorial type theory keeps the same idea but permits each $X(U)$ to be a category rather than merely a set:

$$X : \mathcal{K}^{\text{op}} \longrightarrow \mathbf{Cat}.$$

Now a stage may contain objects, arrows between observations, and higher coherence inherited from the ambient categorical structure. Restriction is a functor. It transports both observations and their comparisons. Presheaf maps are natural family maps, so they too act coherently at every stage.

There is also a useful change-of-base operation. Given $F : \mathcal{A} \rightarrow \mathcal{B}$ and a presheaf X on $\mathcal{B}$, precomposition produces a presheaf on $\mathcal{A}$:

$$F^* X := X \circ F^{\text{op}}.$$

The active construction realizes this whole operation by reusing the existing pullback of directed $\mathbf{Cat}$ -valued families. It does not invent a second calculus for presheaves. That economy will matter repeatedly: local geometry should inherit functoriality from the categorical structure already present.

18.2 The Representable Field Of Probes

Every object U supplies a canonical presheaf

$$yU := \text{Hom}_{\mathcal{K}}(-, U).$$

At a stage V , its objects are precisely the probes $p : V \rightarrow U$. If $q : W \rightarrow V$, restriction sends p to $p \circ q$. Nothing has been added to the category: the presheaf merely organizes all arrows into U as one coherent field.

This is the contravariant Yoneda construction from [Chapter 13](#), now read geometrically. There it served as a universal coordinate system. Here its elements are stages of observation. The two readings are the same mathematics: a map out of a representable is controlled by what happens at the identity probe $\text{id}_U : U \rightarrow U$.

The total category of the family yU has objects (V, p) with $p : V \rightarrow U$. For restriction it is convenient to orient its arrows in the direction in which data are reindexed. Taking the opposite recovers the conventional slice category $\mathcal{K}/U$. Keeping both orientations visible prevents a common mistake: a refinement of a probe and a map in the displayed family carry the same information, but variance determines which way the corresponding arrow points.

A **higher sieve** on U is a Cat-valued coefficient system over this restriction-oriented category of probes. Equivalently, it may be read as a Cat-valued presheaf on the conventional slice. At each $p : V \rightarrow U$ it assigns a category $S(p)$, and a refinement $q : W \rightarrow V$ induces a functor

$$S(p) \longrightarrow S(p \circ q).$$

The category $S(p)$ can retain choices of witnesses and arrows between them. This is why “higher sieve” does not yet mean ordinary sieve. The maximal higher sieve assigns the terminal coefficient category to every probe. If the base probe $V \rightarrow U$ is changed, the entire coefficient system pulls back by postcomposing probes, and the maximal higher sieve remains maximal.

Formal status — checked. Evidence `PSH-YONEDA-HIGHER-SIEVE`. The active construction has a visible category of Cat-valued presheaves, restriction along a functor, the Yoneda presheaf with value $\text{Hom}_{\mathcal{K}}(V, U)$, the restriction-oriented arrow total and opposite slice, and a higher-sieve classifier whose maximal object is stable under pullback. The slice and higher-sieve presentations compare through a shared family representation; they are not declared to be definitionally identical.

18.3 From Witnesses To Membership

An ordinary sieve is what remains when each coefficient category answers only a yes-or-no question. Constructively, “yes or no” does not mean that a Boolean decision has been chosen. It means that the space of answers is a proposition: if it is inhabited, all of its inhabitants agree.

Classically, a sieve R on U is a collection of arrows into U such that

$$p \in R, q : W \longrightarrow V \implies p \circ q \in R.$$

The presheaf formulation replaces the collection by a subterminal value at every probe. Write $R(p)$ for the coefficient at $p : V \rightarrow U$. Its object classifier is the proposition

$$p \in R.$$

The action of the underlying higher sieve carries a witness of $p \in R$ to a witness of $p \circ q \in R$. Closure under refinement is therefore not an extra law written beside the data. It is the functorial action of the data.

Why require a *subterminal category* rather than merely proposition-valued objects? Because a category with a unique object can still have many directed endomorphisms. Ordinary membership is intended to retain no such hidden motion. The active condition combines proposition-valued objects with exact groupoidality, leaving the categorical analogues of the empty and terminal possibilities. The evidence that a coefficient is subterminal is itself a proposition, so it does not create competing ways for one arrow to belong.

Two examples orient the definition.

1. The maximal sieve contains every arrow into U . Its membership proposition is always inhabited.
2. If a monomorphism $j : W \rightarrow U$ is regarded as a subobject, it determines the sieve of arrows $p : V \rightarrow U$ that factor through j . Monicity makes the factorization proposition-valued. Refining a factorization gives another factorization automatically.

The second example is important but not exhaustive. A sieve need not be represented by one monomorphism. It can be a genuinely distributed local question whose answer is stable under refinement without having a single object that names all affirmative probes.

18.4 Pulling Back A Local Question

Suppose R is a sieve on U and $p : V \rightarrow U$. To ask the same question over V , test every $q : W \rightarrow V$ after postcomposition with p :

$$q \in p^*R \quad \Longleftrightarrow \quad p \circ q \in R.$$

This defines a sieve p^*R on V . Indeed, if q belongs and $r : Z \rightarrow W$, then

$$p \circ (q \circ r) = (p \circ q) \circ r$$

belongs by closure of R . More conceptually, the higher-sieve action already postcomposes every probe with p . Pointwise subterminality is preserved by selecting the old witness at that postcomposed probe. Ordinary pullback is therefore inherited structure, not a separately engineered operation.

Theorem 18.1 (pullback of ordinary sieves). For every sieve R on U and arrow $p : V \rightarrow U$, there is an ordinary sieve p^*R on V . Its underlying higher sieve is the higher pullback of the underlying higher sieve of R , and membership at $q : W \rightarrow V$ is old membership at $p \circ q : W \rightarrow U$.

Formal status — checked. Evidence `ORDINARY-SIEVE-PULLBACK`. The active construction supplies the pullback sieve, reuses the whole higher-sieve action, preserves subterminal evidence, and exposes the membership computation. Because an ordinary sieve retains proof fields, identity pullback is not claimed to reconstruct the entire package by raw definitional reduction; this does not change the membership formula above.

Pullback is the reason sieves are the right language for locality. A property stated only at U can be accidental. A sieve records in advance how the property appears after every change of stage. A topology will later decide which such stable questions count as *covering* questions, but the variance has already been settled.

Represented sieves make this calculation visible as ordinary base change. Suppose the sieve R is represented by a monomorphism $j : A \rightarrow U$, and suppose the pullback square

$$\begin{array}{ccc} A \times_U V & \longrightarrow & A \\ \downarrow & \searrow p & \downarrow j \\ V & \longrightarrow & U \end{array}$$

exists. A probe $q : W \rightarrow V$ belongs to p^*R exactly when $p \circ q$ factors through j . By the universal property of the pullback, this is exactly when q factors through $A \times_U V \rightarrow V$. Hence p^*R is represented by the base-changed monomorphism.

This is the bridge between sieve pullback and the usual restriction of an open. The sieve calculation requires only postcomposition and membership. The spatial calculation additionally requires a representing monomorphism and the relevant pullback. When those hypotheses are available, the two descriptions agree; when they are not, the sieve calculation still makes sense.

18.5 Invertibility Before Opens

Let $\mathcal{O}$ be a presheaf of commutative rings on $\mathcal{K}$, let U be a stage, and let $s \in \mathcal{O}(U)$ be a section. For a probe $p : V \rightarrow U$, restriction produces

$$p^*s \in \mathcal{O}(V).$$

Define the **invertibility sieve** of s by

$$D_U(s)(p) := \text{“} p^*s \text{ is a unit in } \mathcal{O}(V)\text{.”}$$

This is a sieve because ring homomorphisms preserve units. If p^*s has an inverse and $q : W \rightarrow V$, applying the restriction homomorphism along q gives an inverse for $(p \circ q)^*s$. Thus invertibility propagates toward finer probes.

Formal status — checked. Evidence `COMM-RING-INVERTIBILITY-SIEVE`. For a selected commutative-ring-valued presheaf and section, the active construction supplies an ordinary sieve, and membership at $p : V \rightarrow U$ computes to unit evidence for the restricted section p^*s .

The notation $D_U(s)$ is familiar from algebraic geometry, but the order of ideas matters. We have not first constructed an open object and then asked which probes land in it. We first have the stable family of all invertibility probes. The question whether that family is represented by an open object is asked afterward.

This reverses a habitual abbreviation. In settings where basic opens are already known to exist, one says “the open on which s is invertible.” The sieve formulation separates two claims hidden in that phrase:

1. invertibility is stable under change of stage, so it defines $D_U(s)$; and
2. the sieve $D_U(s)$ is represented by a suitable open object over U .

The first statement is formal and functorial. The second depends on the geometry of the site. In affine geometry, localization will supply the representing object for a basic invertibility sieve. On a more general site, there may be no single representative, while the sieve itself remains perfectly meaningful.

This point also clarifies the relation with Max Zeuner's constructive account of algebraic geometry. In the locally ringed lattices of Zeuner, the invertibility support of a section is presented as the largest compact open below U on which that section is invertible. In a posetal site, such a largest open represents the sieve $D_U(s)$: a probe lies in the sieve exactly when it factors through that open. Conversely, a compact open representing $D_U(s)$ has the required largest-property interpretation.

The sieve-centered formulation is therefore a generalization, not a rejection, of the compact-open one. It asks the useful comparison question:

■ When is the invertibility sieve $D_U(s)$ representable by a compact open?

On a coherent or qcqs presentation, representability can recover the compact geometry emphasized by Zeuner. On an arbitrary site, the sieve remains the primary object even when that recovery is unavailable. In a higher setting, one may go further and retain a category of invertibility witnesses before subterminality is imposed.

The posetal case makes the comparison exact. Regard a lattice of opens below U as a category, with one arrow $V \rightarrow W$ when $V \leq W$. If an open $A \leq U$ represents $D_U(s)$, then for every $V \leq U$,

$$s|_V \text{ is invertible} \iff V \leq A.$$

Taking $V = A$ shows that $s|_A$ is invertible. Taking any V on which s is invertible shows $V \leq A$. Thus A is the largest invertibility open. Conversely, if a largest such A exists, invertibility is preserved by restriction, so precisely the opens below A belong to $D_U(s)$. The largest open represents the sieve. Compactness is a further finiteness property of that representative, important in coherent algebraic geometry but not needed to define membership.

The distinction separates existence from use. One can calculate with $D_U(s)$, pull it back, and ask whether it covers before finding a representing open. Once a compact representative is constructed, the same sieve acquires the economical lattice presentation. The two layers reinforce rather than compete with one another.

Formal status — mathematical development. The comparison with Zeuner's compact-open support is an attributed reformulation of the representation problem, principally adapted from his definition of locally ringed lattices and his functor-of-points development. The active emdash result in this chapter is the ordinary invertibility sieve. No theorem identifying the general emdash site-relative scheme interface with Zeuner's qcqs schemes is claimed here.

18.6 Categorical Semantics As Executable Mathematics

Presheaves, slices, and sieves are often introduced as an external semantics for some other formal language. That is not the only way to make them computational. In the emdash architecture, the categories and functors just described are themselves expressed in an inner functorial type theory. They are ordinary mathematical objects of that theory: a presheaf is a functor, a slice is a category, and pullback is functorial reindexing.

The inner theory is hosted in an outer dependent logical framework. `Lambdapi`, or the bounded TypeScript emdash Core, supplies explicit binders, checking, rewrite computation, comparison, and unification. Consequently, categorical semantics can be internal enough to calculate with while remaining recognizably the traditional semantics of presheaves and sites.

This is a claim about relative internality, not about replacing every modal or internal language. A modal type theory may provide elegant syntax for local reasoning. Here the chosen route is to make the categorical objects themselves executable and then state locality directly in them. The advantage for the present development is transparency: the probe, its refinement, the sieve pullback, and later the matching family remain visible mathematical data.

Nor does executability settle the metatheory. Local computations accepted by the outer framework do not by themselves prove global normalization, confluence, consistency, or semantic soundness for the combined rewrite system. The categorical constructions and their checked observations are the evidence used here; the larger metatheorems retain their separate boundary.

18.7 From Sieves To Covers

A sieve answers a local question. It does not yet say that the affirmative probes are sufficient to recover information on U . For that, one must select the sieves that count as covers and demand three kinds of stability: the maximal sieve covers, a covering sieve remains covering after pullback, and covers may be refined locally.

Those laws turn a category into a site. They also prepare a more delicate question. Given compatible observations on every probe in a covering sieve, when do they determine one observation on U ? That is descent, and its objects are matching families and their amalgamations.

The next step in the local-to-global spiral therefore does not abandon the sieve for an open. [Chapter 19](#) asks which sieves cover, and what it means for a presheaf to solve every covering question. Geometry will emerge from the answers, but the questions already have their correct functorial shape.

19. Sites, Covers, And Descent

A sieve is a question stable under refinement. It does not yet say that its affirmative probes see enough of the object. The maximal sieve certainly does: it contains every probe. A smaller sieve may also be sufficient, but that is new structure. One must decide which local views are jointly adequate for recovering information at their common target.

A **Grothendieck topology** makes that decision. It selects covering sieves in a way compatible with identities, change of stage, and local refinement. A category equipped with such a topology is a **site**. The word “topology” is appropriate even when the objects are not open subsets of a space. Its role is to identify which families of probes count as local descriptions.

Once a cover has been selected, a presheaf faces a test. Compatible data on the probes of the cover form a matching family. Does that family come from one global datum, and is the global datum determined by it? This is descent. The chapter separates three layers that are often compressed into one phrase:

1. a family or sieve proposed as a cover;
2. a topology generated by chosen proposals; and
3. the locality of a presheaf with respect to the resulting covers.

Sheafification is a fourth layer. It constructs a local object from an arbitrary presheaf and therefore belongs to the next chapter.

19.1 Covering Families And Covering Sieves

Suppose a family of arrows has common codomain U :

$$u_i : U_i \longrightarrow U.$$

It generates a sieve by admitting every probe that factors through one of the u_i . Refinement closure then admits all further composites automatically. The family is a useful presentation of local pieces; the generated sieve is the complete local question determined by that presentation.

More explicitly, a probe $p : V \rightarrow U$ belongs when there are an index i and an arrow $h : V \rightarrow U_i$ such that

$$p = u_i \circ h.$$

Ordinary sieve membership remembers this existential claim only as a proposition. A computational presentation may separately retain the index, the factorization, or algebraic evidence that the family has the desired covering property. Keeping those layers separate prevents two opposite errors: discarding useful witnesses too early, and making the topology depend on accidental choices of witnesses.

Different families can generate the same sieve. Repeating a member changes the list but not the question. Replacing one member by a family that covers it may refine the presentation without changing its eventual force. A sieve also makes arbitrary change of stage immediate: pull it back as in [Chapter 18](#), rather than choosing a new list and proving afresh that the list behaves correctly.

This does not make cover families dispensable. In constructive algebraic geometry, a finite unimodular family or a pair of affine charts can carry valuable computational witnesses. The point is to distinguish the witness from the invariant it presents. One may retain the particular generators for calculation while allowing the topology to speak in terms of the sieve they force to cover.

For an ordinary topological space, a family of open inclusions $U_i \hookrightarrow U$ is covering when their union is U . The associated sieve contains every open $V \hookrightarrow U$ that lies inside some U_i , and then all smaller opens as well. In affine geometry, a finite family of basic opens will later be presented by ring elements satisfying an algebraic cover certificate. Again, the certificate explains why the generated sieve covers; it is not identical to the sieve.

Notice that generation alone says nothing about sufficiency. Any family of arrows generates a sieve. Only the topology declares whether that sieve covers. The separation is what allows the same category of probes to carry different geometries.

At the opposite extreme lies the maximal sieve $\top_U$. Every arrow into U belongs to it. It is generated by the identity probe $\text{id}_U : U \rightarrow U$, since every $p : V \rightarrow U$ factors through the identity. Any reasonable notion of locality must regard this as a cover: observing all of U at once is sufficient to observe U .

19.2 The Three Laws Of A Site

Let $J(U, R)$ be the proposition that the sieve R covers U . To make J a Grothendieck topology, require three laws.

Maximality. The maximal sieve covers:

$$J(U, \top_U).$$

Pullback stability. If R covers U and $p : V \rightarrow U$, then the pulled-back sieve covers V :

$$J(U, R) \implies J(V, p^*R).$$

This is the formal expression of “covers remain covers after changing the stage of observation.” If R is represented by an open $A \rightarrow U$, then under the hypotheses of Chapter 18 the new cover is represented by $A \times_U V \rightarrow V$.

Local character. Suppose R covers U . Let S be another sieve on U . If, for every $p : V \rightarrow U$ belonging to R , the pullback p^*S covers V , then S covers U :

$$J(U, R) \quad \text{and} \quad \prod_{p \in R} J(V, p^* S) \quad \Longrightarrow \quad J(U, S).$$

Local character is the transitivity of coverage. The sieve R says that its probes see all of U . Each of those probes says that S sees all of its domain. Together they say that S sees all of U . The law avoids choosing a single flattened family and consequently works equally well for finite, infinite, or witness-rich presentations.

In family language, suppose the $u_i : U_i \rightarrow U$ cover and, for every i , the arrows $v_{ij} : V_{ij} \rightarrow U_i$ cover U_i . The composites $u_i \circ v_{ij} : V_{ij} \rightarrow U$ should cover U . Let R be the sieve generated by the u_i and S the sieve generated by the composites. The pullback of S to U_i contains the locally covering v_{ij} , while R covers U . Local character is the invariant sieve statement behind this flattening argument. It also covers situations in which there is no preferred single index set of composites.

The three laws are deliberately asymmetric in purpose. Maximality supplies a unit, pullback stability transports a cover, and local character composes local sufficiency. None says that membership in a cover is decidable. Cover evidence remains proposition-valued.

Nor does a Grothendieck topology add new objects or arrows to $\mathcal{K}$. It changes which existing sieves are treated as sufficient. Two topologies on the same category can therefore encode different notions of locality. A covering sieve need not be represented by an open subobject; and when it is represented, the representing open is geometry derived from the site rather than a replacement for the cover predicate. This is the same sieve-before-open discipline that governed $D_U(s)$ in Chapter 18.

There is always a degenerate but useful model: declare every sieve covering. Then all three laws have their unique truth witness. This **chaotic topology** is usually too coarse for geometry, but it proves that the direct definition is inhabited on every category and provides an upper bound for generated topologies.

The opposite extreme declares only maximal sieves covering. Mathematically, every presheaf satisfies the resulting sheaf condition because restriction to the maximal local question loses no data. Enlarging a topology adds covering questions and therefore removes presheaves that fail them. The chaotic topology asks every sieve to be sufficient and is correspondingly severe. These extremes make the variance of the order clear: more covers mean fewer sheaves, even though the topology itself is larger as a cover predicate.

Formal status — checked. Evidence `GROTH-TOPLOGY-SIEVE-LAWS`. The active topology package contains a proposition-valued coverage together with maximality, pullback stability, and local character over ordinary sieves. The maximal sieve and the chaotic topology have direct checked models. This layer neither assumes a global classifier of all sieves nor constructs sheafification.

19.3 Generating The Least Topology

In practice one rarely declares every covering sieve independently. One gives basic covers and closes them under the topology laws. The phrase “the topology generated by these covers” should include two statements:

1. every proposed cover is covering; and
2. no additional covers are accepted except those forced by the three laws.

The active construction makes both statements precise without choosing a syntax of derivation trees. Let

$$G(U, R)$$

be the type of witnesses that R is one of the proposed generating sieves on U . This type need not be a proposition. It may remember which finite family, algebraic certificate, or chart presentation produced the same underlying sieve.

A Grothendieck topology T **accepts** G when every witness in $G(U, R)$ produces evidence that R covers in T . Now define R to cover in the generated topology when it covers in *every* topology accepting G :

$$J_G(U, R) := \prod_{T: \text{GrothTopology}(\mathcal{K})} (T \text{ accepts } G) \longrightarrow T(U, R).$$

This is the intersection of all acceptable topologies. Intersections inherit the three topology laws pointwise. Every generator belongs because every topology under consideration accepts it. And if T accepts the generators, then J_G is below T : a J_G -cover is, by definition, a cover in T .

The order here is an order of consequences. Write $J \leq T$ when every J -cover is a T -cover. A smaller topology makes fewer local families covering and therefore imposes fewer sheaf conditions. The generated topology is the least element among those that accept the proposals. The chaotic topology accepts every proposal, so the class being intersected is never empty.

Witness-rich generation and proposition-valued coverhood now play distinct roles. Several inhabitants of $G(U, R)$ may explain the same proposed cover in different ways. Acceptance must handle each explanation. Once R belongs to J_G , however, its coverhood is a proposition: downstream sheaf reasoning cannot branch on which derivation happened to be supplied. The construction retains evidence where computation may need it and erases choice where geometry should be invariant.

Theorem 19.1 (least generated topology). Every type-valued family of generating sieves determines a Grothendieck topology J_G . The topology accepts every retained generator and is contained in every Grothendieck topology that accepts them.

Formal status — checked. Evidence `GENERATED-GROTH-TOPLOGY`. The generated cover predicate is proved proposition-valued, its three topology laws are constructed, generator inclusion computes, and leastness is a whole pointwise comparison of topologies. The construction is

impredicative: it provides no inductive derivation syntax, induction principle for generation steps, coverhood normalizer, or decision procedure.

The boundary in that note is mathematically significant. An inductive presentation can explain *how* a cover was derived. The intersection presentation proves exactly *which universal property* the generated topology has. For the book's current purpose, leastness is the invariant needed by later geometry. A derivation calculus may still be valuable for automation, but it would be another presentation of the same intended closure, not the definition of a site.

There is no circularity in the universal characterization. One first knows what a Grothendieck topology is from the three laws. One then ranges over all such structures that accept the generators and intersects their cover predicates. The result is certified to satisfy the same laws. This resembles defining a generated algebraic congruence as the intersection of all congruences containing the proposed relations: closure is characterized by leastness before any normal-form algorithm is chosen.

19.4 A Sieve As A Domain Of Local Data

Let R be a sieve on U . The representable presheaf yU contains every probe into U . The sieve selects some of those probes, together with their refinements. To turn that selection into a domain on which a presheaf can be evaluated, form the extension

$$\widehat{R}(V) := \sum_{p: V \rightarrow U} R(p).$$

An object of $\widehat{R}(V)$ is a probe $p : V \rightarrow U$ together with evidence that it belongs to R . Forgetting the evidence gives a whole presheaf map

$$i_R : \widehat{R} \longrightarrow yU.$$

For a $\mathbf{Cat}$ -valued presheaf X , define the category of sections over U in representable form as

$$\text{Sect}_X(U) := \text{Hom}_{\mathbf{Psh}(\mathcal{K})}(yU, X).$$

The ordinary Yoneda lemma identifies this with $X(U)$. Retaining the Hom form has an advantage here: it expresses the local-to-global map without assuming the full $\mathbf{Cat}$ -valued Yoneda equivalence as an active theorem.

The category of matching families on R is

$$\text{Match}_X(R) := \text{Hom}_{\mathbf{Psh}(\mathcal{K})}(\widehat{R}, X).$$

Concretely, a matching family assigns an observation in $X(V)$ to every member $p : V \rightarrow U$ of the sieve, compatibly with every refinement and with the arrows retained by the $\mathbf{Cat}$ -valued coefficients. Compatibility is not a list of equations added afterward. It is the naturality of one presheaf map.

For a cover presented by arrows $u_i : U_i \rightarrow U$, this recovers the usual picture. A matching family begins with local elements

$$x_i \in X(U_i).$$

If the pullbacks $U_i \times_U U_j$ exist, the two restrictions of x_i and x_j to the overlap must agree. But pairwise overlap equations are only a presentation of the deeper condition. A sieve also contains longer refinements, morphisms between probes, and covers in categories without the chosen pullbacks. A map $\widehat{R} \rightarrow X$ packages compatibility with all of them at once.

In the Cat-valued case, the word “agree” may itself have categorical content. A matching object can carry coherent comparison arrows, and a map between matching objects must respect them. This is why the Hom target above is a category rather than merely a set of families.

Precomposition with i_R restricts a global section to its matching family:

$$\rho_{R,X} : \text{Sect}_X(U) \longrightarrow \text{Match}_X(R), \quad \rho_{R,X}(x) = x \circ i_R.$$

This formula is the categorical core of descent. It names both the data to be glued and the direction in which global information becomes local.

19.5 Locality And The Sheaf Condition

The presheaf X is **local at R** when the restriction functor $\rho_{R,X}$ is an equivalence. Every matching family then has an amalgamation, and that amalgamation is unique at the appropriate categorical level.

For a set-valued presheaf, the statement specializes to the familiar pair:

- compatible local elements have a global amalgamation; and
- two global elements with the same local restrictions are equal.

The second clause is often called **separatedness**, while the first is the existence part of gluing. Their conjunction says that restriction is a bijection. For Cat-valued presheaves, a categorical equivalence replaces that bijection: it controls objects, maps, and their inverse laws together. Merely asking for essential surjectivity would give amalgamations without the full uniqueness and functoriality required here.

For a Cat-valued presheaf, the equivalence also retains maps between local families and the higher coherence carried by the Hom categories. Replacing the equivalence by a mere objectwise existence statement would lose this structure.

Given a topology J , say that X is **topology-local** when it is local at every J -covering sieve. This is the sheaf condition in the form used by the current categorical development:

$$\prod_U \prod_R \prod_{c:J(U,R)} \text{IsEquiv}(\rho_{R,X}).$$

The cover witness c indexes the condition but does not become computational data: coverhood is a proposition. What matters computationally is the fixed forward map $\rho_{R,X}$ and its selected inverse behavior.

A topology for which every representable presheaf is local is called **subcanonical**. This is a property of the topology, not part of the definition of a site. It says that the original category embeds into its sheaf semantics without changing the representable probes. The Zariski coverage used later is intended to have this geometric behavior, but no arbitrary Grothendieck topology is subcanonical merely because its three laws hold.

Formal status — checked. Evidence `SIEVE-MATCHING-LOCALITY`. The active construction extends an ordinary sieve to a whole presheaf, includes it in the representable, defines matching and section Hom categories, and makes restriction exact precomposition by that inclusion. Locality at one sieve and simultaneous locality over a topology are active fixed-forward equivalence predicates. No reflector or identification with a separate rigid sheaf category follows at this layer.

This formulation makes the categorical-semantics point particularly direct. There is no need to hide the site behind an abstract modal operator before locality can compute. The actual sieve extension, representable, Hom categories, and restriction functor are objects of the inner functorial type theory. The outer logical framework checks and reduces their selected observations. An internal modal language could summarize this behavior, but the categorical semantics already supplies an executable statement of descent.

19.6 Varying The Cover Question

A single cover is not enough for sheafification. Covers change when their base object is pulled back, and a construction that glues only after all choices have been externalized will carry a growing burden of naturality equations.

It is therefore useful to package an **eligible cover question** as

$$q = (U, R, c),$$

where R is a sieve on U and c says that R covers. These questions themselves form a category. For a fixed presheaf X , matching categories and section categories then vary as Cat-valued families over the whole question category, and restriction becomes one displayed functor between them.

An arrow between questions includes a change of base $p : V \rightarrow U$ and the pulled-back covering question over V . The topology's stability law supplies its eligibility. On local data, the arrow acts by reindexing along the canonical map from the pulled-back sieve extension to the original one. Thus the same base change that transported membership in Chapter 18 now transports the entire descent problem.

The gain is conceptual as well as formal. Pulling back a cover question moves its sieve extension, representable, matching data, sections, and restriction map together. Naturality belongs to the single varying construction. It is not reintroduced as a separate square for every cover chosen later.

The active development realizes this whole varying layer and checks that, at a literal question, the displayed restriction is exactly the precomposition map $\rho_{R,X}$ above. It deliberately stops before supplying glue. A matching family is a well-formed local answer; it is not yet an amalgamation.

19.7 The Construction Still Missing

A sheaf is a presheaf that already solves every covering question. A sheafification must start with an arbitrary presheaf P and construct a new one aP together with a unit

$$\eta_P : P \longrightarrow aP$$

that is universal among maps from P to local presheaves. Merely restating the sheaf condition does not build aP . Nor does choosing an inverse to $\rho_{R,X}$ for an already-local X explain how to add missing amalgamations coherently.

Universality asks for more than a local output. For every topology-local X , precomposition with the unit should give an equivalence

$$\mathrm{Hom}(aP, X) \simeq \mathrm{Hom}(P, X).$$

Thus a map out of P into a local target extends across the completion, and the extension is unique at the whole Hom-category level. This property is what makes sheafification a reflector rather than one arbitrary repair of local data.

The direct construction pursued next treats covers as questions whose solutions may be freely adjoined. It needs a return constructor for old data, a whole glue operation for matching families, a silent law saying that gluing the restriction of an existing section changes nothing, and a recursor expressing the universal property. The varying-cover architecture of this chapter is what lets those constructors be stated once over all eligible questions.

Thus the progression is exact:

sieve $\longrightarrow$ covering sieve $\longrightarrow$ matching family $\longrightarrow$ amalgamation $\longrightarrow$ sheafification.

The first three stages are now in view. The next chapter constructs the last two for the selected Cat-valued setting and then asks for their whole universal property.

20. Sheafification By Cover Completion

A sheaf knows how to answer every covering question. An arbitrary presheaf may answer some and fail others. Sheafification is the passage from the latter to the former, but that phrase conceals two different demands. We must add the missing global sections, and we must add no information beyond what every sheaf is already forced to accept.

The first demand is constructive. Given a matching family on a covering sieve, adjoin an amalgamation. Repeat, because the newly adjoined data may itself occur in another matching family. The second demand is universal. A map from the original presheaf to a sheaf must extend uniquely across the completion. Together they characterize a reflector from presheaves to sheaves.

This chapter carries out that program directly in categorical semantics. The site, covering sieves, representables, matching categories, and restriction functors of Chapters 18 and 19 remain visible. They are not first encoded into a modal object language. The construction instead gives a categorical higher-inductive signature with three memorable operations:

return, glue, silent.

Return preserves old data. Glue supplies answers to covering questions. Silent says that asking a question whose answer is already known has no observable effect. A recursor then proves that this is the free such completion, and the free completion assembles into the desired Cat-valued sheafification functor.

20.1 From A Condition To A Construction

Fix a category $\mathcal{K}$, a Grothendieck topology J , and a Cat-valued presheaf P . Write $a_J P$, or simply aP , for the presheaf to be completed. There must first be a whole presheaf map

$$\eta_P : P \longrightarrow aP.$$

This is **return**. It does not assert that P was already local. It embeds the observations we began with among the observations generated by the completion.

Now take an eligible covering question $q = (U, R, c)$. Recall the categories

$$\text{Match}_{aP}(R) = \text{Hom}(\widehat{R}, aP), \quad \text{Sect}_{aP}(U) = \text{Hom}(yU, aP).$$

The second constructor is a functor

$$\text{glue}_q : \text{Match}_{aP}(R) \longrightarrow \text{Sect}_{aP}(U).$$

It turns compatible local data into a global section. Notice the recursion: the matching family already takes values in aP , not merely in P . A newly glued section can therefore participate in a later matching family. This is why one application of a nonrecursive repair operation would not express the

truction being made here.

The functorial type is equally important. In a set-valued account, glue may look like a function between sets. With Cat-valued coefficients, matching families have arrows between them, and amalgamation must transport those arrows coherently. The constructor acts on the entire matching category. It does not merely select an object-level amalgamation and leave its behavior on maps to be reconstructed later.

Finally let

$$\rho_q : \text{Sect}_{aP}(U) \longrightarrow \text{Match}_{aP}(R)$$

be restriction. The **silent** constructor is the path

$$\text{glue}_q \circ \rho_q = \text{id}_{\text{Sect}_{aP}(U)} . \quad (20.1)$$

If a global section is restricted to a cover and then glued again, nothing changes. The cover was consulted, but its answer was ignored because the global section was already present.

This return/glue/silent pattern is conceptually adapted from Pierre-Marie Pédrot's computational account of free sheaves in *Pursuing Shtuck*. Pédrot describes the last equation as the erasure of a silent transition: a branching computation that does not depend on its returned witness should be indistinguishable from the unbranched computation. Here the pattern is placed directly over actual covering-sieve questions. Its branches are whole matching maps $\widehat{R} \rightarrow aP$, and its output is a whole section $yU \rightarrow aP$.

For a concrete picture, suppose a cover of U is presented by two probes

$$u_0 : U_0 \longrightarrow U, \quad u_1 : U_1 \longrightarrow U.$$

A matching family contains local observations x_0 and x_1 whose restrictions agree wherever the two probes meet, together with compatibility under every further refinement in the generated sieve. Glue adjoins a section x over U with those local observations. If x_0 or x_1 was itself produced by gluing a finer cover, the recursive constructor accepts it without flattening the history into a chosen list of basic pieces. If the entire matching family came by restricting an old x , silent identifies the newly adjoined amalgamation with x .

That last clause is what prevents completion from accumulating duplicate global answers. Without silent, one could glue the same restricted section again and again, building formally distinct trees that encode no new local information. With an indiscriminate equation saying that every two gluings are equal, on the other hand, one would destroy genuine distinctions between matching families. Equation (20.1) is narrow: it removes precisely the detour that starts from an existing global section, passes through its restrictions, and returns by glue.

Classical accounts often construct associated sheaves by a plus operation on matching families and then iterate that operation. Such a presentation is valuable, especially when quotient objects and their exactness properties are already available. The direct completion takes a different route. It exp-

oses the free generators and their necessary path at the outset, so recursion can observe return and glue computationally. No comparison theorem with the classical plus construction is claimed here; the point is that the universal property can be reached without choosing equivalence-class representatives as the operational presentation.

The signature should not be mistaken for an algorithm that searches the site for covers. A glue constructor is available after an eligible question and its cover evidence have been supplied. Coverhood itself may be undecidable, and the generated topology of Chapter 19 provides a universal predicate, not an enumeration. Completion says coherently what to do with every admitted question; it does not promise to discover those questions by normalization.

Nor does the construction privilege the basic generators from which a topology may have arisen. Once the least topology has been formed, a derived cover is as eligible as a generating one. This is essential for locality: pullbacks and composites of covers enter the proof even when they were not in the original presentation. The witness-rich layer may still help a program produce cover evidence, but the free object depends on the invariant covering-sieve predicate.

The terminology “higher-inductive” refers to the shape of this presentation. There is a point-like return constructor, a recursive glue constructor, and a path constructor imposing the silent equation. It does not mean that the entire surrounding theory has been replaced by ordinary HoTT syntax. The objects remain categories and presheaves in functorial type theory, while the outer logical framework supplies the primitive signature, rewrite behavior, and checked equality evidence.

20.2 One Glue Operation Over All Questions

Writing glue_q separately for each cover is useful on paper, but it hides the hardest coherence. A cover can be pulled back along a probe $p : V \rightarrow U$. The matching family moves to the pulled-back sieve, the proposed amalgamation moves to V , and the two orders of those operations must agree.

Chapter 19 packaged all eligible questions into a category $\mathcal{Q}_J$. Over it, matching categories and section categories form two $\mathbf{Cat}$ -valued families,

$$\text{Match}_{aP}, \text{Sect}_{aP} : \mathcal{Q}_J \longrightarrow \mathbf{Cat}.$$

The collection of glue operations is retained as one displayed functor

$$\text{glue}_{\text{all}} : \text{Match}_{aP} \longrightarrow \text{Sect}_{aP} \quad \text{over } \mathcal{Q}_J.$$

Its component at q is glue_q . Its action along arrows of $\mathcal{Q}_J$ is precisely the compatibility of glue with change of cover question. For the canonical pullback of q along p , naturality gives the square summarized by

$$\text{Sect}[p] \circ \text{glue}_q = \text{glue}_{p^*q} \circ \text{Match}[p]. \quad (20.2)$$

Likewise, silent is not stored as an unrelated equation for every U , R , and cover witness. It is one path of displayed endofunctors:

$$\text{glue}_{\text{all}} \circ \rho_{\text{all}} = \text{id}_{\text{Sect}_{aP}} . \quad (20.3)$$

Equation (20.1) is its component at a literal cover. The one whole path also controls arrows between sections and reindexing between cover questions. This is a mathematical economy: coherence is owned at the level where the varying construction lives, rather than repeated as a growing list of component squares.

Formal status — checked categorical-HIT boundary. Evidence `DIRECT-COVER-COMPLETION-HIT` . For every site and Cat-valued presheaf, the active signature provides the completion presheaf, whole unit, whole cover-indexed glue functor, and whole silent path. Pullback compatibility is obtained from displayed-functor naturality, and the result is packaged as an internal direct-cover sheaf structure. The completion and path constructors are primitive at this boundary; locality and the reflector are consequences proved in the following stages, not fields silently included here.

The silent law has an intentional direction. It says that gluing the restriction of a section recovers the section. It does not initially say that restricting the glue of a matching family recovers the family. Supplying both directions as constructors would make locality true by declaration. Supplying only (20.3) leaves room for the geometry of sieves and pullback to prove the opposite composite.

Nor is silent installed as a runtime simplification that erases every visible glue term. It is internal equality evidence between whole functors. Return and recursive glue have selected computational rules under the recursor, while the higher silent coherence remains a path. This distinction avoids choosing one side of every sheaf equation as a universal normal form.

20.3 Why The Completion Is Local

To prove that aP is a sheaf, fix a covering sieve R on U . Equation (20.1) already gives

$$\text{glue}_R \circ \rho_R = \text{id}_{\text{Sect}_{aP}(U)} .$$

The missing law is

$$\rho_R \circ \text{glue}_R = \text{id}_{\text{Match}_{aP}(R)} . \quad (20.4)$$

Take a matching family $m : \widehat{R} \rightarrow aP$ and inspect it at a member $p : V \rightarrow U$ of R . Because R is a sieve, every refinement of p still belongs to R . Consequently the pullback p^*R is maximal on V : its identity belongs, and then so does every probe into V .

The glue operation is natural under this pullback. Thus the restriction to V of the global section $\text{glue}_R(m)$ agrees with gluing the matching data transported to p^*R . But a matching family on the maximal sieve is already the restriction of the section visible at V . The silent law on that pulled-back question removes the redundant glue. At the retained member p , the result is exactly the original component of m .

This calculation works at every member of the sieve and respects every arrow between members. The pointwise paths therefore assemble into a whole transformation

$$\rho_R \circ \text{glue}_R \Longrightarrow \text{id}_{\text{Match}_{aP}(R)},$$

and the strict pointwise-to-whole principle closes it to (20.4). No new naturality square is assumed at this stage. The necessary compatibility came from the one displayed glue functor in the categorical-HIT signature.

Theorem 20.1 (locality of cover completion). For every Cat-valued presheaf P on the site $(\mathcal{K}, J)$, the direct cover completion aP is local at every J -covering sieve. Hence aP is a Cat-valued sheaf.

Formal status — checked. Evidence `DIRECT-COVER-COMPLETION-LOCALITY`. The active derivation constructs the restriction-after-glue path from cover membership, canonical pullback, whole glue naturality, and silent; combines it with the primitive glue-after-restriction path; and produces the existing two-sided fixed-forward locality interface at every eligible question and over the whole topology. It assumes neither generic functor extensionality nor a category-of-elements retraction.

This proof explains why the chosen constructor set is not merely mnemonic. Return begins the free object. Glue supplies a candidate inverse to restriction. Silent gives one inverse law. Sieve stability and change of stage force the other. The sheaf condition emerges from the interaction of the constructors with the site, not from attaching the word “sheaf” to the result.

Existence and uniqueness are therefore not two unrelated postulates. Glue is the existence mechanism. The two inverse laws say that its output is exactly the global datum classified by the matching family and that an already-global datum is unchanged. In the Cat-valued setting those laws also control arrows between families, so “unique” means unique with the categorical coherence visible to the Hom categories, rather than merely unique after forgetting all maps. That coherence is part of the theorem, not expository shorthand.

20.4 The Recursor

Locality shows that aP lands in the right class of objects. Freeness asks whether it lands there in the right way.

First consider a Cat-valued presheaf Y equipped with its own coherent glue and silent operations. Given a seed map

$$f : P \longrightarrow Y,$$

the recursor produces

$$\text{rec}_Y(f) : aP \longrightarrow Y.$$

It is determined by three clauses. On returned data it extends the seed:

$$\text{rec}_Y(f) \circ \eta_P = f. \quad (20.5)$$

On a recursive glue, it first maps the local family into Y and then uses Y 's glue. On the silent path, those two ways of passing through glue agree with the silent path already carried by Y . The first two clauses describe objects and recursive computation; the third is the higher coherence that makes the result a map of the complete algebraic structures.

A topology-local presheaf has canonical such structure. Its restriction functor is an equivalence, so its chosen inverse supplies glue, and one inverse law supplies silent. Thus every sheaf Y is a legitimate target of the recursor without asking a reader to choose fresh gluing operations cover by cover.

The objectwise statement “every seed extends” is still weaker than a reflective universal property. Maps between seed maps must extend too, and the extension must be inverse to restriction on the whole Hom category. For a local target Y , precomposition with the unit is a functor

$$\eta_P^* : \text{Hom}(aP, Y) \longrightarrow \text{Hom}(P, Y), \quad h \longmapsto h \circ \eta_P.$$

Recursion varies functorially in the seed and gives a functor in the opposite direction. Equation (20.5) is the beta law

$$\eta_P^* \circ \text{rec}_Y = \text{id}_{\text{Hom}(P, Y)}.$$

The categorical-HIT uniqueness law gives the eta law

$$\text{rec}_Y \circ \eta_P^* = \text{id}_{\text{Hom}(aP, Y)}.$$

These are equalities of whole functors between Hom categories. They include the action on arrows between presheaf maps, not only a bijection between object-level maps.

Theorem 20.2 (free local completion). If Y is topology-local, then precomposition with η_P is an omega-equivalence

$$\text{Hom}(aP, Y) \simeq \text{Hom}(P, Y). \quad (20.6)$$

Formal status — checked. Evidence `DIRECT-COVER-COMPLETION-UNIVERSALITY`. The recursor extends a whole seed map, computes on return and recursive glue, and carries explicit coherence for the silent path. It varies functorially in the seed. At a topology-local target, whole beta and eta laws exhibit unit precomposition as an omega-equivalence of complete Hom categories. The eta law is scoped to local targets; no uniqueness theorem is asserted for an arbitrary, independently chosen one-sided glue algebra.

Equation (20.6) is the point at which completion becomes sheafification. Many objects might be made local by adding arbitrary data. Only a free local object has this mapping property. The unit remembers how the input enters, and every map to a local target factors through it in the uniquely coherent way.

20.5 From The Universal Property To A Reflector

Let $\mathbf{Sh}_{\mathbf{Cat}}(\mathcal{K}, J)$ denote the category whose objects are $\mathbf{Cat}$ -valued presheaves equipped with topology-locality evidence and whose maps are their underlying presheaf maps. There is an inclusion

$$i : \mathbf{Sh}_{\mathbf{Cat}}(\mathcal{K}, J) \longrightarrow \mathbf{Psh}_{\mathbf{Cat}}(\mathcal{K}).$$

Theorem 20.1 lets $P \mapsto aP$ land in the sheaf category. The recursor makes this assignment functorial on presheaf maps: given $f : P \rightarrow Q$, compose it with η_Q and extend the resulting seed $P \rightarrow aQ$ across aP . Thus there is a functor

$$a : \mathbf{Psh}_{\mathbf{Cat}}(\mathcal{K}) \longrightarrow \mathbf{Sh}_{\mathbf{Cat}}(\mathcal{K}, J).$$

The Hom equivalence (20.6), now read with $Y = iS$ for a sheaf S , is exactly the adjunction

$$a \dashv i.$$

Its unit at P is the return map η_P . Its counit at a sheaf S is the recursor from the identity seed on the underlying presheaf:

$$\varepsilon_S : a(iS) \longrightarrow S.$$

The return beta law shows

$$\varepsilon_S \circ \eta_{iS} = \mathrm{id}_{iS},$$

and the local-target uniqueness law gives the other cancellation. Hence the counit is an equivalence. Sheafification does not alter a sheaf except up to the native categorical equivalence appropriate to this development.

This also explains the familiar idempotent flavor of sheafification. Applying a to an arbitrary P produces a local object. Applying a again therefore meets an object already in the reflective subcategory, and the counit compares $a(aP)$ back to aP by an equivalence. The statement is not that the two presheaves collapse by raw syntax. Reflective idempotence is expressed by the adjunction and its invertible counit, at the same categorical strength as the rest of the theory.

Theorem 20.3 (Cat-valued sheafification reflector). For every site $(\mathcal{K}, J)$, direct cover completion defines a left adjoint to the inclusion of topology-local $\mathbf{Cat}$ -valued presheaves. The counit on every sheaf is an omega-equivalence, so this adjunction is reflective.

Formal status — checked. Evidence `CAT-VALUED-SHEAFIFICATION-REFLECTOR`. The active construction realizes the $\mathbf{Cat}$ -valued sheaf facade as topology-local presheaves, constructs the inclusion and completion functors, supplies their whole adjunction, reduces its unit to return and its

counit to recursion from the identity seed, and proves the two counit cancellations and reflector capability. This is a fixed-site Cat-valued result. It does not yet supply arbitrary coefficient categories, a commutative-ring lift, left exactness, or base-change comparison.

20.6 Categorical Semantics As An Executable Language

There are two legitimate ways to speak about sheafification in type theory. One may design an internal language with a modality whose semantics is a sheaf reflector. Or one may express the categorical semantics itself in a formal language rich enough to compute with categories, presheaves, sieves, and universal properties. This book follows the second route.

The distinction is about where the formalization lives, not about whether it is internal. Within functorial type theory, $\widehat{R}$, yU , matching categories, section categories, glue, and the adjunction are genuine objects and arrows. They are manipulated from inside the categorical theory. That theory in turn lives inside an outer logical framework—`Lambdapi` in the active oracle, and increasingly the explicit emdash Core checked by TypeScript. The outer layer supplies binders, conversion, rewrite rules, unification, and proof checking. Selected categorical observations can therefore reduce and be compared without inventing a second modal surface language first.

The division of labor can be read from one glued section. Its matching family and section are inner categorical objects. The fact that glue is natural in a change of covering question is inner functorial structure. The rule saying how a recursor acts on return or on recursive glue belongs to the outer computation theory. The equality witnessing silent is again an inner path, whose formation and use are certified by the outer checker. Neither level is dispensable, but neither needs to masquerade as the other.

This is **relative internality**. The sheaf construction is internal to the categorical semantics and executable relative to the surrounding logical framework. It is neither an informal metatheoretic diagram nor a claim that all categorical equalities are definitional. Return has a narrow computation rule under recursion; recursive glue has another; silent remains first-class equality evidence; the Hom equivalence packages explicit inverse data.

An internal modal language may still be desirable. It can hide repeated site parameters, support concise local reasoning, or make sheaf semantics available to a different class of programs. What the direct construction shows is that such a language is not a prerequisite for computational sheafification. The ordinary site can remain ordinary, the sieves can remain actual sieves, and the categorical universal property can itself be checked.

This viewpoint also clarifies the comparison with Pédrot. His return/glue/silent pattern reveals the computational content that a purely reflective statement can conceal. Emdash preserves that pattern but moves its indices outward to the category of covering questions and its values upward to categories. The result is not a transcription of his type theory. It is a categorical realization of the same free-construction idea, with whole naturality and whole Hom universality made explicit.

20.7 What Has And Has Not Been Completed

At this point the word *sheafification* is justified for $\mathbf{Cat}$ -valued presheaves on a fixed site. The result is local, functorial, left adjoint to inclusion, and reflective. None of those adjectives should be silently moved to a different coefficient category.

In particular, a commutative-ring-valued presheaf carries operations that must survive completion and continue to satisfy their equations. A reflector on underlying $\mathbf{Cat}$ -valued presheaves does not by itself construct those lifted operations. Nor has the present theorem established left exactness, preservation of finite limits, compatibility with change of site, or a general comparison with classical plus-construction sheafification. Those are separate mathematical obligations.

There is also no claim here about sheafifying every universe of coefficients uniformly. The value category has been fixed to **Cat**, and the sheaf facade has been realized concretely as a presheaf paired with locality evidence. Moving to sets, groupoids with a different equality policy, commutative rings, or a universe of small categories requires a corresponding lifting theorem. The fixed-site theorem is substantial precisely because it keeps that quantification honest.

The next chapter therefore steps sideways before moving further into geometry. It develops commutative algebra by universal property: products, localizations, and the evidence that a ring map sends a chosen element to a unit. That language will let the invertibility sieve $D_U(s)$ of Chapter 18 control affine charts without pretending that $\mathbf{Cat}$ -valued sheafification has already manufactured a structure sheaf of rings.

The local-to-global spiral now has its completion step:

probe $\longrightarrow$ sieve $\longrightarrow$ cover $\longrightarrow$ matching family $\longrightarrow$ free glue $\longrightarrow$ reflective sheaf.

What comes next is the algebra that turns these local questions into affine geometry.

21. Commutative Algebra By Universal Property

Algebraic geometry begins with rings, but it rarely cares how a ring was manufactured. A polynomial algebra may be presented by finite expressions; a localization may be presented by fractions; a quotient may be presented by equivalence classes. These descriptions are indispensable for hand calculation. They are not, however, the invariant meaning of the objects they describe. Change the representation and the same algebra survives.

This matters acutely in a computational foundation. A convenient syntax can make examples reduce, yet it can also make every later construction depend on accidental choices of normal form. Conversely, a universal property can be stated so weakly that it says merely that a map exists, leaving no usable factor and no uniqueness with which to compare two constructions. The middle course taken here is to make the universal property itself data of the theory. The space of admissible factors is not merely inhabited: it is contractible. It therefore has a selected center and a path from every competitor to that center.

That principle gives a representation-free but still computational account of the algebra needed by the geometry ahead. Commutative rings have set-valued carriers and structured maps. Finite unit-ideal certificates carry the algebraic content of basic-open covers. Polynomial algebras are free extensions, and localizations are initial ways of making a chosen element invertible. Unit, zero, and idempotent localizations then show that the interface is not empty formalism. Finally, uniqueness alone constructs the comparison between localization at a product and localization in two stages.

21.1 Rings As Structured Set-Carriers

A commutative ring R consists first of a set $|R|$, with operations

$$0_R, 1_R : |R|, \quad +_R, \cdot_R : |R| \times |R| \longrightarrow |R|, \quad -_R : |R| \longrightarrow |R|,$$

and then the usual associativity, commutativity, unit, inverse, and distributivity laws. Calling the carrier a set is not decorative. It says that equality proofs between ring elements carry no higher ambiguity. The algebraic laws are consequently properties of the selected operations rather than new layers of structure that can vary above a fixed equality.

The definition does **not** require $0_R \neq 1_R$. This convention retains the zero ring, whose carrier has one element and whose two distinguished constants coincide. Excluding it would make some later universal statements awkward or false. Localization at zero, for example, naturally lands in the zero ring: forcing zero to be a unit forces every element to coincide.

A map $h : R \rightarrow S$ is more than a function of carriers. It comes with the five preservation laws

$$h(0) = 0, \quad h(1) = 1, \quad h(x + y) = h(x) + h(y), \quad h(-x) = -h(x), \quad h(xy) = h(x)h(y).$$

Because the target carrier is a set, this preservation evidence is proposition-valued. Two structured maps are equal once their carrier functions agree pointwise. Identities and composites again preserve all five operations, so commutative rings and their structured maps form a one-category, written **CRing**.

This choice of morphism is part of the mathematics. A bare function between carriers forgets the equations that make substitution legitimate. A structured map carries those equations once, after which every derived finite sum, product, unit witness, and factorization can be transported through it. Later base-change arguments therefore do not reopen the ring laws term by term. They use the fact that the arrow already lives in **CRing**.

Sethood plays a second role here. A structured map contains both a function and proofs that it respects the operations. If those proofs carried independent higher data, pointwise equality of the functions would not settle equality of the complete maps. Since equality in the target ring is proposition-valued, preservation proofs create no such ambiguity. This is why the ordinary-looking extensionality principle is available without flattening the ambient functorial type theory.

There is a useful restraint here. Extensionality for maps is not a global principle saying that any two ring packages with equivalent carriers are equal. No structure identity principle for arbitrary rings is being smuggled in. The objects remain chosen carrier-operation-law packages; the homs have the extensionality required to calculate with them.

Formal status — checked. Evidence `COMM-RING-STRUCTURED-CATEGORY`. The active algebra packages a set-valued carrier, operations, and ring laws; admits the zero ring; packages operation-preserving carrier functions as structured maps; proves their extensionality; and assembles them into the one-category **CRing**. No inequality $0 \neq 1$, global equality of ring packages, or general structure identity theorem is assumed.

Several small models keep this abstraction honest. The zero ring computes on the one-point carrier. The two-element ring $\mathbb{F}_2$ computes on the booleans, with exclusive-or as addition and conjunction as multiplication. If R and S are rings, their cartesian carrier $|R| \times |S|$ has componentwise operations and hence a ring structure $R \times S$. A pair of maps induces a componentwise map between such products, and these maps obey whole identity and composition paths.

This last construction should not be overread. We have constructed a product ring and functorial action on paired maps. We have not yet selected projections and proved the entire categorical product universal property inside **CRing**. The componentwise model is exactly what the later split-idempotent calculation needs, and no stronger theorem is required for that calculation.

21.2 Finite Certificates For Covering

The first geometric-looking datum is still entirely algebraic. Let $f_1, \dots, f_n$ be elements of a ring R . They are **unimodular** when one retains coefficients $a_1, \dots, a_n$ and an equality

$$a_1 f_1 + \dots + a_n f_n = 1. \tag{21.1}$$

The coefficients are important. The bare proposition that the f_i generate the unit ideal forgets how that fact was witnessed. Equation (21.1), by contrast, is finite input that can be transported, inspected, and used in a construction. The family together with its coefficients and equality is a **unimodular presentation**.

For two generators the picture is especially sharp. From $af + bg = 1$, any map out of R that makes both f and g vanish would also make 1 vanish. Unless the target has collapsed to the zero ring, that is impossible. Geometrically, no nontrivial affine point can lie outside both regions of invertibility at once. The equation is already the finite algebraic shadow of a covering statement, even though the regions and their topology remain to be constructed.

Every ring map $h : R \rightarrow S$ transports such a presentation. Preservation of finite sums and products turns (21.1) into

$$h(a_1)h(f_1) + \cdots + h(a_n)h(f_n) = 1$$

in S . Thus a certificate does not merely remain true after base change; its chosen witnesses move pointwise with the generators. The singleton family $[1]$ has the canonical certificate $1 \cdot 1 = 1$, and a two-element helper packages the familiar equation $af + bg = 1$.

Why call this cover data? In ordinary affine geometry, (21.1) says that the basic opens $D(f_i)$ cover the whole spectrum. But that geometric conclusion uses meanings that have not yet been introduced in this spiral: a spectrum, basic opens, a topology, and the relationship between unit-ideal generation and coverage. The present layer records precisely the algebraic premise from which those notions will be built. It does not call a finite family a cover by fiat.

There is a second reason to retain the presentation rather than immediately truncate it to a proposition. Different coefficient families may witness the same unit-ideal equation. The classifier of presentations is set-valued, not claimed proposition-valued. Later invariant constructions may forget that choice; earlier computational constructions are allowed to consume it. This separation between witness-rich input and invariant output is the same pattern used for generated topologies in Chapter 19.

Formal status — checked algebraic boundary. Evidence `FINITE-UNIMODULAR-COVER-DATA`. Finite sums and dot products compute on visible families, structured ring maps preserve them, and a finite Zariski presentation retains generators, coefficients, and their unit-ideal law. Such presentations are stable under structured base change and include the singleton $[1]$ and binary $af + bg = 1$ cases. At this layer they are not yet basic opens, covering sieves, localization families, or a Grothendieck topology.

21.3 Free Variables Without A Syntax Of Polynomials

Fix a ring R and a set or groupoid X of variable names. A polynomial algebra on X should contain a base map

$$\iota : R \longrightarrow P$$

and a valuation $v : X \rightarrow |P|$. Its meaning is determined by what happens when the variables are interpreted elsewhere. Given a ring S , a base map $h : R \rightarrow S$, and a valuation $u : X \rightarrow |S|$, consider structured maps $k : P \rightarrow S$ satisfying

$$k \circ \iota = h, \quad k(v(x)) = u(x) \quad (x : X). \quad (21.2)$$

Both equations are retained pointwise, and the complete factor consists of k together with those agreements. The universal property says that the classifier of such factors is contractible for every S , h , and u . There is therefore one coherently selected extension, and every rival extension is equal to it as a structured map with its agreement evidence.

Contractibility is stronger than the phrase “there exists a unique map” when that phrase is read externally. It gives a center of the complete factor classifier and, internally, a path from every other inhabitant to that center. The center can be projected whenever an actual extension is needed; the contraction can be invoked whenever two independently constructed extensions must agree. Because the classifier retains the equations in (21.2), uniqueness does not forget that the comparison lies over R and has the prescribed values on variables.

This is the familiar freeness of $R[X]$, but it does not select a representation of its elements. There is no list of monomials, finitely supported coefficient function, inductive expression grammar, quotient by the ring laws, or preferred normalization order. Those are possible models of the universal property, not fields of the property itself.

The omission is not hostility to syntax. A concrete evaluator might sensibly represent a polynomial by normalized coefficient data, and a parser might let a reader write $x^2 + 2x + 1$. What matters is the direction of dependence. Such a representation should prove that it satisfies (21.2); the geometry should then consume the universal property. The later theory is insulated from whether one implementation uses Horner forms, sparse monomials, or an external computer-algebra package.

For a single variable t , this says that a map out of a supplied $R[t]$ is determined by two observations: what it does to coefficients and where it sends t . Familiar evaluation is recovered by choosing the target value of t . For many variables the same sentence is indexed by X , with no need to choose an ordering at the universal boundary. The interface states exactly what symbolic substitution is meant to accomplish while declining to legislate how symbols are stored.

There is already a closed sanity check. When X is empty, there is no variable data to choose. The ring R itself, with the identity base map, satisfies the polynomial universal property: every $h : R \rightarrow S$ is its own unique extension. This case exercises the complete factor classifier and its contractibility, not merely the formation of a record.

Theorem 21.1 (universal polynomial extension). A supplied polynomial algebra package on (R, X) classifies extensions of every base map and valuation by a contractible factor space. For the empty variable classifier, the identity extension on R supplies a checked model.

Formal status — checked interface and closed model. Evidence `COMM-RING-POLYNOMIAL-UNIVERSALITY`. The active universal property retains base and variable agreements and proves the complete extension space contractible. The empty-variable identity model is checked. No construction of a polynomial algebra for every R and X , concrete monomial syntax, quotient presentation, normalization theorem, runtime rule, or package uniqueness theorem is claimed.

21.4 Making One Element Invertible

Let $f \in R$. A localization of R at f begins with a ring L and a map

$$\ell : R \longrightarrow L$$

for which $\ell(f)$ is a unit. Unit evidence is explicit: it consists of an inverse y and an equality $\ell(f)y = 1$. Commutativity proves that the inverse is unique, and sethood of the carrier proves that the entire evidence is a proposition. Asking for a unit therefore does not introduce a meaningful choice of inverse into the geometry.

The uniqueness calculation is elementary but instructive. If y and z are both inverses to x , then

$$y = y \cdot 1 = y(xz) = (yx)z = (xy)z = 1 \cdot z = z.$$

Associativity and commutativity do the algebraic work; sethood then says that the displayed equality has no further choices. The predicate “ x is invertible” may consequently be used as the fibre of an ordinary sieve, as in Chapter 18, rather than as a higher coefficient carrying distinct inverse witnesses.

The universal property considers any map $h : R \rightarrow S$ for which $h(f)$ is a unit. A factor is a structured map $k : L \rightarrow S$ together with the pointwise triangle

$$k(\ell(x)) = h(x) \quad (x \in R). \tag{21.3}$$

The localization property says that this factor space is contractible. We may write the selected target suggestively as $R[1/f]$, while remembering that the notation names the role of L , not a fraction grammar inside its carrier. Then (21.3) is the invariant content of the usual substitution

$$\frac{x}{f^n} \longmapsto h(x)h(f)^{-n}.$$

The displayed fraction explains the classical formula; it is not used to define the map. Contractibility supplies the map and all comparisons between maps satisfying the same triangle without choosing numerators, denominators, or exponents.

Admissibility belongs to the target map h . The localization package does not choose, for every ring in the universe, whether the image of f is a unit. Rather, when a caller supplies unit evidence, the universal property returns the unique factor. This keeps constructive content visible: testing invertibility

may be undecidable, but using an explicit proof of invertibility is computationally straightforward.

This stronger uniqueness is what makes the interface computationally useful. From a contractible factor space one projects a center, hence an actual map $R[1/f] \rightarrow S$. Given another construction with the same universal property, one obtains maps in both directions. Their composites and the identities are competitors in suitable factor spaces, so uniqueness produces the inverse laws. The universal property is thus a source of programs and equations, not an after-the-fact slogan attached to an opaque object.

It also separates existence from characterization. A **localization package** contains a chosen L , a chosen structure map, its unit evidence, and the contractible factorization theorem. The general interface says what any such package does. It does not yet construct a package for every pair (R, f) . That remaining existence problem can be solved by a fraction model, by a quotient, by a suitable higher-inductive construction, or by importing a certified algebra library, without changing the consumers of localization.

Formal status — checked interface. Evidence `COMM-RING-LOCALIZATION-UNIVERSALITY`. Unit evidence is proposition-valued, and a supplied localization package inverts the chosen element and gives a contractible space of structured factors through every admissible target map. The factor retains a whole ring map and its pointwise triangle. No general existence theorem for arbitrary (R, f) , fraction or power representation, quotient syntax, or equality of arbitrary localization packages is asserted.

21.5 Three Localizations One Can See

The abstract interface has concrete edges where no fraction construction is needed.

First suppose f is already a unit in R . The identity map $R \rightarrow R$ is a localization at f . Every admissible map $h : R \rightarrow S$ factors through the identity by h itself, and extensionality makes that factor unique. In particular, every ring has a canonical identity localization at 1.

At the opposite extreme, localize at 0. If $h : R \rightarrow S$ sends zero to a unit, then 0_S is invertible. Since a ring map preserves zero, one obtains $0_S = 1_S$, and hence every element of S equals zero. The target is contractible as a carrier. It follows that the unique map from R to the zero ring has the localization property at 0. The zero ring is not a nuisance case patched into the theory; it is the correct universal endpoint.

The third case is more revealing. Let $e \in R$ be idempotent, so $e^2 = e$. Consider the fixed-image carrier

$$eR = \{x \in R \mid ex = x\}.$$

It is closed under the inherited additive operations and multiplication. Its zero is 0_R , while its multiplicative unit is e . Scaling defines a ring map

$$R \longrightarrow eR, \quad x \longmapsto ex. \quad (21.4)$$

The image of e under (21.4) is the unit of eR , and any map that makes e invertible factors contractibly through this fixed image. Thus eR is a localization at e , constructed without a quotient and without fractions.

The factor has a simple formula. If $h : R \rightarrow S$ makes the idempotent e invertible, then idempotence of $h(e)$ and cancellation by its inverse force $h(e) = 1$. On an element x fixed by multiplication by e , the factor sends x to $h(x)$. Conversely, the original element r reaches the fixed image as er , and then

$$h(er) = h(e)h(r) = h(r),$$

which is the required triangle. The fixed-point equation retained in the carrier supplies exactly the coherence needed for this formula to define a structured map.

Take now $R = \mathbb{F}_2 \times \mathbb{F}_2$ and $e = (1, 0)$. Componentwise multiplication makes e idempotent, yet boolean discrimination proves $e \neq (0, 0)$ and $e \neq (1, 1)$. Its fixed image consists of the first component with the second forced to zero. This is a closed, genuinely non-endpoint localization: neither the identity localization nor the zero localization is being disguised by notation.

Formal status — checked models. Evidence `COMM-RING-LOCALIZATION-MODELS`. The identity ring localizes at any supplied unit and canonically at 1; the zero ring localizes every ring at 0; and the fixed image eR localizes at an idempotent e . The product ring $\mathbb{F}_2 \times \mathbb{F}_2$ supplies a checked idempotent $(1, 0)$ distinct from both endpoints, so the fixed-image construction has a concrete nondegenerate instance. These models do not amount to arbitrary localization existence.

21.6 Localizing Once Or Twice

Universal properties earn their keep when two descriptions must be compared. Choose a localization of R at f , and then localize its target at the image of g . Also choose a localization of R at the product fg . In customary notation the two targets are

$$R[1/f][1/g] \quad \text{and} \quad R[1/(fg)].$$

No fraction calculation is needed to compare them. In the iterated target, the images of both f and g are units, so their product is a unit. The universal property of $R[1/(fg)]$ therefore gives a forward map

$$\Phi : R[1/(fg)] \longrightarrow R[1/f][1/g].$$

Conversely, if fg is invertible in a commutative ring, then both f and g are invertible: an inverse to f is $g(fg)^{-1}$, and symmetrically for g . The product localization consequently admits first a factor through $R[1/f]$ and then a factor through the localization at the image of g . This gives

$$\Psi : R[1/f][1/g] \longrightarrow R[1/(fg)].$$

This elementary implication is the hinge of the comparison. If w is an inverse to fg , then

$$f(gw) = (fg)w = 1, \quad g(fw) = (fg)w = 1.$$

It converts one unit witness into the two witnesses demanded by the staged universal properties. No appeal to prime ideals, open subsets, or a spectrum is involved; the overlap theorem is already present in commutative algebra.

The two composites are not reduced by a hidden fraction normalizer. Instead, $\Psi\Phi$ and the identity are factors of the same map through the product localization, so contractibility identifies them. The other direction needs one additional step: uniqueness at the first localization aligns the maps on the intermediate ring, after which uniqueness at the second localization identifies $\Phi\Psi$ with the identity. Both results are equalities of whole structured ring maps.

Theorem 21.2 (product and iterated localization). For any supplied localizations in the preceding configuration, the canonical comparison maps satisfy both whole cancellation laws and exhibit an omega-equivalence in **CRing**,

$$R[1/(fg)] \simeq R[1/f][1/g]. \quad (21.5)$$

Formal status — checked. Evidence `COMM-RING-ITERATED-LOCALIZATION-EQUIV`. The active comparison constructs canonical forward and reverse structured maps from the supplied universal properties. Contractibility proves their left and right whole-map laws and packages the selected forward map as an omega-equivalence in **CRing**. Equation (21.5) does not identify the carrier packages by raw equality, provide fraction computation, or choose either localization globally.

This proof displays the intended style of computation. A representation-first development might multiply fractions and cancel powers until both composites normalize. Here the observable computation is composition of structured maps, and the universal property closes the comparison. Concrete models remain free to normalize fractions internally; nothing downstream is allowed to depend on that choice.

21.7 From Localization To The Sieve $D(f)$

Localization answers a transformational question: what is the universal ring under R in which f has become invertible? The sieve of Chapter 18 answers a relational question: along which probes is the image of f already invertible? These are two faces of the same algebraic event.

Given a ring map $u : R \rightarrow S$, membership in the sieve $D_R(f)$ is unit evidence for $u(f)$. Whenever a localization $R \rightarrow R[1/f]$ has been supplied, the factorization theorem turns that membership into a structured map

$$R[1/f] \longrightarrow S$$

over R , and contractibility makes all choices of such a factor coherently unique. Conversely, any map over R carries the selected inverse of the localized image of f to an inverse of $u(f)$. The localization therefore represents the question posed by the invertibility sieve when a representing object is available.

The order of ideas is deliberate. The sieve $D_R(f)$ exists from the invertibility predicate alone; it need not wait for a chosen fraction object. A supplied localization then represents that sieve on affine points. Thus the geometry can be organized around **invertibility's sieve**, while localization supplies a computational chart rather than defining openness by decree.

The next chapter makes this bridge precise. It constructs the affine functor of points, reads $D(f)$ as an ordinary sieve on an affine, and shows how unimodular families become finite basic-open covers. The algebra developed here will reappear there not as an internal manual of ring operations, but as the universal language in which affine geometry recognizes its charts.

22. Affine Geometry And The Sieve $D(f)$

An affine scheme is often introduced as a set of prime ideals equipped with a topology and a sheaf. That description is powerful, but it begins at the end of several constructions. Constructively, even the set of prime ideals can be difficult to use: the familiar argument that a proper ideal lies in a prime ideal invokes a choice principle that one may have deliberately declined. More importantly for the present book, a point-set spectrum hides the functorial action that makes geometry computational.

There is another beginning. Instead of asking first which ideal-valued points a ring has, ask how the ring can be mapped into every test ring. Instead of declaring a subset on which an element is non-zero, ask along which of those maps its image becomes invertible. The answers are already organized by composition. They form a functor of points and, inside it, an ordinary sieve

$$D_R(f) = \{h : R \longrightarrow S \mid h(f) \text{ is invertible in } S\}.$$

This sieve is the geometric form of localization. A chosen localization $R \rightarrow R[1/f]$ represents its points: maps out of $R[1/f]$ are exactly maps out of R along which f is a unit. Products encode intersections, $D_R(fg) = D_R(f) \cap D_R(g)$, and finite unit-ideal presentations generate the Zariski topology on the big affine slice. Only after those constructions are in place do we ask for a structure sheaf and its locality.

The order matters. The sieve exists before it is represented by one chart; the topology exists before a sheafification for commutative-ring values has been constructed; and the computing coordinate presheaf exists before it is equipped with supplied sheaf and locality witnesses. Affine geometry becomes executable without pretending that each of its classical existence theorems has already been rebuilt.

22.1 Affines As Questions Of Points

Let $\mathbf{Aff} = \mathbf{CRing}^{\text{op}}$. A ring map $h : R \rightarrow S$ is read geometrically in the opposite direction,

$$\text{Sp}(S) \longrightarrow \text{Sp}(R).$$

The notation $\text{Sp}(R)$ is deliberately functorial. At a test ring S , define

$$\text{Sp}(R)(S) = \text{Hom}_{\mathbf{CRing}}(R, S). \quad (22.1)$$

If $\alpha : S \rightarrow T$, composition sends a point $h : R \rightarrow S$ to $\alpha \circ h : R \rightarrow T$. Thus (22.1) is not a disconnected family of sets. It is the representable presheaf on $\mathbf{Aff}$, with all change-of-test-ring maps supplied by Yoneda.

This definition changes the meaning of “point” in a useful way. A point is not required to land in a field, an algebraic closure, or a two-valued truth object. Every ring S is an admissible stage of observation. Nilpotents, idempotents, infinitesimal extensions, and degenerate rings are visible when the

chosen tests can detect them. A classical prime ideal may later be recovered through a suitable field-like test, but it does not monopolize the notion of observation.

Generalized points can distinguish phenomena that field-valued points erase. For example, a nilpotent element must map to zero in every reduced field, but it may remain visible under a map to a ring with nilpotents. Likewise, an idempotent can reveal a decomposition through product test rings even when a single ordinary point sees only one side. The functor of points does not force one preferred class of tests to carry the entire geometry. It records the response at all stages and lets later hypotheses specify which stages are sufficient for a particular theorem.

There is a useful logical economy in this approach. To compare two candidate affine constructions, one may compare how maps into every test ring are classified rather than inspect a chosen point-set representation. Conversely, an explicit test ring can refute an overstrong identification by exhibiting a point seen on one side and not the other. The functor is therefore both an extensional language and an experimental instrument. The representation theorem below uses the second role directly: it constructs and compares actual maps at an arbitrary supplied test ring.

The functor of points also remembers direction for free. Suppose $R \rightarrow S \rightarrow T$ is a commuting triangle of ring maps. The corresponding geometric arrows compose in the reverse order, and evaluating $\mathrm{Sp}(R)$ simply composes the ring maps. No separate substitution operation has to be appended to the set of points, and no proof is needed that this operation respects identity and composition: those laws are the ordinary Yoneda action already developed in Chapters 13 and 18.

One should not confuse this representable with the completed geometric object traditionally denoted $\mathrm{Spec}(R)$. At this stage we have a functor of affine tests. We have not yet selected a Zariski topology, a structure sheaf, a local-ring condition, or a comparison with a prime-ideal spectrum. The notation records the intended geometry while leaving each additional layer visible.

22.2 Invertibility Is A Sieve

Fix $f \in R$. For every test ring S , let

$$D_R(f)(S) = \sum_{h: R \rightarrow S} \mathrm{Unit}_S(h(f)), \quad (22.2)$$

where $\mathrm{Unit}_S(x)$ is the proposition that x has a multiplicative inverse. An element of (22.2) is therefore a point h together with actual evidence that the image of f is invertible.

The construction is stable under refinement of the test. If $\alpha : S \rightarrow T$, a unit witness for $h(f)$ is carried by α to a unit witness for $\alpha(h(f)) = (\alpha \circ h)(f)$. Hence every further probe of a $D_R(f)$ -point is again a $D_R(f)$ -point. In the geometric category **Aff**, this downward closure says precisely that $D_R(f)$ is a sieve on $\mathrm{Sp}(R)$.

Unit evidence is proposition-valued, so the sieve is ordinary rather than a higher sieve with a non-trivial category of witnesses. The selected inverse is retained during a calculation, but any two inverse witnesses agree. The predicate can therefore be used as a subterminal fibre of the representable without erasing computational access to the inverse when it is needed.

This definition avoids two premature decisions. It does not ask whether invertibility is decidable, and it does not ask whether all the successful probes factor through one previously chosen open object. A caller may supply a unit witness even when no decision procedure exists. The resulting collection of successful probes is meaningful even when no single object represents it.

Invertibility is stronger than nonvanishing. Over a field the two conditions coincide away from zero, which can make the distinction disappear in a point-set picture. Over a general test ring an element may be nonzero without being a unit. The basic open $D_R(f)$ therefore does not classify probes on which f merely survives; it classifies probes on which f has become reversibly usable. That is precisely the condition needed for a map out of a localization.

Two endpoint examples calibrate the definition. Every ring map preserves one, and one has a canonical inverse, so $D_R(1)$ is the maximal sieve. By contrast, membership in $D_R(0)$ forces zero to be a unit in the test ring and hence forces that ring to collapse. The latter sieve is not literally empty when the zero ring is admitted, but it is geometrically empty relative to nondegenerate tests. The convention from Chapter 21 makes this boundary exact rather than exceptional.

In a posetal presentation of geometry, a sieve represented by an open $V \leq U$ consists of precisely the probes that factor through V . Max Zeuner's constructive development organizes affine geometry through the Zariski lattice of compact opens and, in a locally ringed lattice, assigns a section its largest compact-open invertibility support. From the present viewpoint, that compact open is a particularly economical representative of the sieve $D_U(s)$ when such a representative exists. The sieve is primary on a general site; the compact open is the coherent or qcqs form in which all of its probes can be summarized by one lattice element.

This chapter is structurally adapted from the Zariski-lattice, coverage, compact-open, and functor-of-points development in [Zeuner](#). The change of viewpoint is substantial: rather than define invertibility's locus first as a compact open, we define the ordinary sieve of all invertibility probes and ask for representability afterward. Nothing in that change invalidates the compact-open account in its intended scope.

Formal status — mathematical development and attribution boundary. The comparison with compact opens is an attributed reformulation of Zeuner's presentation. The active construction at this point is the ordinary sieve (22.2). No general theorem that $D_U(s)$ is compactly representable, no comparison with Zeuner's qcqs schemes, and no point-set spectrum theorem is claimed.

22.3 Localization Represents The Basic Open

Now suppose a localization has been selected,

$$\iota_f : R \longrightarrow R[1/f].$$

The notation again names a universal role rather than a fraction syntax. By definition, $\iota_f(f)$ is a unit, and every map $h : R \rightarrow S$ for which $h(f)$ is a unit factors contractibly through ι_f .

There are two immediate constructions at every test ring S . A map $k : R[1/f] \rightarrow S$ gives

$$\Phi_S(k) = (k \circ \iota_f, k(\iota_f(f)) \text{ is a unit}) \in D_R(f)(S). \quad (22.3)$$

Conversely, a point $(h, u) \in D_R(f)(S)$ is exactly an admissible target for the localization property. The center of its contractible factor space gives a structured map

$$\Psi_S(h, u) : R[1/f] \longrightarrow S \quad \text{with} \quad \Psi_S(h, u) \circ \iota_f = h. \quad (22.4)$$

The inverse laws require no calculation with fractions. Starting with k , the map k itself is a competitor in the factor space used to select $\Psi_S \Phi_S(k)$. Contractibility identifies the selected factor with k as a whole structured map. Starting with (h, u) , the factor triangle identifies the map component of $\Phi_S \Psi_S(h, u)$ with h ; proposition-valued unit evidence then completes the equality of the dependent pairs.

Theorem 22.1 (pointwise representation of the basic open). For every test ring S and every supplied localization of R at f , the maps (22.3) and (22.4) form an explicit equivalence

$$\mathrm{Hom}_{\mathbf{CRing}}(R[1/f], S) \simeq D_R(f)(S). \quad (22.5)$$

Formal status — checked, pointwise. Evidence AFFINE-BASIC-OPEN-POINT-REPRESENTATION . Both maps and both inverse laws in (22.5) are constructed from the localization universal property, whole ring-map extensionality, and proposition-valued unit evidence. The equivalence is available for each test ring. Both sides retain their functorial action through existing owners, but the active result does not package these components as a whole natural equivalence or equality of presheaves.

That last qualification is not a defect in the mathematical idea. Formula (22.5) is exactly the component expected from representability, and its maps are defined canonically from composition and factorization. A complete presheaf-level equivalence would additionally assemble the components as internal transformations and prove their whole inverse laws in the relevant functor category. The current theorem records the strength that has actually been checked rather than replacing that missing assembly by an external assertion of naturality.

Representability also separates invariance from choice. Two constructions of a ring called $R[1/f]$ need not be definitionally equal as carrier packages. Each nevertheless classifies the same factorization problem, so their universal properties construct comparison maps and the relevant inverse laws. Geometry may consequently use “the chart $D(f)$ ” without requiring every implementation to share one fraction representation or normal form. What is invariant is the question asked by the sieve; a selected localization is a computational coordinate presentation of it.

This reverses a common expository dependency. If one defines $D(f)$ to be the spectrum of a fraction ring, openness is tied immediately to the chosen construction of fractions. If one begins with (22.2), closure under refinement follows from preservation of units alone. The universal property then proves that a fraction model, quotient model, idempotent fixed-image model, or any other certified localization presents the same geometric question. Representation becomes a theorem with executable maps.

The case $f = 1$ recovers the whole affine: the identity localization at an already invertible element represents the maximal invertibility question. The case $f = 0$ reaches the opposite boundary. Localization at zero is the zero ring, so maps out of it represent precisely those test maps under which zero has become a unit—that is, the degenerate stages where the target ring has collapsed. These endpoints arise from the same universal statement as every other basic open.

22.4 Multiplication Computes Intersection

The elementary algebra of units already knows how basic opens intersect. For a fixed point $h : R \rightarrow S$, if $h(fg)$ is invertible with inverse w , then

$$h(f)(h(g)w) = 1, \quad h(g)(h(f)w) = 1.$$

Thus $h(f)$ and $h(g)$ are both units. Conversely, the product of two units is a unit. Since all three unit predicates are propositions, these operations give an equivalence between unit evidence for $h(fg)$ and paired unit evidence for $h(f)$ and $h(g)$.

Holding the underlying point h fixed and summing over all points yields

$$D_R(fg)(S) \simeq \sum_{h:R \rightarrow S} (\text{Unit}_S(h(f)) \times \text{Unit}_S(h(g))). \quad (22.6)$$

The right side is the explicit pointwise intersection of $D_R(f)$ and $D_R(g)$: one point of the whole affine carrying membership in both sieves. No equality of chosen localization packages is needed. If a localization at fg is supplied, Theorem 22.1 represents the left side, and composition with (22.6) represents the intersection by maps out of $R[1/(fg)]$.

This is the geometric face of Theorem 21.2. The algebraic equivalence

$$R[1/(fg)] \simeq R[1/f][1/g]$$

says that entering $D(f)$ and then $D(g)$ has the same coordinate ring as entering their product open at once. In the functor-of-points picture, equation (22.6) says that the two procedures admit the same tests. The first statement compares coordinate rings by whole structured maps; the second compares membership types at every test ring. Together they explain why multiplication is the algebra of intersection.

Formal status — checked, pointwise. Evidence `AFFINE-BASIC-OPEN-INTERSECTION`. The active maps give the equivalence (22.6), and a supplied localization at fg gives an executable two-step representation of its right side with both component inverse laws. This is not yet a whole-presheaf intersection theorem, an external naturality family, a topology, or an appeal to univalence.

The formula is already the meet law of the Zariski lattice. Zeuner writes the standard compact-open support as $D(fg) = D(f) \wedge D(g)$. Here the same law is seen one probe at a time before a compact-open classifier is assumed. When a compact open represents each sieve, pointwise intersection descends to the lattice meet. When no such representative is known, equation (22.6) still calculates the intersection of the sieves themselves.

Multiplication governs finite meets, but covering uses addition as well. A certificate $a_1 f_1 + \cdots + a_n f_n = 1$ says that the chosen basic regions are jointly sufficient. The robust geometric form of that sufficiency is obtained by declaring their arrows to generate a cover and then closing under the Grothendieck laws. The equation supplies finite algebraic evidence; the topology below supplies invariant coverhood. This keeps the meet calculation (22.6) distinct from the join-like operation of generating a covering sieve.

22.5 The Big Affine Slice And Its Coordinates

To pass from one affine to its charts, fix R and consider every ring map $h : R \rightarrow S$. Geometrically it is a chart

$$\mathrm{Sp}(S) \longrightarrow \mathrm{Sp}(R).$$

These charts form the **big affine slice over** $\mathrm{Sp}(R)$. A morphism from the chart $R \rightarrow T$ to the chart $R \rightarrow S$ is a ring map $\beta : S \rightarrow T$ whose triangle over R commutes. Its geometric direction is

$$\mathrm{Sp}(T) \longrightarrow \mathrm{Sp}(S),$$

and it carries the whole structured restriction map $S \rightarrow T$ that coordinates must follow.

There is consequently a tautological commutative-ring-valued coordinate presheaf $\mathcal{O}_{\mathrm{coord}}$. It sends the chart $R \rightarrow S$ to S and the geometric arrow represented by $\beta : S \rightarrow T$ to the same structured ring map β . At the whole chart $R \rightarrow R$ its value is R . At the basic-open chart $R \rightarrow R[1/f]$ its value is the chosen localization ring. Nothing is defined only on objects: the restriction maps and their identity and composition laws are part of the existing whole functorial structure.

Theorem 22.1 now receives its geometric reading. The chart

$$\mathrm{Sp}(R[1/f]) \longrightarrow \mathrm{Sp}(R)$$

has, at every test ring S , exactly the points of the sieve $D_R(f)$. The active theorem proves this statement componentwise; the big slice supplies the chart object and its coordinate restriction. Representation is therefore not used to define the sieve, but once a localization is selected it produces the expected geometric chart.

The overlap comparison from Chapter 21 also lifts into the slice. The two whole ring maps between $R[1/(fg)]$ and $R[1/f][1/g]$ become chart arrows in opposite geometric directions. Applying $\mathcal{O}_{\text{coord}}$ to those arrows computes back to the same structured comparison maps, and their already-proved cancellation laws give an equivalence of the coordinate rings. No new overlap equation is copied into the chart record; it is inherited from the algebra that owns it.

Why use the big slice rather than immediately restrict to the small category of basic opens of $\text{Sp}(R)$? The big slice accepts every R -algebra as a chart. Base change, comparison maps, degenerate targets, and future affine realizations can therefore be expressed without first proving that each object belongs to a chosen basis. A small site is often more economical for compactness or cohomological arguments, but equivalence between the two presentations is a theorem. Starting big keeps the functor-of-points semantics literal and postpones that theorem rather than assuming it.

The price is controlled redundancy. Many big-slice objects describe regions already covered by smaller basic charts, and equivalent localizations may appear as distinct packages. The coordinate presheaf handles this without quotienting the objects: it follows every whole ring map, while universal properties supply comparisons where needed. A later basis theorem may show that suitable redundancies are invisible to local data. They are not erased at the computational boundary.

Formal status — checked. Evidence `AFFINE-BIG-SLICE-COORDINATES`. The conventional big affine slice, the whole coordinate presheaf, literal charts, localization charts, commuting chart arrows, and the two product/iterated-localization overlap directions are active. Coordinate restriction along a chart arrow computes to its supplied whole ring map. The big slice is not identified with a small site of opens and does not by itself provide a topology, sheaf, locally ringed space, or complete scheme.

22.6 Finite Families Generate The Big Zariski Topology

The algebraic certificate from Chapter 21 becomes geometric on every chart. Let $h : R \rightarrow S$ be an object of the big affine slice, and choose a finite family $f_1, \dots, f_n \in S$ together with coefficients satisfying

$$\sum_i a_i f_i = 1. \quad (22.7)$$

For each f_i , also choose a universal-property localization of S at f_i . It determines a whole chart arrow

$$\text{Sp}(S[1/f_i]) \longrightarrow \text{Sp}(S) \longrightarrow \text{Sp}(R). \quad (22.8)$$

Equation (22.7) is the finite certificate that these basic charts cover. To turn that statement into a topology without discarding its computational input, first call a sieve on the chart a **generator** when it contains every arrow in one selected family (22.8). The generators remember the family, coefficients, localization packages, and literal containments. They are witness-rich data, not merely truth values.

Now intersect all Grothendieck topologies on the big affine slice that accept those generators. The result is a lawful topology, denoted $J_{\text{Zar}}^{\text{big}}(R)$. Every selected finite family covers in it by construction, and it is least in the precise sense that

$$J_{\text{Zar}}^{\text{big}}(R) \leq J$$

for every other Grothendieck topology J accepting the same generators. The presentation witnesses remain available at the generating boundary, while the statement that a sieve covers is proposition-valued. This is the same useful separation encountered in Chapter 19: rich evidence enters the generator; invariant coverhood leaves it.

Theorem 22.2 (the generated big Zariski topology). The selected finite unimodular localization families on all charts of the big affine slice generate a Grothendieck topology with computing generator inclusion and the leastness property above.

Formal status — checked. Evidence AFFINE-BIG-ZARISKI-TOPLOGY . Each chosen localization is a whole internal chart arrow whose coordinate restriction is the localization map. The generator retains finite containment, and the generic generated-topology construction supplies a lawful least topology. No global choice of localization packages, cover-derivation syntax, coverhood decision procedure, subcanonicity theorem, sheafification, or comparison with the small Zariski site is asserted.

This construction parallels the finite Zariski coverage in Zeuner's functorial account, but the book-keeping is arranged around sieves. A finite family presents probes that must cover; Grothendieck closure then adds all maximal, pulled-back, and locally implied covers. In a coherent setting the same information may be summarized by joins in the Zariski lattice. The big site keeps the probes and their restriction maps explicit, which is exactly what the later computational structure presheaf consumes.

The Grothendieck closure is doing real work. A displayed family such as (22.8) is only one cover presentation on one chart. Pulling a covering sieve back along a chart arrow must again cover, and a sieve covered locally by covering pullbacks must cover globally. Stability and local character come from the generated topology, not from repeatedly manipulating the coefficients in (22.7). When an explicit mapped family is desired, structured ring maps transport the unimodular coefficients, while target localization packages are supplied rather than chosen globally. The topology can therefore be invariant even though computational presentations retain choices.

Leastness is equally important. One could declare every sieve to cover and satisfy the Grothendieck axioms vacuously. The chaotic topology is a useful feasibility model, but it forgets Zariski geometry. Intersecting all accepting topologies includes exactly the consequences forced by the selected finite

families and the topology laws. The construction is impredicative rather than an inductive syntax of derivations, so it proves the universal property of generated coverhood without claiming a normalizer or decision procedure.

A small but concrete example comes from $R = \mathbb{F}_2 \times \mathbb{F}_2$. The complementary idempotents $e = (1, 0)$ and $1 - e = (0, 1)$ satisfy $e + (1 - e) = 1$. Their fixed-image localizations therefore give two selected basic charts in a finite Zariski cover. Since $e(1 - e) = 0$, their algebraic intersection is represented by the zero localization. The corresponding overlap coordinate ring computes to the zero ring. Even this degenerate overlap is informative: the two charts are disjoint pieces of the product affine, and that fact is witnessed by restriction maps rather than inferred from an external picture of points.

22.7 The Structure Sheaf Is A Separate Commitment

The coordinate presheaf $\mathcal{O}_{\text{coord}}$ computes, but computation alone does not prove descent. To call it a structure sheaf on the generated big site, one needs commutative-ring-valued sheaf semantics and a comparison identifying the selected sheaf with these coordinates.

The Cat-valued reflector constructed in Chapter 20 does not automatically supply that result. Lifting it to commutative-ring values would require the ring operations and laws to survive the completion coherently, and the active development has not claimed such a lift. The affine interface therefore makes the missing theorem an explicit input. An **affine structure-sheaf presentation** first supplies a reflective sheaf theory for commutative-ring-valued presheaves on the exact topology $J_{\text{Zar}}^{\text{big}}(R)$. It then selects a sheaf object $\mathcal{O}$ and a whole computational isomorphism from its included presheaf to $\mathcal{O}_{\text{coord}}$.

The comparison is whole functorial data, not a list of unrelated object-by-object ring isomorphisms. Its component at a chart compares the selected structure sheaf with the chart ring, while its internal transformation action retains compatibility with every restriction. This is strong enough for downstream computation, but it is supplied evidence. The package does not construct the reflector or prove from first principles that the coordinate presheaf satisfies descent.

The whole comparison matters whenever a section is transported before it is inspected. An object-wise list of ring isomorphisms could identify $\mathcal{O}(U)$ with the coordinate ring of each chart and still fail to commute with restriction. A transformation in the presheaf category carries that compatibility internally. Its cancellation laws let a calculation move to the transparent coordinate presheaf, compute there, and return to the selected sheaf without adding a new naturality square at every use.

Formal status — checked, assumption-explicit. Evidence `AFFINE-STRUCTURE-SHEAF-PRESENTATION`. Given a reflective commutative-ring-valued sheafification capability, a sheaf object, and a whole computational isomorphism to the coordinate presheaf, the active presentation constructs the corresponding reflective ringed big site and exposes the comparison at every chart. Neither the capability nor the comparison is constructed by this affine layer.

There is a second locality condition that should not be confused with sheaf descent. Let U be a chart, let $s \in \mathcal{O}_{\text{coord}}(U)$, and choose a localization of the chart ring at s . Restricting a localization element along every member of the sieve $D_U(s)$ gives a coherent matching family. Cartier locality says that this restriction functor is an equivalence: every coherent family on the invertibility sieve glues uniquely to an element of the localized ring.

The active affine locality capability supplies this whole equivalence for every chart, section, and supplied localization package. Its inverse is a whole glue functor, and both composite-functor laws are retained. Yet $D_U(s)$ need not cover U . Cartier locality describes the coordinate ring of a basic-open region; ordinary sheaf descent describes reconstruction from a covering sieve. A stalk-local-ring theorem is different again. Keeping these three statements separate prevents the word “local” from doing more work than the mathematics.

This distinction mirrors the sieve-first viewpoint. Sheaf descent starts with a sieve certified to cover U and asks whether compatible data on that cover have a unique global amalgamation. Cartier locality starts with an arbitrary section s and studies the possibly noncovering region where s is invertible. Its global object is not a section over all of U but an element of the localized coordinate ring. A local-ring condition, finally, controls how unit evidence can be found locally from algebraic alternatives such as the invertibility of a sum. The three interfaces cooperate in scheme theory, but none is a synonym for either of the others.

22.8 A Thin Computational Affine Presentation

Once the preceding commitments have been named, the affine package itself can be small. For a ring R , an affine-scheme presentation consists of

$$\text{AffPres}(R) = (\text{structure-sheaf presentation on } J_{\text{Zar}}^{\text{big}}(R), \text{coordinate localization locality}). \quad (22.9)$$

The ring R already determines the big affine slice, its coordinate presheaf, the whole chart, and the generated topology. The first entry of (22.9) relates a supplied reflective structure sheaf to those computing coordinates. The second supplies the whole restriction-and-glue equivalence on every basic invertibility sieve. There is no benefit in copying chart actions, overlap maps, or coherence equations into a larger record: they are already inherited from the functors and universal properties that own them.

Formal status — checked, assumption-explicit. Evidence `AFFINE-THIN-SCHEME-PRESENTATION`. The active affine presentation pairs the supplied whole structure-sheaf presentation with supplied whole coordinate localization locality. It projects the exact generated big-Zariski ringed site, the whole coordinate comparison, and locality at each selected chart and localization. The product ring $\mathbb{F}_2 \times \mathbb{F}_2$ supplies a closed reviewer in which the two capabilities remain explicit inputs while the complementary-idempotent cover, chart rings, restriction maps, and zero overlap compute.

The smallness of (22.9) is evidence of internal organization, not of missing coherence being ignored. The topology owns pullback stability and local character. The coordinate presheaf owns restriction. Localization owns basic charts, and the whole locality equivalence owns glue over $D(s)$. An affine presentation selects only the capabilities that cannot yet be derived. Duplicating their component operations would create new obligations to prove that the copies agree with these established owners.

Calling (22.9) an affine scheme is therefore a qualified statement. It is a computational presentation whose assumptions are visible. The active work does not construct commutative-ring-valued sheafification, prove coordinate locality, compare the big site with the small Zariski site, construct stalks, establish a stalk-local-ring theorem, or build a representation-independent category of affine schemes. Nor does it prove Zeuner's comparison between locally ringed lattices and functorial qcqs schemes.

What it does construct is already a coherent geometric chain:

$$\text{ring map} \longrightarrow \text{affine probe} \longrightarrow D(f) \longrightarrow R[1/f] \longrightarrow \text{basic chart} \longrightarrow J_{\text{Zar}}^{\text{big}}.$$

Every arrow in that chain carries computation. Composition changes probes; unit preservation restricts the sieve; localization selects factors; multiplication computes intersections; the coordinate presheaf computes chart restriction; and generated topology turns finite algebraic certificates into coverhood. The supplied sheaf and locality capabilities begin exactly where the constructed chain ends.

The next chapter starts from the complementary direction. Instead of fixing a ring and generating its affine world, it assumes a global ringed object and a covering sieve, selects affine charts inside that cover, and asks which restrictions and overlaps can be inherited from the global object. That global-first viewpoint is the bridge from one affine presentation to site-relative schemes.

23. Schemes From Covering Charts

An atlas is persuasive because its pieces are familiar. If a space can be covered by affine charts, one is tempted to say that the scheme is simply the charts plus instructions for gluing them. But this description suppresses an important choice of direction. We may begin with charts and try to construct a global object, or we may begin with a global object and recognize some of its regions as affine. The two directions ask for different theorems.

This chapter follows the second, **global-first** direction. A global object X is already present in a ringed site, with one structure presheaf $\mathcal{O}$ and one covering sieve $\mathcal{R}$ on X . Two selected members $u_0 : U_0 \rightarrow X$ and $u_1 : U_1 \rightarrow X$ will generate that sieve constructively. Each chart is then compared, as a whole functorial restriction, with affine coordinates. Local-ring behaviour is imposed on the actual slice over X . The result is a binary, site-relative computational scheme presentation.

Starting globally pays a coherence dividend. Restriction maps already belong to $\mathcal{O}$; their identity and composition laws are already functor laws. An intersection, once selected as an actual product in the slice over X , inherits its two projections and both maps on rings. There is no reason to copy those maps into an atlas record and then ask whether the copies agree. The global object is the common source from which they are computed.

The price is equally clear. This chapter does not construct X from an abstract atlas. Its slice sites, affine-basis comparisons, and one selected chart intersection are assumption-explicit. “Scheme” here means a scheme presentation relative to those supplied categorical semantics, not yet a representation-independent category identified with every classical or functorial definition of schemes.

23.1 Two Directions Through An Atlas

Suppose first that two affine objects U_0 and U_1 have been given, along with a candidate overlap U_{01} , maps to the charts, and a transition between coordinate descriptions. The atlas-first problem is to construct an object X for which the diagram is effective: U_0 and U_1 should cover X , U_{01} should be their intersection, and functions agreeing on the overlap should glue uniquely. Even for two charts, that is a colimit and descent theorem. For more charts one must also control triple overlaps and co-cycles. Merely storing the expected diagrams does not prove that a global object exists.

Now reverse the situation. Let X already be an object of a category $\mathcal{K}$, and let $u_i : U_i \rightarrow X$ be actual arrows. Their intersection, if the needed product exists in the slice $\mathcal{K}/X$, is no longer an invented compatibility object. It is the categorical object

$$U_{01} = U_0 \times_X U_1. \quad (23.1)$$

If a single presheaf $\mathcal{O}$ is already defined on $\mathcal{K}$, then its values $\mathcal{O}(U_i)$ and $\mathcal{O}(U_{01})$ and its two restriction maps are forced by (23.1). Repeated restriction agrees because $\mathcal{O}$ is a functor. Global-first geometry therefore shifts the hard question. We no longer ask whether chart data can be glued to

create X ; we ask whether selected regions of X really are affine and whether they cover in the chosen topology.

Max Zeuner develops two constructive architectures for qcqs schemes: finite affine covers of locally ringed lattices and affine compact-open covers of functors of points, followed by a comparison theorem between them. The finite-cover rhythm is an important model for the present exposition, but the emdash construction changes both its starting point and its classifier of locality. The global object and its ringed categorical semantics are supplied first; coverhood is carried by an ordinary sieve; and the locus where a section is invertible is the sieve $D_U(s)$ of all successful probes. A compact open can represent that sieve in a coherent setting, but such a representability theorem is not assumed merely in order to speak about the sieve.

The distinction is not a contest between definitions. A global-first package is useful when a semantic object has already been built and one wants a computational account of its charts. An atlas-first theorem is indispensable when the charts are the input from which the object must be created. Zeuner's comparison explains why two mature theories of qcqs schemes agree. Here the more modest task is to identify exactly how far one can travel on the first route with the current owners.

Attribution and comparison boundary. This chapter comparatively adapts the finite affine-cover and functor-of-points architecture of **Zeuner**, especially Sections 3.3, 4.2, 5.1, and 5.3. It does not import the theorem that affine charts glue to a qcqs scheme, the compact open classifier, or the equivalence between functorial and locally ringed-lattice schemes. The site-relative global-first presentation below is the active emdash result.

23.2 One Covering Sieve, Two Generators

Let $\mathcal{A}$ be a reflective commutative-ringed site with base category $\mathcal{K}$. Its included structure presheaf will be written

$$\mathcal{O} : \mathcal{K}^{\text{op}} \longrightarrow \mathbf{CRing}.$$

Choose an object $X \in \mathcal{K}$, an ordinary sieve $\mathcal{R}$ on X , and evidence that $\mathcal{R}$ covers X in the selected Grothendieck topology. This is already global geometric data. Every arrow $q : V \rightarrow X$ in $\mathcal{R}$ is a region of X , and every further restriction of q remains in $\mathcal{R}$. Grothendieck stability also says that pulling $\mathcal{R}$ back along any arrow into X produces a covering sieve on its domain.

A selected chart is initially nothing more than a selected member of this sieve. Thus an arrow $u : U \rightarrow X$ becomes a chart when accompanied by $u \in \mathcal{R}$. The name does not make U affine. It records that u is one of the regions accepted by the global cover; affineness requires the separate comparison developed in Section 23.5.

Select two such members $u_0 : U_0 \rightarrow X$ and $u_1 : U_1 \rightarrow X$. Their membership alone does not say that they cover. A sieve can contain two arrows while also containing regions that factor through neither one. The computational generation condition supplies the missing direction: for every $q : V \rightarrow X$ in $\mathcal{R}$, it returns a Boolean side b , a map $h : V \rightarrow U_b$, and the triangle

$$q = u_b \circ h. \quad (23.2)$$

In dependent notation, the content is

$$\prod_{q:V \rightarrow X} (q \in \mathcal{R}) \longrightarrow \sum_{b:\mathbf{2}} \sum_{h:V \rightarrow U_b} (q = u_b h). \quad (23.3)$$

Equation (23.3) is stronger computationally than the mere assertion that some chart contains every covered region. Given a membership witness, one can execute the selection, inspect which chart was chosen, and use the actual factor map. No propositional truncation hides the branch. Conversely, because both u_0 and u_1 are themselves members and a sieve is closed under precomposition, every arrow factoring through either chart belongs to $\mathcal{R}$. The retained sieve is therefore exactly generated, in this witness-rich sense, by the two selected arrows.

There are two cautions. First, (23.3) does not construct a second sieve: it explains the already-selected covering sieve $\mathcal{R}$. Its coverhood comes from the global package, not from the unsupported observation that two arrows have been named. Second, a general member q need not itself be affine. It is a refinement of one affine generator once the relevant Boolean branch is computed. Affineness belongs to the generator's whole realization and does not automatically descend to every arbitrary arrow without another theorem.

Formal status — checked. Evidence `GLOBAL-RINGED-COVER-BINARY-GENERATION`. The global package retains the reflective ringed site, object, ordinary covering sieve, structure presheaf, and Grothendieck-stable pullbacks. Binary generation computes a selected chart, factor map, and triangle for every sieve member. The active result does not infer affineness from sieve membership, construct an atlas-first object, or turn arbitrary refinements into affine charts.

The choice to retain a sieve rather than just a two-element family has a further advantage. Pulling the cover back along a region does not require a new ad hoc list of chart fragments. The sieve pullback contains exactly the arrows whose composites lie in $\mathcal{R}$, and Grothendieck stability makes it covering. When an explicit generator is needed, (23.3) still computes a factor through U_0 or U_1 . Invariant coverhood and executable chart selection coexist without being collapsed into the same representation.

23.3 Local Rings Without Stalks

A sheaf of rings is not automatically a sheaf of local rings. Descent says that compatible sections glue; locality says, roughly, that algebraic alternatives can be resolved after passing to a cover. Classically the latter is often phrased by requiring every stalk to be a local ring. A computational site can express the needed alternatives directly, before stalks have been constructed.

Let T be a Grothendieck topology on a category and let $\mathcal{O}$ be a commutative-ring-valued presheaf. For $s \in \mathcal{O}(U)$, recall the ordinary sieve $D_U(s)$ of arrows $q : V \rightarrow U$ along which $s|_q$ is invertible. Two support laws are automatic in the ordinary algebraic account: one is invertible everywhere, and a product becomes invertible precisely where both factors do. In sieve notation these give the expected equations

$$\begin{aligned} D_U(1) &= \top, \\ D_U(st) &= D_U(s) \cap D_U(t). \end{aligned} \tag{23.4}$$

Equation (23.4) is mathematical orientation here; the local-ring interface below packages the two nonautomatic topology laws, not new whole sieve equalities for one and products.

The local-ring content lies in the two remaining directions. If zero becomes invertible at U , then U is locally void: the literal empty sieve must cover U . And if $s + t$ is invertible, there must be a covering sieve $\mathcal{S}$ such that every $q : V \rightarrow U$ in $\mathcal{S}$ comes with a selected alternative

$$s|_q \text{ is invertible} \quad \text{or} \quad t|_q \text{ is invertible.} \tag{23.5}$$

The disjunction in (23.5) is a Boolean-indexed dependent pair. It remembers which summand is usable and retains its unit evidence. Thus the condition is the executable Kripke--Joyal form of

$$D_U(0) = \perp, \quad D_U(s + t) \leq D_U(s) \vee D_U(t). \tag{23.6}$$

No raw union of sieves has to be constructed for (23.6). The right side is presented by a chosen cover subordinate to the two alternatives, and a branch is requested only after an actual member of that cover is supplied. Nor is the choice erased into a mere proposition. This is exactly the kind of local information a later calculation can consume: restrict to a cover member, inspect the side, and use the selected inverse.

The formulation is categorical semantics, not an auxiliary modal object language. Objects, arrows, sieves, restrictions, unit witnesses, covers, and branches all live inside the functorial theory represented in the outer logical framework. `Lambdapi`'s conversion and unification rules execute the transparent projections and functor actions; the same interfaces can be targeted by an explicit TypeScript core. Computation therefore does not require replacing the site by a primitive modality. It comes from making the categorical data sufficiently internal and functorial.

Formal status — checked, topology-local. Evidence `TOPOLOGY-LOCAL-RING-CERTIFICATE`. The presentation executes empty-cover nontriviality and coverwise Boolean splitting of an invertible sum. It is a site-level local-ring certificate for a whole commutative-ring presheaf. It does not construct stalks, prove equivalence with stalk-local rings, form a raw sieve join, decide unit evidence, or identify the chosen topology with the classical Zariski topology.

This condition must be attached to the correct object. Our global structure presheaf lives on $\mathcal{K}$, whereas the local geometry of X lives on the slice $\mathcal{K}/X$. The next section constructs the presheaf restriction to that whole slice and makes its sheaf boundary explicit. Only then can (23.5) be read as local-ring behaviour of the global object X rather than of an unrelated family of rings.

23.4 Restricting The Whole Structure

For any $U \in \mathcal{K}$, the conventional slice $\mathcal{K}/U$ has a domain functor

$$\text{dom}_U : \mathcal{K}/U \longrightarrow \mathcal{K}, \quad (q : V \rightarrow U) \longmapsto V. \quad (23.7)$$

Precomposition with its opposite constructs the ambient restriction

$$\mathcal{O}|_U = \mathcal{O} \circ \text{dom}_U^{\text{op}} : (\mathcal{K}/U)^{\text{op}} \longrightarrow \mathbf{CRing}. \quad (23.8)$$

At a slice object $q : V \rightarrow U$, formula (23.8) computes to $\mathcal{O}(V)$. At a triangle over U , it computes the corresponding restriction homomorphism. Identity, composition, and naturality are inherited from ordinary functor composition. Thus the whole presheaf needed on a chart slice is constructed, not copied object by object.

What (23.8) does **not** construct is a topology and sheaf theory on $\mathcal{K}/U$. The active interface asks for a reflective commutative-ringed site on that actual slice and a whole isomorphism

$$\iota_U \mathcal{O}_U \cong \mathcal{O}|_U \quad (23.9)$$

in the commutative-ring presheaf category. Here $\mathcal{O}_U$ is the selected structure sheaf of the supplied slice site and ι_U includes it as a presheaf. The topology, reflector, and sheaf object on the left of (23.9) are hypotheses. The right side and its restriction action are the computing ambient object.

The word “whole” in (23.9) matters. A separate ring isomorphism at every slice object would not ensure compatibility with restriction. A functor-level isomorphism carries that compatibility internally and supplies inverse laws in the presheaf category. One may compute using the transparent right side, then pass back to the selected sheaf semantics without introducing a new naturality square for each calculation.

For the whole object X , the topology-local ring certificate of Section 23.3 is attached to the computing presheaf $\mathcal{O}|_X$ using the topology of the supplied slice site. The resulting local presentation combines supplied reflective semantics on $\mathcal{K}/X$, a whole bridge to ambient restriction, and executable local-ring forcing on that bridge's target. It does not claim that the slice topology was induced from the ambient topology or that a general sheaf-pullback theorem has been proved.

23.5 When A Selected Region Is Affine

Return to one selected cover member $u : U \rightarrow X$. To call it affine, it is not enough to attach a ring name R to U . We must compare the actual global structure restricted over U with the affine coordinates developed in Chapter 22.

Begin with the supplied reflective slice presentation (23.9). Choose a ring R and a thin affine-scheme presentation for R . Its big affine slice has the generated Zariski topology and coordinate presheaf $\mathcal{O}_{\text{coord},R}$. Next choose a whole basis functor

$$i : \mathbf{Aff}/\text{Sp}(R) \longrightarrow \mathcal{K}/U. \quad (23.10)$$

The direction of (23.10) says that affine coordinate probes are realized as actual regions over U . It is accompanied by two supplied comparisons. The first is a sheaf-basis equivalence along i : restriction relates the selected sheaf categories on the affine basis and on the actual slice. This is an equivalence of sheaf semantics along the displayed functor, not an assertion that the two base categories are themselves equivalent. The second is a direct whole isomorphism from the ambient restriction to the included presheaf of the selected affine presentation. That affine presentation already retains its own whole coordinate comparison. Together they form the chain

$$i^*(\mathcal{O}|_U) \cong \iota_R \mathcal{O}_R^{\text{aff}} \cong \mathcal{O}_{\text{coord}, R}. \quad (23.11)$$

Equation (23.11) is the computational heart of the affine label. On every affine probe, the structure inherited from the global object agrees with the coordinate ring; on every arrow between probes, the agreement respects the restriction homomorphism. Composing the supplied first bridge with the affine scheme's retained coordinate bridge derives the displayed ambient-to-coordinate comparison. Nothing has to be postulated separately for individual chart arrows.

An **affine realization** of u retains exactly these inputs: the supplied reflective site on $\mathcal{K}/U$, the coordinate ring R , its thin affine presentation, the basis functor (23.10), and the whole basis realization. The region u is now affine in the precise, site-relative sense that its actual restricted structure is realized by affine coordinates along the selected basis.

Formal status — checked, assumption-explicit. Evidence `WHOLE-SLICE-AFFINE-REALIZATION`. Whole ambient restriction is constructed by precomposition with the slice-domain functor. The reflective slice, sheaf-basis equivalence, affine scheme presentation, basis functor, and direct ambient-to-affine-underlying comparison are supplied; composition with the affine presentation's retained bridge derives a whole coordinate isomorphism. No induced slice topology, raw base-category equivalence, arbitrary basis theorem, or general transport of local exactness is claimed.

Applying this package to both u_0 and u_1 , and adjoining the generation witness (23.3), gives a binary affine-cover presentation. It retains two actual regions of the one global object, two coordinate rings, and two whole affine realizations. For an arbitrary $q \in \mathcal{R}$, the Boolean generator computes which affine chart it refines and exposes that chart's already-owned realization and coordinate ring. It still does not declare q itself affine.

23.6 The Site-Relative Scheme Total

The pieces can now be gathered without enlarging their claims. A binary site-relative scheme presentation consists of the following dependent data:

$$\begin{aligned} \text{Scheme}_{\mathcal{K}}^{(2)} = \sum_{(\mathcal{A}, X, \mathcal{R})} & \text{(whole-slice local presentation of } X, \\ & u_0, u_1 \in \mathcal{R}, \\ & \text{constructive generation as in (23.3),} \\ & \text{whole affine realizations of } u_0, u_1). \end{aligned} \tag{23.12}$$

The first summation variable includes the global reflective ringed site, distinguished object, covering sieve, and proof of coverhood. The dependent certificate then adds local-ring forcing on the actual slice and the binary affine atlas. Because each later field is indexed by the object selected before it, malformed combinations are not representable: an affine realization cannot silently refer to a different chart, site, or global object.

Theorem 23.1 (global-first binary scheme presentation). Given the data in (23.12), there is one transparent site-relative scheme total that retains the global ringed object exactly once and exposes its structure presheaf, whole object, covering sieve, local-ring certificate, two selected generators, and two whole affine realizations through their existing owners.

Formal status — checked, site-relative. Evidence `BINARY-SITE-RELATIVE-SCHEME`. The constructor and all named observations compute through the dependent total. The global structure presheaf is inherited rather than stored again; the local and atlas halves remain their exact existing packages. No overlap, transition, cocycle, gluing field, scheme morphism, effectivity theorem, or representation-independent category of schemes is added.

The absence of overlap and cocycle fields is deliberate. Suppose a section is restricted from X to U_0 , from U_0 to an overlap region, and perhaps farther to another refinement. These maps are all values of the same contravariant functor $\mathcal{O}$. Their agreement with direct restriction is the functor's composition law. Adding parallel restriction maps to (23.12) would create a second source of truth and an obligation to compare it with the first.

The theorem is nevertheless conditional in meaningful ways. It does not construct the global object, the reflective slice sites, or the affine-basis equivalences. It packages them at endpoints where every downstream action can compute. Nor does it say that all finite atlases reduce to two charts. The binary case is the active vertical slice: rich enough to expose genuine covering, local forcing, affineness, refinement, and intersection, but narrow enough that the supplied boundary remains visible.

The qualifier **site-relative** prevents a more subtle overstatement. The chosen topology decides which sieves cover and the supplied basis semantics decides what counts as an affine chart. Another site may present equivalent geometry, but (23.12) alone does not construct that equivalence. In particular, it does not identify this presentation with Zeuner's compact-open qcqs schemes, with classical locally ringed spaces, or with all local functors on the big Zariski site.

23.7 The Intersection Belongs To The Global Object

Although an overlap should not be copied into the scheme total, a consumer may still need an explicit one. For the two slice objects $u_0, u_1 \in \mathcal{K}/X$, supply a selected binary product with its whole universal property,

$$u_{01} = u_0 \times u_1 \quad \text{in } \mathcal{K}/X. \quad (23.13)$$

Its domain in $\mathcal{K}$ is an actual object U_{01} equipped with arrows $p_i : U_{01} \rightarrow U_i$ whose composites to X agree. Applying the whole domain functor to the product projections derives these arrows; no base-level projections are supplied separately.

The global structure presheaf then computes three rings and two homomorphisms:

$$\mathcal{O}(U_0) \xrightarrow{p_0^*} \mathcal{O}(U_{01}) \xleftarrow{p_1^*} \mathcal{O}(U_1). \quad (23.14)$$

The variance in (23.14) is the familiar geometric one: an arrow from the intersection to a chart restricts functions from the chart to the intersection. Every term is evaluated from an existing whole owner. The overlap ring is not a new coordinate choice, and the homomorphisms are not transition fields. They are the value and arrow action of $\mathcal{O}$ on the selected categorical intersection.

Formal status — checked after a supplied product. Evidence `ACTUAL-BINARY-CHART-OVERLAP`. A selected binary product in the conventional slice retains its whole universal property. Its two slice projections, underlying arrows, overlap ring, and both restriction homomorphisms are derived. The active owner does not construct arbitrary pullbacks, prove every pair of charts has an intersection, identify the overlap as affine or as a localization, or add an atlas-gluing theorem.

This is the exact sense in which coherence is inherited. If another selected restriction $W \rightarrow U_{01}$ is introduced, the maps from the chart rings to $\mathcal{O}(W)$ agree with the composites through $\mathcal{O}(U_{01})$ because presheaf action respects composition. If triple intersections are later supplied, the same principle handles their restriction diagrams. Existence of the needed limits remains a separate categorical hypothesis; compatibility of the resulting restriction maps does not.

One should also resist a plausible but invalid inference: because U_0 and U_1 have affine realizations, U_{01} has not thereby been proved affine. In familiar schemes the intersection of two affine opens is often quasi-affine and, under additional separatedness or basic-open hypotheses, admits more precise affine presentations. None of those theorems is smuggled into (23.13). Chapter 24 supplies a much narrower Laurent-coordinate adapter for the selected overlap used in the projective-line presentation.

23.8 What Has Been Built, And What Has Not

The global-first construction has now crossed a genuine threshold. It starts with one ringed object and proves that a selected binary covering sieve is generated by two regions whose whole restrictions have affine coordinate realizations. It attaches an executable local-ring condition to the actual slice

over the object. It packages those data in one dependent total. And, when a product is supplied in that slice, it computes the chart intersection, its ring, and both restriction maps from the global presheaf.

The resulting chain is worth reading from left to right:

$$\begin{aligned}
 \text{global ringed object} &\longrightarrow \text{covering sieve} \\
 &\longrightarrow \text{two constructive generators} \\
 &\longrightarrow \text{whole affine realizations} \\
 &\longrightarrow \text{site-relative scheme total} \\
 &\longrightarrow \text{inherited intersection.}
 \end{aligned}
 \tag{23.15}$$

At each arrow, either a new hypothesis is named or a new construction is performed. The global object, reflective slice semantics, affine presentations, basis equivalences, local-ring certificate, and selected slice product are supplied. Sieve pullbacks, whole ambient restrictions, generator refinements, coordinate comparisons, total projections, overlap arrows, and ring restrictions are derived. This ledger is mathematical content: it says which theorems a future construction must prove before an input can disappear.

Several larger results remain outside (23.15). There is no constructor taking two abstract affine schemes and transition data to their glued global object. There is no effectivity or independence-of-atlas theorem, no arbitrary finite atlas interface, no category of scheme morphisms, and no proof that changing the selected site preserves the notion. There is no comparison with a prime-spectrum locally ringed space, with Zeuner's locally ringed lattices, or with functorial qcqs schemes. There is also no general theorem representing every invertibility sieve by a compact open.

Those absences locate the work rather than diminish it. Classical definitions often bundle the results of several deep comparison theorems into one word. The site-relative presentation instead exposes a computational semantic boundary: once the global object and its honest affine comparisons are available, restrictions and overlaps should be inherited, not restated. Conversely, when only charts are available, one still owes a gluing theorem.

The next chapter tests this boundary on the first non-affine object one wants to draw: the projective line. Two affine-line charts should meet where their coordinates are invertible, and the transition should send one coordinate to the inverse of the other. The current library can package that Laurent calculation on an inherited actual overlap once the global projective-line presentation has been supplied. It does not yet construct that global object from the two charts, nor does it build `Proj` or general projective space. That precise mixture of calculation and boundary is where the global-first method is most revealing.

24. The Projective Line And The Boundary Of Construction

The projective line is the first scheme whose picture seems to demand gluing. One affine coordinate sees every finite point and misses infinity; a second coordinate sees infinity and misses a different point. Where both coordinates are visible, each is the inverse of the other. The whole geometry is contained in that short sentence, but its formal meaning divides into several tasks that are easy to conflate.

One may construct the two affine lines, construct their principal regions, identify those regions by inversion, and then prove that the identification is effective. That is the atlas-first route. Or one may begin with a global object already carrying two affine-line charts, take their actual intersection, and check that its two inherited coordinate descriptions are Laurent coordinates. That is the global-first route. The present development reaches the second route exactly. It does not silently acquire the first.

This distinction makes the projective line an unusually revealing example. The coordinate calculation is not deferred: it is constructed from polynomial and localization universal properties. The common region is not an invented overlap: it is an actual selected intersection of charts in the global slice. Yet the global object itself remains supplied. The calculation is complete at its stated boundary, and the boundary says precisely what a later construction must add.

24.1 Two Affine Views Of One Line

Let A be a commutative ring. The familiar projective-line atlas has two charts

$$U_0 \simeq \operatorname{Spec} A[t], \quad U_1 \simeq \operatorname{Spec} A[u]. \quad (24.1)$$

In homogeneous coordinates $[x_0 : x_1]$, the first coordinate is $t = x_1/x_0$ where x_0 is invertible, and the second is $u = x_0/x_1$ where x_1 is invertible. Both descriptions apply precisely where neither homogeneous coordinate vanishes. Thus the expected intersection has the two coordinate presentations

$$U_{01} \simeq D(t) \subseteq U_0, \quad U_{01} \simeq D(u) \subseteq U_1, \quad u = t^{-1}. \quad (24.2)$$

In ordinary algebraic notation its ring is therefore written

$$A[t, t^{-1}] \simeq A[u, u^{-1}]. \quad (24.3)$$

These formulas are a specification, not yet a construction. The symbols $D(t)$ and $D(u)$ can name open subspaces only after one knows what represents the corresponding invertibility conditions. The isomorphism in (24.3) can be written down informally, but a computational account should explain why its maps exist and which equations they satisfy. Finally, even perfect overlap data do not by themselves produce the union $U_0 \cup U_1$.

The finite-cover rhythm here follows the constructive scheme architecture developed by [Zeuner](#): global schemes are recognized through finite affine covers, and open or functorial presentations are compared only by an explicit theorem. The present chapter borrows that rhythm and its careful separation of presentation from comparison. Its projective-line and Laurent constructions are not taken from Zeuner's thesis, and no part of Zeuner's qcqs comparison theorem is claimed below.

Status of (24.1)–(24.3). These are the standard mathematical model for the chapter. The checked artifact has no closed term denoting $\mathbf{P}_A^1$, no general `Proj`, and no polynomial or Laurent expression syntax whose normal forms are the displayed rings. Its owners instead work with rings, structured maps, and their universal properties directly.

24.2 Laurent Maps Without Fractions

Begin with two supplied one-variable polynomial algebras over A . Write them conventionally as $A[t]$ and $A[u]$, although what is retained formally is a base map, a distinguished variable, and the universal property of a one-variable polynomial algebra. Select a localization of each algebra at its distinguished variable:

$$\iota_t : A[t] \longrightarrow L_t, \quad \iota_u : A[u] \longrightarrow L_u. \quad (24.4)$$

The element $\iota_u(u)$ is a unit in L_u . Its chosen inverse therefore defines a valuation of the one-variable family in L_u . Polynomial universality selects the unique structured map

$$\varphi_{tu} : A[t] \longrightarrow L_u, \quad \varphi_{tu}(t) = \iota_u(u)^{-1}, \quad (24.5)$$

with the prescribed action on A . The image of t is again a unit. The localization universal property now extends (24.5) to a whole ring map

$$\Phi_{tu} : L_t \longrightarrow L_u. \quad (24.6)$$

Reversing the two polynomial and localization presentations constructs $\Phi_{ut} : L_u \rightarrow L_t$. Nothing in this construction parses a Laurent polynomial, reduces a fraction, or chooses representatives. The inverse coordinate is selected as an element of the target ring; polynomial universality constructs the first map; localization universality constructs the extension. The factor triangle records, for every element of $A[t]$, that the extension after ι_t agrees with φ_{tu} .

This order matters. If one began with a formula on fractions, one would still have to prove that it respects the localization relation and the ring operations. The universal property performs exactly that proof while also choosing the map. The formula $t \mapsto u^{-1}$ is not discarded; it appears as the named coordinate equation (24.5), now attached to a whole structured map.

Theorem 24.1 (canonical Laurent transition). For two supplied one-variable polynomial algebras over A and supplied localizations at their coordinates, there is a canonical whole map from the first localization to the second sending the first coordinate to the inverse of the second. Reversing the inputs gives the opposite orientation, and each map carries its whole localization-factor agreement.

Formal status — checked. Evidence `LAURENT-TRANSITIONS-BY-UNIVERSALITY`. The construction is transparent and rule-free. It supplies neither a global projective object nor a theorem that the two maps in (24.6) are inverse for arbitrary polynomial and localization presentations.

24.3 One Overlap Ring, Not Two Isomorphic Copies

The generic construction still has two target rings, L_t and L_u . A global scheme supplies something stronger. Let U_0 and U_1 be its selected charts, let U_{01} be their selected actual intersection, and evaluate the single global structure presheaf:

$$R_0 = \mathcal{O}(U_0), \quad R_1 = \mathcal{O}(U_1), \quad L = \mathcal{O}(U_{01}). \quad (24.7)$$

Contravariance gives the two inherited restrictions

$$\rho_0 : R_0 \longrightarrow L, \quad \rho_1 : R_1 \longrightarrow L. \quad (24.8)$$

Suppose now that R_0 and R_1 are supplied as one-variable polynomial algebras over the same base ring A , with coordinates t and u , and that the literal maps ρ_0 and ρ_1 are supplied as their localizations at those coordinates. The Laurent construction no longer produces maps between two disconnected candidates for the overlap. Both of its endpoints reduce to the exact ring L . It constructs two endomorphisms

$$\Theta_{tu} : L \longrightarrow L, \quad \Theta_{ut} : L \longrightarrow L, \quad (24.9)$$

each expressing one coordinate in terms of the inverse of the other.

The final comparison is deliberately assumption-explicit. A Laurent overlap presentation retains whole paths

$$\Theta_{tu} = \text{id}_L, \quad \Theta_{ut} = \text{id}_L. \quad (24.10)$$

The maps in (24.9) are constructed; the paths in (24.10) are supplied. This is the exact honest boundary. Polynomial and localization universality tell us how to extend prescribed coordinate values, but arbitrary supplied presentations of two maps into one ring need not automatically assert that the resulting endomorphisms are its identity. Retaining the whole paths says that these two coordinate descriptions really are the two Laurent views of the same inherited ring.

Theorem 24.2 (literal common-overlap coordinates). Two literal maps into one ring, each presented as a one-variable polynomial chart followed by localization at its coordinate, determine canonical coordinate-inversion endomorphisms of that ring. A supplied Laurent overlap presentation identifies both endomorphisms wholly with its identity.

Formal status — checked and assumption-explicit. Evidence `LAURENT-COMMON-OVERLAP`. The identity paths are whole paths of structured maps, not elementwise equations gathered into an external compatibility square. No claim is made that every pair of localization presentations admits those paths.

24.4 The Sieve Beneath The Principal Region

The notation $D(t)$ in (24.2) is best read through the principle developed in Chapters 18 and 22. Before it is an open, invertibility is an ordinary sieve. On the chart U_0 , the sieve $D_{U_0}(t)$ asks of a probe $v : V \rightarrow U_0$ whether the restricted coordinate $t|_V$ is a unit. It is defined whether or not the site possesses a chosen open object representing that question.

Localization is the algebraic representation theorem. A map out of $A[t, t^{-1}]$ is the same data as a map out of $A[t]$ for which t becomes invertible. Thus the selected restriction $\rho_0 : R_0 \rightarrow L$, when supplied as localization at t , gives the overlap ring the universal property expected of the region $D(t)$. The second restriction gives the same actual ring the universal property expected of $D(u)$.

This is the projective-line instance of the sieve-first insight:

$$\begin{aligned} \text{invertibility question} &\longrightarrow \text{ordinary sieve} \\ &\longrightarrow \text{represented principal region.} \end{aligned} \tag{24.11}$$

The first arrow is definitional; the second is a theorem or supplied presentation. In the present package, the actual geometric intersection is already selected, and its two ring restrictions are presented as the relevant localizations. The Laurent identities then say that the two coordinate answers agree on that one region.

There is a useful restraint here. The package does not prove a general site-level equality between every selected chart intersection and an abstract sieve classifier. It connects the actual overlap to the principal-open story at the literal coordinate-ring and restriction-map endpoints. Chapter 22's pointwise representability theorem explains why those localization endpoints have the intended invertibility meaning. A whole comparison between arbitrary site presentations would be additional geometry.

24.5 Laurent Coordinates On The Actual Scheme Overlap

Return to a binary site-relative scheme S . Its global object and structure presheaf are already retained once. Its two chart realizations are already affine. After a selected product in the slice supplies the actual intersection Ω , the ring L and both maps (24.8) are derived by evaluating that single presheaf. The Laurent adapter adds only a common base ring A and a coordinate presentation of those exact endpoints:

$$\text{Laurent}(S, \Omega) = \sum_{A:\text{CommRing}} \text{LaurentOverlap}(A, R_0, R_1, L, \rho_0, \rho_1). \tag{24.12}$$

The dependency in (24.12) is the point. One cannot quietly substitute an isomorphic overlap ring, replace a restriction map, or attach the coordinates to a different pair of charts. The types refer to the rings and maps already computed from S and Ω . Conversely, the adapter does not pretend to disc-

over a common base ring or polynomial structures automatically. Those are the mathematical coordinate data being supplied.

Theorem 24.3 (actual-overlap Laurent adapter). Given a supplied binary site-relative scheme and its selected actual chart intersection, a common base ring and Laurent coordinate presentation can be attached directly to the literal chart rings, overlap ring, and inherited restriction maps, without duplicating any of those global owners.

Formal status — checked. Evidence `ACTUAL-SCHEME-LAURENT-OVERLAP`. This is a thin dependent adapter. It does not add a transition or cocycle field to the general scheme presentation, construct the overlap, infer Laurent coordinates from arbitrary charts, or glue a global scheme from them.

24.6 The Supplied Projective-Line Total

All the pieces can now be named as one object. For an ambient category $\mathcal{K}$, define the supplied projective-line presentation by the dependent total

$$\mathbf{PLine}_{\text{sup}}(\mathcal{K}) = \sum_{S:\text{Scheme}_{\mathcal{K}}^{(2)}} \left(\sum_{\Omega:\text{Overlap}(S)} \mathbf{Laurent}(S, \Omega) \right). \quad (24.13)$$

The first component is the already-global site-relative scheme of Chapter 23. It owns the ringed object, local-ring certificate, covering sieve, two constructively generating charts, and their whole affine realizations. The second component selects their actual intersection. The third says that the two actual structure-ring restrictions present one-variable polynomial charts over a common base and satisfy the Laurent identities on the shared ring.

Nothing else is required because nothing else is free-floating. The overlap projections belong to the selected product. The ring restrictions belong to the structure presheaf. Their functoriality belongs to that presheaf as well. The coordinate-transition maps are constructed by the universal properties in Section 24.2. The identity comparisons are retained by the Laurent package. Adding another transition map or cocycle field would create a second account of data that the global object already owns.

Theorem 24.4 (supplied projective-line capability). From an element of (24.13), the global scheme, actual overlap, common base ring, and exact Laurent package are recovered by projection. Its coordinate-inversion endomorphisms are the internally constructed maps of Theorem 24.2 and carry the supplied whole identity paths.

Formal status — checked, supplied, and conditional. Evidence `SUPPLIED-P1`. The total and its observations compute by dependent-pair projection. It neither constructs a closed global object nor proves projectivity or non-affineness.

$$\begin{aligned} \text{supplied global scheme} &\longrightarrow \text{actual chart intersection} \\ &\longrightarrow \text{two localization presentations} \\ &\longrightarrow \text{Laurent coordinate identities.} \end{aligned} \quad (24.14)$$

Read from top to bottom, every arrow in (24.14) has a different logical force. The first passage uses a selected limit in the slice. The second adds algebraic presentations to maps already inherited from the global object. The third constructs transition maps and retains their whole comparison with identity. None of them reverses the chain and constructs the global scheme from the local data.

24.7 What A Construction Of The Line Would Require

There are two natural ways to cross the remaining boundary. The first is atlas-first. Construct $\text{Spec } A[t]$ and $\text{Spec } A[u]$, represent the two invertibility sieves by their localizations, construct the inversion comparison, and prove that the diagram glues effectively. The result must carry a structure sheaf whose restrictions recover the two affine sheaves, a local-ring certificate, and a proof that the selected charts cover. One must then compare the constructed object, wholly, with the supplied presentation (24.13). Merely packaging two charts and an isomorphism cannot replace this effectivity theorem.

The second route is graded. Give $A[x_0, x_1]$ its standard grading and form

$$\mathbf{P}_A^1 = \text{Proj } A[x_0, x_1]. \quad (24.15)$$

Here ordinary localization is only an intermediate step. For a homogeneous element x_i , one localizes the graded ring and then takes its degree-zero part. The two standard regions should compute as

$$\begin{aligned} D_+(x_0) &\simeq \text{Spec } A[x_1/x_0], \\ D_+(x_1) &\simeq \text{Spec } A[x_0/x_1]. \end{aligned} \quad (24.16)$$

Their common region is obtained by inverting the displayed ratio, returning exactly the Laurent calculation of this chapter. A complete `Proj` construction therefore needs graded commutative rings, homogeneous localization, a degree-zero functor, the irrelevant ideal or its covering condition, a structure sheaf, and the relevant local-ring and descent theorems. None of these objects is hidden inside the ungraded Laurent owner.

The same route explains projective n -space:

$$\mathbf{P}_A^n = \text{Proj } A[x_0, \dots, x_n]. \quad (24.17)$$

Its $n + 1$ standard charts have coordinates x_j/x_i for $j \neq i$; on intersections, ratios invert and compose. The binary projective-line package is therefore the smallest nontrivial test of the coordinate machinery, not an implementation of the graded theory in disguise. A future construction should instantiate the existing site-relative and Laurent boundaries and prove a whole comparison with them, rather than introduce a competing notion of scheme.

That comparison must recover, chart by chart and on the actual intersections, the same structure-presheaf restrictions and Laurent identities. Until it does, (24.17) describes the destination rather than a second implementation.

Formal status — mathematical development and research boundary. Equations (24.15)–(24.17) describe the standard graded construction and the intended next theorem. No active emdash owner defines graded rings, homogeneous localization, degree zero, an irrelevant ideal, Proj , or general projective space. No non-affineness theorem is claimed, even for the supplied line.

24.8 The Boundary Is Part Of The Mathematics

The achievement of this chapter is not that a familiar object has been renamed as a dependent record. It is that the local coordinate calculation takes place on the actual inherited overlap of one global structure presheaf. The two restriction maps are not copied. The Laurent transitions are not postulated as unstructured functions. The equation $u = t^{-1}$ is realized by polynomial and localization universality and compared wholly on one literal ring.

At the same time, existence remains visible. The global object, its cover, its affine realizations, the selected intersection, and the Laurent identity paths are supplied at the points where the current theory needs them. This prevents a computational presentation from being mistaken for an atlas-effectivity theorem. It also turns future work into a precise mathematical program: build the missing global object, then discharge the assumptions of the existing consumer.

The larger methodological lesson reaches beyond projective geometry. Actual presheaves, sieves, sites, rings, and categorical limits can be computationally internal because they live inside a functorial type theory whose equations are checked by an outer logical framework. One need not replace them with an abstract modal object language in order to compute. But internal computation does not abolish mathematical hypotheses. It makes their location observable.

The third spiral of the book therefore ends where a good construction should end: not with a vague promise that gluing will work, and not with a denial that gluing matters, but with a coordinate theorem on an honest overlap and a clear account of the global theorem still owed.

The fourth spiral changes direction without abandoning that discipline. It returns to the book's first contrast—reversible paths and noninvertible arrows—and asks what computation survives when directed structure is viewed groupoidally, freely inverted, and then placed back inside a genuinely lax higher-categorical comparison. The next chapter begins with the smallest bridge: paths in a product and transport through their two coordinates.

25. Paths And The Groupoidal Shadow

A directed arrow remembers which way it points. An equality path may also be followed, but it can be reversed. Functorial type theory keeps these two forms of motion in one language without identifying them. This is important both mathematically and computationally. Directed categories are needed for maps that need not be invertible: inclusions, restriction maps, substitutions, and the morphisms of algebra and geometry. Paths are needed for identity, transport, and homotopy. If every arrow were silently turned into a path, the directed theory would disappear. If paths could not enter the categorical calculus, the homotopical theory would become a detached second foundation.

The bridge is the path category. For a groupoidal classifier A , the category $\mathbf{Path}(A)$ has elements of A as objects and equality evidence as arrows:

$$\mathbf{Obj}(\mathbf{Path}(A)) = A, \quad \mathbf{Hom}_{\mathbf{Path}(A)}(x, y) = (x = y). \quad (25.1)$$

Reflexivity is the identity arrow. Categorical composition has the same mathematical effect as path concatenation, with a proved comparison between the two presentations. Every arrow in $\mathbf{Path}(A)$ is invertible because every path has a reverse. Thus equality can be observed through the same object, hom, functor, transformation, and higher-action interfaces used by the directed theory.

This chapter develops three consequences of that bridge. First, paths in a product split into their two coordinate paths and can be reconstructed from them. Second, dependent transport over a product can be performed directly or one coordinate at a time, and the resulting routes form a coherent diamond. Third, a generic directed laxity cell becomes invertible when it is realized in a path category. The point is not to add parallel copies of products, transport, or functoriality for the groupoidal world. It is to show how one iterable categorical calculus changes character when its homs are paths.

25.1 Equality In Categorical Form

Let $f : A \rightarrow B$ be an ordinary function between groupoidal classifiers. Its action on equality is familiar:

$$p : x = y \quad \longmapsto \quad \mathbf{ap}_f(p) : f(x) = f(y). \quad (25.2)$$

In the categorical presentation, (25.2) is the arrow action of a whole functor

$$\mathbf{Path}(f) : \mathbf{Path}(A) \longrightarrow \mathbf{Path}(B). \quad (25.3)$$

The adjective *whole* matters. Equation (25.3) is not merely a function on objects together with a separately stated congruence lemma. It is the first action of an internal path-category construction. Equality between functions therefore has a next action, equality between those equalities has another, and the same mechanism remains available at every represented hom level. Functoriality is not reconstructed from a finite record each time the dimension rises.

This is the **groupoidal shadow** of ordinary function action. It should not be confused with the groupoidification of a directed category. The path category begins with a type-like classifier and exposes its existing equalities as arrows. Groupoidification, studied in Chapter 27, begins with directed arrows and freely realizes them as paths. The first construction reveals identity already present; the second imposes invertibility on motion that was not initially invertible.

The path category also gives a common surface for equality induction and categorical induction. Fix $x : A$. The objects of the outgoing-path category are pairs (y, p) with $p : x = y$. Its distinguished object (x, refl_x) is initial in the groupoidal sense: a family over outgoing paths is determined by what it does there, and its value at (y, p) is obtained by transport along p . In the directed presentation this is the `PathOut` principle of Chapter 5. In the equality presentation it is the usual eliminator J . They are two views of one operation, not competing eliminators.

The computational boundary is precise. Primitive right-based J computes at reflexivity. Structured categorical transport and structured `PathOut` induction are transparent presentations of the same movement, and there are paths comparing each with primitive J . Those comparisons are mathematical equalities; the book does not promote all three surface expressions to one runtime normal form. This separation lets conversion remain controlled while still proving that the categorical and groupoidal readings agree.

25.2 A Product Path Has Two Coordinates

Consider two groupoidal classifiers A and B . There are two evident ways to form a category of paths of pairs:

$$\text{Path}(A \times B) \quad \text{and} \quad \text{Path}(A) \times \text{Path}(B). \quad (25.4)$$

Their objects are the same pairs. Their category heads, however, are not declared definitionally equal. The comparison is instead carried by a canonical functor

$$\chi_{A,B} : \text{Path}(A \times B) \longrightarrow \text{Path}(A) \times \text{Path}(B), \quad (25.5)$$

which is judgmentally the identity on objects. For $s = (a_0, b_0)$ and $t = (a_1, b_1)$, its action on homs reads

$$(s = t) \longrightarrow (a_0 = a_1) \times (b_0 = b_1). \quad (25.6)$$

The forward map *splits* a path of pairs by projecting it to the two coordinates. Conversely, a pair

$$p : a_0 = a_1, \quad q : b_0 = b_1 \quad (25.7)$$

can be *joined* to a path

$$\langle p, q \rangle : (a_0, b_0) = (a_1, b_1). \quad (25.8)$$

The construction of (25.8) uses the stable path view of a dependent pair. A path in a sigma type consists of a base path together with a path over it in the fibre. For an ordinary product the fibre is constant, so the second component of (25.7) is converted into the required path-over and the two pieces are assembled. Splitting after joining and joining after splitting are both propositionally the identity. Hence, for every pair of endpoints, (25.6) is a type equivalence:

$$((a_0, b_0) = (a_1, b_1)) \simeq ((a_0 = a_1) \times (b_0 = b_1)). \quad (25.9)$$

Formal status — checked. Theorem 25.1 (product-path closure). Evidence `GROUPOIDAL-PRODUCT-CLOSURE`. The canonical comparison (25.5) is judgmentally identity on objects, and its actual action on every hom has an explicit split/join equivalence. The theorem does not add a category-head rewrite between the two categories in (25.4), nor does it postulate a whole inverse functor or a complete equivalence of categories.

The restraint in the final sentence is useful. The mathematical content needed by later transport arguments is already present: a product path may be read componentwise and component paths may be assembled. Turning that fact into a new definitional equality of large category expressions would choose a global normal form and create interactions with every consumer of products and paths. Homwise equivalence records the invariant without forcing such a choice.

This example illustrates what **groupoidal closure** means in the present development. It is not the assertion that every categorical former preserves path structure. It is a checked comparison for a selected former—in this case, products—showing how the path realization is recovered from the component realizations. Other formers require their own comparison or a later general closure theorem.

25.3 Three Routes Through Dependent Transport

The split/join theorem becomes more informative when the codomain depends on the pair. Let

$$P : A \times B \longrightarrow \mathcal{U} \quad (25.10)$$

be a groupoidal family, let $p : a_0 = a_1$ and $q : b_0 = b_1$, and take $u : P(a_0, b_0)$. Joining p and q gives the simultaneous path (25.8), so primitive equality transport gives a direct route

$$T_{\text{dir}}(p, q, u) : P(a_1, b_1). \quad (25.11)$$

There are also two broken-line routes through the square of indices. The base-first route moves from (a_0, b_0) to (a_1, b_0) along p , then from (a_1, b_0) to (a_1, b_1) along q . The fibre-first route changes the B -coordinate before the A -coordinate:

$$\begin{aligned} T_{A;B}(p, q, u) &: P(a_0, b_0) \longrightarrow P(a_1, b_0) \longrightarrow P(a_1, b_1), \\ T_{B;A}(p, q, u) &: P(a_0, b_0) \longrightarrow P(a_0, b_1) \longrightarrow P(a_1, b_1). \end{aligned} \quad (25.12)$$

All three expressions have the same endpoint, but they are not selected as one judgmental normal form. Equality induction on p and q instead proves two comparisons

$$T_{\text{dir}}(p, q, u) = T_{A;B}(p, q, u), \quad T_{\text{dir}}(p, q, u) = T_{B;A}(p, q, u). \quad (25.13)$$

At reflexivity, direct transport and both sequential routes reduce to the original u . The general comparisons are then generated by the same J principle. Reversing the first path in (25.13) and following the second gives the coherence edge between the broken-line routes:

$$T_{A;B}(p, q, u) = T_{B;A}(p, q, u). \quad (25.14)$$

Equations (25.11)–(25.14) form the transport diamond

$$\begin{array}{ccc} & T_{\text{dir}}(p, q, u) & \\ \swarrow & & \searrow \\ T_{A;B}(p, q, u) & & T_{B;A}(p, q, u), \end{array} \quad (25.15)$$

where the lower edge is induced by the two displayed comparisons. The diamond answers the ordering question raised by product transport. There is no need to declare that transport is first- A -then- B , nor that it is first- B -then- A . Direct transport is the neutral centre. Both sequential calculations are valid, and their agreement is retained as data that can itself be transported or acted upon at a higher level.

Nothing Gray-like is required for this theorem. Gray interchange concerns directed two-dimensional composition when comparison cells need not be invertible. Here the base arrows are equality paths and the two routes are compared by equality induction. Chapter 28 will return to the directed interchange after the groupoidal examples have made this distinction visible.

Formal status — checked. Theorem 25.2 (the product-transport diamond). Evidence GROUPOIDAL - PRODUCT - TRANSPORT . Transport along the joined product path agrees with both coordinate orders, and the comparisons compose to the coherent lower edge (25.14). The construction uses primitive right-based equality induction; it introduces no product-specific transport axiom and no second eliminator.

25.4 One Transport, Three Presentations

The family P in (25.10) can also be presented categorically. Apply the path category to every fibre and use path action to obtain a displayed category over $\text{Path}(A \times B)$. A path $r : s = t$ in the base then acts on a fibre object $u : P(s)$ by displayed functorial transport. In symbols, the three readings are

$$\begin{aligned} T_J(r, u) &= \text{primitive equality transport,} \\ T_{\text{disp}}(r, u) &= \text{displayed categorical action,} \\ T_{\text{out}}(r, u) &= \text{structured induction over outgoing paths.} \end{aligned} \quad (25.16)$$

The second line sees r as an arrow of the path category. The third sees (t, r) as an object of the outgoing-path category based at s . These descriptions organize transport differently, but both compare to the first:

$$T_{\text{disp}}(r, u) = T_J(r, u), \quad T_{\text{out}}(r, u) = T_J(r, u). \quad (25.17)$$

Thus the displayed and `PathOut` interfaces are not alternative axioms for transport. They are categorical structures whose computation is justified by the primitive equality eliminator. Conversely, (25.17) shows that J is not stranded in a purely syntactic equality layer. It can be used through the same displayed-action and section interfaces that support directed dependent type theory.

There is a deliberate difference between *computes* and *agrees*. Primitive J computes judgmentally on reflexivity. The two structured expressions in (25.16) agree propositionally with it, including at the reflexive case; they are not installed as additional runtime reductions. This is enough to move between representations in a proof while avoiding three rival normal forms for the same operation.

The result is a useful design test. A computational homotopy layer need not duplicate a primitive eliminator for every categorical presentation of a family. It can keep one J , expose structure through functorial interfaces, and prove that the interfaces return to the same transport. The product diamond then becomes a substantive consumer of the arrangement rather than a second definition of it.

25.5 When Directed Laxity Becomes Pseudo

We can now return to the whole path action (25.3). In the directed calculus, a functor carries a compositor comparing two ways of acting on composable arrows. For $p : x = y$ and $q : y = z$, specialize that generic compositor to $\text{Path}(f)$. Its source and target have the familiar readings

$$\text{ap}_f(p) \cdot \text{ap}_f(q) \quad \text{and} \quad \text{ap}_f(p \cdot q), \quad (25.18)$$

where $\cdot$ denotes path concatenation in diagrammatic order. Formally, the kernel retains represented postcomposition expressions as the compositor's runtime endpoints. Separate paths compare those endpoints with the readable `ap`/concatenation expressions in (25.18). The presentation is therefore recognizable without selecting the notation of (25.18) as a second normal form.

For an arbitrary directed codomain, the compositor is a directed cell and need not have an inverse. Here its codomain is

$$\text{Hom}_{\text{Path}(B)}(f(x), f(z)) = (f(x) = f(z)). \quad (25.19)$$

A cell between the two arrows in (25.18) is consequently an equality between paths. Path symmetry supplies its inverse. No inverse field is postulated and no second pseudofunctor hierarchy is introduced. The generic lax witness has become a pseudo witness because it landed in a groupoidal hom.

This gives a simple three-way interpretation of the same internal action:

$$\begin{array}{ll} \text{directed target} & : \quad \text{the compositor may be noninvertible,} \\ \text{path target} & : \quad \text{the compositor is invertible by symmetry,} \\ \text{selected strict profile} & : \quad \text{the compositor specializes to identity.} \end{array} \quad (25.20)$$

Strictness and pseudo-functoriality are therefore not obtained by erasing the generic witness. They are two special behaviours of it. This is especially important when a strict specialization is also available. The witness cell remains visible before that specialization, while an explicit strict profile can state exactly where identity behaviour is intended. Chapter 28 will use that distinction when comparing Cartesian and Gray-style closure.

The compositor is retained before it is capped at the particular path p . At that whole level it is a transformation, functorial in the first path. Taking its next hom action yields a functor from paths between paths to paths between the corresponding compositor endpoints. Thus the construction has not discarded higher input merely to prove pointwise invertibility. It preserves one explicit next stage of the same internal action, and the generic action can be iterated further.

Formal status — checked. Theorem 25.3 (path-realized pseudo-laxity). Evidence `PATH-PSEUDO-LAXITY`. The generic compositor of $\text{Path}(f)$ decodes to an equality between paths; symmetry gives its inverse; its formal endpoints compare propositionally with the two expressions in (25.18); and its whole form retains one next hom action. No new rewrite or unification rule, pseudofunctor classifier, inverse record, or complete weak ω -groupoid theorem is claimed.

Theorem 25.3 is small but conceptually decisive. It shows that the directed and groupoidal layers do not need unrelated accounts of coherence. A witness generated by the directed internal-action machinery can be read as a path when the target says that its arrows are equalities. Invertibility then comes from the target's geometry. The change from lax to pseudo is semantic in the best computational sense: it is witnessed by the actual type of the cell.

25.6 Closure, Truncation, And Free Inversion

Three nearby operations can now be separated cleanly.

Groupoidal closure starts with groupoidal data and asks whether a former preserves it. Theorem 25.1 answers this for the selected product comparison, hom by hom. **Truncation** starts with a groupoidal classifier and reflects it to a prescribed homotopy level. The active tower and its eliminators were introduced in Chapter 7; Chapter 26 will use the 0-truncated Integer line when calculating the loop space of the Circle. **Groupoidification** starts with a directed category and freely makes its arrows invertible, together with the coherence required by composition. That is neither path exposure nor truncation, and it will occupy Chapter 27.

The boundaries of this chapter are just as important as its theorems. The product comparison is identity on objects and an equivalence on each hom, but is not promoted to a whole category equivalence. The transport comparisons are propositional rather than competing conversion rules. The path-valued compositor retains higher action, but does not by itself establish a complete weak ω -groupoid semantics or a global normalization theorem. These are not missing qualifications attached after the mathematics; they locate the exact reusable interface that has been checked.

These distinctions suggest a useful discipline for working between the two layers. When equality-local data should participate in directed machinery, place it in a path category and preserve the whole categorical action for as long as possible. Do not cap immediately at objects or arrows if a later argument may need the next hom action. When a former such as a product has both a groupoidal and a categorical presentation, compare the actual object and hom actions before considering a new equality of classifier heads. And when a directed comparison lands in a path-valued hom, obtain invertibility from path reversal rather than copying the comparison into an unrelated pseudo structure.

The common theme is that coherence should be **observed where it lives**. Product paths live homwise, so their split/join theorem is homwise. Dependent transport lives in a fibre, so its order comparison is an equality in that fibre. The functor compositor lives one dimension above arrows, so its path-valued realization is an equality between paths and retains a next action. None of these facts is improved by flattening it prematurely into an external collection of equations. Keeping the owner whole makes both the computation and its possible iteration visible.

This is also why the word *shadow* does not mean approximation. The path category forgets none of the equality evidence it exposes. Rather, it changes the angle from which that evidence is viewed: a path becomes an arrow, a path-over becomes displayed action, and higher equality becomes higher categorical action. What remains outside the shadow is genuinely directed information—arrows that were never equalities in the first place. Free inversion will act on precisely that remainder.

The next chapter gives that interface its first geometric test. The Circle is generated by one point and one reversible loop. Mapping it into a type X should therefore amount to choosing a point of X and a loop at that point. When X is the universe of sets and the loop acts by successor, repeated transport records winding number. The result is the Integer line—not because the Circle was declared to be arithmetic, but because groupoidal motion can be followed forward and backward.

26. The Circle And The Integer Line

The walking endomorphism of Chapter 8 has one point and one directed generator. Every based arrow is a finite forward composite, so its arithmetic is the arithmetic of natural numbers. The Circle also has one point and one generator, but its generator is an equality path. It can be followed in either direction. Forward powers are joined by powers of the inverse, and the corresponding arithmetic becomes the integer line.

This resemblance is exact enough to be useful and dangerous enough to demand care. The Circle is not obtained by replacing the natural-number answer in the WalkingEnd theorem with integers. Its loop is intrinsically groupoidal; its eliminator must act on dependent paths; and the inverse generator is derived from path reversal rather than postulated as another arrow. The integer answer is then *calculated* by a universal-cover encode–decode argument. Only in Chapter 27 will the relationship with the directed walking endomorphism be promoted to a universal free-inversion theorem.

The calculation has four layers. First, successor on natural numbers is localized to an invertible shift, producing an internal Integer classifier. Second, the Circle is given its point, loop, and dependent computation. Third, univalence turns integer successor into the monodromy of a family over the Circle; transport in this family records winding number. Finally, integer-indexed loop powers decode winding numbers back to paths. The two directions are inverse not only at the base point but fibre-wise over every endpoint of the Circle.

26.1 Integers By Inverting Successor

The Integer classifier is not introduced as a new datatype with positive and negative constructors. It is obtained from the sequential telescope

$$\mathbb{N} \xrightarrow{\text{succ}} \mathbb{N} \xrightarrow{\text{succ}} \mathbb{N} \xrightarrow{\text{succ}} \dots \quad (26.1)$$

by the set-truncated telescope-localization construction of Chapter 7. Write

$$\text{Integer} = \text{Tel}(\mathbb{N}, \text{succ}). \quad (26.2)$$

A representative at stage n with value x will be denoted $[n, x]$ and has the intended reading $x - n$. The telescope constructor identifies the diagonal step

$$[n + 1, x + 1] = [n, x]. \quad (26.3)$$

Thus $[0, 0]$ represents zero, $[0, x]$ represents the nonnegative integer x , and $[1, 0]$ represents negative one. No subtraction operation is needed to define the carrier. The notation $x - n$ explains the invariant respected by (26.3); the formal object is the localized telescope itself.

This presentation separates arithmetic content from notation. A signed datatype would choose, at the outset, between a nonnegative and a negative constructor and would then require normalization at their boundary. The telescope instead records a history: n applications of the inverse shift and a

ent natural value x . The diagonal equality (26.3) performs the cancellation that signed notation normally hides. A later theorem may choose canonical representatives, but the Circle proof does not depend on that choice.

The construction also isolates exactly what “integer” means at this stage. The proof needs a set with a distinguished zero and an invertible successor, together with an eliminator that respects the localization equation. It does not need addition, multiplication, order, or the universal property of the group completion of the natural-number monoid. Those structures may be built later from the same carrier. Declining to presuppose them makes the eventual loop calculation more informative: integer behaviour is forced by reversible successor, not smuggled in through a ready-made ring.

The forward telescope action descends to integer successor. Shifting the stage supplies its inverse, predecessor:

$$\text{succ}, \text{pred} : \text{Integer} \longrightarrow \text{Integer}, \quad \text{succ pred} \simeq \text{id}, \quad \text{pred succ} \simeq \text{id}. \quad (26.4)$$

Consequently successor is retained as a type equivalence. Univalence turns that equivalence into a path

$$\text{ua}(\text{succ}) : \text{Integer} = \text{Integer}, \quad (26.5)$$

and transport along (26.5) agrees with the actual successor function. This is the path that will drive the universal cover.

The telescope is set-truncated by construction. Its dependent eliminator is therefore restricted to set-valued motives. To define a section over all integers, it is enough to define it on every stage representative $[n, x]$, give a dependent path over (26.3), and show that each target fibre is a set. This restriction is not an inconvenience hidden by notation: it will match exactly the one-dimensional boundary of the Circle loop space used by the decoder.

Formal status — checked. Evidence `INTEGER-LOCALIZATION-LINE`. The Integer carrier is a transparent facade over the successor telescope; successor and predecessor have explicit inverse paths; successor is a retained equivalence and universe path; and set-targeted elimination computes on stage representatives. No addition, signed normal-form equivalence, ordered-ring structure, or universal additive-group-completion theorem is claimed here.

26.2 A Circle That Computes On Its Loop

The groupoidal Circle is generated by

$$\text{Circle} : \mathcal{U}, \quad \text{base} : \text{Circle}, \quad \text{loop} : \text{base} = \text{base}. \quad (26.6)$$

Its inverse loop is simply loop^{-1} , obtained by equality symmetry. It is not a second constructor. The signature also states its selected one-dimensional boundary: every path type of the Circle is a set. This evidence is later checked from another direction when the based loop space is computed to be the set-valued Integer classifier.

The dependent eliminator has the usual geometric form. Given a family $D : \mathbf{Circle} \rightarrow \mathcal{U}$, an element $b : D(\mathbf{base})$, and a dependent path

$$\ell : \text{PathOver}_D(\mathbf{loop}; b, b), \quad (26.7)$$

it produces a section

$$\text{circle_ind}(D, b, \ell) : \prod_{x:\mathbf{Circle}} D(x). \quad (26.8)$$

The section computes judgmentally at the point constructor. More significantly, its canonical dependent action computes judgmentally at the path constructor:

$$\begin{aligned} \text{circle_ind}(D, b, \ell)(\mathbf{base}) &\equiv b, \\ \text{apd}(\text{circle_ind}(D, b, \ell), \mathbf{loop}) &\equiv \ell. \end{aligned} \quad (26.9)$$

The second rule retains the full path-over type in (26.7); it does not erase the transport of endpoints. It is also narrowly owned by the Circle eliminator and generating loop. An arbitrary dependent function on the Circle does not acquire this reduction merely because it is evaluated on **loop**.

Ordinary recursion is the constant-family case of (26.8). For a point $b : B$ and loop $\ell : b = b$, it gives a function

$$\text{circle_rec}(B, b, \ell) : \mathbf{Circle} \rightarrow B. \quad (26.10)$$

Its dependent action inherits the second reduction in (26.9), represented as the constant-family path-over built from ℓ . The familiar ordinary equation

$$\text{ap}(\text{circle_rec}(B, b, \ell), \mathbf{loop}) = \ell \quad (26.11)$$

is derived propositionally. It is not a second runtime rule. This distinction is easy to miss on paper because (26.9) and (26.11) express the same mathematical computation. In the formal calculus they are different observers: **apd** sees the primitive dependent constructor action, while ordinary **ap** is reconstructed through constant-family transport.

Dependent computation is the stronger statement. In (26.7), the endpoint b is transported around the nontrivial base loop before it is compared with itself. The path-over remembers that movement even when the family is not constant. Ordinary **ap** sees only the constant-family shadow after the transport has been converted back into a path of B . If one installed only (26.11), the universal-cover decoder would still need a separate principle to control the function family in (26.20). Rule (26.9) supplies that control at the actual higher-constructor owner.

This is the first HIT in the book whose higher constructor has a selected judgmental dependent beta. The lesson is not that every appealing higher equation should become a rewrite. The safe unit of computation is the action of the eliminator on its own constructor, with its full dependent type ret-

ained. Readable constant-family equations may remain propositional when a second reduction would duplicate normal forms or disturb unrelated equality proofs.

Formal status — checked. Evidence `CIRCLE-HIT-COMPUTATION`. Point beta and dependent loop beta are runtime computations at their stable owners; the named dependent beta is reflexivity after reduction. Constant-family recursion inherits the dependent computation. Its ordinary `ap` equation remains a checked propositional path, and unrelated sections do not collapse to the supplied loop datum.

Attribution and adaptation boundary. The Circle signature and the universal-cover rhythm below structurally adapt the [HoTT Book](#), Sections 6.2 and 8.1. The present account uses the active emdash dependent computation boundary, successor-localized Integer rather than the Book's signed/quotient presentation, and whole categorical realizations. It does not import the HoTT Book's flattening proof or silently claim all of its later homotopy-group consequences.

26.3 The Universal Cover As Monodromy

The classical universal cover of the circle may be pictured as a helix over a circle. Following the positive loop raises the lift by one level; following the negative loop lowers it. Type theory replaces the helix by a family whose fibres are integers and whose monodromy is successor.

Equation (26.5) supplies exactly the loop in the universe needed by Circle recursion. Define

$$\text{Code} : \text{Circle} \longrightarrow \mathcal{U} \quad (26.12)$$

by

$$\text{Code}(\text{base}) \equiv \text{Integer}, \quad \text{ap}(\text{Code}, \text{loop}) = \text{ua}(\text{succ}). \quad (26.13)$$

The loop equation in (26.13) is the ordinary `ap` observation of Circle recursion and is therefore propositional, consistently with (26.11). Transport along it nevertheless has the intended computational content:

$$\begin{aligned} \text{transport}^{\text{Code}}(\text{loop}, z) &= \text{succ}(z), \\ \text{transport}^{\text{Code}}(\text{loop}^{-1}, z) &= \text{pred}(z). \end{aligned} \quad (26.14)$$

Univalence is essential here. Successor is not the identity function on integers, so an ordinary reflexive universe path could not encode the desired monodromy. The equivalence-to-path direction of univalence turns the actual self-equivalence into a loop of classifiers, and its transport comparison returns the underlying successor map.

For any endpoint $x : \text{Circle}$, a path $p : \text{base} = x$ can now be lifted into the code family. Start at integer zero and transport it along p :

$$\text{encode}_x(p) := \text{transport}^{\text{Code}}(p, 0) : \text{Code}(x). \quad (26.15)$$

Encoding reflexivity computes to zero. Encoding the generating loop is propositionally successor of zero. More generally, concatenating a positive loop applies successor and concatenating an inverse loop applies predecessor. The encoder is therefore the winding-number observer: it converts abstract groupoidal motion into a point of the localized integer line.

The calculation can be followed compositionally. If a based path first follows p and then follows q , transport in the code family first lifts zero along p and then acts along q . Each occurrence of **loop** contributes successor, and each occurrence of **loop**⁻¹ contributes predecessor. Adjacent inverse pairs cancel through (26.4). Thus a composite such as

$$\mathbf{loop} \cdot \mathbf{loop}^{-1} \cdot \mathbf{loop} \cdot \mathbf{loop}$$

is observed as two. This example is intuition rather than a claim that every path arrives as a parsed word. The encode map works on arbitrary equality evidence; the word picture explains its behaviour on paths constructed from the generator and its inverse.

The family point of view also explains the name *cover*. Over each Circle point there is an Integer fibre, and travelling once around the base permutes that fibre by successor. What is constructed here is the type-theoretic family and its monodromy. No topological space of real numbers, local triviality atlas, or external covering-space apparatus is assumed.

It is tempting to stop at the base fibre and define only

$$(\mathbf{base} = \mathbf{base}) \longrightarrow \mathbf{Integer}. \quad (26.16)$$

That specialization is the desired forward map, but it is too narrow for the hard inverse proof. Path induction cannot directly simplify an arbitrary loop whose two endpoints have both been fixed at the base. The crucial move, as in the HoTT encode–decode method, is to retain the endpoint x and work fibrewise with (26.15).

26.4 Decoding Integer Powers

At the base point, decoding should send an integer to the corresponding power of the generating loop. Natural powers are obtained by repeatedly appending **loop**; inverse powers repeatedly append **loop**⁻¹:

$$\begin{aligned} \mathbf{loop}^0 &= \mathbf{refl}_{\mathbf{base}}, \\ \mathbf{loop}^{n+1} &= \mathbf{loop}^n \cdot \mathbf{loop}, \\ \mathbf{loop}^{-(n+1)} &= \mathbf{loop}^{-n} \cdot \mathbf{loop}^{-1}. \end{aligned} \quad (26.17)$$

The telescope presentation asks for a slightly subtler definition. A stage representative $[n, x]$ should decode to the loop power corresponding to $x - n$. Rather than first choosing a signed normal form, the construction recurses simultaneously on n and x . Along the diagonal it cancels one positive and one negative step, so that

$$\text{power}(n + 1, x + 1) \equiv \text{power}(n, x). \quad (26.18)$$

The coherence required by the telescope relation (26.3) is therefore literal reflexivity. Integer elimination then gives the based decoder

$$\text{decode}_{\text{base}} : \text{Integer} \longrightarrow (\text{base} = \text{base}), \quad (26.19)$$

with the expected computations at zero, nonnegative representatives, and negative one. The required target is a set because the Circle signature says that its path types are sets. This is the exact point where the truncation level of the telescope eliminator and the dimension of the Circle meet.

The three boundary cases make the construction concrete. At stage zero, $[0, x]$ decodes to the x th positive power. At value zero, $[n, 0]$ decodes to the n th inverse power. When both indices are successors, the definition removes one loop and one inverse loop simultaneously and returns to the preceding stage. These are computations of the representative-level decoder, not a post hoc proof that two separately normalized signed expressions happen to agree.

The decoder must now be generalized over the endpoint, just as the encoder was. Consider the family

$$M(x) := \text{Code}(x) \longrightarrow (\text{base} = x). \quad (26.20)$$

At the base, the desired inhabitant is (26.19). To apply Circle induction one must show that this function returns to itself over **loop**. Transport in the domain of (26.20) uses predecessor, transport in the codomain appends the loop, and the required comparison is therefore the cancellation law

$$\text{loop}^{z-1} \cdot \text{loop} = \text{loop}^z. \quad (26.21)$$

Positive and negative cases are proved from path composition, reversal, and inverse cancellation. The resulting dependent loop datum feeds the Circle eliminator and produces

$$\text{decode}_x : \text{Code}(x) \longrightarrow (\text{base} = x) \quad (26.22)$$

for every x . This is the step that turns the obvious based loop-power function into a morphism of the entire path fibration and code family.

Endpoint generalization is therefore not merely a clever way around a weak induction tactic. It states the invariant at its natural level. The encoder and decoder are maps between two families over the Circle: the outgoing-path family $x \mapsto (\text{base} = x)$ and the code family $x \mapsto \text{Code}(x)$. The loop coherence for (26.22) says that decode commutes with their monodromies. Once the whole family map exists, the based loop function is obtained by ordinary specialization rather than by fixing endpoints before the structure has been built.

26.5 The Two Round Trips

The composite from paths to codes and back is now the easy direction. For $p : \mathbf{base} = x$, ordinary endpoint path induction reduces p to reflexivity. Encoding reflexivity is zero and decoding zero is reflexivity, so

$$\mathbf{decode}_x(\mathbf{encode}_x(p)) = p. \quad (26.23)$$

The reverse composite begins at the base. One proves by natural and telescope induction that positive loop powers encode to $[0, n]$, inverse powers encode to $[n, 0]$, and the general simultaneous power encodes to its own representative $[n, x]$. Integer induction then gives

$$\mathbf{encode}_{\mathbf{base}}(\mathbf{decode}_{\mathbf{base}}(z)) = z. \quad (26.24)$$

To extend (26.24) from the base fibre to every x , observe that every $\mathbf{Code}(x)$ is a set. At the base this is the sethood of **Integer**; the statement that a fibre is a set is itself propositional, so **Circle** induction propagates it around the generating loop without a new choice of coherence. The desired equality is likewise proposition-valued. A second **Circle** induction therefore yields

$$\mathbf{encode}_x(\mathbf{decode}_x(c)) = c \quad (26.25)$$

for all endpoints and all codes.

Equations (26.23) and (26.25) package an endpoint-dependent family of equivalences. At the base point it gives the central calculation

$$(\mathbf{base} = \mathbf{base}) \simeq \mathbf{Integer}. \quad (26.26)$$

Now form the categorical realization $\mathbf{Circle}_{\mathbf{cat}} := \mathbf{Path}(\mathbf{Circle})$. Its based hom carrier is definitionally the same loop space, so there is also

$$\mathbf{Hom}_{\mathbf{Circle}_{\mathbf{cat}}}(\mathbf{base}, \mathbf{base}) \simeq \mathbf{Integer}. \quad (26.27)$$

This is the precise meaning of the shorthand “ $\mathbf{Hom}(\mathbf{Circle}) = \mathbb{Z}$.” It concerns the based endomorphism carrier of the path category, not the type of all self-maps of the **Circle**.

The result is retained at three levels. Equation (26.26) is an intrinsic type equivalence. Equation (26.27) reads the same carrier as a categorical hom. Applying the whole path-category action to the selected encoder gives a categorical equivalence

$$\mathbf{Hom}_{\mathbf{cat}}(\mathbf{base}, \mathbf{base}) \simeq_{\omega} \mathbf{Path}(\mathbf{Integer}), \quad (26.28)$$

whose forward functor acts by the encoder and retains higher equality action. Neither category head is rewritten to the other. The selected one-dimensional **Circle** evidence and the equivalence with the set **Integer** also give two independent proofs that the based hom is a set.

The distinction among these three packages prevents an easy overstatement. A `TypeEquiv` is enough to transport properties of the carrier and to select the encode/decode functions. The categorical hom reading says where that carrier occurs in the directed language. The whole categorical equivalence adds action on equalities between loops and on their iterated equalities. None of them, by itself, is a judgmental identification of the two category expressions, and none yet says that the equivalence preserves a separately packaged group operation. Each level answers a different downstream question without forcing the strongest possible interface on every reader.

Formal status — checked. Theorem 26.1 (the Circle loop space). Evidence `CIRCLE-LOOP-INTEGER`. Endpoint-dependent encode and decode are inverse. Their based specializations form an explicit `TypeEquiv` between the intrinsic loop space and successor-localized Integer; the categorical hom has the same carrier; and a separate whole equality-valued categorical equivalence is retained. No category-head rewrite or group-structure preservation theorem is included in this result.

26.6 Monodromy Beyond Successor

The universal cover is one instance of a general construction. Let A be a groupoidal classifier and let $e : A \simeq A$ be a self-equivalence. Univalence turns e into a universe path, and Circle recursion constructs a family

$$\begin{aligned} \text{Mon}_e : \text{Circle} &\longrightarrow \mathcal{U}, & \text{Mon}_e(\text{base}) &= A, \\ \text{ap}(\text{Mon}_e, \text{loop}) &= \text{ua}(e). \end{aligned} \tag{26.29}$$

Transport around the actual loop agrees with the forward map of e . Taking $A = \text{Integer}$ and $e = \text{succ}$ recovers the code family above. Taking another automorphism produces another local system on the same Circle without changing the HIT.

There is also a directed shadow. Restrict the family along the canonical map from the walking endomorphism to the Circle. The resulting directed representation remembers A at its point and the univalence path of e at its forward generator. The checked restriction–extension comparison recovers that whole representation, rather than only its two displayed components. Chapter 27 will explain the universal mapping theorem that makes this restriction canonical; here it serves as a geometric consumer of monodromy.

Formal status — checked. Evidence `CIRCLE-MONODROMY`. A selected self-equivalence determines a Circle-indexed groupoid family; its base and loop observations have the expected values; transport around the loop agrees with the equivalence's forward map; and whole restriction recovers the corresponding `WalkingEnd` representation. The result is a consumer of the concrete `WalkingEnd–Circle` universality theorem, not a second primitive monodromy axiom.

26.7 Connected Without Choosing Paths

The Circle has one point constructor, so one expects every point to be reachable from the base. A function choosing an actual path $\mathbf{base} = x$ for every x would say too much: it would contract the Circle and destroy its nontrivial loop space. The correct statement is mere connectedness:

$$\prod_{x:\mathbf{Circle}} \|\mathbf{base} = x\|_{-1}. \quad (26.30)$$

At the base, reflexivity supplies the truncated witness. Every fibre in (26.30) is a proposition, so there is a unique dependent path over the generating loop. Circle induction then constructs the section without ever choosing an untruncated global path.

This statement has a concrete truncation consequence. Let $\|\mathbf{Circle}\|_0$ be the classified set truncation and take the image of $\mathbf{base}$ as centre. Mere connectedness can be eliminated into equality inside this set, because its path types have the required truncation level. First one obtains a path from the centre to the image of every Circle point; set-truncation induction then extends it to every point of the truncation:

$$\mathbf{isContr}(\|\mathbf{Circle}\|_0). \quad (26.31)$$

Contractibility is retained as evidence. The carrier of the set truncation is not judgmentally replaced by Unit. This preserves the distinction between a universal construction characterized by elimination and a convenient chosen normal form.

Connectedness and the loop calculation complement rather than contradict one another. Equation (26.30) says that the Circle has only one component after paths are merely inhabited; equation (26.26) says that the ways of returning to the base retain an entire Integer classifier. Set truncation forgets those different return paths while preserving the component, which is why it is contractible even though the Circle itself is not. The proof performs that forgetting through the truncation eliminator instead of declaring the loops irrelevant in the original type.

Formal status — checked. Evidence `CIRCLE-CONNECTED-TRUNCATION`. The propositional truncation of each based path fibre gives mere connectedness, and restricted truncation elimination proves the set truncation contractible. The result selects no global untruncated path and adds no rewrite from the set-truncated Circle to Unit.

26.8 From Counting To Free Inversion

We can now place the two arithmetic calculations side by side:

$\mathbf{shape}$ walking endomorphism Circle	$\mathbf{generator}$ directed arrow invertible path	$\mathbf{based\ hom\ classifier}$ $\mathbb{N}$ Integer.	(26.32)
--	---	---	---------

The change from $\mathbb{N}$ to Integer is not an analogy imposed after the proof. It is the effect of reversibility inside the proof. Positive composites of the directed generator remain distinct natural powers because no inverse exists. Positive and negative powers of the Circle loop cancel because path symmetry supplies an inverse. The telescope relation makes the same cancellation computational on integer representatives.

The canonical map from the walking endomorphism sees only the upper half of this arithmetic. Its n th directed power maps to the nonnegative integer $[0, n]$ and to the n th positive Circle loop. Nothing in the directed source names $[n, 0]$ or loop^{-n} : those elements appear because the target is groupoidal. Free inversion must therefore do more than preserve the old powers. It must add the reverse motion and impose the cancellations that make it inverse, while retaining how all of this acts on higher cells.

This observation also explains why the loop-space calculation is such a good test for the universal property. If restriction from the Circle to the walking endomorphism forgot too much, an extension could choose incompatible actions on negative powers. If it imposed too much, ordinary directed representations whose generator lands in a groupoidal target might fail to extend. The correct theorem says that the image of the one directed generator already determines the whole reversible action: its inverse and all integer powers are forced by the target's path structure. Chapter 27 establishes that claim at the level of whole mapping objects.

Several boundaries remain explicit. Integer has not yet been packaged as an additive group, so Theorem 26.1 does not separately prove that path composition corresponds to integer addition. The chapter does not provide a generic HIT declaration language, a proof that every categorical former has a groupoidal specialization, or a global normalization theorem for computational homotopy type theory. And although the Circle is connected and has a set-valued loop space, the selected results here are not advertised as a complete formal calculation of every homotopy group.

What has been obtained is enough for the next question. The walking endomorphism maps to the Circle by sending its directed generator to the generating loop. Is every map from the walking endomorphism into a groupoidal target extended uniquely, in the appropriate whole sense, across this free inversion? And can the same principle be stated for an arbitrary directed category? Chapter 27 turns the arithmetic evidence of this chapter into that universal property.

27. Free Inversion And Groupoidification

Chapter 26 calculated what happens when one directed endomorphism acquires an inverse: natural powers expand to integer powers. But a universal construction cannot be characterized by its elements alone. It must say what maps *out* of the realized object are, how they vary, and why their restriction remembers exactly the original directed data.

The relevant operation is **groupoidification**. Starting from a directed category C , it produces a groupoidal classifier $\mathbf{Groupoidify}(C)$ and a whole functor

$$\eta_C : C \longrightarrow \mathbf{Path}(\mathbf{Groupoidify}(C)). \quad (27.1)$$

The functor η_C is the unit of free inversion. Its object action names represented points; its arrow action turns a directed arrow into a path; its iterated hom action carries represented higher cells; and its compositor records how images of composable arrows compare with the image of their composite.

The universal property is tested against a groupoidal target G . A map $h : \mathbf{Groupoidify}(C) \rightarrow G$ restricts along (27.1) to a path-valued functor on C . Conversely, any $F : C \rightarrow \mathbf{Path}(G)$ extends to a map from the groupoidification. The checked theorem says that restriction and extension are inverse at the level of whole mapping categories:

$$\mathbf{Map}_{\mathbf{Grpd}}(\mathbf{Groupoidify}(C), G) \simeq_{\omega} \mathbf{Fun}(C, \mathbf{Path}(G)). \quad (27.2)$$

Before studying the arbitrary category C , we will derive (27.2) twice for finite shapes. The walking endomorphism tests inverse powers at one point. The walking arrow tests a generator whose end-points differ. Together they show why the general constructor must be a whole functor rather than a set of objects followed by an unrelated graph of arrows.

There are two complementary ways to recognize a free object. A constructor presentation says how its points and paths are generated and how its eliminator computes. A mapping property says that every interpretation of the generators extends, and that the extension is unique. Either view alone can hide an error. Constructors without uniqueness may admit unintended extra maps; an opaque equivalence of mapping carriers may assert the right answer without explaining how the generators compute. The constructions below keep both: point and dependent-cell beta rules on one side, whole beta/eta uniqueness on the other.

The word *whole* also changes the level of the claim. We are not merely counting functions in and out. The left and right sides of (27.2) are categories with homs, higher homs, and internal action. Restriction and extension act on those levels before they are packaged as an equivalence. This is what makes groupoidification reusable in later mathematics: a proof between representations is itself transported across the universal property.

27.1 The One-Point Test: WalkingEnd And Circle

There is a canonical functor

$$w : \mathbf{WalkingEnd} \longrightarrow \mathbf{Circle}_{\text{cat}} \quad (27.3)$$

sending the unique point to **base** and the directed generator to **loop**. Its action on the n th directed power is the n th positive Circle power. Following this action by the Circle encoder gives the canonical inclusion of natural numbers into the localized Integer line:

$$\text{encode}(w(\mathbf{gen}^n)) = [0, n]. \quad (27.4)$$

The comparison is retained not only for each n but as equality of the two whole carrier functions from **WalkingEnd** based arrows to Integer: map through the Circle and encode, or normalize to a natural number and include it. This is the arithmetic shadow of free inversion.

Now let G be any groupoidal target. Precomposition with w restricts a function $h : \mathbf{Circle} \rightarrow G$ to a functor

$$R_G(h) : \mathbf{WalkingEnd} \longrightarrow \mathbf{Path}(G). \quad (27.5)$$

Because restriction itself is a whole functor, it also acts on equalities between Circle functions and on their higher equalities. Evaluating its whole comparison at the walking point returns $h(\mathbf{base})$. Its generator observation is naturally dependent: the endpoint of the loop changes when the whole functor changes, so the comparison is a path-over whose target is $\mathbf{ap}_h(\mathbf{loop})$.

The inverse construction reads a path-valued **WalkingEnd** representation F . It extracts the image of the walking point and the image path of the generator, then applies Circle recursion:

$$E_G(F)(x) := \mathbf{circle_rec}(G, F(\mathbf{pt}), F(\mathbf{gen}), x). \quad (27.6)$$

The object action in (27.6) is only the first layer. The extension varies as a whole functor in F . A transformation between two representations in $\mathbf{Path}(G)$ is pointwise equality-valued, hence pointwise invertible. The whole univalence machinery turns it into a path of representations, and applying (27.6) gives the corresponding path of Circle functions. One more hom action remains available after that step.

The categorical-HIT uniqueness clauses state

$$R_G E_G = \text{id}, \quad E_G R_G = \text{id} \quad (27.7)$$

as paths between whole functors. Their projections recover the expected base and generator equations, but the whole statements are stronger: they also control transformations and higher action. Thus

$$\mathbf{Map}_{\mathbf{Grpd}}(\mathbf{Circle}, G) \simeq_{\omega} \mathbf{Fun}(\mathbf{WalkingEnd}, \mathbf{Path}(G)). \quad (27.8)$$

It is important that (27.8) holds for every groupoidal G , not only for the Circle itself or for the Integer classifier used in Chapter 26. A path-valued **WalkingEnd** representation in G consists of an object $a : G$ and a loop $p : a = a$, together with the functorial action inherited from the source.

Circle recursion turns exactly this pair into a function from the Circle. Conversely, evaluation at the base and action on the loop recover the pair. The whole theorem says there is no additional choice hidden at transformations or higher paths.

The one-point result also illustrates why free inversion differs from merely adding a formal inverse symbol. Once p^{-1} exists in G , all negative powers and their cancellation laws are already determined by equality. The extension does not ask the representation to supply an independent image for every negative word. This economy is the universal content behind the arithmetic passage from natural to integer powers.

Formal status — checked. Evidence `WE-GROUP-COMPLETION`. The functor (27.3) sends every directed natural power to the corresponding positive Circle power. For every groupoidal G , whole restriction and whole Circle extension give the fixed-forward equivalence (27.8), with beta/eta, dependent generator projections, and retained higher action. This theorem is the free inversion of one source shape, not by itself the generic category-indexed construction.

27.2 The Two-Endpoint Test: WalkingArrow And Interval

One point can conceal an important issue: every generator in WalkingEnd is an endomorphism. A generic directed arrow has a source and target that need not coincide. The next source shape is therefore

$$\text{src} \xrightarrow{\text{edge}} \text{tgt}, \quad (27.9)$$

the walking arrow. It is not an ad hoc three-field record. It is obtained from the join of two terminal categories, and its generator is a projection of the whole cross action. Consequently the source already retains a next hom action beyond the displayed edge.

Its groupoidal counterpart is the interval HIT:

$$\text{Interval} : \mathcal{U}, \quad i_0, i_1 : \text{Interval}, \quad \text{seg} : i_0 = i_1. \quad (27.10)$$

For a family $D : \text{Interval} \rightarrow \mathcal{U}$, the eliminator takes $b_0 : D(i_0)$, $b_1 : D(i_1)$, and a dependent path over **seg**. It computes judgmentally at both endpoints and at the canonical dependent action on the segment. As for the Circle, the ordinary constant-family `ap` equation is retained propositionally rather than installed as a second runtime normal form.

The comparison functor

$$j : \text{WalkingArrow} \longrightarrow \text{Path}(\text{Interval}) \quad (27.11)$$

sends source to i_0 , target to i_1 , and edge to **seg**. For any groupoidal G , restricting along j records the two endpoint values of an Interval function and its action on the segment. Extension uses Interval recursion on exactly those three pieces. The whole beta/eta paths give

$$\text{Map}_{\text{Grpd}}(\text{Interval}, G) \simeq_{\omega} \text{Fun}(\text{WalkingArrow}, \text{Path}(G)). \quad (27.12)$$

The right side of (27.12) has a particularly direct reading. A functor from the walking arrow chooses two objects $a_0, a_1 : G$ and one path $p : a_0 = a_1$. A transformation between two such functors chooses endpoint components compatible with that path, and its next action records equality of such compatibility data. The left side organizes the same information as a whole function on the Interval. Thus the equivalence tests not just two-point recursion but the dependent geometry of the segment.

This is where an underlying-graph construction would first become visibly insufficient. It could remember a source vertex, target vertex, and edge, but it would have to recover from elsewhere how transformations act at both endpoints and over the edge. In the present construction those observations are projections of whole functor paths. Endpoint variation is not metadata attached after the free groupoid has been formed.

Equation (27.12) is stronger evidence for the intended operation than the observation that the interval is contractible. Contractibility describes the homotopy type after it has been formed. The mapping theorem says why its two points and one path are freely generated by a directed arrow. It also tests the endpoint transport that an endomorphism-only theorem cannot see.

Formal status — checked. Evidence `WALKING-INTERVAL-GROUPOIDIFICATION`. The interval has judgmental endpoint and dependent-segment computation. Restriction and extension are inverse as whole functors for every groupoidal target, with separate endpoint and generator projections and retained next action. The two endpoints remain genuinely distinct in the tested interface.

27.3 One Whole Constructor For An Arbitrary Category

The two examples suggest what the generic signature must retain. For every category C , there is a primitive groupoidal HIT **Groupoidify**(C) and the whole unit (27.1). The constructor is not split into a point constructor for every object, a path constructor for every arrow, and an external list of coherence constructors. Those observations are projections of one iterable functor.

At the first two levels, the unit reads

$$\begin{aligned} x \in \text{Obj}(C) &\longmapsto \eta_C(x) : \mathbf{Groupoidify}(C), \\ f : x \rightarrow y &\longmapsto \eta_C(f) : \eta_C(x) = \eta_C(y). \end{aligned} \tag{27.13}$$

Reapplying the generic hom action exposes the image of a source 2-cell, and the process continues at represented higher levels. The use of the word “category-indexed” refers to this complete source C ; it does not mean that only the object set of C is retained.

This arrangement should be read as an indexed HIT signature. Formation is primitive at the groupoidal level; the whole unit is its constructor owner; and recursion supplies elimination. The theory does not pretend to construct the carrier by taking an external quotient of strings of arrows. Such a quotient would immediately face choices of word representation, normalization, and higher coherence. Instead, the computational interface specifies what represented cells do, while the mapping theorem specifies their universal uniqueness.

Nor does one need a primitive symbol named $\mathbf{Groupoidify}_n$ at every dimension. The first hom action of η_C already turns source arrows into paths. Its next hom action handles arrows between arrows, and iteration continues through the ordinary categorical classifiers. This is the same compression principle used throughout the book: internalization plus iteration replaces an indefinitely growing external declaration schema.

Given $F : C \rightarrow \mathbf{Path}(G)$, the recursor produces

$$\mathbf{rec}_F : \mathbf{Groupoidify}(C) \longrightarrow G. \quad (27.14)$$

It computes judgmentally on every represented source object, and its canonical dependent action on every represented source arrow computes to the corresponding arrow of F embedded in the constant motive:

$$\begin{aligned} \mathbf{rec}_F(\eta_C(x)) &\equiv F(x), \\ \mathbf{apd}(\mathbf{rec}_F, \eta_C(f)) &\equiv \mathbf{constPathOver}(F(f)). \end{aligned} \quad (27.15)$$

Arbitrary points and arbitrary paths of the groupoidification do not match these constructor reductions. The rules compute because the inputs are manifestly represented by the whole unit.

The recursor is itself organized as a whole extension functor

$$E_{C,G} : \mathbf{Fun}(C, \mathbf{Path}(G)) \longrightarrow \mathbf{Map}_{\mathbf{Grpd}}(\mathbf{Groupoidify}(C), G). \quad (27.16)$$

Its object action is (27.14), its first hom action maps transformations of representations to equalities of recursor functions, and a next action is retained. This target-varying mapping object is the computational eliminator needed by the universal property; a bare function for each individual F would not express how extension behaves on proofs between representations.

The requirement that G be groupoidal is doing real work. Each directed arrow $F(f)$ lands in a path category and is therefore invertible, so the recursor may interpret the inverse motion freely added to the source. If the target were an arbitrary directed category, a chosen image of f would not in general determine an image for its formal inverse. The right side of (27.16) is consequently not the ordinary category of all directed functors $C \rightarrow G$; it is the category of functors into the path realization of a groupoidal target.

At the same time, nothing forces the original arrows of C to have been invertible. The unit accepts all of them. Groupoidification changes their ambient interpretation rather than filtering the source. This is precisely the opposite of taking the core.

27.4 Composition Is Represented, Not Forgotten

Suppose $f : x \rightarrow y$ and $g : y \rightarrow z$ are composable arrows in C . Since η_C is a whole functor, its internal action supplies a compositor

$$\phi_{g,f} : \eta_C(g) \cdot \eta_C(f) \Longrightarrow \eta_C(g \circ f). \quad (27.17)$$

The codomain of the unit is a path category. Therefore the directed compositor is realized as an equality between paths and is invertible by symmetry, exactly as in Chapter 25. It is nevertheless an explicit cell: it is not definitionally collapsed to an identity term. Its whole transformation form retains a next hom action.

This observation explains why the construction begins with C , not with its underlying graph. A graph-level free groupoid would create inverse paths for edges but would then need composition, identities, and every represented higher relation reintroduced by hand. The unit functor already knows those relations. The recursor computation in (27.15) says that a path-valued representation consumes them using its own whole action.

Identity and composition are therefore not quotient equations checked after paths have been created. They are visible through functorial action. At an identity arrow the selected unit profile supplies the appropriate unit behaviour. At a composite, (27.17) compares the composite of the two image paths with the image of the source composite. A representation F carries the corresponding compositor, so extending F respects the same boundary. The generic theorem is sensitive to the categorical structure of C , not only to its collection of arrows.

In a strict specialization the two endpoints displayed in (27.17) may become convertible. That does not make the compositor term itself an identity, and the active negative check confirms it is not collapsed to one. This is useful for eventual migration away from historical global strict endpoint cuts: the coherence witness and its next action are already present rather than having to be reconstructed from an equation that erased them.

Nor is an infinite record of associators and higher laws added beside the unit. The compositor in (27.17) is projected from the same internal-action calculus used for ordinary and displayed functors. Its next action provides the first higher observation. This does not amount to a completed independent metatheory of all weak ω -groupoids, but it demonstrates that free inversion has not discarded composition or capped coherence at arrows.

27.5 The Generic Mapping Theorem

Restriction is obtained by applying the path-category action to a map $h : \mathbf{Groupoidify}(C) \rightarrow G$ and precomposing with η_C :

$$R_{C,G}(h) = \text{Path}(h) \circ \eta_C. \quad (27.18)$$

Like extension, $R_{C,G}$ is a whole functor. The categorical-HIT beta and eta clauses are whole paths

$$R_{C,G}E_{C,G} = \text{id}, \quad E_{C,G}R_{C,G} = \text{id}. \quad (27.19)$$

The first says that extending a complete path-valued representation and then observing it on every represented source cell returns the original representation. The second is uniqueness: a groupoidal map is determined by its restriction to the generating whole functor. Projecting (27.19) gives point-wise equalities, but the statements themselves compare endofunctors of mapping categories.

Packaging restriction as the selected forward direction and extension as its inverse gives the promised arbitrary-source theorem:

$$R_{C,G} : \text{Map}_{\text{Grpd}}(\text{Groupoidify}(C), G) \simeq_{\omega} \text{Fun}(C, \text{Path}(G)). \quad (27.20)$$

Formal status — checked. Theorem 27.1 (generic free inversion at the fixed-forward boundary). Evidence `GENERIC-GROUPOIDIFICATION-MAPPING`. Formation, whole unit, point and dependent first-cell computation, whole target extension/restriction, beta/eta, explicit compositor, and retained next actions are active for arbitrary C and groupoidal G . The theorem is a fixed-forward `OmegaEquivAlong` of mapping categories. Source action, `Groupoidify_func`, and the packaged adjunction remain outside this boundary.

The adjective *fixed-forward* records real data, not a weakness in the equivalence. The forward functor in (27.20) is specifically restriction along η_C ; it is not merely some equivalence between carriers. What remains unbuilt is variation in the source category C itself.

Equation (27.20) is also more than an objectwise recursor theorem. Its left side contains maps $\text{Groupoidify}(C) \rightarrow G$ as objects and equality action between them as homs. Its right side contains whole path-valued functors and their transformations. The extension functor transports those transformations to equalities of recursive maps; restriction transports equalities of maps back to transformations. The beta and eta paths compare the resulting whole endofunctors, not only their values at one selected representation.

This strength is why the remaining qualification must be phrased carefully. Formation depends on any supplied category C , and the theorem quantifies over arbitrary C and G . Nothing here is restricted to the objects of a single category. The missing operation is instead *functorial dependence on C* : given a functor between two source categories, the current public interface has not yet packaged its induced groupoidal map and higher action as one functor on the category universe.

This distinction between quantification and functorial packaging recurs throughout category theory. A theorem may be uniform in an arbitrary parameter before the parameter has been made the object action of an internal functor. Here the formula for source action is already visible, but its identity, composition, and higher observations have not yet been promoted. Keeping those stages separate lets (27.20) be used now without pretending that its future naturality proofs have already computed.

The target side, by contrast, is complete at the mapping-object boundary needed here. For each groupoidal G , extension and restriction are whole functors between the displayed mapping categories, and their inverse laws are whole paths. The chapter can therefore compare concrete target presentations and transport higher evidence even while source functoriality remains a later construction.

27.6 Recovering The Interval From The Generic Theorem

The generic construction and the concrete interval were formed independently, so their comparison is a useful recovery test. Specialize `Groupoidify` to `WalkingArrow`. Extending the concrete unit (27.11) through the generic recursor gives

$$u : \text{Groupoidify}(\text{WalkingArrow}) \longrightarrow \text{Interval}. \quad (27.21)$$

In the other direction, extend the generic unit through the concrete Interval recursor:

$$v : \text{Interval} \longrightarrow \text{Groupoidify}(\text{WalkingArrow}). \quad (27.22)$$

The generic beta/eta laws and the Interval beta/eta laws identify the restrictions of the two composites with the relevant identity representations. Their whole uniqueness clauses then give

$$v \circ u = \text{id}, \quad u \circ v = \text{id}. \quad (27.23)$$

Hence

$$\text{Groupoidify}(\text{WalkingArrow}) \simeq \text{Interval}. \quad (27.24)$$

Formal status — checked. Evidence `GROUPOIDIFICATION-INTERVAL-RECOVERY`. The maps (27.21) and (27.22) are selected by the two whole extension owners; both composites have whole and pointwise cancellation paths; and the result is packaged as a `TypeEquiv`. The two HIT classifier heads remain non-convertible.

The lack of a definitional identification is desirable. The concrete Interval offers a compact two-endpoint eliminator. The generic object records that it arose by applying one operation to a source category. Equivalence shows that the presentations have the same groupoidal content without forcing every calculation to unfold through the same syntax.

The comparison illustrates a general method for relating independently useful HIT presentations. Map each presentation into the other by the appropriate recursor. Do not try to compare raw constructors syntactically. Instead, restrict each composite to the generating whole representation and use the two uniqueness principles to identify it with the identity. This method preserves the computational advantages of each presentation while producing an explicit equivalence that later proofs can transport across.

It also validates the generic construction against the feature that motivated the Interval: distinct endpoints. The recovered equivalence could not pass the endpoint and segment projections if the generic unit had silently reduced the source to one-object endomorphism data. `WalkingEnd/Circle` and `WalkingArrow/Interval` are therefore complementary regression theorems, not two decorative examples of the same calculation.

27.7 Three Operations With Different Directions

Groupoidification sits near two other operations used in the book. Their similar notation can obscure opposite universal roles:

operation	effect	characteristic map
$\text{Core}(C)$	retain invertibles	$\iota_C : \text{Path}(\text{Core } C) \rightarrow C$
$\ A\ _n$	forget above n	$ - _n : A \rightarrow \ A\ _n$
$\text{Groupoidify}(C)$	freely invert	$\eta_C : C \rightarrow \text{Path}(\text{Groupoidify } C).$

(27.25)

The core deletes noninvertible motion and keeps what was already reversible. Truncation preserves the groupoidal direction but forgets homotopy above a chosen level. Groupoidification keeps every represented directed arrow and adds inverse motion coherently. Applying truncation after groupoidification is meaningful; identifying the two operations would lose the order of those steps. The table suppresses only information already stated in prose: the inputs to Core and Groupoidify are directed categories, while the input to truncation is groupoidal.

Their mapping directions make the distinction even sharper. A map from a groupoidification into G is classified by a path-valued representation of the original directed category. A map from an n -truncation into an n -truncated target is classified by a map from the original groupoidal type. The core instead supplies a groupoidal object mapping into the original category by retaining already-invertible arrows. These point in reflective, truncating, and core-inclusion directions respectively; substituting one for another would reverse which information is freely added and which is discarded. The present statement concerns those maps and mapping properties, not a packaged three-adjunction chain.

There is one final boundary. A functor $H : C \rightarrow D$ should induce a map $\text{Groupoidify}(C) \rightarrow \text{Groupoidify}(D)$ by extending the composite $\eta_D \circ H$. Identity and composition paths should then follow from whole uniqueness. That construction is the planned source action. Until its higher action has been checked, there is no public whole functor

$$\text{Groupoidify}_{\text{fun}} : \text{Cat} \longrightarrow \text{Grpd} \quad (27.26)$$

and therefore no packaged adjunction with the path-category functor. The mapping theorem (27.20) is already valid for every fixed C and G ; it should not be weakened to “object-only.” But neither should that family of theorems be renamed a completed functorial left adjoint before source action, unit/counit observations, and triangle laws are assembled.

The proposed source action has a canonical formula, so the deferral is not a lack of mathematical direction. Given $H : C \rightarrow D$, extend $\eta_D \circ H$ across $\text{Groupoidify}(C)$. Whole uniqueness should then prove identity and composition laws and supply higher action. What remains is to carry out that construction at the same computational standard as (27.15)–(27.20), rather than postulating a functor and an adjunction record whose fields merely restate the desired answer.

The next chapter turns from inversion to interchange. Here the unit compositor is invertible because it lands in a path category; in a genuinely directed two-dimensional target it need not be. The Gray direction asks how that noninvertible cell is projected from the same whole internal action.

28. Laxity, Interchange, And The Gray Direction

Functoriality is often printed as an equation:

$$F(g) \circ F(f) = F(g \circ f). \quad (28.1)$$

In a directed higher setting, the more informative object is the comparison cell that (28.1) would erase. Its source is the composite of the two separate arrow actions, its target is the action on the composite arrow, and its next action records how that comparison varies. When the cell is invertible, one has pseudo-functorial behaviour. When it is an identity, one has a selected strict behaviour. When it is merely directed, one has genuine laxity.

The previous chapters repeatedly changed the *target* of such a witness. In a path category it became invertible. In a strict computational profile it specialized to identity. This chapter keeps the target directed and asks where the noninvertible cell itself comes from. The answer is not an extra square axiom. It is a component of the same whole internal action that already supports dependent transport and ordinary naturality.

The geometric test is the walking square. Let $I = \mathbf{WalkingArrow}$ and consider a selected tensor-shaped category $I \otimes_R I$. Its four vertices and four boundary arrows are derived from coevaluation. The two routes from the source corner to the opposite corner need not be equal. A directed **inter-changer** compares them. Recovering that cell from whole laxity, and retaining one further action, is the checked centre of the chapter.

28.1 Whole Laxity Before Components

Let $F, G : A \rightarrow B$ be functors and let $\epsilon : F \Rightarrow G$ be a transformation. The off-diagonal action of ϵ does more than assign a component at each object. Given an arrow $f : X \rightarrow Y$, it supplies a cell

$$\epsilon[f] : F(X) \longrightarrow G(Y) \quad (28.2)$$

in the appropriate internal hom. Now compose on the target side with an arrow $g : Y \rightarrow Z$. There are two whole functors, varying in f , whose readable values are

$$G(g) \circ \epsilon[f] \quad \text{and} \quad \epsilon[g \circ f]. \quad (28.3)$$

More explicitly, after X , g , and the transformation have been fixed, the two expressions in (28.3) are the object actions of functors

$$\mathrm{Hom}_A(X, Y) \longrightarrow \mathrm{Hom}_B(FX, GZ).$$

The comparison between them is therefore allowed to vary over the *whole* hom category. This is the categorical content hidden by the familiar pointwise naturality equation: the input arrow is still an object of an internal hom and may itself have higher arrows.

The internal displayed action supplies a whole transformation from the first functor to the second. Its component is the post/left laxity cell

$$\lambda_{\epsilon, g, f}^{\text{post}} : G(g) \circ \epsilon[f] \Longrightarrow \epsilon[g \circ f]. \quad (28.4)$$

There is a target-internalized mirror producing the pre/right comparison. These are not two unrelated naturality squares. Both are ordinary specializations of a displayed transformation acting in a dependent hom. The displayed owner first retains transport in the source and target fibres; the ordinary surfaces appear only after the relevant families are specialized to representables.

The pre/right mirror fixes the target side and varies an arrow entering the source. Its readable comparison has the complementary form

$$\epsilon[f] \circ F(h) \Longrightarrow \epsilon[f \circ h].$$

Post/left and pre/right are skew views of one off-diagonal action. They need not be identified by a pointwise square equation before their whole owners have been compared. The explicit orientation becomes important when one of the two views is selected for the Gray interchanger.

This order of construction matters. If one wrote only the component (28.4), the variable f and its higher arrows would already have been capped. By retaining the whole transformation first, the ordinary hom action may be applied again. The next level acts on a cell between f_0 and f_1 and returns a cell between the corresponding laxity boundaries. Iteration, not a manually appended coherence record, supplies the higher observation.

The usual functor compositor is obtained by taking ϵ to be the identity transformation of F . At the appropriate identity component, (28.4) reads

$$\phi_{g, f}^F : F(g) \circ F(f) \Longrightarrow F(g \circ f). \quad (28.5)$$

Thus the compositor is an identity-transformation specialization of whole naturality. It is not postulated independently of the transformation calculus. The same provenance is what allowed Chapter 25 to realize it as a path and Chapter 27 to retain it on the groupoidification unit.

Formal status — checked. Evidence `FUNCTORD-WHOLE-LAXITY`. The displayed internal action owns a whole laxity transformation. Ordinary post/left and pre/right transformations are transparent specializations; their capped cells recover the displayed witness; and the ordinary functor compositor is the identity-transformation specialization. The next hom action remains available. No independent naturality square is added.

28.2 Strictness As A Property And Profile

Strictness should constrain the compositor already obtained in (28.5), not manufacture a second compositor beside it. For a cell $c : x \rightarrow y$, write

$$\text{IsStrictCell}(c) \equiv \sum_{p:x=y} (c = \text{pathToHom}(p)).$$

The endpoint path is explicit. This matters because the prototype still has historical cuts that identify several strict endpoints judgmentally; the property continues to say something meaningful after those cuts are moved to their proper profiles. Applying it to every binary compositor gives

$$\begin{aligned} \text{IsStrictFunctor}(F) &\equiv \prod_{f,g} \text{IsStrictCell}(\phi_{g,f}^F), \\ \text{StrictFunctor}(A, B) &\equiv \sum_{F:\text{Functor}(A,B)} \text{IsStrictFunctor}(F). \end{aligned} \tag{28.6}$$

Thus the evidence itself is the proof-carrying code; there is no second grammar of strict functors. In the logical framework the outer $\text{StrictFunctor}(A, B)$ name is nevertheless kept as a rigid record-like head whose carrier is exactly the second line of (28.6). This retains the indices A and B when a rule must invert a package. Its constructor and projections recover the ambient functor and its proof.

A stable view, written $\text{strict}(S)$, places such a package back in the ambient functor classifier. When the package constructor is visible, point and hom action compute to the packaged carrier. When a strict package is itself returned by higher action, the stable head remains visible instead of prematurely erasing the evidence discriminator.

There is a deliberate limit to the present computation. The former rule that reduced every selected compositor to a literal identity has been retired: an arbitrary proof of $\text{IsStrictCell}(c)$ identifies c with an equality-induced arrow, and does not justify that blanket rewrite by subject reduction. A later consolidation will move the historical global strict cuts to evidence-bearing functor and transformation profiles. The current package records the correct semantic information without anticipating that normal-form migration.

This still requires no second functor theory. Define the profiled internal hom

$$\text{GrayHom}_{\text{lax}}(A, B) \tag{28.7}$$

to have strict packages as objects and the existing ambient transformation categories between their stable views as homs:

$$\begin{aligned} \text{Obj}(\text{GrayHom}_{\text{lax}}(A, B)) &\equiv \text{StrictFunctor}(A, B), \\ \text{Hom}(S, T) &\equiv \text{Transf}(\text{strict } S, \text{strict } T). \end{aligned} \tag{28.8}$$

Identity and composition delegate to the ambient functor category. Homs between transformations are the existing modification categories, and every subsequent hom is reused. A whole inclusion acts as the identity on this shared transformation tower.

The word *lax* in (28.7) describes the arrow profile. Objects are functors equipped with strictness evidence; arrows are the ambient transformations whose off-diagonal action retains laxity. The category is not definitionally the ambient functor category, and an arbitrary ambient functor is not silently accepted as one of its objects.

The selected identity illustrates the separation. It is the ordinary identity functor paired with supplied strictness evidence. Gray curry, uncurry, join maps, and transformation graphs follow the same pattern: first construct the ordinary whole functor, then retain an **IsStrictFunctor** proof about its existing compositor. The proof may initially be supplied where its full internal derivation is not yet needed, but it can never conceal an independent forward cell because the property fixes that cell in its own type.

This profile architecture scales better than duplicating products, pullbacks, transformations, and modifications for each preservation mode. Further properties may refine the same ambient carriers and share the same hom tower; duplication is reserved for a genuinely different classifier or computation.

Formal status — checked. Evidence `GRAY-COMPUTATIONAL-PROFILE`. `IsStrictFunctor` constrains the existing extracted compositor, and `StrictFunctor` is its exact carrier/evidence package behind a rigid indexed facade. Constructor-visible stable-view action computes to the carrier. `GrayHom_lax` reuses the complete ambient transformation and higher-hom tower; no duplicate strict-functor grammar, compositor, modification hierarchy, or broad category-head conversion is introduced. Reflection of strict evidence into judgmental computation remains part of the later global-cut migration.

28.3 One Selected Right Closure

The tensor-shaped category used here is characterized through one right closure. For categories A, B, C , the checked boundary is

$$\text{GrayHom}_{\text{lax}}(A \otimes_R B, C) \simeq_{\omega} \text{GrayHom}_{\text{lax}}(A, \text{GrayHom}_{\text{lax}}(B, C)). \quad (28.9)$$

The tensor head $A \otimes_R B$ is stable and distinct from the Cartesian product. Curry and uncurry in (28.9) are whole functors between the two profile categories, each paired with supplied **IsStrictFunctor** evidence. Their composites are compared with the appropriate identity functors by whole beta and eta paths, and the resulting fixed-forward equivalence retains hom action.

Equation (28.9) is an equivalence of profiled mapping *categories*. On the left, objects are strict packages for maps out of the selected tensor and arrows are ambient transformations. On the right, an outer strict package selects, at each object of A , an inner strict package $B \rightarrow C$; its arrows are allowed the lax transformation behaviour retained by (28.7). Curry and uncurry transport not only objects but this transformation tower.

The adjective *right* fixes which variable is moved into the internal hom. A mirror closure would instead expose the opposite orientation and its corresponding lax or oplax convention. The present theorem chooses one of these directions rather than asserting that they have already been related.

Coevaluation is not another primitive:

$$\mathrm{coev}_{A,B} : A \longrightarrow \mathrm{GrayHom}_{\mathrm{lax}}(B, A \otimes_R B) \quad (28.10)$$

is curry applied to the strict identity package on $A \otimes_R B$. Dually, evaluation is uncurry applied to the strict identity package of the internal hom. These two maps give introduction and elimination for the selected tensor boundary.

The introduction/elimination reading is concrete. Coevaluation builds a generic tensor point by currying the identity, while evaluation consumes a profiled inner map by uncurrying the identity. Their whole beta/eta paths are the computational boundary through which the walking-square observations are derived. The square is not added beside the closure.

Formal status — checked. Evidence `GRAY-RIGHT-CLOSURE`. Whole strict curry and uncurry package the right-closure equivalence (28.9), with whole beta/eta and hom action. Coevaluation and evaluation are derived at selected identity codes. The tensor remains distinct from the Cartesian product and the beta comparison is equality evidence, not a competing object-level run-time fold.

There is a complementary combinatorial route to Gray products. In [Hadzihanovic](#), Gray products are constructed on directed cell complexes and oriented cubes are studied as shapes in their own right. That perspective supplies an important model-independent picture of the square and its higher-dimensional successors. The emdash result has a different boundary: it begins with one profiled internal hom and selects the right closure (28.9). It does not construct the tensor combinatorially or prove agreement with Hadzihanovic's Gray product.

28.4 The Walking Square From Coevaluation

Take $I = \mathrm{WalkingArrow}$, the join-derived directed interval of Chapter 27, and form

$$\mathrm{Square}_R := I \otimes_R I. \quad (28.11)$$

Evaluating coevaluation at the two outer endpoints produces two strict inner functors $I \rightarrow \mathrm{Square}_R$. Evaluating those functors at the two inner endpoints gives four vertices $v_{00}, v_{01}, v_{10}, v_{11}$. Their arrow actions on the walking generator give the horizontal edges a_0, a_1 . The outer generator acts through coevaluation as one whole transformation between the inner functors; its two components give the vertical edges b_0, b_1 :

$$\begin{array}{ccc} v_{00} & \xrightarrow{a_0} & v_{01} \\ b_0 \downarrow & & \downarrow b_1 \\ v_{10} & \xrightarrow{a_1} & v_{11} \end{array} \quad (28.12)$$

The first index in v_{ij} records the outer endpoint and the second records the inner endpoint. Thus a_0 and a_1 are the inner generator evaluated in the two outer fibres. The arrows b_0 and b_1 are not obtained by applying two unrelated maps: they are the two object components of the *same* outer transformation. This shared owner is precisely what supplies a naturality comparison between the two boundary routes.

Every displayed object and arrow in (28.12) is therefore an observation of coevaluation and the existing walking generator. None is postulated as a standalone tensor constructor. The four vertex normal forms remain pairwise distinct, and the selected square does not convert to $I \times I$.

The two routes around the boundary are

$$a_1 \circ b_0 \quad \text{and} \quad b_1 \circ a_0. \quad (28.13)$$

In a Cartesian product these would be forced to commute by the relevant strict interchange. Here their difference is the feature being measured.

28.5 The Oriented Interchanger

Let ϵ be the whole outer transformation generated by coevaluation, and let g be the inner walking arrow. The post/left laxity action from Section 28.1 has readable direction

$$G(g) \circ \epsilon[-] \implies \epsilon[g \circ -]. \quad (28.14)$$

Evaluate this whole transformation at the identity of the outer source endpoint. Its component is the square's interchanger:

$$\chi : a_1 \circ b_0 \implies b_1 \circ a_0. \quad (28.15)$$

The formal source and target are owned by the stable transport functors of the internal action; (28.13) gives their readable composite presentations under the current strict endpoint conversions. The direction (28.15), rather than the choice of a terminology convention in isolation, is why the internal hom is named **GrayHom**_{lax}.

The cell χ is not an independently declared filler. The whole post/left transformation exists first; χ is its identity component. Nor does the component end the construction. The transformation's next hom action is retained as a functor, and evaluating that next owner at its identity recovers χ . This confirms that the square has not capped the common action calculus one dimension too early.

The retained action can accept a higher arrow of the input hom and return a higher cell between interchanger boundaries. In a larger cubical consumer it would be the route by which faces of a three-dimensional comparison are observed. This chapter checks only the owner and its identity observation; it does not claim that a walking cube, all of its faces, or a general cubical coherence theorem has been constructed.

Finally, the interchanger is not collapsed to an identity term. Its post/left provenance also remains distinct from the pre/right mirror. If the same cell were realized in a path-valued target, symmetry would provide an inverse as in Chapter 25. Inside the directed square, no such inversion is assumed.

Formal status — checked. Theorem 28.1 (the walking-square interchanger). Evidence GRAY-WALKING-INTERCHANGER . The four vertices and two coordinate arrow families are derived from right coevaluation. The oriented nonidentity interchanger is the identity component of the existing whole post/left laxity transformation, remains distinct from its pre/right mirror, and retains one next hom action. No square filler, endpoint rewrite, or unification rule is added independently.

28.6 What This Does And Does Not Call Gray

The low-dimensional picture agrees with the characteristic reason for using a Gray rather than Cartesian tensor: currying a strict map produces strict inner functors while allowing transformation-level laxity, and $I \otimes_R I$ exposes a directed interchanger instead of forcing a commuting square. This is enough to test the architecture of profiles, closure, and extracted laxity.

Terminology in the literature can exchange *lax* and *oplax* when tensor variance or diagrammatic composition conventions are reversed. The invariant statement here is the displayed direction (28.14): postcomposition by the target action points toward the action on the composite. That typed direction, not the name alone, is the comparison surface for any later mirror closure or literature equivalence.

It is not enough to assert a full Crans–Gray monoidal structure. Such a claim would require at least:

- the mirror closure and a settled comparison between the two orientations;
- functorial action of the tensor in both parameters;
- associativity and unit data, with their higher coherences;
- compatibility of curry, uncurry, evaluation, and coevaluation with those actions; and
- a comparison with an established combinatorial or globular construction in the dimensions being claimed.

None of these is manufactured by adding fields to Theorem 28.1. They are new whole constructions. The tensor in this chapter is consequently written $\otimes_R$: it records the selected right closure rather than pretending that the mirror and monoidal boundaries have already been built.

There is a second historical boundary. Some ambient functoriality and naturality endpoints are still identified by global prototype conversion rules. The strict property now records where the compositor cell itself is constrained by semantic evidence, and the unprofiled interchanger remains nonidentity, but the eventual migration must both re-home endpoint conversions at explicit strict profiles and decide where that evidence should be reflected into judgmental computation. This chapter does not perform that repository-wide normal-form change.

28.7 The End Of The Fourth Spiral

The fourth spiral began by placing equality inside the directed language. Product paths showed groupoidal closure without a category-head rewrite. The Circle turned reversible motion into the Integer line. Groupoidification then freely realized arbitrary directed cells as paths. The Gray square now returns to a target where the comparison cell is directed and worth keeping.

These movements are not separate foundations. They are different readings of one iterable calculus:

target or profile	behaviour of the comparison cell	(28.16)
arbitrary directed target	lax, possibly noninvertible	
path-valued target	pseudo, invertible by symmetry	
strict property package	equal to an equality-induced cell.	

This table also explains why the historical strict endpoint conversions do not invalidate the experiment. They may simplify the written source and target of a comparison, but they do not turn the unprofiled cell into the identity. The path target, strict property, and arbitrary directed target still select observably different behaviours of the retained witness.

The three rows of (28.16) should be read as operations on one owner, not as three parallel theories. The arbitrary directed cell is primary. A path-valued target changes its hom into equality and thereby supplies an inverse. A strict package retains an endpoint path and identifies the same cell with the arrow induced by that path; judgmental reflection is deliberately deferred. Neither specialization requires the ambient compositor, its whole transformation, or its next action to be redeclared.

This suggests a discipline for extending the higher theory. First locate the whole internal action before introducing a pointwise coherence name. Second select the target or property profile that gives the desired invertibility or strictness. Third derive the geometric consumer—such as the walking square—from evaluation, coevaluation, and existing generators. Finally check both a non-collapse boundary and one further hom action. A pointwise cell without its owner is too easy to postulate; an owner without a concrete consumer is too easy to misorient.

The mirror Gray closure can follow the same discipline. Its construction should begin from the pre/right whole action, not by reversing the arrow in (28.15) after the fact. Comparing the two closures will then be a theorem about their whole curry and evaluation structures. Likewise, a future associator for the tensor should be derived at the mapping-category level before its components are named. This keeps higher coherence attached to the operations that generate it.

The walking square is therefore valuable even though it is small. It tests all of the architectural seams at once: strict packages versus ambient functors, profiled homs versus duplicated hierarchies, closure versus an opaque tensor, whole transformations versus square axioms, and directed cells versus paths. Passing that test does not finish Gray theory, but it gives the unfinished theory a computational spine.

The result is deliberately asymmetric. Groupoidal realization explains how directed coherence may become invertible; the Gray direction explains why it should not have been erased beforehand. Chapter 29 now uses the same retained action recursively: a dependent arrow between dependent arrows becomes a simplex with another dimension. Chapter 30 returns to universal structure and keeps the same higher action through products, pullback base change, and dependent products. The appendices then give the ledger of exact notation, evidence, provenance, computation, and remaining research boundaries for all six spirals.

29. Simplexes From Dependent Homs

A directed edge has two vertices and one arrow. A directed triangle adds three edges and a comparison between a composite edge and a direct one. A tetrahedron adds four triangles and a cell relating their comparison data. The pattern continues, but its usual description becomes increasingly combinatorial: every new dimension carries many faces, and those faces must agree on all of their shared lower faces.

Functorial type theory offers another description. Begin with a category C . Choose an object, then an outgoing arrow, then an outgoing arrow in the category of outgoing arrows, and continue. Each new choice is made in a category whose objects already contain the entire previous boundary. Its arrows contain a base arrow together with a dependent arrow above transport. The next hom action therefore supplies the next coherence without asking for a separately written coherence record.

This chapter relates that native dependent presentation to the familiar combinatorics of standard simplexes. Injective monotone maps give a computing category of faces. Directed joins give the finite ordinal categories $\Delta[n]$. Representables give standard semisimplices. Iterated outgoing paths give the native dependent cells. Finally, a structural recursion builds one canonical dependent simplex inside every $\Delta[n]$ and maps it along every functor $\Delta[n] \rightarrow C$.

The result is deliberately semisimplicial: faces are present, degeneracies are not. It is also deliberately weaker than a categorical normal-form theorem. The construction computes one canonical simplex and all of its nonempty face observations in variable dimension; it does not yet identify the whole mapping category $\text{Functor}(\Delta[n], C)$ with a whole category of dependent simplexes.

29.1 The Shape Before The Coordinates

Write $\Delta[n]$ for the finite ordinal category

$$0 \longrightarrow 1 \longrightarrow \cdots \longrightarrow n, \quad (29.1)$$

including the unique composite arrow $i \rightarrow j$ whenever $i \leq j$. A functor $H : \Delta[n] \rightarrow C$ is the conventional categorical presentation of an n -simplex in C . At dimension two it selects three vertices, the three arrows

$$p_{01} : x_0 \rightarrow x_1, \quad p_{12} : x_1 \rightarrow x_2, \quad p_{02} : x_0 \rightarrow x_2, \quad (29.2)$$

and whatever comparison is selected by the functorial profile between $p_{12} \circ p_{01}$ and p_{02} . At dimension three, the four restrictions of H to three vertices are its triangular faces.

There are already three distinct notions in this paragraph.

- The category $\Delta[n]$ is the *ordinal shape*.

- The representable $\mathbf{Hom}(-, [n])$ on the category of injective ordinal maps is the *standard semisimplex*.
- The data obtained by repeatedly entering an outgoing-path category is the *dependent simplex*.

The first is a finite source category. The second records how all smaller faces enter that source. The third is a native normal form for the data seen inside a target. The value of the construction below is not that these three expressions can be printed. It is that their face and higher-cell operations are owned by functors already present in the theory.

The ordinal shapes themselves grow by directed join:

$$\Delta[0] \equiv \mathbf{1}, \quad \Delta[n+1] \equiv \Delta[n] \star \mathbf{1}. \quad (29.3)$$

The new terminal vertex receives one arrow from every old vertex. Thus joining with $\mathbf{1}$ adds exactly the new final vertex and all arrows that point toward it. The construction is directed: it does not add inverse arrows from the new vertex back into the old ordinal.

29.2 Faces Form A Computing Category

Use the augmented cardinal convention. The natural number m represents the finite ordinal with m vertices, so zero is the empty ordinal, one is a vertex, two is an edge, and three is a triangle. An injective monotone map from the p -vertex ordinal to the n -vertex ordinal can be encoded by a word of length n : at each target position, either skip that position or keep it.

The structural constructors have the form

$$\begin{aligned} \text{skip} : \mathbf{Face}(p, n) &\longrightarrow \mathbf{Face}(p, n+1), \\ \text{keep} : \mathbf{Face}(p, n) &\longrightarrow \mathbf{Face}(p+1, n+1). \end{aligned} \quad (29.4)$$

The all-keep word is identity. Composition substitutes one word into the kept positions of another. Its four skip/keep cases are ordinary structural recursion, so closed faces compute rather than requiring a theorem for every pair of dimensions.

These codes are classified as sets before becoming public face maps. The classification removes unwanted higher ambiguity in the combinatorial index, while restricted recursion preserves computation on visible constructors. They form the homs of the internal augmented semi-simplex category Δ_+^{inj} :

$$\mathbf{Obj}(\Delta_+^{\text{inj}}) = \mathbb{N}, \quad \mathbf{Hom}(p, n) = \mathbf{Path}(\mathbf{Face}(p, n)). \quad (29.5)$$

Identity and composition are the identity and composition of face codes. The homs are locally discrete, but the enclosing category remains an ordinary internal category, so functors and transfor on it use the generic iterable action calculus.

The standard n -simplex is now Yoneda:

$$\Delta[n] := \text{Hom}_{\Delta_+^{\text{inj}}}(-, n+1). \quad (29.6)$$

The shift by one converts from dimension to vertex count. Evaluating $\Delta[n]$ at a p -vertex ordinal returns the code of a p -vertex face of $[n]$. Restriction is composition of face codes. A groupoid-valued semisimplicial diagram is consequently a functor

$$X : (\Delta_+^{\text{inj}})^{\text{op}} \longrightarrow \mathbf{Grpd}, \quad (29.7)$$

and postcomposition with the path-category operation realizes all its levels and face maps as a Cat-valued presheaf. Because realization is one whole postcomposition functor, arrows between diagrams and their higher action are retained as well.

Formal status — checked. Evidence `SEMISIMPLICIAL-FACE-SUBSTRATE`. Skip/keep face codes, their composition, the augmented injective index category, join-built ordinal shapes, Yoneda standard semisimplices, and whole groupoid-to-Cat diagram realization are active. No degeneracy maps or full simplex category are asserted.

29.3 A Simplex Is An Iterated Outgoing Path

The combinatorial index says which face is selected. It does not yet explain why a higher simplex should have the right dependent boundary. That explanation begins with the outgoing-arrow category from Chapter 5:

$$\text{PathOut}_C(x) = \sum_{y:C} \text{Hom}_C(x, y). \quad (29.8)$$

An object is an endpoint y and an arrow $p : x \rightarrow y$. An arrow between (y, p) and (z, q) contains an arrow $r : y \rightarrow z$ together with a cell from $r \circ p$ to q . Thus one step into `PathOut` adds a vertex, the edge from the fixed source, and the comparison that makes the resulting triangle coherent.

This observation can be iterated. Put

$$S_0(C) := C. \quad (29.9)$$

After choosing a flag $s_k \in \text{Obj}(S_k)$, define

$$S_{k+1}(C; s_0, \dots, s_k) := \text{PathOut}_{S_k}(s_k). \quad (29.10)$$

A zero-simplex is an object x_0 of C . A one-simplex is an object $e_{01} = (x_1, p_{01})$ of $\text{PathOut}_C(x_0)$. A two-simplex is an object t_{012} of $\text{PathOut}_{\text{PathOut}_C(x_0)}(e_{01})$. Unpacked readably, it contains an edge p_{02} and an arrow from e_{01} to e_{02} ; the latter contains p_{12} and a two-cell

$$\alpha_{012} : p_{12} \circ p_{01} \Longrightarrow p_{02}. \quad (29.11)$$

No triangle record has been introduced. Equation (29.11) is the dependent fibre component of an arrow in a Sigma total.

The generic calculation is worth stating. Let E be a Cat-valued family on K , and consider total objects (x, u) and (y, v) . Their hom in the total category has the native presentation

$$\text{Hom}_{\Sigma E}((x, u), (y, v)) \simeq \sum_{p:x \rightarrow y} \text{Hom}_{E(y)}(E[p](u), v). \quad (29.12)$$

The second factor is precisely a dependent hom. Specializing E to the representable family $\text{Hom}_C(x_0, -)$ gives (29.11). Nesting this hom slice beneath the next outgoing-path Sigma gives tetrahedra, then higher simplexes. The recursion is therefore semantic before it is coded: every stage is built from the existing `Hom`, dependent `Sigma`, and dependent-hom owners.

With both total endpoints fixed, (29.12) projects (p, α) to p and, through covariant fibre action, to $E[p](u)$ in the already fixed fibre $E(y)$. The latter is internal transport, not the independently varying simplex target supplied by the outer `PathOut Sigma` in (29.8).

29.4 Why A Tetrahedron Has Four Surfaces

An ordinary globular arrow has two endpoints, whereas a tetrahedron has four faces. The recursive triangle category supplies the difference through two nested fibrations.

Write

$$S_1 = \text{PathOut}_C(x_0), \quad S_2 = \text{PathOut}_{S_1}(e_{01}).$$

For a visible $e_{01} = (x_1, p_{01})$, a triangle in S_2 has the nested form

$$t_{012} = (e_{02}, q_{012}), \quad e_{02} = (x_2, p_{02}), \quad q_{012} = (p_{12}, \alpha_{012}).$$

The outer pair remembers the target edge e_{02} . The inner Hom-of-Sigma pair remembers the base edge $e_{12} = (x_2, p_{12})$ and comparison α_{012} .

Now take two triangles $t_{012}, t_{013} \in S_2$. An arrow

$$\Theta : t_{012} \longrightarrow t_{013} \quad (29.13)$$

is the volume of the tetrahedron 0123. Its ordinary source and target are the faces 012 and 013. Two whole line projections provide the remaining faces:

$$\begin{aligned} d_{02}(t_{01i}) &= e_{0i}, & d_{02}[\Theta] &= t_{023}, \\ d_{12}(t_{01i}) &= e_{1i}, & d_{12}[\Theta] &= t_{123}. \end{aligned}$$

The first is the target projection of the outer `PathOut` `Sigma`. The second uses the base-arrow projection of the inner Hom-of-Sigma, whose dependent fibre is organized by `homd_int`. Pairing them gives one whole boundary functor. Their shared vertices and edges are consequently preserved by ordinary functor action rather than imposed by a hand-written boundary equation.

An ordinary functor $F : C \rightarrow D$ maps the whole recursive triangle category by iterated `PathOut` action. Within its fixed-endpoint Hom-of-Sigma slice, the displayed part is mapped by the existing internal dependent action: the base cell is retained while the fibre cell is sent through the next displayed hom action. Applying the hom action once more remains meaningful. This is the same iteration that produced the laxity witness of Chapter 28; here its geometric reading is a higher simplex.

Formal status — checked. Evidence `DEPENDENT-SIMPLEX-INTERNAL-ACTION`. The fixed-endpoint dependent hom is the active hom of a Sigma total and retains its base/transport observations. The recursive `PathOut` triangle category has whole target-line and base-line projections whose hom actions expose faces 023 and 123. A visible tetrahedral constructor computes through the existing displayed internal action, and one further hom action is retained. No stand-alone tetrahedron filler or coherence record is added.

29.5 Codes Without A Second Semantics

Equations (29.9)-(29.10) are dependent in a strong sense: the category at the next stage depends on the previously selected object. Ordinary recursion into a fixed codomain cannot store that changing type directly. An internal code is useful here, but only if it remembers the native category rather than interpreting a parallel syntax of cells.

The intrinsic code has two constructors. Its zero case is indexed by C . Its successor stores an existing code indexed by K and a flag $x \in \text{Obj}(K)$, and is indexed by $\text{PathOut}_K(x)$. Schematically,

$$\begin{aligned} \text{zero}_C &: \text{Code}(C, 0; C), \\ \text{step}(c, x) &: \text{Code}(C, n + 1; \text{PathOut}_K(x)) \quad (c : \text{Code}(C, n; K)). \end{aligned} \tag{29.14}$$

The semicolon records the already-decoded category. Public code packaging may hide K , but decoding merely projects that index. It does not traverse a syntax tree and rebuild `Hom`, `Sigma`, or `PathOut`. This makes the code an internal witness to the changing boundary, not a second definition of what a simplex means.

Faces recurse simultaneously on the flagged code and the skip/keep word. There are three structural situations.

1. Skipping the newest vertex selects a face of the fixed flag and returns a constant whole functor.
2. Keeping the newest vertex after skipping its predecessor selects the corresponding face through the first projection of `PathOut`.
3. Keeping both newest vertices maps the whole outgoing path by the recursively selected lower face functor.

The third case is where higher action matters: a face is not only a function on stored points, but a functor on the outgoing-path category. The result retains its own hom action. Direct and sequential face presentations are not globally collapsed to one judgmental normal form; the structural recursion provides the selected whole observation.

29.6 The Ordinal Source Grows By A Transformation

The code recursion describes arbitrary flags. To compare it with the standard ordinal, one needs a canonical flag in every $\Delta[n]$. The directed join equation (29.3) supplies the first step. Extend the identity of the old ordinal across $\Delta[n] \star \mathbf{1}$. The old observed outgoing-path map and the primitive join outgoing-path map are related by one whole transformation.

Suppose a nonzero stage has already produced:

$$d, \quad F, G : K \longrightarrow B, \quad \epsilon : F \Longrightarrow G. \quad (29.15)$$

For a selected old source $s \in \text{Obj}(K)$, the new code and source are

$$d' := \text{step}(d, F(s)), \quad s' := (G(s), \epsilon_s). \quad (29.16)$$

The second expression is an object of $\text{PathOut}_B(F(s))$: its endpoint is $G(s)$ and its outgoing arrow is the component of the whole transformation. For the next flag, lift ϵ through `PathOut`. The lift is again a whole transformation, so its component supplies the next cell and its hom action remains available.

The first stage uses the identity-join comparison. Every later stage repeats the same `PathOut` lift. This makes (29.16) a structural successor, not a table with separate clauses for triangles, tetrahedra, and four-simplexes.

29.7 The Four-Simplex As A Decisive Finite Test

Dimension four is the first compact test that combines the recursive source, a genuinely higher component, every coface, arbitrary target mapping, and a retained next action. Beginning with the canonical source edge and triangle, the join comparison is lifted three times:

$$\begin{aligned} \epsilon_1 &:= \text{identity-join outgoing-path comparison}, \\ \epsilon_2 &:= \text{PathOutLift}(\epsilon_1, e_{01}), \\ \epsilon_3 &:= \text{PathOutLift}(\epsilon_2, t_{012}), \\ \omega_{01234} &:= (\epsilon_3)_{s_{0123}}. \end{aligned} \quad (29.17)$$

The component ω_{01234} is the fourth-level cell. Pairing it with its endpoint constructs an object of the existing native four-simplex classifier; it is not supplied as an opaque filler.

For every functor

$$H : \Delta[4] \longrightarrow C, \quad (29.18)$$

the existing mapped-code action sends this single source to a dependent four-simplex in C . The five skip-one-vertex codes expose its tetrahedral faces

$$0123, \quad 0124, \quad 0134, \quad 0234, \quad 1234. \quad (29.19)$$

Native Sigma projections separately expose the source, target, base tetrahedron, and readable dependent top component. These two views have the same intended geometry, but they retain different construction histories. The development does not force every code-selected face to be judgmentally equal to every native projection.

The same construction is checked for an arbitrary target category, a selected computationally strict target map, and an exact path-category target. A wrong recursive source is rejected, the top cell is not identified with an arbitrary replacement, and the next action of ϵ_3 remains available.

Formal status — checked. Evidence `ORDINAL-DEPENDENT-FOUR-SIMPLEX`. One canonical four-simplex is constructed from the generic join comparison and repeated whole `PathOut` lift. It maps under every $H : \Delta[4] \rightarrow C$, exposes all five cofaces, passes strict and path-valued profile checks, remains noncollapsed, and retains one higher action.

29.8 The Variable-Dimensional Theorem

The finite calculation is not the definition. It validates the structural successor that Nat recursion can iterate. Let

$$\mathbf{Obs}(C, n) := \sum_{c : \text{DependentSimplexCode}(C, n)} \mathbf{Obj}(\text{decode}(c)). \quad (29.20)$$

This is the present object package called `DependentSimplexObservation`. It contains both the intrinsic boundary code and one object of its native decoded category.

Formal status — checked. Theorem 29.1 (the variable-dimensional ordinal dependent simplex). Evidence `ORDINAL-DEPENDENT-SIMPLEX-RECURSION`. For every natural number n there is a canonical $s_n \in \mathbf{Obs}(\Delta[n], n)$ computed by Nat recursion and the structural successor (29.16). Every $H : \Delta[n] \rightarrow C$ induces a canonical observation $H_*(s_n) \in \mathbf{Obs}(C, n)$. Every nonempty injective face code has a whole face observation. Base and successor computations, selected source objects through dimensions zero to four, wrong-index rejection, noncollapse of the successor cell, and one generic next action are checked.

The theorem is uniform in n , but its validation claim is deliberately finite where readability matters. The source and successor are genuinely variable-dimensional; the explicit zero-through-four checks show that the recursor reaches the existing native classifiers and the expected finite geometry. They are not an induction theorem identifying the new source judgmentally with every earlier hand-written presentation.

The theorem also separates construction from observation. The canonical source lives once in $\Delta[n]$. Mapping it by H uses the generic mapped decoder; it does not reconstruct the source in C . Face restriction then uses the generic code action. Thus source recursion, target mapping, and face selection are three composable whole operations.

29.9 Comparison With Other Recursive Presentations

Semisimplicial types are a well-known stress test for dependent type theory. The boundary of an $(n + 1)$ -simplex depends on all lower dimensions, and the coherence needed at the next stage depends on that boundary. Different approaches make different parts of this dependency primitive.

Kolomatskaia and Shulman's displayed type theory presents semisimplicial types through a compact displayed or cone interface [12](#). Herbelin and Ramachandra reconstruct the frame, restriction, and coherence dependencies through iterated parametricity, first for semisimplicial and semicubical sets and then in an indexed very dependent formulation [13](#), [14](#). These comparisons clarify why neither an ordinary fixed-codomain Nat recursor nor a flat record of faces is sufficient.

The emdash construction chooses another ownership boundary. Categories and category-valued families are already internal types. `PathOut` is already a Sigma of a representable. A hom in that Sigma already exposes a dependent hom. Whole functor and transfor action already iterate. The semantic simplex is therefore built from these owners directly. The code layer is introduced only to internalize the changing native category in variable dimension.

This does not make the external combinatorics disappear. Face codes remain the clean way to name arbitrary restrictions, and the semi-simplex category organizes them globally. Nor does it prove that every other presentation is equivalent. It shows a computational bridge at a precise point: the same simplex can be observed combinatorially by injective faces and natively as an iterated dependent outgoing path.

29.10 The Whole Mapping-Category Boundary

The strongest tempting statement is

$$\mathbf{Functor}_{\text{cat}}(\Delta[n], C) \simeq \mathbf{DependentSimplex}_{\text{cat}}(C, n). \quad (29.21)$$

Equation (29.21) is not yet a theorem. Its right side is intentionally written with a future name. The active `DependentSimplexObservation(C, n)` in (29.20) is a groupoid-valued object total; it is not a whole category with the arrows and higher cells required by (29.21).

Constructing the right side requires a category whose objects recover (29.20), whose homs express compatible transformations of all dependent frames, and whose higher action agrees with the native internal-action tower. One must then construct comparison functors in both directions, whole beta and eta witnesses, and compatibility with face action. An objectwise decoding function, even one that computes in every dimension, is not that equivalence.

The other major absent operation is degeneracy. The current index has injective monotone maps only. Adding surjections would require a computational account of repeated vertices and identity cells compatible with the dependent recursion. No degeneracy law is inferred merely because each ambient category has identities.

Consequently this chapter does not claim a full simplicial object, arbitrary horn fillers, a Kan complex, a Segal or Rezk theorem, complicial structure, or a comparison with Street orientals. The existing two-dimensional path-groupoid horn fillers are bounded consumers, not a general consequence of Theorem 29.1.

29.11 The End Of The Fifth Spiral

The earlier chapters moved repeatedly between points and arrows, local data and whole action, directed cells and paths. Simplexes make that movement recursive. A point becomes an outgoing arrow. An outgoing arrow becomes an arrow between outgoing arrows. The base and fibre of that arrow become two faces of the next cell. Its whole image supplies another face. Repeating the same construction raises the dimension without changing foundations.

The combinatorial and dependent views play complementary roles. Face words say which vertices survive. Yoneda packages all such restrictions into the standard semisimplex. Directed join constructs the ordinal source. Dependent hom and Sigma explain the cell above its boundary. Whole transformation action constructs the successor. The code merely remembers which native category the next step inhabits.

This is the computational lesson of Theorem 29.1. Higher-dimensional data need not be introduced as an ever-growing list of coherence fields when the theory already knows how a dependent arrow acts. The retained action is the resource from which the next dimension is observed.

The lesson is also a boundary. Constructing one canonical simplex in every dimension is not the same as classifying all simplexes. Faces are not degeneracies. A semisimplicial substrate is not a Kan or Segal theory. By keeping those differences visible, the checked recursion can serve as a foundation for later simplicial methods without being mistaken for their completion.

The next chapter changes scale rather than dimension. It returns to the structural cuts of Chapters 9 and 12 and asks how products, terminality, and adjunctions behave when the objects being varied are themselves arrows in a slice. The same insistence on whole action will turn finite cartesian structure into the indexed chain $\Sigma_u \dashv u^* \dashv \Pi_u$.

30. Cartesian Structure And Dependent Products

A product is usually introduced by a diagram and a universal property. A pullback is introduced by a square. A dependent product is introduced as a right adjoint to substitution. These descriptions are mathematically correct, but they can hide the common computational question:

What happens when a universal construction is immediately eliminated, and which whole operation must remain visible so that the reduction can be iterated at the next categorical level?

This chapter answers that question for selected cartesian and indexed structure. It begins with binary products and the empty product. It then moves to slices, where postcomposition is always available and pullback reindexing is selected structure. A second selected right adjoint completes the chain

$$\Sigma_u \dashv u^* \dashv \Pi_u. \quad (30.1)$$

The three functors in (30.1) act between whole slice categories. Their units, counits, off-diagonal actions, and mate operations therefore retain higher arrows. That retained action is what distinguishes an internal computational presentation from a collection of objectwise bijections.

30.1 A Chosen Product Is A Whole Functor

Let C be a category. A chosen binary-product structure begins with a whole functor

$$P : C \times C \longrightarrow C. \quad (30.2)$$

For objects A, B write $A \times B = P(A, B)$. The two projections are not unrelated arrows chosen afresh at every pair. They are components of whole transformations

$$\kappa_1 : P \Rightarrow \text{pr}_1, \quad \kappa_2 : P \Rightarrow \text{pr}_2. \quad (30.3)$$

Thus a change in either input already has a natural action. If $f : X \rightarrow A$ and $g : X \rightarrow B$, pairing supplies

$$\langle f, g \rangle : X \longrightarrow A \times B. \quad (30.4)$$

In the internal presentation, pairing itself is a whole transformation between represented Hom families. Fixing X, A, B projects it to a functor

$$\text{Hom}_C(X, A) \times \text{Hom}_C(X, B) \longrightarrow \text{Hom}_C(X, A \times B). \quad (30.5)$$

Consequently a higher arrow between two pairs of legs is sent to a higher arrow between their pairings. The operation is not capped at (30.4).

This selected product functor must be kept distinct from $\mathbf{ProductCat}(A, B)$, the category whose objects and arrows are pairs. The latter exists for every A and B and is used to state the source of P ; it does not choose products inside an arbitrary C .

30.2 The Triangular Product Cuts

Došen's cut-elimination perspective motivates antecedential product operations that make projection followed by further composition into one visible cut. For a suitable arrow h , write

$$K_1^a(h) = h \circ \kappa_1, \quad K_2^a(h) = h \circ \kappa_2. \quad (30.6)$$

The basic product reductions are

$$\begin{aligned} K_1^a(h) \circ \langle f, g \rangle &\rightsquigarrow h \circ f, \\ K_2^a(h) \circ \langle f, g \rangle &\rightsquigarrow h \circ g. \end{aligned} \quad (30.7)$$

The right-hand sides are smaller cuts: the product introduction has disappeared, and the unobserved component is absent. Pairing also distributes through precomposition,

$$\langle f, g \rangle \circ k \rightsquigarrow \langle f \circ k, g \circ k \rangle, \quad (30.8)$$

while projection postcomposition accumulates,

$$h \circ K_i^a(f) \rightsquigarrow K_i^a(h \circ f). \quad (30.9)$$

Finally, pairing the two projections removes the detour entirely:

$$\langle \kappa_1, \kappa_2 \rangle \rightsquigarrow \text{id}_{A \times B}. \quad (30.10)$$

These rules do more than prove that a product exists. They select a normal form for arrows built from projections, pairing, and composition. The generic action of the whole functor P agrees at proof time with the triangular map

$$P[f, g] \doteq \langle K_1^a(f), K_2^a(g) \rangle, \quad (30.11)$$

where $\doteq$ records proof-time agreement rather than a competing runtime orientation.

There is also a transparent unpairing functor that postcomposes an arrow with both projections. Pairing followed by unpairing and unpairing followed by pairing have checked pointwise inverse paths. Equality of the two whole functor composites is a stronger packaging step and is not inferred merely from those pointwise paths.

Formal status — checked. Evidence `TRIANGULAR-BINARY-PRODUCTS`. A selected whole product functor, whole projections, whole represented-family pairing, stable K_1^a/K_2^a and pairing observations, beta, distribution, eta, and selected normalization rules are active. The generic produ-

ct-functor action has an explicit proof-time comparison with the triangular map. No free-cartesian syntax or global commuting decision procedure is claimed.

30.3 The Empty Product

A chosen terminal object t is the empty product. Its computational structure is one whole transformation

$$! : \text{id}_C \Rightarrow \text{Const}_t. \quad (30.12)$$

The component at A is the canonical arrow

$$!_A : A \longrightarrow t. \quad (30.13)$$

Because (30.12) is whole, an arrow $h : A \rightarrow B$ has an off-diagonal observation, and the canonical terminal cut computes:

$$!_B \circ h \rightsquigarrow !_A. \quad (30.14)$$

The unrestricted uniqueness statement

$$f = !_A \quad (f : A \rightarrow t) \quad (30.15)$$

is not oriented as a variable-headed rewrite. Instead the entire Hom classifier $\text{Hom}_C(A, t)$ is contractible, recentered at the component $!_A$. Equation (30.15) is then a derived equality path. In particular $!_t = \text{id}_t$ is derived without forcing every visible endomorphism of a terminal object to rewrite before its context is known.

Formal status — checked. Evidence `TERMINAL-OBJECT-COMPUTATION`. The selected terminal object has a whole canonical-arrow transformation, computing component and off-diagonal action, the cut (30.14), and contractible Hom classifiers. Arbitrary uniqueness is equality evidence; there is no bare-variable unifier or variable-headed runtime rule.

The thin selected cartesian package merely pairs the existing binary-product and terminal-object capabilities. It adds no product, projection, terminal arrow, rewrite rule, or unification rule of its own.

30.4 Direct And Weighted Products

Chapter 16 describes weighted limits through representability. A binary product can also be presented by a suitable weight and a strict **Deflso** comparison. The direct triangular interface and the weighted interface answer different implementation questions:

- the triangular interface chooses the runtime projection/pairing calculus;
- the weighted interface packages a stronger representability comparison in the general profunctor machinery.

The active adapter therefore takes the weighted comparison as supplied data and records paths identifying its two projections with the triangular ones. It does not fabricate a weighted witness from pointwise beta and eta alone. This preserves both useful presentations without making either an alias for the other.

30.5 Slices And The Always-Existing Σ_u

Fix an object X of C . The slice C/X organizes arrows ending at X . Its objects may be written

$$a = (A \xrightarrow{p} X). \quad (30.16)$$

For $u : X \rightarrow Y$, postcomposition is always available:

$$\Sigma_u : C/X \longrightarrow C/Y, \quad (A \xrightarrow{p} X) \longmapsto (A \xrightarrow{u \circ p} Y). \quad (30.17)$$

These functors are the arrow action of one whole covariant family

$$\text{SliceSigma}(C) : C \longrightarrow \text{Cat}. \quad (30.18)$$

The fibre at X is exactly C/X . Hence the domain of (30.17) is preserved definitionally, and the structure arrow computes by ordinary postcomposition; no equality transport is inserted to repair its endpoint.

The internal slice is directed. A slice arrow has an underlying base arrow and a comparison cell in an iterated Hom. In a locally discrete category that cell amounts to the usual commuting equality. In a general higher category it must not be erased or replaced by a manually supplied equation.

30.6 Pullback As Chosen Base Change

Postcomposition does not choose a pullback domain. A chosen pullback structure supplies the opposite-variance whole family

$$\text{SliceBaseChange}(PB) : C^{\text{op}} \longrightarrow \text{Cat}, \quad (30.19)$$

again with exact fibre C/X . Its action at $u : X \rightarrow Y$ is

$$u^* : C/Y \longrightarrow C/X. \quad (30.20)$$

The defining universal relationship is the existing adjunction

$$\Sigma_u \dashv u^*. \quad (30.21)$$

Let its unit and counit be

$$\eta^u : \text{id}_{C/X} \Rightarrow u^* \Sigma_u, \quad \varepsilon^u : \Sigma_u u^* \Rightarrow \text{id}_{C/Y}. \quad (30.22)$$

They are actual whole transformations. Their off-diagonal actions are the Došen operations

$$\gamma_u^c(k) = \eta^u[k], \quad \varphi_u^a(h) = \varepsilon^u[h]. \quad (30.23)$$

For arrows of the appropriate slice types, the full rectangular cuts compute:

$$\begin{aligned} \varphi_u^a(h) \circ \Sigma_u(\gamma_u^c(k)) &\rightsquigarrow h \circ \Sigma_u(k), \\ u^*(\varphi_u^a(h)) \circ \gamma_u^c(k) &\rightsquigarrow u^*(h) \circ k. \end{aligned} \quad (30.24)$$

Identity instances recover the ordinary unit and counit components and both triangle identities. The readable slice terms normalize through opposite restriction-oriented Sigma categories, so the active rectangle rules match the exact surviving endpoints rather than defined aliases.

30.7 Mates And The Pullback Object

Fix a in C/X and g in C/Y . The adjunction gives whole mate functors

$$\begin{aligned} \Phi_{a,g} : \text{Hom}_{C/Y}(\Sigma_u a, g) &\longrightarrow \text{Hom}_{C/X}(a, u^* g), \\ \Gamma_{a,g} : \text{Hom}_{C/X}(a, u^* g) &\longrightarrow \text{Hom}_{C/Y}(\Sigma_u a, g). \end{aligned} \quad (30.25)$$

Their point formulas are

$$\Phi(h) = u^*(h) \circ \eta_a^u, \quad \Gamma(k) = \varepsilon_g^u \circ \Sigma_u(k). \quad (30.26)$$

The stable point and whole presentations compare at proof time with these semantic bodies. Both point composites and both whole functor composites reduce to identity. At the whole level, exact raw Hom-category guards retain a and g after opposite normalization; a wildcard-only cancellation rule would not be safe.

Now write $g : Z \rightarrow Y$. The selected pullback object is simply the slice object

$$u^*(g) = (P \xrightarrow{\pi_1} X). \quad (30.27)$$

The counit component

$$\varepsilon_g^u : \Sigma_u(u^* g) \longrightarrow g \quad (30.28)$$

has an underlying arrow $\pi_2 : P \rightarrow Z$ and a directed cell

$$g \circ \pi_2 \Longrightarrow u \circ \pi_1. \quad (30.29)$$

Thus the pullback object, both projections, and the square are observations of the selected slice object and counit. They are not independent fields.

A cone from a to g is exactly an object of

$$\mathrm{Hom}_{C/Y}(\Sigma_u a, g). \quad (30.30)$$

The universal lift is Φ , and cone recovery is Γ . There is no pullback-specific cone constructor accepting two legs and a square witness. In a locally discrete category, (30.29) and the projection cells recover the ordinary strict equations.

Formal status — checked. Evidence `PULLBACK-SLICE-BASE-CHANGE`. Chosen pullbacks form one whole exact-slice base-change family with $\Sigma_u \dashv u^*$, actual unit/counit transfors, full rectangular cuts, point and guarded whole mate cancellation, and derived pullback object/projection/square observations. Generic family substitution `Pullback_catd` remains a different construction.

30.8 Dependent Products In Slices

A category with chosen pullbacks need not have dependent products. The extra structure selects a third whole slice family

$$\mathrm{SliceDependentProduct}(DP) : C \longrightarrow \mathrm{Cat} \quad (30.31)$$

with exact fibre C/X . Its action at $u : X \rightarrow Y$ is

$$\Pi_u : C/X \longrightarrow C/Y. \quad (30.32)$$

The selected relationship is the second adjunction

$$u^* \dashv \Pi_u. \quad (30.33)$$

Write its unit and counit as

$$\eta^{\Pi,u} : \mathrm{id}_{C/Y} \Rightarrow \Pi_u u^*, \quad \varepsilon^{\Pi,u} : u^* \Pi_u \Rightarrow \mathrm{id}_{C/X}. \quad (30.34)$$

Again these are the generic actual adjunction transformations. Their stable off-diagonal observations satisfy

$$\begin{aligned} \varphi_{\Pi,u}^a(h) \circ u^*(\gamma_{\Pi,u}^c(k)) &\rightsquigarrow h \circ u^*(k), \\ \Pi_u(\varphi_{\Pi,u}^a(h)) \circ \gamma_{\Pi,u}^c(k) &\rightsquigarrow \Pi_u(h) \circ k. \end{aligned} \quad (30.35)$$

The associated whole Hom correspondence is

$$\mathrm{Hom}_{C/X}(u^* a, b) \simeq \mathrm{Hom}_{C/Y}(a, \Pi_u b). \quad (30.36)$$

The transparent transpose and untranspose functors have the familiar formulas

$$h \longmapsto \Pi_u(h) \circ \eta_a^{\Pi,u}, \quad k \longmapsto \varepsilon_b^{\Pi,u} \circ u^*(k), \quad (30.37)$$

and retain higher Hom action. The generic adjunction profunctor comparison owns the varying-end-point whole equivalence; no parallel mate theory is introduced.

Formal status — checked. Evidence `SLICE-DEPENDENT-PRODUCTS`. A selected coherent dependent-product structure supplies the whole exact-slice family Π_u , the existing adjunction $u^* \dashv \Pi_u$, actual whole unit/counit transfors, both full rectangular cuts, transparent whole mate action, and the generic whole Hom comparison. It adds no specialized composite-point or identity-action rule.

30.9 The Three-Adjoint Chain

Combining (30.21) and (30.33) gives the promised chain

$$\begin{aligned} \Sigma_u &: C/X \longrightarrow C/Y, \\ u^* &: C/Y \longrightarrow C/X, & \Sigma_u \dashv u^* \dashv \Pi_u. \\ \Pi_u &: C/X \longrightarrow C/Y, \end{aligned} \tag{30.38}$$

The left adjunction says that changing a structure arrow by postcomposition is freely related to pulling an object back. The right adjunction says that maps out of a pullback are represented by a dependent product. Both statements are internal equivalences of Hom categories with executable triangular cuts.

The neutral total capability

$$\text{SliceDependentProducts}(C) \tag{30.39}$$

packages a chosen pullback structure together with its indexed chosen dependent-product structure. The name is deliberately weaker than "locally cartesian closed category." To reach that conventional package the development should still select the intended finite-limit convention and derive exponentials in each slice. Beck-Chevalley comparison, Frobenius reciprocity, pushout duality, and comparison with independently selected weighted pullbacks are later coherence layers.

They are later layers, not reasons to replace the three whole families. Each future comparison has well-typed endpoints built from the existing Σ_u , u^* , and Π_u actions.

30.10 The End Of The Sixth Spiral

The chapter began with the smallest structural detour:

$$\kappa_i \circ \langle f, g \rangle.$$

It ended with adjunctions between categories of structured arrows. The same discipline governed every stage:

- keep the constructor or whole universal operation visible;

- reduce elimination after introduction to a smaller cut;
- use proof-time comparison when neither presentation should erase the other;
- retain higher action before projecting to points; and
- state coherence not yet constructed as a boundary rather than as an implicit equation.

The result is not a claim that all cartesian higher-category theory has been completed. It is a checked computational spine on which the next coherence questions can be asked without changing what products, pullbacks, or dependent products mean.

31. Additive, Abelian, And Homological Computation

How can a connecting homomorphism be computed while keeping the universal properties that explain it? An element chase suggests a procedure: choose a preimage, apply a differential, lift into a subobject, and pass to a quotient. Each instruction conceals a mathematical obligation. A preimage may exist only after an epimorphic cover; a lift must respect relations; a quotient map requires an annihilation equation. The choices must also agree with the homology objects already in use.

The answer developed here has two complementary levels. Internal factor spaces give the selected lifts, quotients and reconstruction laws needed by a constructive chase. Coherent kernel and cokernel presentations make these operations whole functors. Their adjunctions then build homology as a whole functor on an existing category of zero-composition diagrams. Addition and monic/epic cancellation supply the exactness arguments; actual transformations retain the naturality of the resulting operations.

The same constructions have an executable interpretation in polynomial Freyd categories. There the result is a bounded long exact sequence with retained maps, equations, and exactness witnesses. Its formal reconstruction retains the same raw chain spine and constructs homology and exactness at every interior position, relative to explicitly adopted equations and selected-provider semantics. The generic five-term homology window and its three interior exactness results are also checked, as is finite iteration over coherent rows. Attaching the final symbolic zero-endpoint theorem to the conventional bounded display remains a separate, deferred interface. The formal-status notes describe the current research calculus, with the foundational qualifications recorded in [Appendix F](#).

31.1 Addition Belongs To The Homs

A preadditive category has a set-valued abelian group $\text{Hom}_C(A, B)$ for every pair of objects, and composition is bilinear. Thus parallel arrows can be added and subtracted, and

$$h \circ (f + g) = h \circ f + h \circ g, \quad (f + g) \circ k = f \circ k + g \circ k. \quad (31.1)$$

There is a zero arrow for every ordered pair of objects. Composing a zero arrow on either side gives zero. The set-valued hypothesis concerns these particular Homs; it does not impose a global truncation on the ambient functorial type theory.

To obtain an additive category, supply the selected cartesian structure of [Chapter 30](#). A binary product becomes a biproduct. If its projections are p_A, p_B , define the injections by

$$\iota_A = \langle \text{id}_A, 0 \rangle, \quad \iota_B = \langle 0, \text{id}_B \rangle.$$

The product laws give the four projection–injection equations, and bilinearity gives

$$\iota_A \circ p_A + \iota_B \circ p_B = \text{id}_{A \oplus B}. \quad (31.2)$$

Consequently arrows out of the same object $A \oplus B$ are assembled by $f \circ p_A + g \circ p_B$. Product and coproduct are two universal uses of this selected object. No equality between independently chosen objects is needed. The chosen terminal object is also initial: its identity is zero, which forces every arrow from it to be the corresponding zero arrow. We therefore write this object as 0 .

Finite-free modules give a concrete model. Direct sum supplies the product, rank zero supplies the zero object, and matrix addition supplies the Hom groups. This explains why block matrices become useful later: they express operations already available in any additive category.

Formal status — checked. Evidence `ADDITIVE-HOM-BIPRODUCTS`. `PreadditiveCategory` supplies the abelian Hom structures and bilinearity; `AdditiveCategory` combines them with selected cartesian structure. The biproduct and initial-zero consequences are derived. The finite-free and Freyd additive instances reuse their existing matrix and quotient owners.

31.2 Kernels And Cokernels As Internal Factor Spaces

Fix an arrow $d : B \rightarrow D$. A kernel test consists of an arrow $h : T \rightarrow B$ and a path $d \circ h = 0$. For a proposed kernel embedding $k : Z \rightarrow B$, the factors of that test form the internal space

$$\sum_{u:T \rightarrow Z} (k \circ u = h). \quad (31.3)$$

A selected computational kernel retains Z, k , the annihilation path $d \circ k = 0$, and a contraction of (31.3) for every test. Its center gives the lift u together with the reconstruction. Its contraction supplies uniqueness when another arrow has the same reconstruction.

This is an instance of the ordinary internal Hom fibre. More generally, postcomposition by k has a fibre over any h , whether or not a kernel is involved. Precomposition has the dual fibre: given $q : B \rightarrow Q$ and $h : B \rightarrow T$, its points are arrows $v : Q \rightarrow T$ with $v \circ q = h$. A cokernel of $d : A \rightarrow B$ makes these latter fibres contractible whenever $h \circ d = 0$.

The quantifier over tests matters. Recording $d \circ k = 0$ says that k is a candidate cone. An effective weak kernel additionally supplies a factor for every annihilated test. A genuine computational kernel supplies the contractible factor spaces, including uniqueness. A finite list of successful tests does not provide this all-test operation.

These packages also explain how choices are compared. Two supplied kernels of the same arrow factor through each other. Uniqueness identifies the two composites with identities, producing an isomorphism with both inverse laws. The same argument applies to cokernels. Later constructions can therefore retain a supplied choice and compare it with another, instead of replacing its object silently.

Formal status — checked. Evidence `INTERNAL-KERNEL-COKERNEL-FACTORS`. `HomPostcompFactor` and `HomPrecompFactor` are transparent internal fibres. `ComputationalKernel` and `ComputationalCokernel` supply selected factors, reconstruction,

and uniqueness. The map/isomorphism owners construct comparisons between supplied universal objects.

The associated equations are theorem-level paths. Projecting a constructed factor computes; arbitrary reconstruction and uniqueness are consumed as equality evidence. These factor spaces are the operational interface to universality, not a requirement that every later construction be assembled as an isolated arrow calculation.

For coherent families, use the arrow-diagram category $\mathcal{D}_C = \text{Functor}(\mathbf{2}, C)$, where $\mathbf{2}$ is the walking arrow. The zero object gives whole functors

$$J(A) = (A \longrightarrow 0), \quad I(A) = (0 \longrightarrow A).$$

A kernel presentation supplies a whole $K : \mathcal{D}_C \rightarrow C$ and an adjunction $J \dashv K$ over the original selected kernels. A cokernel presentation supplies $Q : \mathcal{D}_C \rightarrow C$ and $Q \dashv I$ over the original selected cokernels. Evaluating their counit and unit gives the kernel embedding and cokernel projection. Transposition gives the universal lifts and colifts. Thus the adjunction and factor-space descriptions are two interfaces to the retained universal operations.

The presentation includes coherent structure: it is not an automatically constructed witness for every previously supplied kernel or cokernel family. Proof-time usability relates its observations to those original choices, while the whole functor heads remain available for the generic adjunction cuts. Conversely, the adjunctions supply operational universal records at their actual objects, so a later construction need not move between two independently selected kernels merely to use universality.

Formal status — checked. Evidence `WHOLE-KERNEL-COKERNEL-PRESENTATIONS`. The two presentation classifiers supply actual functors and adjunctions, with object, structural-arrow and map usability for the original selections. Their mate and record interfaces retain the universal factors. Existence of a coherent presentation remains an explicit input, not a consequence inferred from finite matrix tests.

31.3 Normality And The Image Comparison

A pre-Abelian capability combines additive structure with selected kernels and cokernels. Abelian structure adds the normal universal operations. For a monomorphism $m : A \rightarrow B$, an arrow $h : T \rightarrow B$ killed by the selected cokernel of m has a selected lift through m . Dually, an arrow out of the source of an epimorphism descends through it when it kills its selected kernel.

Monicity and epicity themselves are cancellation operations. They consume equalities after postcomposition or precomposition and return equalities of the original arrows. Normality supplies additional existence data. In particular, knowing that m is monic does not establish the cokernel-zero test required to lift a given h .

For $f : A \rightarrow B$, form

$$\text{Coim}(f) = \text{Coker}(\text{Ker}(f)), \quad \text{Im}(f) = \text{Ker}(\text{Coker}(f)).$$

Kernel and cokernel universality construct a comparison χ_f and a factorization

$$A \xrightarrow{p_f} \text{Coim}(f) \xrightarrow{\chi_f} \text{Im}(f) \xrightarrow{i_f} B, \quad f = i_f \circ \chi_f \circ p_f. \quad (31.4)$$

The factorization already exists at the pre-Abelian boundary. With Abelian normality, the comparison is proved monic and epic. A monic epic arrow has an inverse: lift its codomain identity through the monomorphism, using epicity to establish the normal test, and use cancellation for the other inverse law. Applying this construction to χ_f gives the coimage–image isomorphism.

This formulation retains the two objects and the comparison between them. The isomorphism is computed from universal operations and cancellation; there is no identification of independently selected image and coimage objects by fiat.

Formal status — checked. Evidence `ABELIAN-NORMALITY-IMAGES`. `ComputationalAbelianCategory` retains one pre-Abelian capability and its normal lift/colift families. The image, image-bimorphism, and bimorphism owners derive (31.4) and the comparison isomorphism. These generic capabilities remain explicit when passed to a concrete implementation.

31.4 Whole Homology Retains Its Cycles And Boundary

Consider a chain pair

$$A \xrightarrow{d_{n+1}} B \xrightarrow{d_n} D, \quad d_n \circ d_{n+1} = 0.$$

Select the cycles as the kernel $k_n : Z_n \rightarrow B$ of d_n . The chain law is exactly the test required to lift d_{n+1} into this kernel. Write the resulting boundary arrow as $b_n : A \rightarrow Z_n$. Homology is its selected cokernel:

$$k_n \circ b_n = d_{n+1}, \quad q_n : Z_n \longrightarrow H_n, \quad q_n \circ b_n = 0. \quad (31.5)$$

The whole homology result retains these universal objects and their reconstructions. Its cycle object, boundary, homology object, and projection are observations of that result. Supplying a whole homology value therefore fixes the cycle kernel as well as the final quotient.

A map of chain pairs is represented by a middle component $f : B \rightarrow B'$ and two existing Hom-factor points. Their arrows supply the neighboring components; their reconstruction paths give the familiar chain-map compatibilities. The lower compatibility makes $f \circ k_n$ land in the target cycles, so kernel universality constructs a cycle map z . The upper compatibility and kernel uniqueness then show that z preserves boundaries. Cokernel universality gives the induced homology map, characterized by

$$H_n(f) \circ q_n = q'_n \circ z. \quad (31.6)$$

These constructions give a useful selected-record interface. Its map laws follow from uniqueness and compare maps between the supplied homologies. Other choices receive comparison isomorphisms, preserving the distinction between an invariant up to isomorphism and the particular object used in a calculation.

Formal status — checked. Evidence `SELECTED-HOMOLOGY-MAPS`. `ComputationalHomologyAt` retains the actual cycle kernel and boundary cokernel. The chain-pair-map and homology-map owners construct the cycle action, boundary compatibility, induced map, and its laws. The whole functor below supplies the primary interface for coherent families.

Now let a category $\mathcal{B}$ parameterize the data. Supply actual functors $A : \mathcal{B} \rightarrow C$ and $D : \mathcal{B} \rightarrow \mathcal{D}_C$, together with an actual transformation $h : J \circ A \Rightarrow D$. The kernel adjunction constructs the boundary transformation directly:

$$\beta = K(h) \circ \eta_A : A \Longrightarrow K \circ D.$$

At a point this is precisely the lift of the incoming differential into cycles. But the formula is already a composition of whole transformations; there is no additional family of naturality squares to supply. Introducing its walking-arrow diagram gives a functor $\text{Arr}(\beta) : \mathcal{B} \rightarrow \mathcal{D}_C$. Define

$$H_{A,D,h} = Q \circ \text{Arr}(\beta) : \mathcal{B} \longrightarrow C.$$

This is a derived whole functor, not a new primitive homology algorithm. Its identity and composition computations are the existing generic functor computations, including the accumulation direction $H(g) \circ H(f) \longrightarrow H(g \circ f)$. Its Hom actions remain available at further dimensions. The selected-record views recover its actual cycles, boundary and quotient; in particular, the record's homology object is the original $H[b]$, not a replacement object reached by transport.

There is also one native source for these coherent diagrams:

$$\mathcal{Z}_C = (J \downarrow \text{id}_{\mathcal{D}_C}).$$

This uses the existing internal Hom-comma construction. An object is (A, d, h) with $h : J(A) \Rightarrow d$. Its source component is the incoming arrow; naturality and the zero-object property give the zero composite with the differential of d . Maps and their compatibility come from the native Hom structure, not a newly introduced datatype of hand-written commuting squares. In the ordinary-target profile used here, where C is used as a one-category, the tautological whole transformation is available. Applying the preceding family construction to it gives $H : \mathcal{Z}_C \rightarrow C$.

This is whole one-degree homology on zero-composition diagrams. It is not a claim that every possible category of bounded or unbounded complexes, or a global graded homology functor, has already been packaged. The local ordinary-target hypothesis and the two coherent presentations stay explicit; the polynomial Freyd interface fills its ordinary-target profile from its existing set-valued Hom presentation.

Formal status — checked. Evidence `WHOLE-HOMOLOGY-FUNCTOR`. `homology_family_func` derives the whole functor; its boundary is an actual transformation. `zero_arrow_cone_homology_func` specializes it to the native source with the ordinary-target profile. The family-record projection returns the same H object. Reviewers exercise nonidentity action, generic identity/composition and further Hom action.

31.5 Exactness Is A Property Of The Actual Boundary

For $A \xrightarrow{i} B \xrightarrow{p} D$ with zero composite, let $b : A \rightarrow \text{Ker}(p)$ be the boundary in the chosen homology. Exactness at B is the statement that this b is epic. The definition uses the actual kernel factor and its cancellation operation. A short exact row additionally retains monicity of i and epicity of p .

In an Abelian category, this definition admits the following useful local description. Every $h : T \rightarrow B$ with $p \circ h = 0$ factors through i after an epimorphic cover: there are $e : T' \rightarrow T$, epic, and $a : T' \rightarrow A$ with

$$h \circ e = i \circ a. \quad (31.7)$$

If b is epic, pull it back along the kernel lift of h to obtain these data. Conversely, apply such a family of covers to the kernel embedding. The resulting factor and the epic cover prove that the original boundary b is epic. This is a constructive replacement for choosing an elementwise preimage in a diagram chase.

There is also a sufficient dual criterion. A test $h : B \rightarrow T$ killed by i can be extended through p after a monomorphism $m : T \rightarrow U$:

$$m \circ h = a \circ p \quad (a : D \rightarrow U). \quad (31.8)$$

A family of these extensions proves the same exactness property. The checked development uses this direction on the cokernel side of the snake proof; it does not claim a separately packaged equivalence for the dual criterion.

For a short exact row, its source is isomorphic to the selected kernel of its projection, and the selected cokernel of its inclusion is isomorphic to its target. The first forward map is the original boundary b . The second inverse lands in that cokernel. Neither comparison chooses a section of the projection into the middle object. This matters already for the polynomial row $0 \rightarrow R \xrightarrow{x} R \rightarrow R/(x) \rightarrow 0$.

Formal status — checked. Evidence `SELECTED-EXACTNESS-COVERS`. `ComputationalExactAt` is indexed by the supplied whole homology. The exactness-cover owners prove the local-cover equivalence; the extension owners prove the sufficient dual criterion. Short-exact comparison isomorphisms retain the original boundary and quotient factors.

31.6 Fibre Products, Pushouts, And Stability

The cover argument requires genuine fibre products. Given $f : X \rightarrow Z$ and $g : Y \rightarrow Z$, form the difference arrow

$$[f, -g] : X \oplus Y \longrightarrow Z.$$

Its selected kernel is a fibre product. Composing the kernel embedding with the biproduct projections gives $p : E \rightarrow X$ and $q : E \rightarrow Y$, with $f \circ p = g \circ q$. An equalizing pair becomes a kernel test by pairing its legs. The kernel factor then reconstructs both projections. Replacing the genuine kernel with a weak kernel gives a weak pullback, with the weaker factor capability retained explicitly.

The direct dual constructs the pushout of $f : Z \rightarrow X$ and $g : Z \rightarrow Y$ as the selected cokernel of

$$\begin{bmatrix} f \\ -g \end{bmatrix} : Z \longrightarrow X \oplus Y.$$

Its injections have the required compatibility, and its cofactor operation reconstructs both legs of every compatible cocone. The matrix notation is the biproduct notation of Section 31.1; it does not require a new datatype for commuting squares.

Abelian normality proves the stability facts needed by the chase. A pullback projection opposite an epimorphism is epic. A pushout injection opposite a monomorphism is monic. These properties are derived from the selected difference kernel or cokernel, its universal factor spaces, and the normal lift/colift operations. They are available with the same whole fibre product or pushout that supplies the structural maps.

Formal status — checked. Evidence `ABELIAN-FIBER-PUSHOUT-STABILITY`. The computational fibre-product and pushout owners construct both reconstructions. The Abelian stability owner proves the indicated epicity and monicity. All are transparent constructions over the existing additive and universal-property layers.

31.7 Constructing The Snake Arrow

The generic snake begins with a particularly small input:

$$A \xrightarrow{\delta} B \xrightarrow{\beta} X \xrightarrow{\lambda} D, \quad \lambda \circ \beta \circ \delta = 0. \quad (31.9)$$

Select the cokernel $\epsilon : B \rightarrow \text{Coker}(\delta)$ and the kernel $\mu : \text{Ker}(\lambda) \rightarrow X$. Universality gives

$$\begin{aligned} \alpha : A &\longrightarrow \text{Ker}(\lambda), & \mu \circ \alpha &= \beta \circ \delta, \\ \gamma : \text{Coker}(\delta) &\longrightarrow D, & \gamma \circ \epsilon &= \lambda \circ \beta. \end{aligned} \quad (31.10)$$

The intended connecting arrow goes from $\text{Ker}(\gamma)$ to $\text{Coker}(\alpha)$. These are new universal constructions on the induced maps in (31.10), so their identities must be kept visible. Write $\iota : \text{Ker}(\gamma) \rightarrow \text{Coker}(\delta)$ and $\pi : \text{Ker}(\lambda) \rightarrow \text{Coker}(\alpha)$ for their structural arrows.

Pull back ϵ along ι . This gives $p_1 : E \rightarrow \text{Ker}(\gamma)$ and $p_2 : E \rightarrow B$, with $\iota \circ p_1 = \epsilon \circ p_2$. Stability makes p_1 epic. Push out μ along π . Its injections $q_1 : X \rightarrow Q$ and $q_2 : \text{Coker}(\alpha) \rightarrow Q$ satisfy $q_1 \circ \mu = q_2 \circ \pi$, and q_2 is monic.

The existing annihilation and image consequences establish the two normal tests. Normal-epi colifting first constructs $u : \text{Ker}(\gamma) \rightarrow Q$. Normal-mono lifting then constructs ∂ with

$$u \circ p_1 = q_1 \circ \beta \circ p_2, \quad q_2 \circ \partial = u. \quad (31.11)$$

This order explains the role of the cover. There is no chosen section of p_1 , and hence no instruction to select a global preimage in E . The normal universal operations descend and lift an arrow only after the corresponding test has been proved.

There is a useful reconstruction before leaving the cover. The arrow $\beta \circ p_2$ is killed by λ , so its selected kernel lift is $L : E \rightarrow \text{Ker}(\lambda)$ with $\mu \circ L = \beta \circ p_2$. Postcomposing the two candidates below with monic q_2 , and using (31.11) and pushout compatibility, proves

$$\partial \circ p_1 = \pi \circ L. \quad (31.12)$$

The equality retains the original cover, lift, and connecting arrow. It is especially useful when the connecting construction must later be compared with cycles and homology.

Formal status — checked. Evidence `ABELIAN-SNAKE-CONNECTING`. `AbelianSnakeTriple` retains (31.9). The connecting-result owner constructs both normal factors and their reconstructions. The covered-reconstruction owner proves (31.12) by cancelling the existing monic q_2 ; no connecting formula, normal test, or factor is postulated.

31.8 The Six-Term Exact Sequence

Theorem 31.1 (selected six-term snake sequence). In a supplied computational Abelian category, the triple (31.9) determines the sequence

$$\begin{aligned} \text{Ker}(\alpha) &\longrightarrow \text{Ker}(\beta) \longrightarrow \text{Ker}(\gamma) \\ &\xrightarrow{\partial} \text{Coker}(\alpha) \longrightarrow \text{Coker}(\beta) \longrightarrow \text{Coker}(\gamma), \end{aligned} \quad (31.13)$$

with four adjacent-zero paths and exactness at its four interior objects. All maps and exactness witnesses refer to the universal objects selected by the construction. The input does not assert that the two outer arrows are monic or epic, so no zero endpoints are added to (31.13).

The kernel-side arrows follow from δ and ϵ by kernel universality; the cokernel-side arrows follow from μ and λ by cokernel universality. Their reconstruction paths give the four adjacent-zero equations by cancellation. Exactness then needs a further argument at each interior object.

At $\text{Ker}(\beta)$, take a test killed by the next arrow. The canonical cokernel row for δ gives an epic cover and a δ -preimage. Equation (31.10) and monicity of μ force that preimage into $\text{Ker}(\alpha)$. Kernel uniqueness identifies the resulting map with the covered test, establishing the local-cover criterion.

At $\text{Ker}(\gamma)$, pull a test killed by ∂ back along p_1 . On that cover, its lifted L -component is killed by π . The canonical cokernel row for α supplies a second epic cover and an α -preimage. Subtracting its δ -image from the covered p_2 -leg produces an arrow killed by β . Its factor through $\text{Ker}(\beta)$ reconstructs the test after both covers. Composition of those covers completes the proof.

The two cokernel positions use the extension criterion. At $\text{Coker}(\alpha)$, push out q_2 along a test killed by ∂ , then use a canonical kernel-row extension. A corrected difference kills β and descends through its cokernel. Epic cancellation identifies the resulting coextension. At $\text{Coker}(\beta)$, a test killed by the incoming arrow extends through λ after a monomorphism; the reconstruction for γ makes it descend through $\text{Coker}(\gamma)$.

The result stores the six-term snapshot together with exactness evidence indexed by that snapshot. This dependency is part of the theorem: a proof about another pair of arrows cannot be inserted into the result merely because its object names look similar.

Formal	status	—	checked.	Evidence	ABELIAN-SNAKE-SIX-TERM-EXACT .
<code>abelian_snake_six_term_exact_result</code>				constructs	<code>AbelianSnakeSixTermExactResult</code>

from the original Abelian capability and triple. Its projections recover the canonical five arrows, four zero paths, and four exactness instances. This generic snake theorem is independent of the concrete polynomial tests below.

31.9 Exact Homology Windows And Bounded Assembly

A bounded complex retains a finite family of objects and differentials, together with the adjacent-zero laws. A degree-wise short exact sequence retains three such complexes and their inclusion and projection maps, with a short exact row in each degree. In the coherent formal interface, the rows are whole functors into the native zero-cone category and their maps are actual transformations. Four consecutive rows provide one homology window; shifting the window reuses three of those rows.

Theorem 31.2 (selected homology window). In an ordinary-target computational Abelian category with supplied coherent kernel and cokernel presentations, such a window determines

$$\begin{aligned} H_n(A) &\xrightarrow{H(i_n)} H_n(B) \xrightarrow{H(p_n)} H_n(C) \\ &\xrightarrow{\partial_n} H_{n-1}(A) \xrightarrow{H(i_{n-1})} H_{n-1}(B). \end{aligned} \tag{31.14}$$

The three adjacent composites are zero, and the sequence is exact at its three interior objects. Its homologies are the actual whole-H observations of Section 31.4. No replacement homology object or endpoint transport is needed to state the connecting map.

Here is the constructive idea behind the direct connecting operation. Pull back the row projection $p_n : B_n \rightarrow C_n$ along the actual cycle embedding $k_C : Z_n(C) \rightarrow C_n$. Write its legs as $e : E \rightarrow Z_n(C)$ and $b : E \rightarrow B_n$, so $p_n b = k_C e$; the first leg is epic. Applying the middle differential to b gives an arrow killed by the next row projection. That row's kernel property lifts it through its original inclusion into B_{n-1} . The following chain law and monicity then force this lift into the actual cycles of A . Write the resulting cycle arrow as $c : E \rightarrow Z_{n-1}(A)$.

The normal universal operations descend $q_A c$ along the cover, and the upper neighboring row proves that the resulting map kills the source boundary. Cokernel universality therefore gives ∂_n , with

$$\partial_n \circ q_C \circ e = q_A \circ c.$$

All tests needed for these operations are derived. The caller does not provide a section of e , a connecting arrow, or extra normal-test equations. Since $q_C \circ e$ is epic, the covered formula also characterizes the resulting arrow.

The exactness proofs use the same cover-and-correction method. At $H_n(B)$, represent a test by a middle cycle after an epic cover. Its projection is a boundary; lift a boundary preimage through the upper row, then subtract its middle differential. The corrected cycle has zero projection and hence comes from the left column.

At $H_n(C)$, a class killed by ∂_n has a covered lift whose left-column cycle is a boundary. Subtract the corresponding inclusion from its middle-column lift. The corrected lift is a middle cycle and reconstructs the original class under $H(p_n)$.

At $H_{n-1}(A)$, a class killed by $H(i_{n-1})$ becomes a middle boundary after a cover. Projecting a boundary preimage to the right column gives a cycle. The covered connecting formula sends its class back to the original left-column class. These all-test constructions establish the three exactness predicates at the same H objects.

Formal status — checked. Evidence `HOMOLOGY-DIRECT-CONNECTING` and `HOMOLOGY-WINDOW-EXACTNESS`. The direct connecting factor, source-boundary descent and covered reconstruction are constructed at the retained homologies. The first, second and third window owners prove all three interiors; `homology_whole_exact_window` packages those original witnesses. These are generic formal results, not deductions from polynomial examples.

Naturality is also a whole interface. For a window family over $\mathcal{B}$, let $R, L : \mathcal{B} \rightarrow \mathcal{Z}_C$ be its right and left vertical zero-diagram functors in degrees n and $n - 1$, respectively. Composing them with H gives the source and target homology functors. The connecting interface is an actual transformation

$$\partial_w : H \circ R \Longrightarrow H \circ L.$$

Its component observation computes to the direct connecting construction. Generic transfor action supplies the naturality and further Hom-action interfaces; no separately supplied naturality square or transformation constructor is part of the input. Whole column observations retain the original row functors, with proof-time comparisons for the alternative Hom-action projection order.

Formal status — checked. Evidence `HOMOLOGY-CONNECTING-TRANSFORMATION`. The whole source and target are composites with the original `H`. `homology_window_connecting_transf` has the retained component computation and ordinary off-diagonal/higher transfor action. The whole transformation is present from the outset; pointwise naturality proofs are not additional caller inputs.

The native implementation retains a snake-based trace of the same public homology operation. A useful comparison in that trace is the canonical monomorphism

$$j_A : H_{n-1}(A) \longrightarrow \text{Coker}(d_n^A). \quad (31.15)$$

It is induced from the actual cycle embedding and boundary cokernel. Its monicity follows generically from pushout stability.

Formal status — checked. Evidence `HOMOLOGY-COKERNEL-INCLUSION`. The homology-inclusion owner constructs j_A at the supplied whole homology and proves it monic, retaining the original cycles and quotient.

If s compares source cycles with the snake kernel and v compares the snake cokernel with the differential cokernel, the retained native reconstruction is

$$j_A \circ \partial_n \circ q_C = v \circ \partial \circ s. \quad (31.16)$$

This explains the implementation trace without making its intermediate isomorphisms the public definition of the formal connecting interface. The supplied-model interpretation in Section 31.10 records the connection to the native selected arrow explicitly.

Finite formal assembly starts with an actual H-inclusion arrow. Each window extension prepends its three arrows and the three existing zero/exactness annotations to a retained continuation. A field-indexed Nat iterator repeats this operation over coherent rows and transformations. Adjacent windows share their boundary arrow; they are not independent windows joined by object-equality fields. For two windows, the result has eight objects, seven arrows and six interior exactness annotations.

Formal status — checked. Evidence `HOMOLOGY-BOUNDED-ITERATOR`. `homology_row_field_span_exact_tail` iterates any finite number of windows. Its data projections retain the actual arrows and original annotations. The conventional zero-ended symbolic theorem, including its final endpoint-proof attachment, is deferred; it is not implied merely by this checked iterator.

The native polynomial implementation already assembles the complete bounded display. It constructs each degree's homologies and ordinary induced maps once, then shares them between neighboring windows. Exactness at $H_n(B)$ and $H_n(C)$ comes from the window at n ; exactness at $H_n(A)$ comes from the window at $n + 1$. Its outside-support endpoints retain their true neighboring differentials and explicit zero-identity witnesses, not replacement zero presentations.

These are whole operations in the computational category interface. A retained program declares its kernel, cokernel, normal-factor and other prerequisites; compilation lowers the operation to the native algebra engine. Observations return stored homologies and windows. Direct and compiled execution are compared on complete serialized results.

This operation-and-prerequisite design takes inspiration from [CAP](#) and the separation of categorical algorithms from ring computations in [homalg](#). The native emdash implementation owns its polynomial algorithms; those reference systems are not required at runtime.

The examples exercise nonsplit polynomial rows, nonzero boundaries and nonzero connecting maps. An independent field-linear calculation over $\mathbb{Q}$ compares results in explicit quotient coordinates, including both signs and endpoint cases. It uses constant polynomials, where the coefficient ring is already $\mathbb{Q}$; it does not specialize a polynomial variable and assume that exactness survives.

Executable reviewers:

- `tests/v3_2_algebra_polynomial_freyd_homology_connecting_tests.ts`
- `tests/v3_2_algebra_polynomial_freyd_long_exact_tests.ts`
- `tests/v3_2_algebra_polynomial_freyd_long_exact_category_tests.ts`
- `tests/v3_2_algebra_polynomial_freyd_long_exact_differential_tests.ts`

They check selected-object reuse, bounded output, compiled agreement and coordinate comparisons. Native endpoint computation and the checked generic window theorem must not be confused with the deferred final symbolic endpoint attachment. The [worked nonsplit example](#) below follows one such computation through to its whole-H interpretation.

31.10 Presentations, Matrices, And Formal Replay

Concrete computation needs an effective representation. A Freyd presentation over a commutative ring R has a relation map $\rho_P : R^{r_P} \rightarrow R^p$. A raw morphism from P to Q retains a generator matrix F and a relation-coefficient matrix W satisfying

$$\rho_Q W = F \rho_P. \quad (31.17)$$

Equality of represented morphisms has its own witness. Two generator matrices F, G agree in the quotient when a retained matrix H satisfies $\rho_Q H = F - G$. The relation witness in (31.17) and this equality witness have different jobs. The first makes a raw map well formed; the second compares quotient maps.

For the supported polynomial provider, effective syzygy and membership operations supply these witnesses. Kernels use the two-weak-pullback Freyd construction; normal lifts and colifts consume the corresponding explicit agreements. A successful computational decision supplies data for the next map. A failed decision remains negative computational information, while an unsupported operation or exhausted resource does not prove nonexistence.

The formal interface can use a supplied finite-free weak-kernel capability and explicit raw agreements. This yields the witnessed Freyd universal operations. It does not decode an arbitrary propositionally truncated quotient equality into a chosen raw witness. Native effectiveness and this formal input contract are separate aspects of the implementation.

The constructive Freyd-category account in Posur explains why relation witnesses, weak-kernel factor operations and effective agreement solving must be distinguished. The broader algorithmic setting is developed in the homalg axiomatic setup.

The proof-CAS workflow first replays a whole selected computation and compares its complete serialization. It then presents the exact matrix equations, with fixed coefficient bindings, for explicit adoption into a proof environment. Running the computation alone changes no proof assumptions. Adoption is a recorded trust decision; the subsequent typecheck checks the construction relative to those adopted equations.

The whole-H interface adds a further connection. A supplied coherent Freyd homology model retains its kernel and cokernel operations and their presentations. Its formal H points and induced maps can be interpreted at the native results already stored by a replay. With the same model's normality enhancement, this includes every retained connecting window and the complete displayed arrow order. The interpretation does not run homology or connecting again, or select another kernel to make the types fit.

Three kinds of information remain distinct. Matrix equations justify particular raw morphisms and agreements. Selected-provider contracts supply all-test universal operations. Whole-model interpretations identify the formal H observations, model-side short-exact rows and connecting arrows with the retained computational data. The latter two are explicitly recorded trusted presentation semantics, not theorems inferred from a finite list of equations. Model normality is a typed supplied input.

Formal status — checked. Evidence `FREYD-WHOLE-HOMOLOGY-MODEL`. `FreydHomologyModel` packages the supplied coherent data. Its object, complete-arrow and connecting observations use the original H; the normality enhancement retains its original kernel/cokernel selections. This is an interface for a supplied model, not a closed construction of a polynomial Freyd Abelian model from matrix equations alone.

Without normality, the workflow still exposes the useful point/induced-map prefix and reports that connecting interpretation was not requested. With normality, it checks that every displayed arrow is the original retained arrow before returning the complete observation view. Repeating an adoption reuses existing claims rather than silently adding duplicate assumptions.

Those equations can also be consumed as data. A transparent introduction packages a retained coefficient matrix and its equation as the original raw agreement value. The whole inventory can therefore expose its degreeewise, connecting and snake witnesses as typed raw terms, with every label retained even when identical terms share one representation. This construction needs no additional trust decision.

Formal status — checked. Evidence `FREYD-RAW-WITNESS-INTRO`. The explicit-matrix introduction returns the original raw agreement and preserves its coefficient witness. It does not decode a truncated quotient equality or turn a literal matrix equation into an unproved semantic-composition equation.

The adopted equations construct a formal raw spine. Each displayed arrow is built by (31.17) using the original generator and relation data. For consecutive arrows $F : P_2 \rightarrow P_1$ and $G : P_1 \rightarrow P_0$, the retained equation

$$\rho_{P_0} H = GF - 0 \quad (31.18)$$

constructs their original Freyd chain-pair agreement. Its right-hand side is the formal matrix composite, so it addresses the composed arrows in the term itself. The bounded constructors assemble these morphisms and laws into one sequence. The displayed left-to-right order is translated into the formal convention in which index zero names the rightmost object.

The completed term is checked as a whole. This goes beyond checking a list of equation types: the constructors must fit together at their actual endpoints, with every projected arrow preserved. Taken by itself, this establishes a formal chain spine. Section 31.11 adds the separate selected-homology and exactness evidence.

Formal status — checked. Evidence `FREYD-FORMAL-RAW-SPINE`. `comm_ring_presentation_morphism_from_matrices` and `comm_ring_freyd_chain_pair_from_matrices` return the existing formal owners, which assemble raw morphisms and adjacent-zero agreements through the existing bounded constructors. No primitive matrix algorithm, quotient decoder, or new Core structure is introduced.

The executable bridge and its trust boundary have separate reviewers:

- `tests/v3_2_algebra_formal_freyd_long_exact_tests.ts`
- `tests/v3_2_algebra_formal_freyd_spine_tests.ts`

Their live consumers check the constructed whole term, including nontrivial bounded and short-tail cases, relative to the explicitly adopted equations.

31.11 What A Boundary-Epicity Witness Proves

At an interior position, the native exactness calculation retains a boundary $F : P \rightarrow Z$. Write Q for the relation matrix of its target presentation Z . Its epimorphism data provide matrices U, V with

$$QU + FV = \text{id}. \quad (31.19)$$

The identity says that every target generator is the sum of a target relation and a boundary image. The canonical cokernel of F has relation matrix $[Q \ F]$, so the vertical block satisfies

$$[Q \ F] \begin{bmatrix} U \\ V \end{bmatrix} = \text{id}. \quad (31.20)$$

Thus its cokernel projection agrees with zero. This is exactly the existing formal Freyd epimorphism witness. The explicit constructor derives it from (31.19), together with the original relation-preservation law for F ; the block identity itself is not an opaque declaration of epicity.

The whole-long-exact consumer applies this construction to each retained interior boundary. It uses the boundary's own target cycle presentation and original morphism, reusing an exactly matching relation equation or explicitly adopting a missing one. It does not rerun homology separately for each witness. Each resulting epicity term is then checked in the extended proof environment.

Equation (31.19) proves epicity onto Z , but exactness needs more: Z must have the kernel universal property for the outgoing arrow, and F must reconstruct the incoming arrow through its kernel embedding. The bridge therefore retains the two actual weak pullbacks used to construct the cycle presentation. Each carries an operation that factors every admissible test, together with its reconstruction law. Their ranks and projection matrices are observations of those same packages.

The native factor algorithms are bound to these packages by an explicit trust decision. A finite collection of matrix equations does not prove the all-test law. The adopted provider semantics name the represented ring, the selected data, and the retained factor operation. They are kept distinct from the computed equations used for individual morphisms and agreements.

This distinction also prevents an invalid change of scalars. Multiplication by x has zero kernel over $k[x]$, but becomes a zero map after passing to $k[x]/(x)$. Its selected kernel cannot simply be carried across that quotient. The polynomial provider is therefore interpreted over its original coefficient ring. Modules presented by relations over that ring, including $k[x]/(x)$ as a $k[x]$ -module, remain ordinary supported inputs.

Formal	status	—	checked.	Evidence	FREYD-FORMAL-BOUNDARY-EPICITY .
				<code>comm_ring_freyd_epimorphism_from_matrices</code>	constructs
				<code>CommRingFreydEpimorphismWitness</code>	from the block equation and original morphism law. This
proves formal epicity relative to the supplied equations. The provider and homology constructions					
below supply the separate cycle and boundary data needed to use it as exactness.					

The explicit-equation bridge and its application to every retained interior boundary are exercised by:

- `tests/v3_2_algebra_formal_freyd_epimorphism_tests.ts`
- `tests/v3_2_algebra_formal_freyd_long_exact_epimorphisms_tests.ts`

These reviewers separately check original-morphism preservation, adoption bindings, and the constructed formal witness terms.

With those cycle choices fixed, the homology constructor accepts the original raw boundary and its reconstruction agreement, then builds the existing witnessed cokernel of that boundary. It does not replace a raw relation witness merely because another one represents the same quotient map. The epimorphism term from (31.19) is then exactness at precisely this stored boundary.

Formal status — checked. Evidence `FREYD-SELECTED-HOMOLOGY-EXACTNESS`. The selected-kernel and homology constructors retain the two weak pullbacks, their universal operations, the supplied raw boundary, its reconstruction, and its cokernel. The exactness introduction reuses the existing epicity witness at that same boundary; it adds no opaque exactness theorem.

The whole-result consumer performs this construction at every retained interior position. One proof environment contains the original spine and the homology/exactness terms for each displayed pair. The two-degree nonsplit example checks the spine and all twelve interior terms together, without another kernel or homology selection. The executable reviewers are:

- `tests/v3_2_algebra_formal_freyd_actual_homology_tests.ts`
- `tests/v3_2_algebra_formal_freyd_long_exact_homology_tests.ts`

Their conclusion is relative to the recorded equation and provider assumptions, not a generic theorem about arbitrary Abelian categories. The same reviewers cover the retained whole-model point, induced-map and connecting interpretations described in Section 31.10, including explicit coverage, unchanged selections and rejection of mismatched model inputs.

A Nonsplit Example From Computation To Whole H

Take $R = \mathbb{Q}[x]$ and $S = R/(x)$, always regarded as an R -module. Consider the complexes supported in degrees one and zero:

$$A_{\bullet} = (R \xrightarrow{x} R), \quad B_{\bullet} = (R \xrightarrow{x} R), \quad C_{\bullet} = (S \xrightarrow{0} S).$$

Multiplication by x defines the inclusion $i : A_{\bullet} \rightarrow B_{\bullet}$, and quotient projection defines $p : B_{\bullet} \rightarrow C_{\bullet}$. In both supported degrees the row is

$$0 \longrightarrow R \xrightarrow{x} R \longrightarrow S \longrightarrow 0.$$

This row is nonsplit. An R -linear map $s : S \rightarrow R$ would satisfy $xs(\bar{1}) = s(x\bar{1}) = 0$. Since multiplication by x is injective on R , every such map is zero; none is a section of the quotient projection. The computation therefore cannot obtain its connecting map by choosing a linear section.

The homology calculation gives

$$\begin{aligned} H_1(A) &= H_1(B) = 0, & H_0(A) &\cong H_0(B) \cong S, \\ H_1(C) &\cong S, & H_0(C) &\cong S. \end{aligned}$$

After omitting the leading zero terms and identifying the displayed homologies with S , the long exact sequence reads

$$0 \longrightarrow S \xrightarrow{\partial_1} S \xrightarrow{0} S \xrightarrow{\text{id}} S \longrightarrow 0, \quad \partial_1 = \text{id}.$$

To see the connecting map, represent a class of $H_1(C)$ by $\bar{r}$, lift it to $r \in B_1$, and apply the differential. The result $xr \in B_0$ is the image of $r \in A_0$, whose homology class is again $\bar{r}$. Replacing the representative by $r + xu$ changes the resulting element of A_0 by the boundary xu , so the class is independent of that choice. This is a calculation with a representative, not a linear section $S \rightarrow R$. The native algorithm implements the corresponding universal lifting and descent operations.

The actual matrices retain more information than this simplified display. The differential of C is stored as $[x]$ between presentations with relation matrix $[x]$; it is zero as a quotient map, not as a raw matrix. The returned connecting and induced maps are

$$\partial_1 = [1], \quad H_0(i) = [x], \quad H_0(p) = [1].$$

The selected $H_0(C)$ has one generator and relation matrix $[x \ x]$. Its cokernel construction retains both the original relation and the image of the incoming differential. The two columns generate the same relation submodule as $[x]$, but the result is not silently replaced by that smaller presentation. Neighboring windows and formal observations refer to the original selected object.

The same distinction explains a concrete adjacent-zero witness. The composite $H_0(i) \circ \partial_1$ has raw matrix $[x][1] = [x]$, while its target $H_0(B)$ has relation matrix $\rho = [x]$. The coefficient matrix $[1]$ witnesses

$$\rho[1] = [x] = [x][1] - [0].$$

This is an instance of (31.18). Reifying the coefficient witness gives a formal zero-composite agreement without falsely reducing the raw matrix $[x]$ to zero. A relation-preservation witness for each individual arrow still has its separate role in (31.17).

Now interpret the retained calculation in a supplied coherent model M . Its whole functor H_M is the construction of Section 31.4, obtained by applying the whole cokernel functor to the boundary diagram. In particular, the two inputs relevant to the nonzero connecting map are

$$H_1(C) : \quad 0 \longrightarrow S \xrightarrow{[x]} S, \quad H_0(A) : \quad R \xrightarrow{[x]} R \longrightarrow 0.$$

The original raw arrows and chain agreement introduce each input into the native zero-diagram category. Applying H_M gives its formal homology point. Applying the Hom action of H_M to an introduced chain map gives the formal induced map. The connecting component has those whole-H points as its source and target; its family-level owner is the actual connecting transformation of Section 31.9. No second H object is chosen merely to state the component.

For this example the proof-CAS workflow first replays the whole categorical program and checks agreement with the selected result. Explicit adoption then makes the computed equations available in the proof environment. The formal constructors build the bounded raw sequence and its six interior homology/exactness pairs from the retained maps, boundary witnesses and selected universal-provider contracts.

The model-observation stage adds a different kind of binding. Write P_z for the reified selected homology presentation of an input z , and $\text{obs}(f)$ for a complete arrow observation in the existing arrow-object carrier: source, target and arrow together. The bindings have the form

$$H_M[z] = P_z, \quad \text{obs}(\partial_{1,M}) = \text{obs}(\partial_{1,\text{native}}).$$

Here the native arrow is the reified [1] between the two retained homologies. These are explicitly adopted model interpretations, not new runtime rewrites that make every occurrence of H_M call the CAS. The coherent model and its normality enhancement remain supplied inputs.

The complete consumer keeps eighteen H observations: twelve degree-wise points for the three complexes in degrees $-1, 0, 1, 2$, and six homologies of the adjacent pairs in the long exact sequence used to express interior exactness. It also retains eight degree-wise induced-map observations and three connecting observations. Only the connecting map at degree one is nonzero. All seven displayed arrows refer to these original observations. During interpretation the test forbids calls that would rerun homology, connecting or the kernel/weak-pullback algorithms; mismatched selections are rejected and existing adopted claims are reused.

The runnable entry is `polynomialFreydHomologyFixture('two')`. Its fixture, displayed-matrix regression and complete formal/model consumer are, respectively:

- `tests/v3_2_algebra_polynomial_freyd_homology_fixtures.ts`
- `tests/v3_2_algebra_polynomial_freyd_long_exact_tests.ts`
- `tests/v3_2_algebra_formal_freyd_long_exact_homology_tests.ts`

Formal status — mathematical development. This worked module calculation is exercised by the named native regression and formal/model consumer. It illustrates the checked conditional interfaces above; it is not a closed construction of the supplied model, a correctness proof for all native algorithms, or a proof of the deferred symbolic endpoint theorem. The native result retains its two actual outside-support zero homologies and their witnesses.

31.12 Boundaries Of The Integration

The generic and selected-result developments now meet at a useful computational interface, but their conclusions must be kept separate.

- Whole formal H and connecting use coherent K/Q presentations, a derived H and a whole connecting transformation. They do not construct a coherent presentation for every older universal-operation family.

- Generic exactness supplies three exact window interiors and a finite iterator retaining their arrows and evidence, not the final conventional zero-ended symbolic theorem.
- Native bounded computation supplies the complete selected sequence, interior witnesses and outside-support zero data. Its tests do not prove a theorem for every category.
- Retained proof-CAS interpretation supplies formal raw terms, selected interior exactness and model $H/\text{map}/\text{connecting}$ observations. It does not extract arbitrary quotient witnesses or construct a closed model.

The remaining endpoint issue is an interface problem, not an absent native endpoint calculation. Separate formal zero-object lemmas are available, but their final symbolic attachment to the conventional bounded display is deferred. The checked iterator and practical model interpretation do not require that deferred attachment as an input. This edition states the boundary rather than replacing the actual H endpoints by convenient zero objects.

Formal status — research boundary. The deferred claim is the final zero-ended bounded theorem with its symbolic endpoint evidence, not the checked window and finite-iterator interfaces above. Coherent model existence, arbitrary quotient effectiveness and a general normalization calculus for homological algebra remain separate obligations.

The current formulation is a constructive reference implementation with whole functors and transformations. Its algebraic reconstruction and exactness laws still use explicit paths. A later, more economical record and projection design may improve the dependent interfaces; a Došen-style normalization theorem for this whole homological layer has not been proved. Unbounded complexes, derived categories and spectral sequences are also later developments, not implicit consequences of the bounded result.

A possible later computation mode would represent a universal diagram and solve its witness equations once. To turn that calculation into a theorem about arbitrary Abelian categories, it would also need an encoding of the premise and an exact-functor interpretation. Adding zero-composition relations alone does not encode exactness hypotheses. Rational examples cannot serve as universal instances for categories with integral torsion.

Such a mode would also require its own category constructor. A one-sided Freyd presentation, an image/subquotient representation, and a free Abelian Adelman construction have different data and computational hypotheses. Likewise, a generalized inverse in a relation calculus must be shown to recover an ordinary arrow before it can replace a lift in the present interface. These are prospective comparisons with the retained construction, not completed layers of the kernel.

The relevant reference distinctions are developed in Posur's constructive methods, image completion, and free Abelian categories for theorem proving.

Formal status — research boundary. Universal-diagram solving and generalized-morphism compilation are future interfaces. Their interpretation, effective solving capabilities, and recovery of ordinary arrows are additional obligations. The current concrete Freyd provider remains an executable model with explicitly recorded trust and selection boundaries.

Appendix A. Notation

The book follows `reports/REPORT_EMDASH_V3_2_CANONICAL_SURFACE_SYNTAX_2026-06-05.md`. This appendix records the compact notation used in the mathematical line of the book. It is a reading guide, not a proposal to make every glyph parser syntax.

Book notation	Reading	Current implementation witness
$a \rightarrow_C b$	an arrow of C from a to b	<code>Hom C a b</code>
$F : A \vdash B$	a functor from A to B	<code>Functor A B</code>
$E : K \vdash \text{Cat}$	a Cat-valued directed family	<code>Catd K</code>
$E[f]$	functorial action of a family on a base arrow	<code>catd_transport_func</code>
H_x	the based hom-category $\text{Hom}_W(*, x)$	<code>Hom_cat WalkingEnd_cat walking_base x</code>
$W, *, \ell$	WalkingEnd, its base, and its directed generator	<code>WalkingEnd_cat , walking_base , walking_loop</code>
$\text{Path}(A)$	the category whose objects are elements of A and whose arrows are equality paths	<code>Path_cat A</code>
$S^1, \text{base}, \text{loop}$	the groupoidal Circle, its point, and its generating path	<code>Circle_grpd , circle_base , circle_loop</code>
$\mathbb{Z}$	the successor-localized Integer classifier	<code>Integer_grpd</code>
I, i_0, i_1, seg	the groupoidal interval, its endpoints, and generating path	<code>Interval_grpd , interval_i0 , interval_i1 , interval_seg</code>
$\text{Groupoidify}(C)$	free groupoidal realization of a directed category C	<code>Groupoidify C</code>
$\eta_C : C \rightarrow \text{Path}(\text{Groupoidify}(C))$	the whole free-inversion unit	<code>groupoidify_unit_func C</code>
Code	the Nat-valued directed family over W	<code>walking_Code_catd</code>
$\text{encode}_x(p)$	apply $\text{Code}[p]$ to zero	<code>walking_encode</code>
ℓ^n or $\text{power}(n)$	the n th generator-prefix power	<code>walking_power</code>
$\text{decode}_x(c)$	the object action of the contextual decoder	<code>walking_directed_decode_funcd</code>
ν_p	the directed normalization cell $p \rightarrow \text{decode}_x(\text{encode}_x(p))$	<code>walking_directed_normalization_cell</code>
$\simeq$	an explicitly stated equivalence interface	in Theorem 8.1, <code>walking_hom_nat_type_equiv</code>

$F[f]$	functor action on an arrow	<code>fapp1_fapp0</code> and its iterable <code>fapp1_func</code> owner
$u_*(g)$	postcomposition by u , namely $u \circ g$	<code>hom_postcomp_fapp0</code>
$u^*(h)$	precomposition by u , namely $h \circ u$	<code>hom_precomp_along_fapp0</code>
$\eta[f]$	off-diagonal action of a transfor on $f : x \rightarrow y$	<code>tapp1_fapp0</code>
$\chi_{(p,u)}^\Phi$	displayed transport-comparison component	<code>fdapp1_int_cell</code>
$\phi_{g,f}^F : F[g] \circ F[f] \Rightarrow F[g \circ f]$	the functor compositor extracted from whole laxity	<code>fapp1_compositor</code>
$\text{GrayHom}_{\text{lax}}(A, B)$	strict-functor objects with the ambient lax-arrow and higher-hom tower	<code>GrayHom_lax A B</code>
$A \otimes_R B$	the stable tensor head selected by the checked right Gray closure	<code>GrayTensor_R A B</code>
$P : A \rightsquigarrow B$	a Cat-valued profunctor on $A^{\text{op}} \times B$	<code>Prof A B</code>
U_A	the unit hom profunctor	<code>Unit_prof A</code>
$P \otimes_B Q$	selected fixed-middle profunctor tensor	<code>Prof_tensor P Q</code>
$F \dashv G$	adjunction data with selected triangle cuts	<code>Adjunction F G</code>
T, η, μ, f^*	a selected monad, its unit/multiplication, and Kleisli extension in the ambient category	<code>Monad ,</code> <code>unit_monad_transf ,</code> <code>mult_monad_transf ,</code> <code>kleisli_extend_fapp0</code>
$A \times B = P(A, B)$	a selected product object from a whole product functor	<code>BinaryProducts C P</code>
$\langle f, g \rangle$	selected triangular pairing into $P(A, B)$	<code>binary_products_pair_fapp0</code>
$K_1^a(h), K_2^a(h)$	postcomposition with the selected first or second projection	<code>binary_products_K1a_fapp0 ,</code> <code>binary_products_K2a_fapp0</code>
$!_A : A \rightarrow t$	canonical arrow to a selected terminal object	<code>terminal_arrow_fapp0</code>
C/X	the conventional internal slice over X	<code>Slice_cat C X</code>
$\Sigma_u : C/X \rightarrow C/Y$	slice postcomposition along $u : X \rightarrow Y$	<code>slice_sigma_func C X Y u</code>
$u^* : C/Y \rightarrow C/X$	chosen pullback/base-change action when the endpoints are slice categories	<code>slice_base_change_func C PB X Y u</code>
$\Pi_u : C/X \rightarrow C/Y$	chosen slice dependent product, right adjoint to base change	<code>slice_dependent_product_func C PB DP X Y u</code>

$\Sigma_u \dashv u^* \dashv \Pi_u$	the selected three-adjoint slice chain	<code>slice_base_change_adjunction , slice_dependent_product_adjunction</code>
$\text{SliceDependentProducts}(C)$	thin total pairing chosen pullbacks with their indexed dependent-product structure	<code>SliceDependentProducts C</code>
$\text{Cone}_W(F)$	the weighted-cone profunctor	<code>WeightedCone_prof F W</code>
$\text{IsWeightedLimit}(F, W, L)$	a chosen representation of weighted cones	<code>IsWeightedLimit_cov_comp F W L</code>
$\text{Cocone}_W(F)$	the opposite-dual weighted-cocone profunctor	<code>WeightedCocone_prof F W</code>
$A \star B$	directed join with left-to-right cross arrows	<code>Join_cat A B</code>
$\Delta[n]$	the join-built finite ordinal category with $n + 1$ vertices	<code>DirectedSimplex_cat n</code>
$\Delta[n]$	the standard representable semisimplex $\text{Hom}(-, n + 1)$	<code>StandardSimplex (succ n)</code>
$\text{Face}(p, n)$	injective monotone maps from the p -vertex ordinal to the n -vertex ordinal	<code>FaceCode p n</code>
$\text{Obs}(C, n)$	a dependent-simplex code paired with one object of its native decoded category	<code>DependentSimplexObservation C n</code>
$X : \mathcal{K}^{\text{op}} \rightarrow \text{Cat}$	a Cat-valued presheaf on $\mathcal{K}$	<code>Psh K</code>
yU	the representable presheaf $\text{Hom}_{\mathcal{K}}(-, U)$	<code>yoneda_psh K U</code>
p^*R	pullback of the sieve R by postcomposing its probes with p	<code>sieve_pullback K V U p R</code>
$D_U(s)$	the sieve of probes along which the restricted section is invertible	<code>comm_ring_psh_invertibility_sieve K O U s</code>
$J(U, R)$	the proposition that R covers U in a Grothendieck topology	<code>groth_topology_covers K J U R</code>
$\widehat{R}$	the whole presheaf extension of a sieve into its representable	<code>ordinary_sieve_extension_psh K U R</code>
$\rho_{R,X}$	restriction from sections over U to matching families on R	<code>ordinary_sieve_local_precomp K U R X</code>
$a_J P$ or aP	direct Cat-valued cover completion of P on the fixed site $(\mathcal{K}, J)$	<code>DirectCoverCompletionPsh K J P</code>
$\eta_P : P \rightarrow aP$	return/unit of direct cover completion	<code>direct_cover_completion_unit K J P</code>
glue_q	whole amalgamation functor for one eligible cover question	<code>direct_cover_completion_glue_func K J P U R covers</code>

CRing	the one-category of set-carrier commutative rings and structured maps	<code>CommRing_cat</code>
$\sum_i a_i f_i = 1$	a finite unimodular certificate for the generators (f_i)	<code>CommRingUnimodularPresentation R n generators</code>
$R[X]$	a supplied free commutative R -algebra on the variable classifier X	<code>CommRingPolynomialAlgebra R X</code>
$R[1/f]$	the target of a supplied universal-property localization at f	<code>comm_ring_localization_target R f localization</code>
$R[1/(fg)] \simeq R[1/f][1/g]$	whole product/iterated-localization comparison	<code>comm_ring_iterated_localization_comparison_omega_equiv</code>
$\mathrm{Sp}(R)(S)$	affine S -points, namely structured maps $R \rightarrow S$	<code>AffineSpecPoint R S</code>
$D_R(f)(S)$	affine S -points at which the image of f is a unit	<code>AffineSpecBasicOpenPoint R f S</code>
$\mathcal{O}_{\mathrm{coord}}$	the whole coordinate-ring presheaf on the big affine slice over $\mathrm{Sp}(R)$	<code>affine_spec_coordinate_psh R</code>
$J_{\mathrm{Zar}}^{\mathrm{big}}(R)$	the least topology generated by selected finite unimodular localization charts	<code>affine_spec_big_zariski_topology R</code>
$\mathcal{O} _U$	the whole ambient structure presheaf restricted along the domain functor of $\mathcal{K}/U$	<code>reflective_comm_ringed_site_slice_ambient_psh K A U</code>
$\mathrm{Scheme}_{\mathcal{K}}^{(2)}$	a binary site-relative scheme presentation over the selected categorical semantics	<code>BinarySiteRelativeSchemePresentation K</code>
$U_0 \times_X U_1$	a selected actual product of two chart objects in the conventional slice $\mathcal{K}/X$	<code>BinarySchemeChartOverlapPresentation K S</code>
$A[t, t^{-1}] \rightarrow A[u, u^{-1}]$	the canonical coordinate-inversion map between supplied one-variable localizations	<code>comm_ring_laurent_transition_map A P Q LP LQ</code>
$\mathrm{Laurent}(S, \Omega)$	a common base and Laurent presentation on the literal rings and restrictions of the actual overlap Ω	<code>BinarySchemeLaurentOverlapPresentation K S overlap</code>
$\mathrm{PLine}_{\mathrm{sup}}(\mathcal{K})$	an already-global binary scheme, its actual overlap, and its Laurent coordinate package	<code>SuppliedProjectiveLinePresentation K</code>

The homological notation in [Chapter 31](#) keeps selected objects and their structural arrows visible.

Book notation	Reading	Current implementation witness
$A \oplus B, 0$	the selected biproduct and zero object	<code>AdditiveCategory</code>
$\sum_{u:T \rightarrow Z}(k \circ u = h)$	the internal factor fibre of h through k	<code>HomPostcompFactor</code>
$\mathcal{D}_C = \mathrm{Functor}(\mathbf{2}, C)$	the category of walking-arrow diagrams	<code>Functor_cat</code> <code>WalkingArrow_cat C</code>

$J \dashv K, Q \dashv I$	coherent kernel and cokernel presentations over the original selections	<code>KernelPresentation ; CokernelPresentation</code>
$\mathcal{Z}_C = (J \downarrow \text{id}_{\mathcal{D}_C})$	the native category of one-degree zero-composition diagrams	<code>ZeroArrowCone_cat</code>
$\beta = K(h) \circ \eta_A$	the whole boundary transformation of a coherent family	<code>kernel_family_boundary_transf</code>
$H = Q \circ \text{Arr}(\beta)$	whole homology of that family	<code>homology_family_func</code>
$k_n : Z_n \rightarrow C_n$	the actual cycle kernel embedding	<code>computational_homology_cycle_embedding</code>
$b_n : C_{n+1} \rightarrow Z_n$	the lifted incoming differential	<code>computational_homology_boundary</code>
$q_n : Z_n \rightarrow H_n$	the selected cokernel of that boundary	<code>computational_homology_projection</code>
$\chi_f : \text{Coim}(f) \rightarrow \text{Im}(f)$	the canonical coimage-image comparison	<code>coimage_image_comparison</code>
∂, ∂_n	respectively the snake arrow and the homology connecting component	<code>abelian_snake_connecting ; homology_record_connecting</code>
$\partial_w : H \circ R \Rightarrow H \circ L$	the whole connecting transformation of a window family	<code>homology_window_connecting_transf</code>
$\text{obs}(f)$	a complete arrow observation, retaining source, target and arrow in the existing arrow-object carrier	<code>FreydArrowObservation ; freyd_raw_arrow_observation</code>
$\rho_Q W = F \rho_P$	relation preservation for a column-matrix raw morphism	<code>CommRingPresentationMorphism</code>
$\rho_Q H = F - G$	a coefficient witness for agreement of raw representatives	<code>CommRingPresentationMorphismAgreement</code>
$QU + FV = \text{id}$	a boundary-epicity block equation	<code>CommRingFreydEpimorphismWitness</code>

Matrix products use the same composition order as categorical arrows: the rightmost matrix acts first. A selected presentation is not identified with another merely because both represent isomorphic modules.

The bounded executable text bridge uses four intrinsic categorical lambda modes:

```

λf  x : A. ...
λn  k : K. ...
λfd a : E. ...
λnd k : K. ...

```

The superscript belongs to the lambda: it specifies ordinary functorial, natural/indexed, displayed-functorial, or displayed-natural variation. The classifier annotation after the variable may be omitted when an expected classifier supplies it bidirectionally, but the binder mode itself is not inferred from

that annotation. Thus the book's mathematical telescope declaration $k :^n K$ corresponds to binding with $\lambda^{\wedge n} k : K. \dots$; the two notations have the same mode reading without being character-for-character surface syntax. Ordinary object binding in the outer logical framework uses its ordinary dependent lambda rather than a categorical $\wedge_o$ mode.

Composition is written in diagrammatic reading order as $g \circ f$: first f , then g . For the concrete model **BNat**, this agrees with the implemented convention $g \circ f = g + f$, where Nat addition recurses in its left argument.

The two star actions are deliberately variance-separated. If $g : w \rightarrow x$ and $u : x \rightarrow y$, then $u_*(g) : w \rightarrow y$. If $u : x \rightarrow y$ and $h : y \rightarrow z$, then $u^*(h) : x \rightarrow z$. Thus the formula $f^*(g) = g \circ f$ names **precomposition** and belongs to `hom_precomp_along_*`; it is not the postcomposition owner with a typographic variation.

The same superscript star is conventional for pullback reindexing. Context disambiguates it: $u^*(h) = h \circ u$ between ordinary Hom objects means precomposition, while $u^* : C/Y \rightarrow C/X$ between slice categories is the selected base-change functor. Monad extension $f^* : TX \rightarrow TY$ is a third typed use, always accompanied by a monad endofunctor T .

These formulas are categorical, not specifically functor-categorical. Their general reading takes w, x, y, z to be objects and the displayed arrows to be arrows of an arbitrary ambient category K . The specialization $K = \mathbf{Cat}$ makes those objects categories and those arrows functors; that is the currently checked product/projection instance used in Chapter 9.

$\mathbf{Path}(A)$ denotes the equality-path category on a carrier A . It must not be confused with a directed hom-category. In particular, the objects of H_x are arrows of W , whereas arrows of H_x are directed 2-cells between those arrows.

Formal status — mathematical development with a checked executable subset. The four categorical binder spellings, neutral application, and selected structural forms are implemented only for the reviewed profiles. The rest of the appendix remains mathematical notation, not an assertion that the complete book grammar is parsed.

Appendix B. Emdash Evidence

The prose cites stable evidence IDs rather than brittle line numbers. The register behind this appendix names a claim's status, its active owner, and the diagnostic or reviewer surface that exercises it. During assembly, the table below is generated directly from `book/evidence.json` ; editing the table here is therefore neither necessary nor possible.

An **owner** is the declaration that supplies the interface used by the book. A **reviewer/check** is a separate executable occurrence or deliberately searched check phrase. “Formal consequence” rows name checked premises but also say that the displayed theorem has not yet been packaged. “Research boundary” rows deliberately have no purported proof owner.

Here **MathOps** means the discipline of keeping formal owners, executable reviews, exposition, and published artifacts connected without confusing their authority. The chain is intentionally one-way:

```
owner -> reviewer -> register -> prose -> artifact
```

Later stages may detect drift or preserve provenance, but they do not become new proof authorities. In particular, a reproducible PDF certifies the book artifact, not the mathematics printed inside it.

Chapter 31 uses two complementary evidence routes. The register records `Lambdapi` declarations and focused formal reviewers, including the generic snake and homology-window theorems, whole `H/`connecting interfaces, finite iteration and transparent matrix introductions. Native computations, categorical-program comparisons and field differentials have their own TypeScript reviewers, named in the prose. A test of a polynomial example is not entered as a generic categorical theorem. Likewise, a formal term built after explicit provider adoption is checked relative to those assumptions; the register does not turn that trust decision into a proof of the native algorithm's universal correctness. The deferred symbolic endpoint-proof attachment is not entered as a checked consequence of the finite iterator or native zero-endpoint tests.

Formal status — checked. This appendix describes traceability; the accompanying emdash artifact remains the proof authority. The evidence checker validates every row before this generated view is assembled.

Evidence	Status	Claim	Owners	Reviewer/check evidence
TT-EQUALITY-INDUCTION	checked	The equality-local foundation has reflexivity, right-based dependent equality induction, nondependent path action, and dependent path action.	eq_refl emdash3_2.lp ind_eqr emdash3_2.lp eq_ap emdash3_2.lp eq_apd emdash3_2.lp	eq_apd examples/path_category.lp

TT-ELEMENTARY-INDUCTION	checked	Empty, Unit, Bool, and Nat have decoded classifiers, and the nontrivial elementary inductive carriers have dependent eliminators with constructor computation.	Empty_grpd emdash3_2.lp Unit_grpd emdash3_2.lp Bool_grpd emdash3_2.lp Nat_grpd emdash3_2.lp empty_elim emdash3_2.lp bool_elim emdash3_2.lp nat_elim emdash3_2.lp	nat_elim emdash3_2_checks.lp
TT-SIGMA-PI-PATHS	checked	The groupoid layer has dependent-pair and dependent-function classifiers, observational Sigma paths, and a checked happly/funext equivalence for Pi paths.	sigma_Fst emdash3_2.lp sigma_Snd emdash3_2.lp PiHapply emdash3_2.lp PiFunext emdash3_2.lp pi_happly_type equiv emdash3_2.lp	PiFunext examples/pi_funext.lp
CAT-ITERATED-HOMS	checked	A category has an object classifier and category-valued homs, so higher cells are represented by iterating Hom.	Cat emdash3_2.lp Obj emdash3_2.lp Hom emdash3_2.lp	Hom emdash3_2_checks.lp
CAT-PATH-CATEGORY	checked	Path(A) internalizes equality as a groupoidal category, and an ordinary function induces an iterable functor between path categories.	Path_cat emdash3_2.lp Path_cat_func emdash3_2.lp path_map_func emdash3_2.lp	path_map_func examples/path_category.lp
CAT-FUNCTOR-CALCULUS	checked	Ordinary functors expose object and iterated-hom action, with generic identity and composition computation rather than constructor-specific functor laws.	Functor emdash3_2.lp fapp0 emdash3_2.lp fapp1_func emdash3_2.lp fapp1_fapp0 emdash3_2.lp id_func emdash3_2.lp comp_cat_fapp0 emdash3_2.lp	fapp1_fapp0 emdash3_2_checks.lp

CAT-PRODUCT-CALCULUS	checked	Binary product categories have projection functors and a stable two-sided product-map constructor whose object, arrow, and projection observations compute componentwise.	Product_cat emdash3_2.lp Product_projL_ func emdash3_2.lp Product_map_fu nc emdash3_2.lp	text product/projec tion cut used by the book emdash3_2_chec ks.lp text General two-sided product maps used by pro- functor rein- dexing emdash3_2_chec ks.lp
TRIANGULAR-BINARY-PRODUCTS	checked	A selected whole product functor $P:C \times C \rightarrow C$ exposes whole projection transfors, represented-family pairing, stable K1a/K2a and pairing heads, run-time beta/distribution/eta computation, and transparent whole unpair action without duplicating the ambient product-category calculus.	BinaryProduct s emdash3_2_tria ngular_binary_ products.lp binary_product s_proj1_transf emdash3_2_tria ngular_binary_ products.lp binary_product s_proj2_transf emdash3_2_tria ngular_binary_ products.lp binary_product s_pair_transf emdash3_2_tria ngular_binary_ products.lp binary_product s_K1a_fapp0 emdash3_2_tria ngular_binary_ products.lp binary_product s_K2a_fapp0 emdash3_2_tria ngular_binary_ products.lp	binary_product s_pair_fapp0 examples/trian gular_binary_p roducts.lp binary_product s_map_fapp0 examples/trian gular_binary_p roducts.lp

TERMINAL - OBJECT - COMPUTATION	checked	A selected terminal object exposes one whole transformation from the identity functor to the constant terminal functor, computing components and off-diagonal action; every $\text{Hom}_C(A,t)$ is contractible at the canonical arrow, yielding arbitrary-arrow uniqueness without a variable-headed runtime rule.	TerminalObject emdash3_2_terminal_objects.lp terminal_arrow_transf emdash3_2_terminal_objects.lp terminal_hom_contr emdash3_2_terminal_objects.lp terminal_unique_path emdash3_2_terminal_objects.lp	terminal_arrow_transf examples/terminal_objects.lp terminal_unique_path examples/terminal_objects.lp
PULLBACK - SLICE - BASE - CHANGE	checked	Chosen pullbacks form one whole opposite-variance exact-slice family $u:C/Y \rightarrow C/X$ with $\Sigma_u \dashv u$, actual whole unit/counit transfor, full Došen rectangles, proof-time whole/point semantic comparison, guarded whole mate cancellation, and pullback object/projection/square observations derived from $u^*(g)$ and the counit rather than from a cone record.	PullbackStructure emdash3_2_pullbacks.lp SliceBaseChange_catd emdash3_2_pullbacks.lp slice_base_change_adjunction emdash3_2_pullbacks.lp slice_base_change_gamma emdash3_2_pullbacks.lp slice_base_change_transpose_func_semantic_path emdash3_2_pullbacks.lp pullback_square_readable_cell emdash3_2_pullbacks.lp	slice_base_change_gamma examples/pullbacks.lp pullback_lift_slice examples/pullbacks.lp

SLICE-DEPENDENT-PRODUCTS	checked	A selected coherent dependent-product structure supplies one whole covariant exact-slice family $\Pi_u:C/X\rightarrow C/Y$ and the existing adjunction $u^*\dashv\Pi_u$; its actual unit/counit tapp1 observations, both full Došen rectangles, component triangles, transparent whole mate action, generic Hom comparison, and thin total SliceDependentProducts package are active.	DependentProductStructure emdash3_2_slice_dependent_products.lp SliceDependentProduct_catd emdash3_2_slice_dependent_products.lp slice_dependent_product_adjunction emdash3_2_slice_dependent_products.lp slice_dependent_product_gamma emdash3_2_slice_dependent_products.lp slice_dependent_product_hom_prof_comparison emdash3_2_slice_dependent_products.lp SliceDependentProducts emdash3_2_slice_dependent_products.lp	slice_dependent_product_gamma examples/slice_dependent_products.lp slice_dependent_product_hom_prof_comparison examples/slice_dependent_products.lp
CUT-PRODUCT-PROJECTION	formal-consequence	In the Cat-specialized case, for $h:A_0\rightarrow A_1$, $k:B_0\rightarrow B_1$, and $g:A_1\rightarrow C$, the readable equation $\pi_1^{(g)} \text{ composed with } h \times k \text{ equals } \pi_1'^{(g)}$ (g composed with h); its owner-aligned projection and nested-precomposition forms are checked, while the literal raw projection-composite equality is not packaged.	hom_precomp_along_fapp0 emdash3_2.lp Product_projL_func emdash3_2.lp Product_map_functor emdash3_2.lp comp_assoc emdash3_2.lp	text product/projection cut used by the book emdash3_2_checks.lp

CAT-HOM-CUTS	checked	Represented-hom postcomposition and precomposition have distinct stable full and capped owners with identity, consecutive-action, adjacent-cut, and proof-time ordinary-composition comparisons.	hom_postcomp_f unc emdash3_2.lp hom_postcomp_f app0 emdash3_2.lp hom_precomp_al ong_func emdash3_2.lp hom_precomp_al ong_fapp0 emdash3_2.lp	text Hom-action functoriality joins for postcomposition emdash3_2_checks.lp text Hom-action functoriality joins for precomposition emdash3_2_checks.lp
CAT-TRANSFOR-CALCULUS	checked	Transformations form the next hom between functors and expose point components together with higher naturality action.	Transf emdash3_2.lp tapp0_fapp0 emdash3_2.lp tapp1_fapp0 emdash3_2.lp	tapp1_fapp0 emdash3_2_checks.lp
CAT-DIRECTED-FAMILIES	checked	A Cat-valued directed family has fibres and functorial reindexing, while a family morphism has fibre functors and directed comparison cells.	Catd emdash3_2.lp Fibre_cat emdash3_2.lp catd_transport _func emdash3_2.lp Functord emdash3_2.lp fdapp1_int_cell emdash3_2.lp	fdapp1_int_cell examples/dependent_hom_laxity.lp
CAT-SIGMA-PI	checked	Directed families have a Sigma total category, canonical transport arrows, and a Pi section category with coherent evaluation.	Sigma_cat emdash3_2.lp sigma_transport_arrow emdash3_2.lp Pi_cat emdash3_2.lp piapp0 emdash3_2.lp	sigma_transport_arrow emdash3_2_checks.lp

CAT - FIBREWISE - CONTEXT	checked	Fixed-base fibrewise products of displayed families have displayed projections and pairing whose fibre, base-arrow, internalized-cell, higher, and projection-after-pairing observations compute componentwise; swap and diagonal are derived from those owners.	Product_projL_ funcd emdash3_2.lp Product_projR_ funcd emdash3_2.lp Product_pair_f uncd emdash3_2.lp	Product_projL_ funcd_higher_c heck emdash3_2_chec ks.lp Product_projR_ funcd_higher_c heck emdash3_2_chec ks.lp Product_pair_f uncd_higher_ch eck emdash3_2_chec ks.lp text The canonical internalized cell of displayed pairing is componentwise. emdash3_2_chec ks.lp text Whole displayed universal-property betas. emdash3_2_chec ks.lp
CAT - BASE - CHANGE - TOTALIZATION	checked	Pullback totalization sends a base-changed dependent pair (a, u) to $(F[a], u)$ and sends its total arrow by the functorial base action while retaining the fibre component.	sigma_pullback _total_func emdash3_2.lp	text Pullback totalization exposes exactly its base-changed dependent pair. emdash3_2_chec ks.lp

CAT-DISPLAYED-EVALUATION	checked	Constant-domain displayed evaluation projects fibrewise to ordinary functor evaluation and retains generic base/higher action, while displayed terminal weakening projects to the ordinary terminal functor.	Eval_funcd emdash3_2.lp Terminal_funcd emdash3_2.lp	text The stable displayed evaluator projects to ordinary product evaluation. emdash3_2_checks.lp Displayed_eval_higher_checks.lp text Displayed terminal weakening projects to the ordinary unique functor. emdash3_2_checks.lp
CAT-REPRESENTABLE	checked	The fixed-source representable family has fibre $\text{Hom}(x,y)$, and its source action is precomposition.	Rep_catd emdash3_2.lp Rep_transport_func emdash3_2.lp	Rep_catd examples/path_induction_transitivity.lp
LOGIC-TRUNCATION-PREDICATE	checked	Contractibility and recursive truncation evidence define the proposition, set, and groupoid levels as properties of existing classifiers.	IsContr emdash3_2.lp IsTruncGrpd emdash3_2.lp IsPropGrpd emdash3_2.lp IsSetGrpd emdash3_2.lp	IsTruncGrpd examples/truncation_monotonicity.lp
LOGIC-NAT-SETHOOD	checked	Unit and Empty are proposition-valued and Nat is set-valued by explicit internal evidence.	unit_is_prop emdash3_2_nat_arithmetic.lp empty_is_prop emdash3_2_nat_arithmetic.lp nat_is_set emdash3_2_nat_arithmetic.lp	nat_is_set examples/walking_endomorphism_nat_prerequisites.lp
LOGIC-TRUNCATION-EVIDENCE-PROP	checked	For every truncation level and classifier, the classifier of truncation evidence is proposition-valued.	is_trunc_grpd_evidence_is_prop emdash3_2.lp	is_trunc_grpd_evidence_is_prop examples/truncation_evidence_property.lp

EQUIV - TYPE	checked	TypeEquiv packages a forward map with contractible homotopy fibres and exposes a selected inverse with both inverse paths.	HFiber emdash3_2.lp IsEquivMap emdash3_2.lp TypeEquiv emdash3_2.lp type_equiv_fro m emdash3_2.lp type_equiv_left emdash3_2.lp type_equiv_right emdash3_2.lp	TypeEquiv examples/type_ equiv_algebra. lp
EQUIV - TYPE - ALGEBRA	checked	Type equivalences have reflexivity, symmetry, and categorical-order composition with checked forward-map computation.	type_equiv_refl emdash3_2.lp type_equiv_sym emdash3_2.lp type_equiv_com p emdash3_2.lp	type_equiv_com p examples/type_ equiv_algebra. lp
UNIV - GROUPOID	checked	The groupoid universe has a decoder-oriented univalence equivalence between universe paths and TypeEquiv, with named round trips and transport agreement.	grp_d_univalence_type_equiv emdash3_2.lp grp_d_equiv_path emdash3_2.lp grp_d_equiv_path_idtoequiv emdash3_2.lp idtoequiv_grp_d_equiv_path emdash3_2.lp	grp_d_univalence_from_decoder examples/grp_d_univalence_decoder.lp
UNIV - TRUNCATED	checked	For packaged n-truncated classifiers, package equality is equivalent to TypeEquiv of the retained carriers, with named encode/decode round trips.	trunc_grp_d_univalence_type_equiv emdash3_2.lp trunc_grp_d_idtoequiv emdash3_2.lp trunc_grp_d_equiv_path emdash3_2.lp	trunc_grp_d_univalence_type_equiv examples/truncation_universe_univalence.lp

EQUIV-OMEGA	checked	OmegaEquivAlong carries equality-valued inverse-arrow data, OmegaEquiv packages a selected arrow, and an object path has a transparent computational equivalence package.	OmegaEquivAlong emdash3_2.lp OmegaEquiv emdash3_2.lp object_path_equiv emdash3_2.lp	object_path_equiv examples/equality_valued_omega_equivalence.lp
EQUIV-EVIDENCE-PROP	checked	For every fixed arrow, its equality-valued OmegaEquivAlong evidence is proposition-valued without a finite-dimensional hypothesis.	omega_equiv_along_evidence_is_prop emdash3_2_eq1_evidence_property.lp	omega_equiv_along_evidence_is_prop examples/equality_valued_omega_equivalence_evidence_property.lp
EQUIV-HOM-ACTION	checked	Equality-valued equivalence evidence has a one-way next-hom action, and coherent groupoidality makes directed fibre transport an equivalence.	omega_equiv_along_fapp1 emdash3_2_eq1_hom_action.lp groupoidal_fibre_transport_equiv emdash3_2_eq1_hom_action.lp	omega_equiv_along_fapp1 examples/equality_valued_omega_equivalence_hom_action.lp
EQUIV-ORDINARY-ISO-LIFT	checked	Ordinary categorical isomorphism evidence has a one-way lift to the native equality-valued OmegaEquiv package.	IsoEvidence emdash3_2.lp iso_evidence_omega_equiv emdash3_2.lp	iso_evidence_omega_equiv emdash3_2_checks.lp
UNIV-FULL-OBJECT-ISO	research-boundary	A full native equivalence between arbitrary categorical object equality and ordinary isomorphism evidence is not part of the selected API.	—	—
IND-PATHOUT	checked	PathOut_Z(x) is the Sigma total of the fixed-source representable, with a reflexive object and a canonical rho arrow to every outgoing arrow.	PathOut_cat emdash3_2.lp pathout_refl_object emdash3_2.lp pathout_refl_arrow emdash3_2.lp	pathout_refl_arrow emdash3_2_checks.lp
IND-ARROW	checked	Fixed-source arrow induction extends data at the reflexive outgoing arrow to a section, and the theorem is internalized functorially as fixed-source and varying-source interfaces.	path_ind_section emdash3_2.lp PathInd_function emdash3_2.lp PathInd_transfined emdash3_2.lp	path_ind_section examples/path_induction_transitivity.lp

IND-COMPOSITION	checked	Applying arrow induction to the represented composition motive yields a functor whose object action computes to categorical composition.	CompMotive_cat d emdash3_2.lp path_comp_sec emdash3_2.lp path_comp_fun c emdash3_2.lp	path_comp_fun c examples/path_induction_transitivity.lp
IND-PATH-COMPARISON	checked	For a literal path-category base, structured directed transport agrees propositionally with primitive right-based equality induction.	path_cat_structured_transport_agrees_ind_eqr emdash3_2.lp	path_cat_structured_transport_agrees_ind_eqr examples/groupoidal_structured_j_eq1.lp
IND-GENERAL-INITIALITY	research-boundary	A general theorem identifying all selected categorical induction interfaces with homotopy-initial algebras has not been packaged.	—	—
DHIT-DERIVED-ELIMINATORS	checked	The contextual WalkingEnd eliminator specializes to a dependent section and an ordinary recursor with base and generator observations.	walking_end_ind_sec emdash3_2_walking_end_hit.lp walking_end_rec_func emdash3_2_walking_end_hit.lp walking_end_rec_beta_base emdash3_2_walking_end_hit.lp walking_end_rec_beta_loop_ordinary emdash3_2_walking_end_hit.lp	walking_end_rec_func examples/walking_endomorphism_hit.lp
DHIT-GENERAL-SCHEMA	research-boundary	The selected WalkingEnd interface does not yet constitute a reusable schema for arbitrary directed higher-inductive categories, pushouts, or cell complexes.	—	—

TRUNC - CLOSURE	checked	Recursive truncation evidence is monotone and has checked dependent Pi and same-level dependent Sigma closure operations.	is_trunc_grpd_succ emdash3_2.lp is_trunc_pi emdash3_2.lp is_trunc_sigma a emdash3_2.lp	is_trunc_pi examples/truncation_pi_closure.lp is_trunc_sigma a examples/truncation_sigma_closure.lp
TRUNC - RETRACT	checked	Every recursive truncation level is closed under explicit retractions.	is_trunc_retract emdash3_2_eq1_evidence_property.lp	is_trunc_retract emdash3_2_checks.lp
CAT - DIMENSION	checked	IsNCat defines finite directed height by hom recursion, and every such category has the recursively predicted object-truncation evidence.	IsNCat emdash3_2.lp cat_dim_trunc_level emdash3_2.lp ncat_obj_trunc c emdash3_2_eq1_evidence_property.lp	IsNCat examples/directed_dimension.lp ncat_obj_trunc c examples/equality_valued_omega_equivalence_evidence_property.lp
TRUNC - REFLECTOR	checked	The active groupoidal truncation layer has a classified NType_cat(n) target, a primitive Trunc_ntype(n,A) code with decoded carrier Trunc_grpd(n,A), restricted point-computing induction into classified n-types, and a recursor-derived whole map action with identity, composition, and retained Path action.	NType_cat emdash3_2_truncation_reflector.lp Trunc_ntype emdash3_2_truncation_reflector.lp trunc_ind emdash3_2_truncation_reflector.lp trunc_map_function emdash3_2_truncation_reflector.lp	Trunc_ntype examples/computational_truncation_facade.lp p trunc_ind examples/computational_truncation_facade.lp p trunc_map_function examples/computational_truncation_facade.lp

WE - SIGNATURE	checked	WalkingEnd has an opaque category, base, directed loop, and explicit one-dimensional evidence.	WalkingEnd_cat emdash3_2_walking_end_hit.lp walking_base emdash3_2_walking_end_hit.lp walking_loop emdash3_2_walking_end_hit.lp walking_end_is_one_cat emdash3_2_walking_end_hit.lp	walking_end_is_one_cat examples/walking_endomorphism_hit.lp
WE - ONE - DIMENSIONAL	checked	Every WalkingEnd hom-category is discrete, without WalkingEnd itself becoming discrete.	walking_end_hom_discrete emdash3_2_walking_end_hit.lp	walking_end_hom_discrete emdash3_2_checks.lp
WE - CONTEXTUAL - ELIMINATOR	checked	The contextual WalkingEnd eliminator has base and loop computation at the selected displayed observers.	walking_end_in_d_funcd emdash3_2_walking_end_hit.lp walking_end_in_d_funcd_beta_base emdash3_2_walking_end_hit.lp walking_end_in_d_funcd_beta_loop emdash3_2_walking_end_hit.lp	walking_end_in_d_funcd examples/walking_endomorphism_hit.lp
WE - BNAT - MODEL	checked	BNat is a separate one-object Nat-monoid model and receives a functor from WalkingEnd.	BNat_cat emdash3_2_walking_end_hit.lp bnat_comp_nat_add emdash3_2_walking_end_hit.lp walking_bnat_model_func emdash3_2_walking_end_hit.lp	walking_bnat_model_func examples/walking_endomorphism_hit.lp

WE - CODE	checked	Code sends the walking base to Path(Nat) and the walking loop to the Nat successor functor.	walking_Code_c atd emdash3_2_walking_end_hit.lp walking_Code_beta_base emdash3_2_walking_end_hit.lp walking_Code_beta_loop emdash3_2_walking_end_hit.lp	walking_Code_c atd examples/walking_endomorphism_hit.lp
WE - ENCODE	checked	Encode acts with Code on a based arrow and evaluates at zero.	walking_encode emdash3_2_walking_end_hit.lp	walking_encode examples/walking_endomorphism_hit.lp
WE - POWER	checked	Natural powers map zero to identity and successor to generator-prefix composition.	walking_power emdash3_2_walking_end_hit.lp walking_power_func emdash3_2_walking_end_hit.lp	walking_power examples/walking_endomorphism_hit.lp
WE - SPIRAL	checked	The selected explicit-kappa directed spiral supplies the loop coherence for powers.	walking_power_spiral emdash3_2_walking_end_hit.lp walking_power_spiral_cell emdash3_2_walking_end_hit.lp	walking_power_spiral_cell examples/walking_endomorphism_hit.lp
WE - CONTEXTUAL - DECODER	checked	The contextual decoder maps Code to the based representable and computes to powers at the base.	walking_directed_decode_funcd emdash3_2_walking_end_hit.lp walking_directed_decode_beta_base emdash3_2_walking_end_hit.lp	walking_directed_decode_funcd examples/walking_endomorphism_hit.lp
WE - NORMALIZATION - CELL	checked	Every based arrow has a directed normalization cell toward decode(encode(p)).	walking_directed_normalization_cell emdash3_2_walking_end_hit.lp	walking_directed_normalization_cell examples/walking_endomorphism_hit.lp

WE - NORMALIZATION - PATH	checked	Hom-discreteness converts the directed normalization cell into equality.	walking_direct ed_normalizati on_path emdash3_2_walk ing_end_hit.lp	walking_direct ed_normalizati on_path emdash3_2_chec ks.lp
WE - POWER - ENCODE	checked	For a based endomor- phism p , $\text{power}(\text{encode}(p))$ equals p .	walking_power_ encode_roundtr ip emdash3_2_walk ing_end_hit.lp	walking_power_ encode_roundtr ip examples/walki ng_endomorphis m_hit.lp
WE - ENCODE - POWER	checked	For every natural n , $\text{encode}(\text{power}(n))$ equals n .	walking_encode_ power_roundtr ip emdash3_2_walk ing_end_hit.lp	walking_encode_ power_roundtr ip examples/walki ng_endomorphis m_hit.lp
WE - HOM - NAT - CARRIER	checked	The underlying based- endomorphism car- rier is equivalent to Nat .	walking_hom_na t_type_equiv emdash3_2_walk ing_end_hit.lp walking_hom_na t_omega_equiv emdash3_2_walk ing_end_hit.lp	walking_hom_na t_type_equiv examples/walki ng_endomorphis m_hit.lp
WE - HOM - SETHOOD	checked	The based-endomor- phism carrier is a set both from dimension and from the Nat equivalence.	walking_end_ho m_is_set_from_ dimension emdash3_2_walk ing_end_hit.lp walking_end_ho m_is_set_from_ equiv emdash3_2_walk ing_end_hit.lp	walking_end_ho m_is_set_from_ dimension examples/walki ng_endomorphis m_hit.lp
WE - LOOP - NOT - IDENTITY	checked	The walking generator is not equal to the identity.	walking_loop_n ot_identity emdash3_2_walk ing_end_hit.lp	walking_loop_n ot_identity emdash3_2_chec ks.lp
WE - LOOP - NO - RIGHT - INVERSE	checked	The walking generator has no right inverse.	walking_loop_n o_right_invers e emdash3_2_walk ing_end_hit.lp	walking_loop_n o_right_invers e emdash3_2_chec ks.lp
WE - LOOP - NONINVERTIBLE	checked	The walking generator carries no native ω -equivalence evidence.	walking_loop_n ot_omega_equiv emdash3_2_walk ing_end_hit.lp	walking_loop_n ot_omega_equiv examples/walki ng_endomorphis m_hit.lp

WE - ENCODE - PREFIX	checked	Encoding a generator-prefixed based endomorphism produces the successor of its code.	walking_encode_loop_prefix_path emdash3_2_walking_end_hit.lp	walking_encode_loop_prefix_path examples/walking_endomorphism_hit.lp
WE - STRUCTURED - ENCODER	checked	The based encoder is also packaged as a functor from the based hom-category to Path(Nat).	walking_encode_func emdash3_2_walking_end_hit.lp	walking_encode_func examples/walking_endomorphism_hit.lp
WE - COMPOSITION - ADDITION	formal-consequence	Compatibility of based composition with Nat addition follows from the checked power recursion, inverse laws, and ordinary category/Nat induction, but is not packaged as a monoid equivalence.	walking_encode_loop_prefix_path emdash3_2_walking_end_hit.lp walking_power_encode_roundtrip emdash3_2_walking_end_hit.lp walking_encode_power_roundtrip emdash3_2_walking_end_hit.lp	—
WE - FULL - CATEGORICAL - COMPARISON	research-boundary	A reverse BNat functor, full hom-category equivalence, and functor-category initiality require additional reusable infrastructure.	—	—
WE - GROUP - COMPLETION	checked	The concrete WalkingEnd-to-Circle functor sends every directed natural power to the corresponding nonnegative Circle power, and restriction along it is a whole fixed-forward OmegaEquivAlong from Circle maps into every groupoid G to path-valued WalkingEnd functors; source-functorial generic adjunction packaging remains separate.	walking_to_circle_func emdash3_2_walking_circle_completion.lp walking_to_circle_power emdash3_2_walking_circle_completion.lp walking_circle_groupoidification_hom_omega emdash3_2_walking_circle_universality.lp	walking_to_circle_power examples/walking_circle_completion.lp walking_circle_groupoidification_hom_omega examples/walking_circle_groupoidification_universality.lp

TRANSF - POINT - OFFDIAGONAL	checked	An ordinary transfor has point components and an iterable off-diagonal hom action from $F(x)$ to $G(y)$ along every source arrow x to y .	Transf_cat emdash3_2.lp tapp0_fapp0 emdash3_2.lp tapp1_func emdash3_2.lp tapp1_fapp0 emdash3_2.lp	tapp1_at_transf emdash3_2_checks.lp
TRANSF - STRICT - NATURALITY	checked	Postcomposition and precomposition adjacent to an ordinary transfor's off-diagonal action reduce to the action on the corresponding composite source arrow.	text Full strict naturality for ordinary transforsemdash3_2.lp	text Full strict naturality: post/left accumulation before cappingemdash3_2_checks.lp
TRANSF - HORIZONTAL - CALCULUS	checked	The product-composition action supplies an iterable horizontal composite of a pair of ordinary transforse, with checked point, full off-diagonal, and capped off-diagonal projections.	comp_prod_fapp1_fapp0 emdash3_2.lp	comp_prod_fapp1_fapp0 emdash3_2_checks.lp
TRANSFD - FIBRE - COMPONENTS	checked	A natural family transformation between displayed functors has a transformation in every fibre and a point component at every fibre object, with identity and vertical composition inherited from the generic transfor calculus.	Transfd_cat emdash3_2.lp Fibre_transf emdash3_2.lp Fibre_transf_app emdash3_2.lp	Fibre_transf_app emdash3_2_checks.lp
FUNCTORD - DISPLAYED - LAXITY	checked	For a natural family morphism and a base arrow, the internal displayed hom action supplies a directed component from target transport after the source fibre functor to the target fibre functor after source transport.	functord_transport_lhs_func emdash3_2.lp functord_transport_rhs_func emdash3_2.lp fdapp1_int_cell emdash3_2.lp	fdapp1_int_cell examples/dependent_hom_laxity.lp
FUNCTORD - SIGMA - ACTION	checked	The Sigma-total map induced by a natural family morphism sends a total arrow to the same base arrow paired with the capped internal displayed hom action in the fibre.	sigma_map_func emdash3_2.lp fdapp1_int_hom_fapp0 emdash3_2.lp	sigma_map_transf examples/sigma_total.lp

FUNCTORD- WHOLE-LAXITY	checked	The active internal displayed action supplies a whole laxity transformation, with ordinary post/left and pre/right whole surfaces obtained as transparent specializations; their capped cells return the existing <code>fdapp1_int_cell</code> witness, and the ordinary functor compositor is the identity-transfor specialization.	<code>functord_laxity_transf</code> <code>emdash3_2.lp</code> <code>tapp1_post_laxity_transf</code> <code>emdash3_2.lp</code> <code>tapp1_pre_laxity_transf</code> <code>emdash3_2.lp</code> <code>fapp1_compositor</code> <code>emdash3_2.lp</code>	text Ordinary post/pre laxity and functor compositor <code>emdash3_2_checks.lp</code> <code>fapp1_compositor</code> or <code>examples/dependent_hom_laxity.lp</code>
PROF-CATEGORY	checked	A Cat-valued profunctor from A to B is a directed family on A opposite times B, and vertical profunctor maps are natural family morphisms with the existing identity and composition calculus.	<code>Prof_base</code> <code>emdash3_2.lp</code> <code>Prof_cat</code> <code>emdash3_2.lp</code> <code>ProfMap</code> <code>emdash3_2.lp</code>	text Cat-valued profunctor facade and representable hom action <code>emdash3_2_checks.lp</code>
PROF- REPRESENTABLE	checked	The unit profunctor evaluates to the ambient hom, and reindexing it along endpoint functors gives the binary representable with action by precomposition and postcomposition.	<code>Unit_prof</code> <code>emdash3_2.lp</code> <code>Hom_prof_along</code> <code>emdash3_2.lp</code> <code>Hom_fapp0</code> <code>emdash3_2.lp</code>	<code>Hom_prof_along</code> <code>emdash3_2_checks.lp</code>
PROF- REINDEXING	checked	Profunctors reindex functorially in both endpoints by pullback along an opposite-times-covariant product map, including identity, nesting, and representable accumulation laws.	<code>Prof_reindex</code> <code>emdash3_2.lp</code> <code>Prof_reindex_unc</code> <code>emdash3_2.lp</code>	text Stable profunctor reindexing and its representable fold <code>emdash3_2_checks.lp</code>
PROF- REPRESENTATION -PACKAGE	checked	Representability of a profunctor is expressible as ordinary isomorphism to a conjoint, and a representation package retains a chosen representing functor with that evidence.	<code>Conjoint_prof</code> <code>emdash3_2.lp</code> <code>IsRepresentedBy_iso</code> <code>emdash3_2.lp</code> <code>Representation_iso</code> <code>emdash3_2.lp</code>	<code>Representation_iso</code> <code>emdash3_2_checks.lp</code>

PROF - COMPARISON - BETA-ETA	checked	A computational profunctor comparison exposes mutually inverse push and pull operations on every incoming profunctor map, with runtime beta and eta reductions and functorial reindexing and composition.	ProfComparison emdash3_2.lp prof_comparison_push emdash3_2.lp prof_comparison_pull emdash3_2.lp	text Computational profunctor comparisons and weighted representability emdash3_2_checks.lp prof_comparison_pull emdash3_2_checks.lp
PROF - SHAPED - CELLS	checked	A profunctor cell over endpoint functors is a natural family morphism into a reindexed target; unit-source specialization and narrow cell application provide shaped elements and their action.	Prof_transf_cell emdash3_2.lp Prof_hom emdash3_2.lp Prof_cell_application emdash3_2.lp	text Shaped profunctor cells and elements emdash3_2_checks.lp
PROF - TENSOR	checked	The selected profunctor tensor is an opaque fixed-middle composite with exposed-endpoint reindexing and a fixed-endpoint bifunctorial map action.	Prof_tensor emdash3_2.lp Prof_tensor_functor emdash3_2.lp Prof_tensor_hom emdash3_2.lp	text Primitive profunctor tensor and exposed-endpoint reindexing emdash3_2_checks.lp Prof_tensor_examples/profunctor_weighted_limits.lp
PROF - CLOSED - CALCULUS	checked	The two selected profunctor residuals expose fixed-endpoint evaluation and lambda operations inverse by runtime beta and eta, without asserting semantic end formulas for their opaque objects.	Prof_imply_con emdash3_2.lp Prof_eval_con emdash3_2.lp Prof_lambda_con emdash3_2.lp Prof_imply_con emdash3_2.lp Prof_eval_con emdash3_2.lp Prof_lambda_con emdash3_2.lp	Prof_lambda_con v_map emdash3_2_checks.lp Prof_lambda_con n_map emdash3_2_checks.lp Prof_lambda_con v_map examples/profunctor_weighted_limits.lp

PROF - COYONEDA	checked	Tensoring on either side with the unit profunctor has a natural co-Yoneda map to the original profunctor, with fixed shaped-element beta and naturality-fusion computations.	Prof_coyoneda_cov_transf emdash3_2.lp Prof_coyoneda_con_transf emdash3_2.lp Prof_coyoneda_cov_map emdash3_2.lp Prof_coyoneda_con_map emdash3_2.lp	Prof_coyoneda_cov_transf emdash3_2_checks.lp Prof_coyoneda_con_transf emdash3_2_checks.lp
PROF - GENERAL - COEND	research-boundary	The selected opaque tensor and co-Yoneda computations do not yet provide a general coend or coinserter construction, tensor associativity package, or a bicategory of profunctors.	—	—
YONEDA - FULLY - FAITHFUL	research-boundary	The active representable and co-Yoneda interfaces do not yet package a general fully faithful Yoneda embedding or the full Yoneda equivalence for arbitrary Cat-valued presheaves.	—	—
UCAT - IDENTITY - ISOMORPHISM	mathematical-development	In the ordinary set-valued-hom specialization, a category is a precategory whose identity-to-isomorphism map is an equivalence; its object type is consequently a 1-type.	—	—
UCAT - FUNCTOR - CATEGORY	mathematical-development	An ordinary natural transformation is invertible exactly when its components are, and for a precategory A and univalent category B the functor precategory from A to B is univalent, with functor identity corresponding to natural isomorphism.	—	—

UCAT - ADJOINT - REPRESENTABILITY	mathematical-development	In ordinary univalent 1-category theory, right adjoints are equivalently coherent representations of the contravariant hom functors obtained by fixing the target of the left adjoint.	—	—
UCAT - ADJOINT - UNIQUENESS	mathematical-development	For an ordinary functor whose source is a univalent category, right-adjoint structures are unique in the appropriate identity sense and existence of a right adjoint is a mere proposition.	—	—
UCAT - EQUIVALENCE - CRITERIA	mathematical-development	Ordinary adjoint equivalences correspond to fully faithful and split essentially surjective functors, and between univalent categories mere essential surjectivity suffices.	—	—
UCAT - CATEGORY - IDENTITY	mathematical-development	Identity of ordinary precategories corresponds to isomorphism of precategories, and for univalent categories this agrees with categorical equivalence; the resulting type of categories is a 2-type.	—	—
UCAT - YONEDA	mathematical-development	The ordinary set-valued Yoneda evaluation map is an equivalence, making the Yoneda embedding fully faithful and representing objects unique up to isomorphism, hence up to identity in a univalent category.	—	—
UCAT - STRICT - CATEGORY	mathematical-development	An ordinary HoTT strict category is a precategory whose object type is a set; it need not be univalent, and an ordinary univalent category is strict exactly when it is gaunt.	—	—

UCAT - DAGGER - CATEGORY	mathematical- development	An ordinary dagger precategory has a chosen identity-on-objects contravariant involution; unitary arrows use the dagger as inverse, and dagger univalence identifies object identity with unitary isomorphism.	—	—
UCAT - STRUCTURE - IDENTITY	mathematical- development	For a standard proposition-valued notion of structure over an ordinary univalent category, the precategory of structured objects and structure-preserving arrows is itself univalent.	—	—
UCAT - REZK - COMPLETION	mathematical- development	Every ordinary precategory admits a fully faithful and essentially surjective map into a univalent category, constructible by a Yoneda image or a 1-truncated HIT; univalent categories are exactly the targets that invert all such weak equivalences by precomposition.	—	—
NATIVE - DAGGER - INTERFACE	research-boundary	The active opposite operation does not provide a chosen identity-on-objects involutive self-duality, coherent higher dagger action, a unitary classifier, or native dagger univalence.	—	—
NATIVE - STRUCTURE - IDENTITY	research-boundary	A generic native structure identity theorem requires a directed structure signature, a higher structure-preserving equivalence classifier, qualified base univalence, and coherent off-diagonal and next-hom action.	—	—
NATIVE - REZK - COMPLETION	research-boundary	No selected native saturation predicate, Rezk completion object, unit weak equivalence, or iterable higher mapping property is currently implemented.	—	—

ADJ - TRIANGLE - CUTS	checked	An indexed adjunction exposes stable unit and counit observations whose two component-level triangle cuts reduce to the corresponding functorial composites.	Adjunction emdash3_2.lp unit_adj_transf emdash3_2.lp counit_adj_transf emdash3_2.lp	unit_adj_transf examples/adjunction_triangle_s.lp text Indexed adjunction triangles, opposite, and operation-trust boundary emdash3_2_checks.lp
ADJ - HOM - PROF - COMPARISON	checked	An indexed adjunction supplies a reindexable profunctor comparison between $\text{Hom}_B(\text{LM}, \text{F})$ and $\text{Hom}_A(\text{M}, \text{RF})$, whose push and pull inherit the generic comparison beta and eta laws.	Adjunction_hom_prof_comparison emdash3_2.lp Adjunction_hom_prof_comparison_along emdash3_2.lp	Adjunction_hom_prof_comparison_along emdash3_2_checks.lp
MONAD - TRIANGULAR - COMPUTATION	checked	A selected monad exposes whole unit and multiplication observations and one whole Kleisli-extension operation whose point action supports the ambient Došen beta and accumulation cuts; an adjunction supplies the canonical monad with runtime unit and transparent G-epsilon-F multiplication semantics.	Monad emdash3_2_monads.lp unit_monad_transf emdash3_2_monads.lp mult_monad_transf emdash3_2_monads.lp kleisli_extend_func emdash3_2_monads.lp kleisli_extend_fapp0 emdash3_2_monads.lp adjunction_monad emdash3_2_monads.lp	kleisli_extend_fapp0 examples/monads_comonads.lp adjunction_monad examples/monads_comonads.lp

WEIGHTED-LIMIT-REPRESENTABILITY	checked	A parameterized weighted-cone profunctor is formed by the selected covariant residual, and its computational representation by a functor exposes inverse push and pull operations on every reindexed incoming profunctor map.	WeightedCone_prof emdash3_2.lp IsWeightedLimit_cov_iso emdash3_2.lp IsWeightedLimit_cov_comp emdash3_2.lp weighted_limit_cov_push emdash3_2.lp weighted_limit_cov_pull emdash3_2.lp	text Computational profunctor comparisons and weighted representability emdash3_2_checks.lp weighted_limit_cov_pull emdash3_2_checks.lp weighted_limit_cov_pull examples/profunctor_weighted_limits.lp
WEIGHTED-LIMIT-SPECIALIZATIONS	formal-consequence	Terminal weights and conjoint weights inhabit the selected weighted-limit classifier, so the general right-adjoint preservation construction specializes to the corresponding conical-limit and right-Kan interfaces; this does not assert their missing semantic end identifications.	Terminal_prof emdash3_2.lp Conjoint_prof emdash3_2.lp IsWeightedLimit_cov_comp emdash3_2.lp right_adjoint_preserves_weighted_limit_cov_comp emdash3_2.lp	text Weighted limit and colimit specialization typing emdash3_2_checks.lp
WEIGHTED-END-KAN-SEMANTICS	mathematical-development	With semantic ends and coends, terminal weights recover ordinary cone and cocone categories while conjoint and companion weights recover the standard pointwise right and left Kan extension formulas.	—	—
DEPENDENT-ADJUNCTIONS	research-boundary	A general dependent Sigma-change-of-base-Pi adjunction chain requires base-arrow action, off-diagonal and next-hom coherence, and Beck-Chevalley comparisons beyond the current Sigma and Pi family interfaces.	—	—

OP-DUALITY	checked	Opposite category, functor, transfor, and adjunction operations expose the selected involutive and variance-reversing computations used by the duality arguments.	Op_cat emdash3_2.lp Op_func emdash3_2.lp Op_transf emdash3_2.lp Op_adjunction emdash3_2.lp	Op_adjunction emdash3_2_checks.lp
WEIGHTED-LIMIT-PRESERVATION	checked	The selected profunctor comparison certifying a weighted limit is transported by an indexed right adjoint to a weighted-limit comparison for the composed diagram and cone point.	IsWeightedLimit_cov_comp emdash3_2.lp right_adjoint_preserves_weighted_limit_cov_comp emdash3_2_checks.lp right_adjoint_preserves_weighted_limit_cov_comp examples/profunctor_weighted_limits.lp	
WEIGHTED-COLIMIT-PRESERVATION	checked	Weighted colimits are presented through opposite weighted limits, and the selected left-adjoint preservation certificate is derived by applying right-adjoint preservation to the opposite adjunction.	WeightedColimit_con emdash3_2.lp left_adjoint_preserves_weighted_colimit_con emdash3_2.lp	left_adjoint_preserves_weighted_colimit_con emdash3_2_checks.lp
WEIGHTED-COLIMIT-SPECIALIZATION	formal-consequence	Terminal weights and companion weights inhabit the selected opposite-defined weighted-colimit classifier, so left-adjoint preservation specializes to the corresponding conical-colimit and left-Kan interfaces without supplying semantic coend identifications.	Terminal_profunctor emdash3_2.lp Companion_profunctor emdash3_2.lp WeightedColimit_con emdash3_2.lp left_adjoint_preserves_weighted_colimit_con emdash3_2.lp	text Weighted limit and colimit specialization typing emdash3_2_checks.lp

JOIN-RECURSOR	checked	The primitive directed join has two inclusion functors, an internally natural cross cell, and a nondependent recursor with beta computation on both inclusions and the cross-cell datum.	Join_cat emdash3_2.lp join_cross_tra nsf emdash3_2.lp join_elim_fun c emdash3_2.lp join_elim_cros s_transf emdash3_2.lp	join_elim_cros s_transf emdash3_2_chec ks.lp join_elim_cros s_transf examples/direc ted_join.lp
JOIN-COLLAGE- BOUNDARY	research-boundary	The join recursor has the input shape of the collage of the terminal profunctor, but no object or hom decomposition, mapping-category equivalence, opposite comparison, or dependent collage eliminator is active.	—	—
FORMAL-KERNEL- PRESENTATION	checked	The active v3.2 modules expose categories, iterated homs, functors, transfors, and directed families through explicit classifiers and full or capped application owners, with executable assertions checking representative typing and computation.	Cat emdash3_2.lp Hom_cat emdash3_2.lp fapp1_func emdash3_2.lp tapp1_func emdash3_2.lp Catd emdash3_2.lp	fapp1_fapp0 emdash3_2_chec ks.lp tapp1_fapp0 emdash3_2_chec ks.lp

FORMAL - ELABORATION - BOUNDARY	research-boundary	The renewed TypeScript product implements a bounded direct-TypeScript and categorical-text path through scoped contextual elaboration, backend-neutral explicit Core, and a generic checker/evaluator, together with bounded outer-LF adjunction and dependent-structure declarations, a client-side reviewer, and optional Lambdapi conformance. A complete compiler for the book's canonical surface, arbitrary displayed coherence, a general record or inductive facility, and whole-library transfer are not claimed.	—	—
FORMAL - METATHEORY - BOUNDARY	research-boundary	Local source acceptance, diagnostics, warning inventories, and model examples do not establish global confluence, strong normalization, canonicity, decidability, consistency, or semantic soundness for the whole emdash rewrite and unification theory.	—	—

GROTH- TOPOLOGY- SIEVE-LAWS	checked	An ordinary-sieve Grothendieck topology packages proposition-valued coverhood with maximality, pullback stability, and local character; the maximal sieve and the topology in which every sieve covers have checked direct models.	GrothTopology emdash3_2_site s.lp groth_topology _maximal emdash3_2_site s.lp groth_topology _pullback emdash3_2_site s.lp groth_topology _local_character emdash3_2_site s.lp chaotic_groth topology emdash3_2_site s.lp	GrothTopology examples/groth endieck_topolo gy.lp groth_topology _local_charact er examples/groth endieck_topolo gy.lp
GENERATED- GROTH-TOPOLOGY	checked	Every type-valued family of generating ordinary sieves determines an internally constructed least Grothendieck topology that accepts each retained generator and lies below every topology accepting them.	generated_siev e_cover_is_pro p emdash3_2_gene rated_topologi es.lp generated_grot h_topology emdash3_2_gene rated_topologi es.lp generated_grot h_topology_acc epts_generator s emdash3_2_gene rated_topologi es.lp generated_grot h_topology_lea st emdash3_2_gene rated_topologi es.lp	generated_grot h_topology examples/gener ated_grothendi eck_topologies .lp generated_grot h_topology_lea st examples/gener ated_grothendi eck_topologies .lp

SIEVE- MATCHING- LOCALITY	checked	An ordinary sieve extends to a whole presheaf with a whole inclusion into Yoneda; precomposition defines the section-to-matching restriction functor, locality is its fixed-forward Hom equivalence, and matching, section, and restriction families vary internally over eligible cover questions.	ordinary_sieve _extension_inclusion emdash3_2_sieve_extensions.lp ordinary_sieve _local_precomp emdash3_2_sieve_extensions.lp PshLocalAtOrdinarySieve emdash3_2_sieve_extensions.lp IsTopologyLocalPsh emdash3_2_sieve_extensions.lp DirectCoverQuestionMatching_catd emdash3_2_direct_cover_question_families.lp DirectCoverQuestionSection_catd emdash3_2_direct_cover_question_families.lp direct_cover_question_restriction_funcd emdash3_2_direct_cover_question_families.lp	ordinary_sieve _local_precomp examples/sieve_extensions.lp DirectCoverQuestionMatching_catd examples/direct_cover_question_families.lp direct_cover_question_restriction_funcd examples/direct_cover_question_families.lp
---------------------------------	---------	---	--	---

PSH-YONEDA-HIGHER-SIEVE	checked	Cat-valued presheaves reuse whole directed-family restriction; the Yoneda presheaf evaluates to the represented hom category; the conventional slice is the opposite restriction total; and the maximal Cat-valued higher sieve is stable under pullback.	Psh_cat emdash3_2_presheaves.lp Psh_pullback_functor emdash3_2_presheaves.lp yoneda_psh_functor emdash3_2_presheaves.lp Slice_cat emdash3_2_presheaves.lp HigherSieveClassifier emdash3_2_presheaves.lp maximal_higher_sieve emdash3_2_presheaves.lp	Psh_pullback_functor examples/presheaf_facade.lp HigherSieveClassifier examples/higher_sieve_classifier.lp
ORDINARY-SIEVE-PULLBACK	checked	An ordinary sieve is a Cat-valued higher sieve with pointwise subterminal evidence; pullback reuses the higher-sieve action, preserves that evidence, and computes membership at a probe as old membership at its postcomposition image.	Sieve emdash3_2_sieves.lp ordinary_sieve_pullback_evidence emdash3_2_sieves.lp sieve_pullback emdash3_2_sieves.lp SieveMembership emdash3_2_sites.lp sieve_pullback_membership emdash3_2_sites.lp	sieve_pullback examples/ordinary_sieves.lp SieveMembership examples/grothendieck_topology.lp

COMM-RING-INVERTIBILITY-SIEVE	checked	For a commutative-ring-valued presheaf and a section over U , the invertibility construction produces an ordinary sieve whose membership at a probe computes to unit evidence for the restricted section.	CommRingPshInvertibleAlong emdash3_2_commutative_algebra_presheaves.lp comm_ring_psh_invertibility_sieve emdash3_2_commutative_algebra_presheaves.lp	comm_ring_psh_invertibility_sieve examples/commutative_ring_presheaf_invertibility.lp CommRingPshInvertibleAlong examples/commutative_ring_presheaf_invertibility.lp
DIRECT-COVER-COMPLETION-HIT	checked	For every site and Cat-valued presheaf, the direct cover-completion categorical-HIT boundary provides a whole unit, one cover-question-indexed glue functor, and one whole silent path, with pullback compatibility inherited from displayed functoriality and a packaged internal direct-cover sheaf structure.	DirectCoverSheafStructure emdash3_2_direct_cover_internal_sheaves.lp direct_cover_sheaf_structure _glue_funcd emdash3_2_direct_cover_internal_sheaves.lp direct_cover_sheaf_structure _silent_funcd emdash3_2_direct_cover_internal_sheaves.lp DirectCoverCompletionPsh emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_unit emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_glue_funcd emdash3_2_direct_cover_completion_hit.lp direct_cover_completion_silent_funcd emdash3_2_direct_cover_completion_hit.lp	DirectCoverSheafStructure examples/direct_cover_internal_sheaves.lp direct_cover_completion_glue_funcd examples/direct_cover_completion_hit.lp direct_cover_completion_silent_funcd examples/direct_cover_completion_hit.lp

DIRECT-COVER-COMPLETION-LOCALITY	checked	Canonical cover pull-back, retained-member calculation, whole glue naturality, and silent derive restriction after glue as the second inverse law; the direct cover completion is consequently local at every eligible question and over the whole topology.	direct_cover_completion_restriction_glue_path emdash3_2_direct_cover_completion_locality.lp direct_cover_completion_local_at_question emdash3_2_direct_cover_completion_locality.lp direct_cover_completion_is_topology_local emdash3_2_direct_cover_completion_locality.lp	direct_cover_completion_restriction_glue_path emdash3_2_checks.lp direct_cover_completion_is_topology_local emdash3_2_checks.lp
----------------------------------	---------	--	--	--

DIRECT-COVER-COMPLETION-UNIVERSALITY	checked	The completion recursor extends a whole seed map with return, glue, and silent coherence; it varies functorially in the seed, and at a topology-local target its whole beta and eta laws make unit precomposition an omega-equivalence of complete Hom categories.	<code>direct_cover_completion_rec_emdash3_2_direct_cover_completion_eliminator.lp</code> <code>direct_cover_completion_rec_beta_unit_emdash3_2_direct_cover_completion_eliminator.lp</code> <code>direct_cover_completion_rec_beta_glue_emdash3_2_direct_cover_completion_eliminator.lp</code> <code>direct_cover_completion_rec_beta_silent_emdash3_2_direct_cover_completion_eliminator.lp</code> <code>direct_cover_completion_rec_func_emdash3_2_direct_cover_completion_universality.lp</code> <code>direct_cover_completion_rec_eta_local_func_emdash3_2_direct_cover_completion_universality.lp</code> <code>direct_cover_completion_hom_omega_emdash3_2_direct_cover_completion_universality.lp</code>	<code>direct_cover_completion_rec_beta_glue_examples/direct_cover_completion_eliminator.lp</code> <code>direct_cover_completion_rec_eta_local_func_emdash3_2_checks.lp</code> <code>direct_cover_completion_hom_omega_emdash3_2_checks.lp</code>
--------------------------------------	---------	--	--	--

CAT-VALUED-SHEAFIFICATION-REFLECTOR	checked	At Cat-valued coefficients, direct cover completion forms a functor into topology-local presheaves left adjoint to inclusion; its unit is return, its counit is local recursion from the identity seed, and the two counit cancellations make the adjunction reflective and instantiate the sheafification capability.	CatValuedSheafData emdash3_2_direct_cover_sheafification.lp cat_valued_sheaf_include_psh_func emdash3_2_direct_cover_sheafification.lp direct_cover_sheafification_func emdash3_2_direct_cover_sheafification.lp direct_cover_sheafification_adjunction emdash3_2_direct_cover_sheafification.lp direct_cover_sheafification_reflector emdash3_2_direct_cover_sheafification.lp direct_cover_sheafification_capability emdash3_2_direct_cover_sheafification.lp	direct_cover_sheafification_func emdash3_2_checks.lp direct_cover_sheafification_reflector_at emdash3_2_checks.lp direct_cover_sheafification_capability emdash3_2_checks.lp
-------------------------------------	---------	--	--	---

COMM-RING-STRUCTURED-CATEGORY	checked	Commutative rings have set-valued carriers and retained operations and laws, including the zero ring; operation-preserving carrier maps are extensional structured homs and form the one-category <code>CommRing_cat</code> , while componentwise products and the Boolean-carrier <code>F2</code> ring supply closed models without a claimed categorical-product universal property.	<code>comm_ring_carrier_is_set</code> <code>emdash3_2_commutative_algebra.lp</code> <code>zero_comm_ring</code> <code>emdash3_2_commutative_algebra.lp</code> <code>CommRingHom</code> <code>emdash3_2_commutative_algebra_category.lp</code> <code>comm_ring_hom_ext</code> <code>emdash3_2_commutative_algebra_category.lp</code> <code>CommRing_cat</code> <code>emdash3_2_commutative_algebra_category.lp</code> <code>comm_ring_cat_is_one_cat</code> <code>emdash3_2_commutative_algebra_category.lp</code> <code>comm_ring_product</code> <code>emdash3_2_commutative_algebra_product.lp</code> <code>f2_comm_ring</code> <code>emdash3_2_commutative_algebra_f2.lp</code>	<code>zero_comm_ring</code> <code>examples/commutative_ring_objects.lp</code> <code>CommRing_cat</code> <code>examples/commutative_ring_morphisms.lp</code> <code>f2_comm_ring</code> <code>examples/commutative_ring_split_idempotent_localization.lp</code>
-------------------------------	---------	--	--	--

FINITE- UNIMODULAR- COVER-DATA	checked	Finite sums and dot products compute on finite-family constructors and are preserved by structured ring maps; unimodular presentations retain coefficients witnessing that the generators span one, assemble into set-valued finite Zariski presentations, transport under ring maps, and include singleton and binary constructors.	comm_ring_finite_sum emdash3_2_commutative_algebra_finite.lp comm_ring_finite_dot emdash3_2_commutative_algebra_finite.lp CommRingUnimodularPresentation emdash3_2_commutative_algebra_finite.lp comm_ring_unimodular_map emdash3_2_commutative_algebra_finite.lp CommRingZariskiCoverPresentation emdash3_2_commutative_algebra_finite.lp comm_ring_zariski_cover_map emdash3_2_commutative_algebra_finite.lp comm_ring_binary_zariski_cover emdash3_2_commutative_algebra_finite.lp	CommRingUnimodularPresentation examples/commutative_ring_finite_covers.lp comm_ring_unit_zariski_cover examples/commutative_ring_finite_covers.lp comm_ring_binary_zariski_cover examples/commutative_ring_finite_covers.lp
--------------------------------------	---------	--	--	--

COMM-RING-POLYNOMIAL-UNIVERSALITY	checked	A supplied commutative-ring polynomial algebra retains a base map and variables and makes the structured extension space contractible for every target base map and valuation; the identity ring is a complete checked model for the empty variable classifier.	CommRingPolynomialFactor emdash3_2_commutative_algebra_polynomial.lp IsCommRingPolynomialAlgebra emdash3_2_commutative_algebra_polynomial.lp comm_ring_polynomial_factorization_is_con r emdash3_2_commutative_algebra_polynomial.lp CommRingPolynomialAlgebra emdash3_2_commutative_algebra_polynomial.lp	empty_is_polynomial examples/commutative_ring_polynomial_algebra.lp empty_polynomial examples/commutative_ring_polynomial_algebra.lp comm_ring_polynomial_factorization_is_con r examples/commutative_ring_polynomial_algebra.lp
-----------------------------------	---------	---	--	--

COMM-RING- LOCALIZATION- UNIVERSALITY	checked	Unit evidence in a commutative ring is proposition-valued, and a supplied localization at one element retains an inverting structure map and a contractible classifier of whole structured factors with their pointwise triangles through every admissible target map.	CommRingUnitEvidence emdash3_2_commutative_algebra_localization.lp comm_ring_unit_evidence_is_prop emdash3_2_commutative_algebra_localization.lp CommRingLocalizationFactor emdash3_2_commutative_algebra_localization.lp IsCommRingLocalizationAt emdash3_2_commutative_algebra_localization.lp comm_ring_localization_factorization_is_contr emdash3_2_commutative_algebra_localization.lp CommRingLocalizationAt emdash3_2_commutative_algebra_localization.lp	zero_is_localization_at_point examples/commutative_ring_localization.lp zero_localization_factor_is_contr examples/commutative_ring_localization.lp
---	---------	--	--	--

COMM-RING-LOCALIZATION-MODELS	checked	Identity is a localization at an already invertible element, the zero ring is a localization at zero, and the fixed image of an idempotent is a quotient-free localization at that idempotent; the split element (1,0) in F_2 times F_2 gives a closed idempotent distinct from zero and one.	comm_ring_unit_is_identity_localization emdash3_2_commutative_algebra_localization_unit.lp comm_ring_identity_localization_at_one emdash3_2_commutative_algebra_localization_unit.lp comm_ring_zero_is_zero_localization emdash3_2_commutative_algebra_localization_zero.lp comm_ring_zero_localization emdash3_2_commutative_algebra_localization_zero.lp comm_ring_idempotent_image_is_localization emdash3_2_commutative_algebra_localization_idempotent.lp comm_ring_idempotent_image_localization emdash3_2_commutative_algebra_localization_idempotent.lp f2_split_idempotent_not_zero emdash3_2_commutative_algebra_localization_split.lp f2_split_idempotent_not_one emdash3_2_commutative_algebra_localization_split.lp f2_split_idempotent_localization
-------------------------------	---------	--	--

			emdash3_2_commutative_algebra_localization_split.lp comm_ring_idempotent_localization_at_one_examples/commutative_ring_unit_localization.lp comm_ring_zero_localization_examples/commutative_ring_zero_localization.lp comm_ring_idempotent_image_localization_examples/commutative_ring_idempotent_localization.lp f2_split_idempotent_not_zero_examples/commutative_ring_split_idempotent_localization.lp
--	--	--	---

COMM-RING-ITERATED-LOCALIZATION-EQUIV	checked	For supplied localizations at f, at the image of g, and at fg, the universal properties construct canonical product-to-iterated and iterated-to-product comparison maps whose two whole structured cancellation laws exhibit the selected forward map as an omega-equivalence in CommRing_cat.	CommRingIteratedLocalizationComparison emdash3_2_commutative_algebra_localization_comparison.lp comm_ring_iterated_localization_comparison emdash3_2_commutative_algebra_localization_comparison.lp comm_ring_iterated_localization_comparison_left_law emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_right_law emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv_a_long emdash3_2_commutative_algebra_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv emdash3_2_commutative_algebra_localization_overlap.lp	comm_ring_iterated_localization_comparison_left_law examples/commutative_ring_localization_overlap.lp comm_ring_iterated_localization_comparison_right_law examples/commutative_ring_localization_overlap.lp comm_ring_iterated_localization_comparison_omega_equiv examples/commutative_ring_localization_overlap.lp
--	----------------	---	---	---

AFFINE - BASIC - OPEN - POINT - REPRESENTATION	checked	The affine Yoneda presheaf has S -points $\text{CommRingHom}(R, S)$, its semantic basic open $D(f)$ is the ordinary invertibility sieve, and every supplied localization R to $R[1/f]$ gives, at each test ring S , an explicit TypeEquiv from whole structured maps $R[1/f]$ to S to $D(f)$ -points, with both inverse laws derived from localization contractibility and proposition-valued unit evidence.	<code>affine_spec_functor_of_point s emdash3_2_commutative_algebra_affine_point s.lp affine_spec_basic_open_sieve emdash3_2_commutative_algebra_affine_point s.lp affine_spec_basic_open_point_left_law emdash3_2_commutative_algebra_affine_point s.lp affine_spec_basic_open_point_right_law examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_type_equiv emdash3_2_commutative_algebra_affine_point s.lp affine_spec_basic_open_point_type_equiv emdash3_2_commutative_algebra_affine_point s.lp</code>	<code>AffineSpecPoint examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_left_law examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_right_law examples/commutative_ring_affine_points.lp affine_spec_basic_open_point_type_equiv examples/commutative_ring_affine_points.lp</code>
--	---------	--	--	---

AFFINE - BASIC - OPEN - INTERSECTION	checked	At each test ring S , unit evidence for $h(fg)$ is equivalent to paired unit evidence for $h(f)$ and $h(g)$, yielding an explicit pointwise TypeEquiv from $D(fg)(S)$ to the same-map intersection of $D(f)(S)$ and $D(g)(S)$; a supplied localization at fg represents that intersection through executable maps with both component laws.	<code>affine_spec_ba sic_open_produ ct_unit_type_e quiv emdash3_2_comm utative_algebr a_affine_inter sections.lp affine_spec_ba sic_open_produ ct_point_type_ equiv emdash3_2_comm utative_algebr a_affine_inter sections.lp affine_spec_ba sic_open_inter section_repres entation emdash3_2_comm utative_algebr a_affine_inter sections.lp affine_spec_ba sic_open_inter section_repres entation_left emdash3_2_comm utative_algebr a_affine_inter sections.lp affine_spec_ba sic_open_inter section_repres entation_right emdash3_2_comm utative_algebr a_affine_inter sections.lp</code>	<code>affine_spec_ba sic_open_produ ct_point_type_ equiv examples/commu tative_ring_af fine_intersect ions.lp affine_spec_ba sic_open_inter section_repres entation_left examples/commu tative_ring_af fine_intersect ions.lp affine_spec_ba sic_open_inter section_repres entation_right examples/commu tative_ring_af fine_intersect ions.lp</code>
--	---------	--	---	---

AFFINE-BIG-SLICE-COORDINATES	checked	The conventional big affine slice over $\mathrm{Sp}(R)$ has objects R-algebras and geometric arrows given by commuting structured triangles; its whole $\mathrm{CommRing}$-valued coordinate presheaf evaluates a chart at its ring and restriction at the supplied structured map, includes selected localization charts, and internalizes both product/iterated-localization overlap directions with the existing whole coordinate equivalence.	AffineSpecBigSlice_cat emdash3_2_commutative_algebra_affine_spec. lp affine_spec_coordinate_psh emdash3_2_commutative_algebra_affine_spec. lp affine_spec_chart_arrow emdash3_2_commutative_algebra_affine_spec. lp affine_spec_overlap_forward_chart_arrow emdash3_2_commutative_algebra_affine_spec. lp affine_spec_overlap_reverse_chart_arrow emdash3_2_commutative_algebra_affine_spec. lp affine_spec_overlap_coordinate_omega_equiv emdash3_2_commutative_algebra_affine_spec. lp	affine_spec_coordinate_psh examples/commutative_ring_affine_spec.lp affine_spec_chart_arrow examples/commutative_ring_affine_spec.lp affine_spec_overlap_coordinate_omega_equiv examples/commutative_ring_affine_spec.lp
------------------------------	---------	---	---	---

AFFINE-BIG-ZARISKI-TOPOLOGY	checked	At every chart R to S , each supplied localization in a selected finite unimodular family lifts to a whole chart arrow whose coordinate restriction computes to the localization map; literal finite containment forms witness-rich generators, and their generic intersection constructs the lawful least big-affine Zariski topology with generator coverhood and leastness.	affine_spec_chart_localization_arrow emdash3_2_commutative_algebra_affine_zariski.lp AffineSpecBigZariskiGenerators emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology_covers emdash3_2_commutative_algebra_affine_zariski.lp affine_spec_big_zariski_topology_least emdash3_2_commutative_algebra_affine_zariski.lp	affine_spec_chart_localization_restriction_review examples/commutative_ring_affine_zariski.lp affine_spec_big_zariski_topology_covers examples/commutative_ring_affine_zariski.lp affine_spec_big_zariski_topology_least examples/commutative_ring_affine_zariski.lp
-----------------------------	---------	--	--	---

AFFINE- STRUCTURE- SHEAF- PRESENTATION	checked	Given a supplied reflective CommRing-valued sheafification capability on the exact generated big-affine Zariski topology, a selected sheaf object, and a whole DefIso from its included presheaf to the computing coordinate presheaf, the affine structure-sheaf presentation determines a reflective commutative-ringed site and retains the whole coordinate comparison and its chart components.	AffineStructureSheafPresentation emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_ringed_site emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_coordinate_defiso emdash3_2_commutative_algebra_affine_ringed_sites.lp affine_structure_sheaf_to_coordinate_at emdash3_2_commutative_algebra_affine_ringed_sites.lp	affine_structure_sheaf_ringed_site examples/commutative_ring_affine_ringed_sites.lp affine_structure_sheaf_coordinate_defiso examples/commutative_ring_affine_ringed_sites.lp affine_structure_sheaf_to_coordinate_at examples/commutative_ring_affine_ringed_sites.lp
---	---------	--	---	---

AFFINE - THIN - SCHEME - PRESENTATION	checked	The thin affine-scheme presentation pairs a supplied whole reflective structure-sheaf presentation with supplied whole coordinate-localization locality; it inherits the exact generated big-Zariski ringed site, a whole coordinate DefIso, and fixed-forward localization matching equivalences, while the F2 times F2 reviewer keeps both capabilities explicit and computes its complementary-idempotent cover, chart rings, restrictions, and zero overlap.	AffineCoordinateLocalizationLocality emdash3_2_commutative_algebra_affine_locality.lp affine_coordinate_localization_locality_at emdash3_2_commutative_algebra_affine_locality.lp AffineSchemePresentation emdash3_2_commutative_algebra_affine_schemes.lp affine_scheme_ringed_site emdash3_2_commutative_algebra_affine_schemes.lp affine_scheme_coordinate_defiso emdash3_2_commutative_algebra_affine_schemes.lp affine_scheme_locality_at emdash3_2_commutative_algebra_affine_schemes.lp	affine_coordinate_localization_locality_at examples/commutative_ring_affine_locality.lp f2_split_affine_scheme examples/commutative_ring_affine_schemes.lp affine_scheme_coordinate_defiso examples/commutative_ring_affine_schemes.lp affine_scheme_locality_at examples/commutative_ring_affine_schemes.lp
---------------------------------------	---------	---	---	---

GLOBAL - RINGED - COVER - BINARY - GENERATION	checked	A global reflective CommRinged cover retains one site, whole object, ordinary covering sieve, and whole structure presheaf; Grothendieck stability derives every pullback cover, while witness-rich binary generation computes, for each retained sieve member, a Boolean-selected chart factorization and triangle, and the affine refinement route exposes the selected generator without asserting the arbitrary member affine.	ReflectiveCommRingedSpaceCover emdash3_2_commutative_algebra_ringed_space_covers.lp reflective_comm_ringed_space_cover_underlying_psh emdash3_2_commutative_algebra_ringed_space_covers.lp reflective_comm_ringed_space_cover_pullback_covers emdash3_2_commutative_algebra_ringed_space_covers.lp BinarySelectedCoverGeneration emdash3_2_commutative_algebra_binary_covers.lp binary_selected_cover_generation_at emdash3_2_commutative_algebra_binary_covers.lp binary_affine_cover_refinement_at emdash3_2_commutative_algebra_affine_cover_refinements.lp	reflective_comm_ringed_space_cover_pullback_covers examples/commutative_ring_ringed_space_covers.lp BinarySelectedCoverGeneration examples/commutative_ring_binary_covers.lp binary_selected_cover_generation_at examples/commutative_ring_binary_covers.lp binary_affine_cover_refinement_at examples/commutative_ring_affine_cover_refinements.lp
---	---------	---	--	---

WHOLE-SLICE-AFFINE-REALIZATION	checked	Precomposition with the whole slice-domain functor constructs the ambient CommRing presheaf on K/U; a supplied reflective slice retains a whole DefIso to that target, and supplied sheaf-basis semantics plus an ambient-to-affine-underlying bridge and the affine presentation's retained coordinate bridge derive one whole ambient-to-coordinate DefIso for a selected cover chart.	slice_domain_fun emdash3_2_presheaves.lp reflective_comm_ringed_site_slice_ambient_psh emdash3_2_commutative_algebra_ringed_space_restrictions.lp SuppliedReflectiveCommRingedSlicePresentation emdash3_2_commutative_algebra_ringed_space_restrictions.lp supplied_reflective_comm_ringed_slice_ambient_defiso emdash3_2_commutative_algebra_ringed_space_restrictions.lp SuppliedSheafBasisEquivalenceAlong emdash3_2_site_basis.lp AffineBasisRealizationAlong emdash3_2_commutative_algebra_affine_basis.lp affine_basis_realization_coordinate_defiso emdash3_2_commutative_algebra_affine_basis.lp AffineCoverChartRealization emdash3_2_commutative_algebra_affine_cover_charts.lp
--------------------------------	---------	--	---

			hart_coordinate_defiso emdash3_2_commutative_algebra affine_cover_charts.lp	supplied_reflective_comm_ringed_slice_ambient_defiso examples/commutative_ring_ringed_space_restrictions.lp affine_basis_realization_coordinate_defiso examples/commutative_ring_affine_basis.lp AffineCoverChartRealization examples/commutative_ring_affine_cover_charts.lp affine_cover_charts.lp hart_coordinate_defiso examples/commutative_ring_affine_cover_charts.lp
--	--	--	---	--

TOPOLOGY- LOCAL-RING- CERTIFICATE	checked	A topology-local CommRing presheaf presentation makes a zero-unit stage empty-covering and turns every invertible sum into a selected covering sieve whose members compute a Boolean branch with unit evidence for one restricted summand; the whole-object package attaches that capability to the computing ambient presheaf on the supplied actual slice K/X.	CommRingPshTopologyLocal-RingPresentation emdash3_2_commutative_algebra_local_ringed_sites.lp comm_ring_psh_topology_local_ring_empty_covers emdash3_2_commutative_algebra_local_ringed_sites.lp comm_ring_psh_topology_local_ring_split emdash3_2_commutative_algebra_local_ringed_sites.lp ReflectiveCommRingedWholeObjectLocalPresentation emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ambient_defiso emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ring emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp	comm_ring_psh_topology_local_ring_nontriviality examples/commutative_ring_local_ringed_sites.lp comm_ring_psh_topology_local_ring_split examples/commutative_ring_local_ringed_sites.lp ReflectiveCommRingedWholeObjectLocalPresentation examples/commutative_ring_locally_ringed_space_presentations.lp reflective_comm_ringed_whole_object_local_ambient_defiso examples/commutative_ring_locally_ringed_space_presentations.lp
--	----------------	--	---	---

BINARY-SITE-RELATIVE-SCHEME	checked	A BinarySiteRelativeSchemePresentation totals one existing global reflective CommRinged cover with its whole-object topology-local certificate and a constructively generated binary affine cover; the global structure presheaf, covering sieve, local package, selected charts, and both whole affine realizations remain exact existing owners rather than duplicated overlap, cocycle, transition, or gluing fields.	BinaryAffineCoverPresentation emdash3_2_commutative_algebra_affine_cover_presentations.lp BinaryLocallyRingedAffineCoverPresentation emdash3_2_commutative_algebra_locally_ringed_space_presentations.lp BinarySiteRelativeSchemePresentation emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_underlying_psh emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_local emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_atlas emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_realization0 emdash3_2_commutative_algebra_site_relative_schemes.lp binary_site_relative_scheme_realization1 emdash3_2_commutative_algebra_site_relative_schemes.lp
-----------------------------	---------	---	--

			<code>a_site_relative_schemes.lp</code>	<code>binary_affine_cover_generation</code> <code>examples/commutative_ring_affine_cover_presentations.lp</code> <code>binary_locally_ringed_affine_cover_atlas</code> <code>examples/commutative_ring_locally_ringed_space_presentations.lp</code> <code>BinarySiteRelativeSchemePresentation</code> <code>examples/commutative_ring_site_relative_schemes.lp</code> <code>binary_site_relative_scheme_underlying_psh</code> <code>examples/commutative_ring_site_relative_schemes.lp</code> <code>binary_site_relative_scheme_realization0</code> <code>examples/commutative_ring_site_relative_schemes.lp</code> <code>binary_site_relative_scheme_realization1</code> <code>examples/commutative_ring_site_relative_schemes.lp</code>
--	--	--	---	--

ACTUAL - BINARY - CHART - OVERLAP	checked	Given a selected binary product of the two chart objects in the conventional slice K/X, its whole universal property derives both slice projections and their base arrows; evaluating the single global structure presheaf derives the overlap ring and both restriction homomorphisms, without adding an overlap field to the scheme total or constructing arbitrary pullbacks.	BinaryProductPresentation emdash3_2_finite_limits.lp BinarySchemeChartOverlapPresentation emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart0 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart1 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart1 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_domain emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_ring emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction0 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction1 emdash3_2_commutative_algebra_scheme_chart_overlaps.lp	BinarySchemeChartOverlapPresentation examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart0 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_to_chart1 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_ring examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction0 examples/commutative_ring_scheme_chart_overlaps.lp binary_scheme_chart_overlap_restriction1 examples/commutative_ring_scheme_chart_overlaps.lp
--------------------------------------	---------	--	---	---

LAURENT - TRANSITIONS - BY - UNIVERSALITY	checked	For supplied one-variable polynomial algebras over A and selected localizations at their coordinates, polynomial universality constructs $A[t]$ to $A[u, 1/u]$ with t sent to the chosen inverse of u , localization universality extends it to $A[t, 1/t]$ to $A[u, 1/u]$, reversing the inputs constructs the opposite orientation, and the retained factor agreement is a whole pointwise triangle.	CommRingLaurentLocalization emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_polynomial_coordinate_path emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_transition_map emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_transition_agreement emdash3_2_commutative_algebra_laurent.lp	comm_ring_laurent_polynomial_coordinate_path examples/commutative_ring_laurent.lp comm_ring_laurent_transition_map examples/commutative_ring_laurent.lp comm_ring_laurent_transition_agreement examples/commutative_ring_laurent.lp
--	---------	---	---	--

LAURENT - COMMON-OVERLAP	checked	Given two literal restriction maps R to L and T to L , each supplied simultaneously as a one-variable polynomial chart over one base ring and a localization at its coordinate, the Laurent owner constructs both coordinate-inversion endomorphisms of that exact L ; a Laurent overlap presentation then supplies whole paths identifying both constructed endomorphisms with the identity, rather than a disconnected overlap isomorphism or componentwise square.	CommRingOneVariableLocalizationPresentation emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_common_overlap_transition emdash3_2_commutative_algebra_laurent.lp CommRingLaurentOverlapPresentation emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_overlap_forward_identity emdash3_2_commutative_algebra_laurent.lp comm_ring_laurent_overlap_reverse_identity emdash3_2_commutative_algebra_laurent.lp	CommRingOneVariableLocalizationPresentation examples/commutative_ring_laurent.lp comm_ring_laurent_common_overlap_transition examples/commutative_ring_laurent.lp CommRingLaurentOverlapPresentation examples/commutative_ring_laurent.lp comm_ring_laurent_overlap_forward_identity examples/commutative_ring_laurent.lp comm_ring_laurent_overlap_reverse_identity examples/commutative_ring_laurent.lp
ACTUAL - SCHEME - LAURENT - OVERLAP	checked	For a supplied binary site-relative scheme and selected actual chart intersection, the thin Laurent adapter adds one common base ring and retains a generic Laurent overlap presentation at the literal two chart structure rings, inherited overlap ring, and already-derived restriction homomorphisms; it neither duplicates those global owners nor constructs the coordinate presentation from no data.	BinarySchemeLaurentOverlapPresentation emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_base_ring emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_coordinates emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp	binary_scheme_laurent_overlap_base_ring examples/commutative_ring_scheme_laurent_overlaps.lp binary_scheme_laurent_overlap_coordinates examples/commutative_ring_scheme_laurent_overlaps.lp

SUPPLIED-P1	checked	A SuppliedProjectiveLinePresentation is the transparent dependent total of one already-global binary site-relative scheme, its selected actual chart intersection, and a Laurent-coordinate presentation on the literal inherited restriction maps; its scheme, overlap, base ring, and coordinate package compute by projection, while no closed projective object, Proj construction, gluing theorem, projectivity proof, or non-affineness proof is produced.	SuppliedProjectiveLinePresentation emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_scheme emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_overlap emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_base_ring emdash3_2_commutative_algebra_projective_line.lp supplied_projective_line_coordinates emdash3_2_commutative_algebra_projective_line.lp	supplied_projective_line_scheme examples/commutative_ring_projective_line.lp supplied_projective_line_overlap examples/commutative_ring_projective_line.lp supplied_projective_line_base_ring examples/commutative_ring_projective_line.lp supplied_projective_line_coordinates examples/commutative_ring_projective_line.lp
GROUPOIDAL-PRODUCT-CLOSURE	checked	The canonical comparison from $\text{Path}(A \times B)$ to $\text{Path}(A) \times \text{Path}(B)$ is judgmentally identity on objects, and at each pair of objects its actual hom action has explicit split and join maps forming a TypeEquiv between product paths and pairs of component paths; the distinct category heads are not definitionally identified.	path_product_compare_func emdash3_2_groupoidal_closure.lp path_product_compare_hom_type_equiv emdash3_2_groupoidal_closure.lp path_product_comparison_evidence emdash3_2_groupoidal_closure.lp	path_product_compare_func examples/groupoidal_product_transport.lp path_product_compare_hom_type_equiv examples/groupoidal_product_transport.lp

PATH-PSEUDO-LAXITY	checked	For a raw map h between groupoids, the generic compositor $\text{path_map_func}(h)$ decodes to an equality between paths with a canonical eq_sym inverse; its formal endpoints compare propositionally with the readable $\text{eq_ap}/\text{eq_trans}$ composites, and the whole compositor retains one next hom action.	<code>path_map_compositor_path</code> <code>emdash3_2_path_pseudo_laxity</code> <code>.lp</code> <code>path_map_compositor_inverse</code> <code>emdash3_2_path_pseudo_laxity</code> <code>.lp</code> <code>path_map_compositor_readable</code> <code>emdash3_2_path_pseudo_laxity</code> <code>.lp</code> <code>path_map_compositor_higher_functor</code> <code>emdash3_2_path_pseudo_laxity</code> <code>.lp</code>	<code>path_map_compositor_path</code> <code>examples/path_pseudo_laxity</code> <code>.lp</code> <code>path_map_compositor_inverse</code> <code>examples/path_pseudo_laxity</code> <code>.lp</code> <code>path_map_compositor_higher_functor</code> <code>examples/path_pseudo_laxity</code> <code>.lp</code>
GROUPOIDAL-PRODUCT-TRANSPORT	checked	For a dependent family over a product, transport along the path assembled from two coordinate paths agrees with both sequential coordinate orders; the two comparisons form a coherence diamond, and the existing structured transport and PathOut induction agree with the same primitive right-J transport.	<code>product_transport_direct_agrees_base_then_fibre</code> <code>emdash3_2_groupoidal_closure</code> <code>.lp</code> <code>product_transport_direct_agrees_fibre_then_base</code> <code>emdash3_2_groupoidal_closure</code> <code>.lp</code> <code>product_transport_coherence_diamond</code> <code>emdash3_2_groupoidal_closure</code> <code>.lp</code> <code>product_path_structured_induction_agrees_primitive</code> <code>emdash3_2_groupoidal_closure</code> <code>.lp</code>	<code>product_transport_coherence_diamond</code> <code>examples/groupoidal_product_transport</code> <code>.lp</code> <code>product_path_structured_induction_agrees_primitive</code> <code>examples/groupoidal_product_transport</code> <code>.lp</code>

INTEGER- LOCALIZATION- LINE	checked	The Integer carrier is the transparent set-truncated telescope localization of Nat successor: integer_stage(n,x) represents x-n, successor is an equivalence with executable predecessor, univalence supplies its universe path, and set-targeted dependent and nondependent elimination compute on stage representatives.	Integer_grpd emdash3_2_inte ger_localizati on.lp integer_succ_e quiv emdash3_2_inte ger_localizati on.lp integer_ind emdash3_2_inte ger_localizati on.lp integer_is_se t emdash3_2_inte ger_localizati on.lp	integer_succ_e quiv examples/integ er_localizatio n.lp integer_ind examples/integ er_localizatio n.lp integer_is_se t examples/integ er_localizatio n.lp
CIRCLE-HIT- COMPUTATION	checked	The groupoidal Circle has base and loop constructors, unrestricted dependent elimination, judgmental point computation, and judgmental dependent PathOver computation on the generating loop; constant-family recursion inherits that dependent beta while its ordinary eq_ap loop equation remains propositional.	circle_ind emdash3_2_circ le_hit.lp circle_ind_bet a_loop emdash3_2_circ le_hit.lp circle_rec_bet a_loop_path emdash3_2_circ le_hit.lp	circle_ind_bet a_loop examples/circl e_judgmental_l oop_computatio n.lp circle_rec_bet a_loop_path examples/circl e_loop_space.l p
CIRCLE- MONODROMY	checked	Every selected self-equivalence e of a groupoid A yields a Circle-indexed family with base fibre A and loop monodromy ua(e); transport around the loop agrees with e.to, and restriction along the WalkingEnd-to-Circle map recovers the original directed representation as a whole functor path.	walking_circle _monodromy_cir cle_family emdash3_2_walk ing_circle_mon odromy.lp walking_circle _monodromy_tra nsport_path emdash3_2_walk ing_circle_mon odromy.lp walking_circle _monodromy_res triction_path emdash3_2_walk ing_circle_mon odromy.lp	walking_circle _monodromy_cir cle_family examples/walki ng_circle_grou poidification_ monodromy.lp walking_circle _monodromy_tra nsport_path examples/walki ng_circle_grou poidification_ monodromy.lp walking_circle _monodromy_res triction_path examples/walki ng_circle_grou poidification_ monodromy.lp

CIRCLE-CONNECTED-TRUNCATION	checked	Circle induction constructs were based on connectedness $x \mapsto \lfloor \lfloor \text{base} = x \rfloor \rfloor - 1$; eliminating that evidence into the classified set truncation and then using its restricted induction proves the set truncation of Circle contractible without identifying its carrier judgmentally with Unit.	circle_connected examples/circle_connectedness.lp circle_set_trunc_contract examples/circle_connectedness.lp circle_set_trunc_is_contract examples/circle_connectedness.lp	circle_connected examples/circle_connectedness.lp circle_set_trunc_is_contract examples/circle_connectedness.lp
CIRCLE-LOOP-INTEGER	checked	The opaque groupoidal Circle has a universal Integer cover whose end-point-dependent encode and decode maps are inverse; the intrinsic based-loop carrier and the categorical based Hom carrier are TypeEquiv to the successor-localized Integer, with a whole equality-valued categorical equivalence retained separately.	circle_loop_integer_type_equiv examples/circle_loop_space.lp circle_hom_integer_type_equiv examples/circle_loop_space.lp circle_hom_integer_type_equiv examples/circle_loop_space.lp	circle_loop_integer_type_equiv examples/circle_loop_space.lp circle_hom_integer_type_equiv examples/circle_loop_space.lp
WALKING-INTERVAL-GROUPOIDIFICATION	checked	The groupoidal interval has two endpoints, one generating path, judgmental point and dependent-segment computation, and a whole fixed-forward mapping-object equivalence from maps Interval to G to path-valued functors WalkingArrow to Path(G), with endpoint/generator projections and retained higher action.	interval_ind examples/groupoidal_interval_hit.lp walking_arrow_to_interval_functor examples/walking_interval_groupoidification_hom_omega examples/walking_interval_groupoidification_hom_omega examples/walking_interval_groupoidification_hom_omega	interval_ind examples/groupoidal_interval_hit.lp walking_arrow_to_interval_functor examples/walking_interval_groupoidification_hom_omega examples/walking_interval_groupoidification_hom_omega examples/walking_interval_groupoidification_hom_omega

GROUPOIDIFICATION - INTERVAL - RECOVERY	checked	Specializing generic groupoidification to WalkingArrow and comparing the generic and interval extension owners gives maps $\text{Groupoidify}(\text{WalkingArrow})$ to Interval and back; their whole beta/eta laws yield both cancellation paths and a TypeEquiv without a definitional identification of the two HITs.	groupoidify_walking_to_interval emdash3_2_groupoidification_interval_recovery.lp interval_to_groupoidify_walking emdash3_2_groupoidification_interval_recovery.lp groupoidify_walking_interval_type_equiv emdash3_2_groupoidification_interval_recovery.lp	groupoidify_walking_to_interval examples/generic_groupoidification_interval.lp groupoidify_walking_interval_type_equiv examples/generic_groupoidification_interval.lp
GENERIC - GROUPOIDIFICATION - MAPPING	checked	For every category C and groupoid G , restriction along the whole unit C to $\text{Path}(\text{Groupoidify}(C))$ is a fixed-forward OmegaEquivAlong between groupoidal maps out of $\text{Groupoidify}(C)$ and path-valued functors out of C ; the unit recursor computes on represented objects and dependent first cells and retains higher action.	groupoidify_unit_func emdash3_2_groupoidification_hit.lp groupoidify_extend_func emdash3_2_groupoidification_hit.lp groupoidification_hom_omega emdash3_2_groupoidification_universality.lp groupoidify_unit_compositor_next_func emdash3_2_groupoidification_composition.lp	groupoidification_hom_omega examples/generic_groupoidification.lp groupoidify_unit_compositor_next_func examples/generic_groupoidification.lp

GRAY-COMPUTATIONAL-PROFILE	checked	IsStrictFunctor constrains the existing extracted compositor, StrictFunctor is its exact carrier/evidence Sigma package behind a rigid indexed facade, and constructor-visible strict_functor action computes to the carrier; GrayHom_lax uses those packages as objects and reuses the ambient Transf_cat tower without a duplicate code grammar or compositor.	IsStrictFunctor emdash3_2_gray_profiles.lp StrictFunctor emdash3_2_gray_profiles.lp strict_functor emdash3_2_gray_profiles.lp GrayHom_lax emdash3_2_gray_profiles.lp grayhom_lax_include_func emdash3_2_gray_profiles.lp	IsStrictFunctor examples/gray_profiles.lp strict_functor_evidence examples/gray_profiles.lp GrayHom_lax examples/gray_profiles.lp grayhom_lax_include_func examples/gray_profiles.lp
GRAY-RIGHT-CLOSURE	checked	The selected GrayTensor_R has one profiled right closure: whole curry and uncurry, each paired with supplied IsStrictFunctor evidence, form an OmegaEquivAlong between GrayHom_lax(GrayTensor_R(A,B),C) and GrayHom_lax(A,GrayHom_lax(B,C)), with whole beta/eta and coevaluation/evaluation derived at strict identity packages.	GrayTensor_R emdash3_2_gray_right_closure.lp gray_curry_R_func emdash3_2_gray_right_closure.lp gray_curry_R_is_strict emdash3_2_gray_right_closure.lp gray_right_closure_omega emdash3_2_gray_right_closure.lp gray_coevaluation_R_func emdash3_2_gray_right_closure.lp	GrayTensor_R examples/gray_right_closure.lp gray_right_closure_omega examples/gray_right_closure.lp gray_curry_R_is_strict examples/gray_right_closure.lp gray_coevaluation_R_func examples/gray_right_closure.lp

GRAY-WALKING-INTERCHANGER	checked	In the selected strict-object/lax-arrow Gray right-closure slice, the walking square has four coevaluation-derived vertices and two coordinate arrow families, while its oriented nonidentity interchanger is projected from the existing whole post/left laxity action and retains one next hom action.	gray_square_in ner_src_arrow emdash3_2_gray _walking_squar e.lp gray_square_ou ter_src_arrow emdash3_2_gray _walking_squar e.lp gray_interchan ger emdash3_2_gray _interchanger. lp gray_interchan ger_next_func emdash3_2_gray _interchanger. lp	gray_interchan ger examples/gray_ interchanger.l p gray_interchan ger_next_func examples/gray_ interchanger.l p
SEMISIMPLICIAL-FACE-SUBSTRATE	checked	Injective skip/keep face codes compute, form the locally discrete augmented semi-simplex category, realize selected ordinal shapes by directed join, and define Yoneda standard semisimplices and whole groupoid-valued semisimplicial diagram realization with retained higher action.	FaceCode emdash3_2_semi simplicial_fac e_codes.lp SemiDeltaPlus_ cat emdash3_2_semi simplicial_ind ex.lp DirectedSimple x_cat emdash3_2_simp lex_shapes.lp StandardSimple x emdash3_2_semi simplicial_dia grams.lp	face_vertex_ze ro examples/semis implicial_face _codes.lp semi_delta_edg e_zero_one examples/semis implicial_inde x_category.lp text Ordinary dimension n means n+1 ver tices examples/simpl ex_shapes.lp semisimplicial _grpdp_realized _face_func examples/semis implicial_diag rams.lp

DEPENDENT - SIMPLEX - INTERNAL - ACTION	checked	The fixed-endpoint dependent hom is the existing hom of a Sigma total and retains whole base-arrow and transported-endpoint observations; the recursive PathOut triangle category has whole target-line and base-line projections whose hom actions expose the 023 and 123 faces, while the next internal action maps a visible dependent tetrahedron and remains iterable rather than adding an independent coherence record.	DependentTriangle_cat emdash3_2_dependent_simplex_bridge.lp dependent_triangle_boundary_face_func emdash3_2_dependent_simplex_bridge.lp dependent_simplex2_boundary_line_func emdash3_2_dependent_simplex_faces.lp dependent_tetrahedron_map emdash3_2_dependent_simplex_bridge.lp	text The triangle classifier is the active Hom(Sigma) presentation examples/dependent_simplex_bridge.lp text A further hom action remains available examples/dependent_simplex_bridge.lp dependent_simplex2_boundary_target_action_path_test examples/dependent_simplex_faces.lp dependent_simplex2_boundary_base_action_path_test examples/dependent_simplex_faces.lp
ORDINAL - DEPENDENT - FOUR-SIMPLEX	checked	One canonical ordinal four-simplex is constructed by iterating the whole PathOut transformation lift, maps under every functor from Delta[4], exposes all five tetrahedral cofaces through generic FaceCode action, supports selected strict and exact Path targets, remains noncollapsed, and retains one next action.	pathout_transf_lift emdash3_2_pathout_transformation_lift.lp ordinal_dependent_simplex4_source emdash3_2_dependent_simplex_ordinal_dimension4.lp ordinal_dependent_simplex4_face emdash3_2_dependent_simplex_ordinal_dimension4.lp ordinal_simplex4_top_next_action emdash3_2_dependent_simplex_ordinal_dimension4.lp	pathout_transf_component examples/pathout_transformation_lift.lp ordinal_dependent_simplex4_source examples/dependent_simplex_ordinal_dimension4.lp ordinal_dependent_simplex4_face1234 examples/dependent_simplex_ordinal_dimension4.lp

ORDINAL-DEPENDENT-SIMPLEX-RECURSION	checked	Nat recursion constructs a canonical intrinsic ordinal dependent-simplex source at variable dimension; its structural successor steps the code at F[s] and stores the transformation component epsilon[s], while arbitrary-target observation, generic nonempty-face access, selected computations through dimensions zero to four, noncollapse, and a retained next action are checked.	ordinal_dependent_simplex_source emdash3_2_dependent_simplex_ordinal_recursive.lp ordinal_dependent_simplex_observation emdash3_2_dependent_simplex_ordinal_recursive.lp ordinal_dependent_simplex_face emdash3_2_dependent_simplex_ordinal_recursive.lp ordinal_dependent_simplex_lift_next_action emdash3_2_dependent_simplex_ordinal_recursive.lp	OrdinalDependentSimplexSource examples/dependent_simplex_ordinal_recursive.lp ordinal_dependent_simplex_observation examples/dependent_simplex_ordinal_recursive.lp ordinal_dependent_simplex_lift_next_action examples/dependent_simplex_ordinal_recursive.lp
EH-COMMUTATIVITY	checked	Two 2-endomorphisms of an identity 1-cell commute in the selected Eckmann-Hilton slice.	EH_comm emdash3_2.lp	text Eckmann-Hilton specialization emdash3_2_checks.lp
ADDITIVE-HOM-BIPRODUCTS	checked	Set-valued abelian Hom structures and bilinear composition combine with selected cartesian structure to give biproducts and a terminal/initial zero object.	PreadadditiveCategory emdash3_2_preadditive_categories.lp AdditiveCategory emdash3_2_additive_categories.lp	AdditiveCategory examples/additive_categories.lp comm_ring_finite_free_additive examples/commutative_ring_finite_free_additive.lp

INTERNAL - KERNEL - COKERNEL - FACTORS	checked	Kernels and cokernels use contractible internal Hom-factor fibres, with selected factors, reconstruction, uniqueness and comparison maps between supplied choices.	HomPostcompFactor emdash3_2_hom_factor_spaces. lp HomPrecompFactor emdash3_2_hom_factor_spaces. lp ComputationalKernel emdash3_2_kernels_cokernels. lp ComputationalCokernel emdash3_2_kernels_cokernels. lp computational_kernel_maps_is o emdash3_2_kernel_map_isos. lp computational_cokernel_maps_iso emdash3_2_cokernel_map_isos. lp	ComputationalKernel examples/kernel_maps_cokernels. lp computational_kernel_map examples/kernel_maps. lp computational_cokernel_map examples/cokernel_maps. lp
---	---------	--	---	--

ABELIAN-NORMALITY-IMAGES	checked	Computational Abelian normality constructs the coimage-image comparison, proves it monic and epic, and constructs its isomorphism without identifying the selected objects by fiat.	ComputationalAbelianCategory emdash3_2_abelian_categories .lp coimage_image_comparison emdash3_2_abelian_images.lp coimage_image_comparison_is_monic emdash3_2_abelian_image_bimorphisms.lp coimage_image_comparison_is_epic emdash3_2_abelian_image_bimorphisms.lp computational_abelian_coimage_image_iso emdash3_2_abelian_bimorphisms.lp	coimage_image_comparison examples/abelian_images.lp abelian_coimage_image_comparison_is_monic examples/abelian_image_bimorphisms.lp computational_abelian_coimage_image_iso examples/abelian_bimorphisms.lp
SELECTED-HOMOLOGY-MAPS	checked	Selected homology retains its cycle kernel and boundary cokernel; chain-pair maps induce maps on these homologies with reconstruction and functoriality paths.	ComputationalHomologyAt emdash3_2_computational_homology.lp ComputationalChainPairMap emdash3_2_chain_pair_maps.lp computational_homology_map emdash3_2_homology_maps.lp	ComputationalHomologyAt examples/computational_homology.lp computational_homology_map examples/homology_maps.lp

SELECTED- EXACTNESS- COVERS	checked	Exactness is epicity of the selected boundary; local epic-cover factorization characterizes it, and the dual local-extension family is a sufficient criterion.	ComputationalE xactAt emdash3_2_shor t_exact_sequen ces.lp computational_ exact_at_from_ covers emdash3_2_exac tness_covers.l p computational_ exact_covers_f rom_exact emdash3_2_exac tness_covers_f rom_exact.lp computational_ exact_at_from_ extensions emdash3_2_exac tness_extensio ns_to_exact.lp short_exact_ke rnel_compariso n_iso emdash3_2_shor t_exact_kernel_ comparison.lp short_exact_co kernel_compari son_iso emdash3_2_shor t_exact_cokern el_comparison. lp	ComputationalE xactAt examples/short _exact_sequenc es.lp short_exact_ke rnel_compariso n_iso examples/short _exact_compari sons.lp abelian_snake_ exact_at_coker_ alpha examples/abeli an_snake_exact_ third.lp
-----------------------------------	---------	--	--	--

ABELIAN-FIBER-PUSHOUT-STABILITY	checked	Difference kernels and cokernels construct additive fibre products and pushouts; Abelian normality proves the corresponding pullback-epic and pushout-monic stability.	ComputationalFiberProduct emdash3_2_computational_fiber_products.lp ComputationalPushout emdash3_2_computational_pushouts.lp abelian_fiber_product_epic_projection emdash3_2_abelian_fiber_pushout_stability.lp abelian_pushout_moniconjecture emdash3_2_abelian_fiber_pushout_stability.lp	ComputationalFiberProduct examples/computational_fiber_products_pushouts.lp abelian_fiber_product_epic_projection examples/abelian_fiber_pushout_stability.lp
ABELIAN-SNAKE-CONNECTING	checked	A triple with zero composite constructs the snake connecting arrow, both normal factors and their reconstructions, including the original covered lambda-kernel factor.	AbelianSnakeTriple emdash3_2_abelian_snake_lemma.lp abelian_snake_connecting_result emdash3_2_abelian_snake_connecting_result.lp abelian_snake_connecting_after_p1_path emdash3_2_abelian_snake_covered_reconstruction.lp	abelian_snake_connecting examples/abelian_snake_connecting.lp abelian_snake_connecting_after_p1_path examples/abelian_snake_covered_reconstruction.lp

ABELIAN-SNAKE-SIX-TERM-EXACT	checked	The selected generic six-term snake has five maps, four adjacent-zero paths and exactness at its four interior objects, indexed by the original snapshot.	AbelianSnakeSixTermExactResult emdash3_2_abelian_snake_six_term_exact_result_foundation.lp abelian_snake_six_term_exact_result emdash3_2_abelian_snake_six_term_exact_result.lp	abelian_snake_six_term_exact_result examples/abelian_snake_six_term_exact_result.lp
HOMOLOGY-COKERNEL-INCLUSION	checked	The original cycle embedding induces H into the supplied cokernel of the incoming differential; this map is monic in a computational Abelian category.	computational_homology_cokernel_inclusion emdash3_2_homology_cokernel_inclusion.lp abelian_homology_cokernel_inclusion_monicemdash3_2_abelian_homology_cokernel_inclusion.lp	abelian_homology_cokernel_inclusion_monicemdash3_2_homology_cokernel_inclusion.lp
FREYD-FORMAL-RAW-SPINE	checked	Explicit matrix introductions construct original presentation morphisms and raw chain agreements, which assemble through the existing bounded Freyd-spine constructors.	comm_ring_presentation_morphism_from_matrices emdash3_2_commutative_algebra_freyd_explicit_spines.lp comm_ring_freyd_chain_pair_from_matrices emdash3_2_commutative_algebra_freyd_explicit_spines.lp CommRingFreydBoundedComplex emdash3_2_commutative_algebra_freyd_bounded_complexes.lp	comm_ring_freyd_chain_pair_from_matrices examples/commutative_ring_freyd_explicit_spines.lp

FREYD-FORMAL-BOUNDARY-EPICITY	checked	The identity $QU + FV = \text{id}$ and the original relation-preservation law construct the existing formal Freyd epimorphism witness at that raw morphism.	comm_ring_freyd_epimorphism_from_matrices emdash3_2_commutative_algebra_freyd_explicit_epimorphisms.lp	comm_ring_freyd_epimorphism_from_matrices examples/commutative_ring_freyd_explicit_epimorphisms.lp
FREYD-SELECTED-HOMOLOGY-EXACTNESS	checked	The two supplied whole weak-pullback providers construct the selected kernel; the actual raw boundary and reconstruction construct its homology, and its existing epicity witness is exactness at that same boundary.	CommRingFreydKernelChoices emdash3_2_commutative_algebra_freyd_kernel_choices.lp CommRingFiniteFreeWeakPullbackMatrix-Provider emdash3_2_commutative_algebra_freyd_kernel_choice_providers.lp CommRingFreydSelectedHomologyAt emdash3_2_commutative_algebra_freyd_selected_homology.lp comm_ring_freyd_exactness_from_matrix_providers emdash3_2_commutative_algebra_freyd_actual_homology.lp	comm_ring_freyd_kernel_choices_from_matrix_providers examples/commutative_ring_freyd_kernel_choice_providers.lp comm_ring_freyd_exactness_from_matrix_providers examples/commutative_ring_freyd_actual_homology.lp

WHOLE - KERNEL - COKERNEL - PRESENTATIONS	checked	Supplied coherent kernel/cokernel presentations provide whole functors and adjunctions over the original selected operations; mate and universal-record interfaces retain those objects with proof-time usability.	KernelPresentation emdash3_2_kernel_adjunction_presentations. lp CokernelPresentation emdash3_2_cokernel_adjunction_presentation_s. lp kernel_presentation_record emdash3_2_kernel_adjunction_records. lp cokernel_presentation_record emdash3_2_cokernel_adjunction_records. lp	kernel_presentation_adjunction examples/kernel_adjunction_presentations. lp cokernel_presentation_adjunction examples/cokernel_adjunction_presentations. lp kernel_presentation_record examples/kernel_adjunction_records. lp cokernel_presentation_record examples/cokernel_adjunction_records. lp
WHOLE - HOMOLOGY - FUNCTOR	checked	Whole homology is derived from the boundary transformation $K(h)$ composed with the unit and the whole cokernel functor; its native one-degree source uses the ordinary-target profile, and record views retain the actual H objects.	kernel_family_boundary_transf emdash3_2_homology_families. lp homology_family_func emdash3_2_homology_families. lp zero_arrow_cone_homology_func emdash3_2_zero_arrow_cone_homology. lp homology_family_record emdash3_2_homology_family_records. lp	homology_family_func examples/homology_families. lp zero_arrow_cone_homology_func examples/one_cat_zero_cone_homology. lp homology_family_record examples/homology_family_records. lp

HOMOLOGY-DIRECT-CONNECTING	checked	The direct homology connecting construction derives its normal tests, covered factor and source-boundary descent at the retained H records, with reconstruction and no endpoint transport.	homology_record_connecting emdash3_2_homology_record_connecting.lp homology_record_connecting_covered_reconstruction emdash3_2_homology_record_connecting.lp	homology_record_connecting examples/homology_record_connecting.lp homology_record_connecting_examples/homology_record_connecting_whole.lp
HOMOLOGY-WINDOW-EXACTNESS	checked	The actual five-term whole-H window has all three adjacent-zero laws and exactness at all three interior objects, packaged over the same retained arrows.	homology_whole_window_first_exact emdash3_2_homology_first_exactness.lp homology_whole_window_second_exact emdash3_2_homology_second_exactness.lp homology_whole_window_third_exact emdash3_2_homology_third_exactness.lp homology_whole_exact_window emdash3_2_homology_exact_window_result.lp	homology_whole_connecting_projection_zero examples/homology_window.lp homology_whole_window_first_exact examples/homology_first_exactness.lp homology_whole_window_second_exact examples/homology_second_exactness.lp homology_whole_window_third_exact examples/homology_third_exactness.lp homology_whole_exact_window examples/homology_exact_window.lp

HOMOLOGY-CONNECTING-TRANSFORMATION	checked	A whole connecting transformation between H of the two native vertical columns has the retained direct component computation and generic off-diagonal and higher action; no caller-supplied naturality square is added.	homology_window_source_homology_func emdash3_2_homology_window_connecting_transformation.lp homology_window_target_homology_func emdash3_2_homology_window_connecting_transformation.lp homology_window_connecting_transf emdash3_2_homology_window_connecting_transformation.lp	homology_window_connecting_transf examples/homology_window_connecting_transformation.lp homology_window_right_diagram_action_path examples/homology_window_column_usability.lp
HOMOLOGY-BOUNDED-ITERATOR	checked	A field-indexed Nat iterator assembles finitely many coherent whole- H windows, retaining original arrows and zero/exactness annotations. Separate same- H zero-middle lemmas are checked; the final symbolic endpoint attachment is not claimed.	HomologyRowFieldSpan emdash3_2_homology_row_field_spans.lp ComputationalExactArrowTail emdash3_2_computational_exact_arrow_tails.lp homology_row_field_span_exact_tail emdash3_2_homology_bounded_generator.lp homology_whole_middle_zero emdash3_2_homology_whole_zero.lp	homology_row_field_span_exact_tail examples/homology_bounded_generator.lp generated2 examples/homology_bounded_generator_arrows.lp homology_whole_terminal_middle_zero examples/homology_whole_zero.lp

FREYD-WHOLE-HOMOLOGY-MODEL	checked	A supplied coherent Freyd model exposes whole H, original object/map observations and, with supplied normality and model-side short-exact rows, the direct connecting observation without replacing its universal selections.	FreydHomologyModel emdash3_2_commutative_algebra_freyd_homology_models.lp freyd_homology_model_map emdash3_2_commutative_algebra_freyd_homology_model_maps.lp FreydHomologyModelNormality emdash3_2_commutative_algebra_freyd_homology_model_normality.lp freyd_homology_model_connecting emdash3_2_commutative_algebra_freyd_homology_model_connecting.lp	freyd_homology_model_func examples/freyd_homology_models.lp freyd_homology_model_map examples/freyd_homology_model_maps.lp freyd_homology_model_abelian examples/freyd_homology_model_normality.lp freyd_homology_model_connecting examples/freyd_homology_model_connecting.lp
FREYD-RAW-WITNESS-INTRO	checked	A retained coefficient matrix and its adopted equation construct the existing raw presentation-agreement value at literal source and target presentations.	CommRingPresentationAgreementAtMatrices emdash3_2_commutative_algebra_freyd_explicit_spines.lp comm_ring_presentation_agreement_from_matrices emdash3_2_commutative_algebra_freyd_explicit_spines.lp	comm_ring_presentation_agreement_from_matrices examples/commutative_ring_freyd_raw_witnesses.lp

Appendix C. From The Circle To The Walking Endomorphism—And Back

The proof of Theorem 8.1 is inspired by the encode-decode calculation of the loop space of the circle in the *Homotopy Type Theory* book. This appendix records the correspondence so that the analogy can guide the reader without smuggling groupoidal assumptions into the directed theorem.

The comparison is no longer merely retrospective or prospective. Chapter 26 checks the Circle side of the analogy with a successor-localized Integer classifier, while Chapter 27 proves that mapping out of the Circle into a groupoid is equivalent, as a whole mapping object, to mapping out of WalkingEnd through paths. The appendix therefore records both the deliberate change from inverse to forward powers and the checked free-inversion passage back.

The source reference is revision `578b85cc8d586b1677ec4335148adeb443057d24` of the HoTT Book repository. The detailed, machine-readable adaptation ledger is `book/references/third-party-sources.json`.

C.1 The Rhetorical Spine

The HoTT calculation proceeds through four questions:

1. What are the evident powers of the loop, and how might a loop be assigned a winding number?
2. What does the classical free/universal-cover picture predict?
3. How can the cover be represented internally as a family?
4. How do encode and decode become inverse after the endpoint is generalized?

Chapter 8 retains that sequence. It changes the mathematical answer at every point where the circle proof uses reversibility.

Circle calculation	Walking-endomorphism calculation
A base point and an identity loop in <code>S¹</code>	A base object and a directed arrow <code>ell</code> in <code>W</code>
Loop carrier <code>base = base</code>	Based hom carrier <code>Hom_W(*, *)</code>
Integer-indexed positive and negative powers	Natural-number-indexed forward powers
A cover with integer fibre	A Cat-valued code with <code>Path(Nat)</code> base fibre
Successor equivalence on integers	Successor functor on naturals
A universe path obtained through univalence	A directed endofunctor accepted directly by recursion into <code>Cat</code>
Transport zero along an identity path	Apply the Code functor to zero along a directed arrow
Decode by dependent circle induction	Decode by the contextual displayed eliminator
Hard inverse by generalized path induction	Directed normalization cell, then hom-discreteness
Easy inverse by integer/circle induction	Easy inverse by Nat induction

$\Omega(S^1) \approx \text{Int}$	$\text{Hom}_W(*, *) \approx \text{Nat}$ as carriers
Group structure and inverse loops	Free-monoid direction and a noninvertible generator

The table is a dependency map, not a dictionary identifying the objects.

C.2 Where Univalence Moves

In the circle proof, a family $\text{Code} : S^1 \rightarrow \text{Type}$ must map the loop of S^1 to a loop in the universe. Successor on the integers is an equivalence, and univalence converts that equivalence into the needed universe identity. Its inverse supplies predecessor action.

In the directed proof,

$$\text{Code} : W \longrightarrow \text{Cat}$$

is specified by a category and an endofunctor. Natural-number successor is therefore accepted without being an equivalence. No universe identity and no predecessor are needed.

This does not make univalence irrelevant to functorial type theory. It clarifies its jurisdiction. Equality, transport, equivalence, and univalent universes govern groupoidal identification. A directed family may additionally act along arrows that are not identities and whose action is not invertible. The title of this book refers to a foundation containing both layers.

Formal status — mathematical development. The comparison of the roles of univalence is explanatory. The individual emdash interfaces cited by Chapter 8 are checked; a general metatheorem comparing all directed families with HoTT fibrations is not claimed.

C.3 The Generalized Endpoint

Both proofs teach the same strategic lesson in different formal languages: when a statement about a fixed loop cannot be inducted on, vary its endpoint.

For the circle, one considers all $x : S^1$ and paths $\text{base}=x$. For W , one considers all objects x and based arrows $* \rightarrow x$. The target family is the representable

$$x \longmapsto \text{Hom}_W(*, x).$$

The crucial difference is what generalized elimination produces. Groupoidal path induction produces equality. The contextual directed eliminator first produces coherent arrow action, and at an arbitrary based arrow it yields a directed 2-cell

$$p \longrightarrow \text{decode}_x(\text{encode}_x(p)).$$

Only the separate one-dimensionality premise converts that cell into equality. Thus the directed proof factors the groupoidal conclusion into two conceptually independent steps: normalization and dimension.

C.4 What Was Not Transferred

The adaptation removes, rather than renames, the following ingredients:

- inverse paths and negative powers;
- predecessor action on the code;
- the claim that successor is an equivalence;
- a universe path produced from successor;
- cancellation in a group;
- equality as the first output of the hard inverse;
- the conclusion that the result is already a group isomorphism.

It also adds ingredients absent from the circle proof in this form:

- a separate `BNat` model whose definitional structure is not imposed on the HIT;
- a displayed contextual algebra comparing postcomposition with successor;
- a coherent directed spiral;
- a dimension witness used explicitly at the equality boundary;
- negative results proving the generator has no right inverse or omega-equivalence evidence.

These changes explain why a textual search-and-replace from `S1/Int` to `W/Nat` would be mathematically misleading.

C.5 Stronger Comparisons

The carrier equivalence suggests two different strengthenings.

First, composition/addition compatibility should package the comparison as a monoid isomorphism. Chapter 8 records this as a formal consequence of the checked recursion and inverse laws, but the library does not yet expose the package.

Second, free groupoidal realization should invert the generator without silently adding an inverse inside the original directed category. That comparison is now active. The functor from `WalkingEnd` to the `Circle` sends natural powers to nonnegative `Circle` powers, and restriction along it is a whole mapping equivalence from `Circle` maps into a groupoid to path-valued `WalkingEnd` functors.

Formal status — checked. Evidence `WE-GROUP-COMPLETION`. This is a categorical universal property, not merely a carrier comparison with an integer one-object category. Chapter 27 develops the theorem and its category-indexed successor.

A reverse `BNat` functor, a packaged monoid isomorphism for the Chapter 8 carrier theorem, source functoriality of generic groupoidification, and the resulting adjunction remain future layers. Keeping those claims separate is precisely why the completed `Circle` comparison does not retroactively turn the directed `Nat` proof into a group-valued proof.

C.6 Attribution Method

The subsection order and proof strategy are structural adaptations from the pinned HoTT sources. The prose in this edition is newly written. Each target, source file, source label, and adaptation type is recorded before the adaptation is committed. The book's CC BY-SA 3.0 license and credits preserve the upstream attribution and ShareAlike requirements.

Appendix D. Glossary And Concept Index

This appendix fixes the vocabulary used by the expanded development edition. Each entry points to the place where the idea is constructed or used, rather than to a page number that would change with paper size and typography.

D.1 Glossary

Abelian category, computational. An additive category with selected kernels and cokernels and normal lift/colift operations. Monic and epic cancellation are distinguished from the normality operations that construct factors. See [Chapter 31](#).

Additive category. A category with set-valued abelian Hom groups, bilinear composition and selected finite biproducts. The implementation derives biproducts from its supplied cartesian structure. See [Chapter 31](#).

Affine chart realization. A selected region $U \rightarrow X$ together with a supplied reflective presentation of the actual slice $\mathcal{K}/U$, a coordinate ring and thin affine presentation, a whole affine-basis functor, and a whole comparison from ambient restriction to affine coordinates. The label is site-relative and is not inferred from cover membership alone. See [Chapter 23](#).

Affine scheme, computational presentation. A base ring together with a supplied reflective structure-sheaf presentation on its generated big Zariski site and supplied whole localization locality for the coordinate presheaf. The current interface is assumption-explicit and does not construct sheafification, stalks, or a representation-independent category of affine schemes. See [Chapter 22](#).

Binary site-relative scheme presentation. One supplied global reflective ringed object with topology-local ring behaviour and a covering sieve constructively generated by two whole affine chart realizations. The global structure presheaf is retained once, so restrictions and selected overlaps are inherited rather than duplicated as atlas fields. See [Chapter 23](#).

Adjunction. Functors $F : A \rightarrow B$ and $G : B \rightarrow A$ equipped either with a unit and counit satisfying the two triangle laws or with an equivalent natural hom comparison. In the active calculus, the triangles are universal cuts with selected computational owners. See [Chapter 12](#).

Arrow induction. Extension of data at the reflexive outgoing arrow to a section over `PathOut`. Unlike equality induction, its base category may contain noninvertible arrows. See [Chapter 5](#).

Basic open. Primarily, the ordinary sieve $D_R(f)$ of maps $R \rightarrow S$ that make f invertible. A supplied localization $R \rightarrow R[1/f]$ represents its points at every test ring; a compact open is a further representation when available. See [Chapters 18](#) and [22](#).

Based hom-category. For a selected base object $*$ and endpoint x , the category $H_x = \text{Hom}_W(*, x)$. Its objects are based 1-arrows and its arrows are based 2-cells. See [Chapter 8](#).

BNat. The separate one-object category with $\mathbb{N}$ -valued hom, zero identity, and addition composition. It is a concrete model of the walking signature, not the definition of WalkingEnd. See [§8.1.2](#).

Canonical mathematical surface. The readable notation in which the book states categorical judgments and rule schemas. It maps to stable kernel owners and has a bounded executable subset, but the full notation is broader than the implemented text grammar. See [Appendix G.5](#).

Carrier equivalence. An equivalence between underlying classifiers, here $\text{Hom}_W(*, *) \simeq \mathbb{N}$. It does not by itself package preservation of composition or an equivalence of ambient categories. See [Theorem 8.1](#).

Cat-valued directed family. A functor $E : K \rightarrow \mathbf{Cat}$. It assigns a category $E[k]$ to each base object and a transport functor $E[p]$ to each directed base arrow. See [Chapter 2](#).

Categorical height. The recursive `IsNCat` condition: dimension zero is discreteness, and successor dimension asks every hom-category to have the preceding dimension. See [Chapter 7](#).

Category, native. An object of `Cat`, with iterable category-valued homs. It is not definitionally an ordinary HoTT precategory. See [Chapters 2](#) and [10](#).

Chosen binary product. A selected whole functor $P : C \times C \rightarrow C$ together with whole projections and a represented-family pairing operation. Its triangular beta, distribution, and eta cuts compute. It is distinct from the product category constructor. See [Chapter 30](#).

Circle. The selected groupoidal HIT with one point and one generating path. Its dependent eliminator computes at the point and on dependent action over the loop; its based loop carrier is equivalent to the successor-localized Integer classifier. See [Chapter 26](#).

Compositor. The directed comparison $F[g] \circ F[f] \Rightarrow F[g \circ f]$ obtained by specializing whole transfer laxity to the identity transfer of F . It may become invertible in a path target; selected `IsStrictFunctor` evidence instead compares it with an equality-induced arrow without globally erasing it. See [Chapters 25](#) and [28](#).

Commutative ring. A set-valued carrier with zero, one, addition, negation, multiplication, and the usual commutative-ring laws. The zero ring is retained; structured maps preserve all five operations and form **CRing**. See [Chapter 21](#).

Code. The Cat-valued family over WalkingEnd whose base fibre is $\text{Path}(\mathbb{N})$ and whose generator action is successor. See [§8.1.3](#).

Contractible factor space. The classifier of structure-preserving maps and their required triangles, equipped with a center and a path from every competitor to it. It turns a universal property into both a selected factor and coherent uniqueness. See [Chapter 21](#).

Co-Yoneda cut. Elimination of a representable leg from a profunctor composite. The checked theorem is a shaped, fixed-middle beta/fusion law; a general coend theorem remains separate. See [Chapter 13](#).

Cover. An ordinary sieve selected as locally sufficient by a Grothendieck topology. A family of arrows can present or generate a covering sieve while retaining separate computational witnesses. See [Chapter 19](#).

Cover generation, binary and witness-rich. For every member $q : V \rightarrow X$ of one retained covering sieve, an executable Boolean branch, factor map $V \rightarrow U_b$, and triangle $q = u_b h$ through one of two selected members. It explains the retained sieve without asserting that every member is itself affine. See [Chapter 23](#).

Contextual eliminator. An eliminator that constructs a displayed functor between two varying families from base-fibre data and coherent constructor cells. Sections and ordinary recursors are special cases. See [Chapter 6](#).

Cut elimination. Controlled normalization at the semantic owner of an arrow, family, structural, or universal cut. It does not mean installing unrestricted associativity as a global rewrite. See [Chapter 9](#).

Dagger structure. A chosen contravariant involution on one category, identity on objects in the ordinary presentation and coherent with retained higher action in a prospective native presentation. It is not merely the operation of taking an arbitrary opposite category. See [Chapter 14](#).

Dependent simplex. A recursively flagged object obtained by choosing an object of C , then an object of its outgoing-path category, and continuing. Each successor is a `PathOut`, hence a Sigma of a representable hom; its arrows expose a base cell and a dependent cell above transport. The active variable-dimensional object package is not yet a whole category of all dependent simplexes. See [Chapter 29](#).

Dependent product in slices. A selected whole functor $\Pi_u : C/X \rightarrow C/Y$ right adjoint to pullback base change $u^* : C/Y \rightarrow C/X$. It is distinct from the global section category `Pi_cat` and from a general right-Kan direct image. See [Chapter 30](#).

Directed higher-inductive type/category. A presentation with object, directed-arrow, and possibly higher-cell constructors whose arrow generators are not silently inverted. The current book has one selected implementation, `WalkingEnd`, not a general schema. See [Chapter 6](#).

Directed normalization cell. The cell $p \rightarrow \text{decode}(\text{encode}(p))$ constructed by the contextual decoder before hom-discreteness extracts equality. See [§8.1.4](#).

Discrete category. A category whose hom structure agrees with equality of objects at the selected interface. In a one-dimensional category every hom-category is discrete; this does not make every ambient 1-arrow invertible. See [§7.6](#).

Duality. Either a proof method that transports a theorem through the opposite construction or additional structure comparing a category with an opposite. It never licenses an unannounced variance reversal. See [Chapters 14](#) and [17](#).

Elaborator. The implemented bounded TypeScript layer that interprets directly constructed or parsed categorical surface terms against expected classifiers, selects stable owners, and emits backend-neutral explicit Core. It fails closed when a required coherent construction is absent. It is not a compiler for the whole book surface or a second mathematical kernel. See [Appendix G.5](#).

Evidence status. One of checked, formal consequence, mathematical development, or research boundary. The status describes the relation between prose and the active artifact. See [How to Read](#) and [Appendix B](#).

Face code. A set-classified skip/keep word representing an injective monotone map between finite ordinals. Identity is the all-keep word and composition computes by structural substitution. Face codes are the homs of the augmented semi-simplex category. See [Chapter 29](#).

Explicit emdash Core. The backend-neutral representation produced after elaboration has selected logical and categorical owners and made their arguments explicit. The generic TypeScript LF checks and reduces it; optional deterministic Lambdapi emission is a conformance route, not its definition. See [Appendix G.5](#).

Formal presentation. The layered account relating the canonical mathematical surface, bounded contextual elaboration and outer-LF declaration conveniences, backend-neutral explicit Core, the generic TypeScript LF, the active Lambdapi authority, and separately stated semantic models. The categorical kernel comes first; it is not post-hoc semantics for an unspecified traditional syntax. See [Appendix G](#).

Functor. A map with object and iterated-hom action. Generic functoriality, not constructor-specific laws, owns identity and composition preservation. See [Chapter 2](#).

Generated topology. The least Grothendieck topology accepting a selected type-valued family of generating sieves. The active presentation is the intersection of all accepting topologies, not an inductive derivation syntax. See [Chapter 19](#).

Global-first scheme architecture. An approach that begins with one already-existing global ringed object and recognizes selected regions as covering affine charts. Restriction and overlap coherence are inherited from the global presheaf; constructing the object from abstract chart data remains a separate gluing theorem. See [Chapter 23](#).

Group completion. Free adjoining of inverse motion. The active WalkingEnd–Circle theorem gives this comparison as a whole universal mapping property, and category-indexed groupoidification extends the construction to an arbitrary source category at the target-side mapping boundary. See [§8.1.5](#) and [Chapter 27](#).

Groupoidification. The free realization of directed objects, arrows, and higher cells as groupoidal points, paths, and higher paths. It differs from the core, which retains arrows already invertible, and from truncation, which lowers the homotopy level of groupoidal data. Source functoriality and the packaged adjunction remain future interfaces. See [Chapter 27](#).

Gray profile, selected. The computational full-subcategory facade $\text{GrayHom}_{\text{lax}}(A, B)$ whose objects are ordinary functors paired with `IsStrictFunctor` evidence and whose arrows and higher homs reuse the ambient transfor tower. One checked right closure yields a walking-square interchanger; a full Crans–Gray biclosed monoidal structure is not claimed. See [Chapter 28](#).

Hom action. The functorial action induced on a hom-category. Emdash keeps covariant postcomposition, contravariant precomposition, and simultaneous two-endpoint action as distinct computational owners. See [Chapters 2, 9, and 13](#).

Higher sieve. A Cat-valued coefficient system on the restriction-oriented category of probes into a fixed object; equivalently, a Cat-valued presheaf on the conventional slice. Its values may retain witnesses and arrows between them; it is not automatically an ordinary sieve. See [Chapter 18](#).

Invertibility sieve. For a section s over U , the sieve $D_U(s)$ of all probes $p : V \rightarrow U$ for which p^*s is a unit. An open may represent this sieve, but representability is a further claim. See [Chapter 18](#).

Integer line. The set-truncated telescope localization of natural-number successor. A stage pair (n, x) represents $x - n$; executable successor and predecessor are inverse, and the resulting self-equivalence supplies Circle monodromy. See [Chapter 26](#).

Interchanger. A directed higher cell comparing the two coordinate routes across the selected Gray walking square. In the checked slice it is projected from whole post/left laxity and retains a next action; it is not postulated as an isolated square axiom. See [Chapter 28](#).

Interval, groupoidal. The selected HIT with endpoints i_0, i_1 and one generating path between them. It is equivalent to the groupoidification of the directed WalkingArrow, without the two carriers being definitionally identified. See [Chapter 27](#).

Join. The selected directed category generated by left and right embeddings together with cross arrows from the left side to the right side. Its recursor and three beta observations are checked; a general collage mapping property and dependent eliminator are not. See [Chapter 17](#).

Kan extension. A universal extension along a functor. This edition expresses right and left Kan interfaces as conjoint- and companion-weighted limits and colimits; identifying those interfaces with the full standard pointwise semantics remains mathematical development. See [Chapters 16 and 17](#).

Lower-star action. Postcomposition: if $g : w \rightarrow x$ and $u : x \rightarrow y$, then $u_*(g) = u \circ g : w \rightarrow y$. Its active owners are `hom_postcomp_func` and `hom_postcomp_fapp0`. See [§9.2](#).

Localization at an element. A structured map $R \rightarrow R[1/f]$ that makes f invertible and has a contractible factor space through every map in which the image of f is already invertible. The notation names a universal role, not a required fraction representation. See [Chapter 21](#).

Laurent overlap. Two literal chart restrictions into one actual overlap ring, each supplied as localization of a one-variable polynomial algebra over one common base, together with whole paths identifying both internally constructed coordinate-inversion endomorphisms with the overlap identity. The transition maps are constructed; the identity paths are supplied. See [Chapter 24](#).

Laxity cell. A directed comparison retained where a strict equation would identify its endpoints. Whole laxity varies over an entire hom category, so its component and next action arise from one internal operation rather than a finite list of unrelated coherence fields. See [Chapter 28](#).

Matching family. A whole presheaf map $\widehat{R} \rightarrow X$ assigning local data to every member of a sieve R , compatibly with refinement. A global section restricts to a matching family by precomposition. See [Chapter 19](#).

Monad, computational presentation. An endofunctor T with whole unit and multiplication observations and a whole extension operation $f \mapsto f^*$. The ambient beta and accumulation cuts normalize extension without requiring an explicit Kleisli category. See [Chapter 12](#).

Natural transformation. In the ordinary specialization, a pointwise family $\alpha_x : F(x) \rightarrow G(x)$ satisfying a naturality equation. A native transfor retains an off-diagonal action and iterates to higher hom levels, so the two notions are related but not definitionally identical. See [Chapter 11](#).

Off-diagonal transfor action. If $\eta : F \Rightarrow G$ and $f : x \rightarrow y$, then $\eta[f] : F(x) \rightarrow G(y)$. Adjacent functor actions accumulate into this term by strict naturality. See [Chapter 9](#).

Omega-equivalence. The native recursive equality-valued equivalence interface for categorical cells. It is distinct from a bare carrier `TypeEquiv` and from ordinary isomorphism evidence. See [Chapter 4](#).

Opposite category. The active arrow-reversing construction $C \mapsto C^{\text{op}}$. Opposite duality exchanges selected limit and colimit interfaces while preserving a visible variance ledger. See [Chapter 14](#).

Ordinary sieve. A refinement-closed, proposition-valued family of probes into one object. In the active categorical presentation it is a higher sieve whose every coefficient category is subterminal. See [Chapter 18](#).

Outer-LF declaration convenience. A typed host operation that validates selected higher-level input and expands it into ordinary dependent-LF declarations and rules before explicit Core is checked. The current adjunction and bounded dependent-structure forms add no trusted term node or new categorical semantics. See [Appendix G.5](#).

Path category. $\text{Path}(A)$, the equality-local groupoidal category on a classifier A . It embeds ordinary identity reasoning into the directed calculus without identifying every directed hom with equality. See [Chapters 2](#) and [25](#).

Path-realized pseudo-laxity. The generic directed compositor viewed in a path category. Its underlying cell is an equality and therefore invertible, while its whole higher action is retained. This is target-induced pseudo-functorial behaviour, not a global strictness rule. See [Chapter 25](#).

PathOut. The outgoing-arrow category $\sum_{y:Z} \text{Hom}_Z(x, y)$ at a fixed source x . Its canonical arrow from (x, id_x) to (y, p) drives arrow induction. See [Chapter 5](#).

Precategory, ordinary. A classifier of objects with set-valued homs, identities, composition, and category laws. It is used as a readable one-categorical specialization of the native iterated-hom architecture. See [Chapter 10](#).

Polynomial algebra. A free commutative R -algebra on a variable classifier X , characterized by contractible structured extension spaces for every base map and valuation. The universal interface does not select a monomial representation. See [Chapter 21](#).

Proj. The standard construction of a projective scheme from a graded ring, using homogeneous localization and degree-zero parts on standard regions. It is mathematical development and a research boundary in this edition; the active artifact has no graded `Proj` owner. See [Chapter 24](#).

Projective-line presentation, supplied. One already-global binary site-relative scheme, its selected actual chart intersection, and a Laurent coordinate presentation on the literal inherited restriction maps. It is an end-to-end computational capability, not a construction of $\mathbf{P}^1$, a projectivity or non-affineness proof, or a substitute for `Proj`. See [Chapter 24](#).

Presheaf. A contravariant functor $X : \mathcal{K}^{\text{op}} \rightarrow \mathbf{Set}$ or, in the active higher presentation, to $\mathbf{Cat}$. It assigns observations to stages and functorial restriction to probes. See [Chapters 13](#) and [18](#).

Profunctor. A $\mathbf{Cat}$ -valued functor $A^{\text{op}} \times B \rightarrow \mathbf{Cat}$, contravariant in its first endpoint and covariant in its second. See [Chapter 13](#).

Pullback base change. For chosen pullbacks and $u : X \rightarrow Y$, the whole functor $u^* : C/Y \rightarrow C/X$ right adjoint to slice postcomposition Σ_u . Pullback objects, projections, and their directed square derive from this slice action and its counit. See [Chapter 30](#).

Representable. A family or profunctor obtained from an ambient hom. Its action is composition, which makes it the computational bridge between universal properties and cut elimination. See [Chapters 5](#) and [13](#).

Reflector. A left adjoint to the inclusion of a full subcategory whose counit on objects already in that subcategory is an equivalence. Direct cover completion is a reflector from $\mathbf{Cat}$ -valued presheaves to topology-local ones at the fixed site. See [Chapter 20](#).

Rezk completion. A completion intended to turn the selected weak equivalences into equivalences as seen by saturated targets. The book gives ordinary Yoneda-image and higher-inductive constructions and a prospective native specification; no native implementation is claimed. See [Chapter 15](#).

Runtime rewrite. A directed reduction selecting an intended normal form. It is distinct from proof-time unification and internal propositional equality. See [Appendix E](#).

Saturation. The property that the chosen identity-to-equivalence map is itself an equivalence, or the result of freely enforcing that property by a completion. Saturation is not finite categorical height. See [Chapter 15](#).

Sheaf. A presheaf local at every covering sieve: restriction from global sections to matching families is an equivalence. This condition on an existing presheaf is distinct from a sheafification construction. See [Chapters 19](#) and [20](#).

Sheafification. A reflective free-local completion of a presheaf. In the active Cat-valued construction, return preserves old data, whole glue adjoins amalgamations over eligible cover questions, silent removes redundant restrict-and-glue detours, and the recursor gives the whole Hom universal property. See [Chapter 20](#).

Site. A category equipped with a Grothendieck topology, whose covering sieves satisfy maximality, pullback stability, and local character. A site need not be a poset of open subsets. See [Chapter 19](#).

Semisimplicial diagram. A functor from the opposite augmented category of injective ordinal maps. It has face action but no degeneracy action. The active groupoid-valued diagrams realize level-wise through the path-category functor while retaining whole map and higher action. See [Chapter 29](#).

Standard semisimplex. The Yoneda representable $\text{Hom}(-, n + 1)$ on the augmented semisimplex category. The shift converts dimension n to $n + 1$ vertices. It is distinct from both the ordinal source category $\Delta[n]$ and one native dependent simplex. See [Chapter 29](#).

Terminal object, selected. An object t with one whole canonical-arrow transformation $! : \text{id}_C \Rightarrow \text{Const}_t$. Every $\text{Hom}_C(A, t)$ is contractible at $!_A$; arbitrary uniqueness is equality evidence rather than a variable-headed rewrite. See [Chapter 30](#).

Topology-local ring presentation. A commutative-ring presheaf capability making a zero-unit stage empty-covering and splitting an invertible sum over a selected cover into an executable branch where one restricted summand is a unit. It is a direct site-level forcing condition, not a constructed stalk or a theorem comparing with stalk-local rings. See [Chapter 23](#).

Strict category, ordinary HoTT sense. A precategory whose object classifier is a set. This is object-level truncation and must not be confused with strict transfor computation or strict higher associativity. See [Chapter 14](#).

Strict transfor. A native transfor for which the selected two-sided naturality cuts compute through the global `tapp*` calculus. The adjective does not say that every coherence law in its ambient category is judgmental. See [Chapters 9](#) and [14](#).

Structure identity principle. A theorem identifying equality of structured objects with an appropriate structure-preserving equivalence when the displayed notion of structure is univalent or standard. The ordinary theorem is developed mathematically; a generic native package is a research boundary. See [Chapter 15](#).

Transfor. An arrow in a functor category, generalizing a natural transformation through off-diagonal and higher hom action. See [Chapters 9](#) and [11](#).

Truncation evidence. A recursive property of an existing classifier: at the base level it is contractibility, and at successor levels it truncates all identity classifiers one step lower. The property is distinct from the active classified truncation reflector, which constructs a new truncated target with restricted elimination. See [Chapter 7](#) and [Chapter 26](#).

Unimodular presentation. A finite family (f_i) together with coefficients (a_i) and a retained equation $\sum_i a_i f_i = 1$. It is the witness-rich algebraic input for a finite basic-open cover, not by itself a covering sieve or topology. See [Chapter 21](#).

Univalence. An interface relating identity of classifiers or packages to an appropriate equivalence. This edition has checked groupoid and restricted truncated-universe interfaces but does not claim one universal directed univalence axiom. See [Chapter 4](#).

Unitary arrow. In a dagger category, an arrow whose dagger is an inverse. The ordinary theory is developed in [Chapter 14](#); a native unitary classifier awaits the selected dagger interface and coherent higher action. See [Chapter 14](#).

Upper-star action. Precomposition: if $u : x \rightarrow y$ and $h : y \rightarrow z$, then $u^*(h) = h \circ u : x \rightarrow z$. Its active owners are `hom_precomp_along_func` and `hom_precomp_along_fapp0`. The action is contravariant in u . See [§9.2](#).

Weighted colimit. A representation of a weighted-cocone profunctor. The selected interface and left-adjoint preservation theorem are checked through opposite duality; full coend semantics is not assumed. See [Chapter 17](#).

Weighted limit. A representation of a weighted-cone profunctor, with beta and eta supplied by a chosen profunctor comparison. The selected interface and right-adjoint preservation theorem are checked. See [Chapter 16](#).

WalkingEnd. The opaque one-dimensional directed HIT/category with a base object and a directed generating endomorphism. See [Chapters 6](#) and [8](#).

Yoneda principle. Natural maps out of a representable are determined by their value at the identity. The ordinary equivalence is developed by encode-decode, while the active native theorem is the shaped co-Yoneda cut; full Cat-valued Yoneda remains a named boundary. See [Chapter 13](#).

D.2 Homological Glossary

The homological continuation also uses the following linked terms.

Exactness, selected. Epicity of the actual boundary arrow into the selected cycle kernel. In an Abelian category, local epic-cover factors characterize this property. See [Chapter 31](#).

Freyd presentation. A relation arrow into a generating object. Raw maps retain relation-preservation coefficients; equality of their induced quotient maps has separate coefficient witnesses. See [Chapter 31](#).

Selected homology. A retained cycle kernel, the incoming differential lifted into it, and the cokernel of that boundary. Maps and exactness refer to these same choices. Whole H is the primary coherent-family interface; its record views retain the actual H objects. See [Chapter 31](#) and the [worked non-split example](#).

Coherent kernel/cokernel presentation. Supplied whole functors and adjunctions over the original selected universal operations, with proof-time usability for their observations. Such a presentation is additional coherent structure, not a consequence of finitely many computed matrix equations. See [Chapter 31](#).

Whole homology. The functor obtained by forming the boundary transformation through K and applying Q to its arrow-diagram family. On the native zero-cone category this is a whole one-degree homology functor; it is not an unbounded-complex or global graded-homology package. See [Chapter 31](#).

Local factor cover. An epimorphism $e : T' \rightarrow T$ after which an annihilated test factors through the preceding arrow. It replaces an elementwise choice of preimages in the constructive snake proof. See [Chapter 31](#).

Proof-CAS adoption. The explicit decision to add a computed equation or a selected algorithm's semantic contract to a proof source. Computation alone adds no assumption; subsequent formal checking is relative to the adopted statements. See [Chapter 31](#).

D.3 Index Strategy

This edition uses the linked concept index as its stable index. Terms are curated rather than extracted from raw identifier frequency; synonyms point to one canonical entry, and every destination is an explicit HTML anchor checked by the source gate.

Page numbers are deliberately absent from the source because they depend on the renderer, paper size, and font metrics. A later release tool may resolve these anchors to PDF page labels, but the anchor remains the authority. New index entries should be added when a concept is defined or changes status, not for every occurrence of its implementation name.

Appendix E. Computation And Normalization

The prose uses equations freely, but the implementation distinguishes three ways expressions can agree. That distinction is essential whenever a theorem depends on a particular normal form.

E.1 Three Forms Of Agreement

Runtime reduction selects a computational direction. A rewrite

```
left  ↔  right
```

makes `right` the intended normal form when the left pattern is observed. Constructor beta rules, functor projection rules, and selected cut-elimination rules live here.

Proof-time comparison helps elaboration recognize two typed expressions without making either one compute to the other. Emdash uses narrow `unif_rule` declarations for this purpose. A typed reflexivity proof is the relevant diagnostic; mere conversion testing does not exercise the same interface.

Internal equality is mathematical data in a classifier `x=y`. It can be transported, acted on, inverted, or used in an equivalence proof. A propositional theorem may compare two stable runtime presentations without changing either presentation's reduction behavior.

The book's symbol `=` normally denotes internal equality or an ordinary mathematical equality justified by a formal-status note. It never implies that the source expressions are definitionally identical.

E.2 Semantic Owners

A computational operation should have one owner. Generic functoriality is owned by the `fapp*` calculus; generic naturality by `tapp*`; displayed hom action by `fdapp*` and `tdapp*`; Sigma and Pi expose their own structural projections. Readable aliases route through these owners instead of copying their semantic bodies.

This prevents two kinds of drift:

- competing rewrites can no longer silently choose incompatible normal forms;
- a theorem at the next hom level retains the functor or transfor object it needs for further iteration.

The WalkingEnd development illustrates the policy. The contextual eliminator owns the constructor-specific base and generator observations. It does not restate generic preservation of identity or composition. The decoder's normalization cell is the displayed hom-action of one constructed functor; it is not a custom recursion rule for every arbitrary based arrow.

E.3 Selected Groupoidal And Gray Boundaries

The Circle and Interval illustrate why “constructor computation” must name the observer. Their dependent eliminators reduce at the point constructors, and applying dependent path action to the generating loop or segment reduces to the supplied `PathOver` datum. Ordinary constant-family recursion inherits that dependent computation. Its familiar homogeneous `ap` equation is instead an internal equality derived through the constant-family `PathOver` bridge; it is not a second rewrite competing for the same observation.

Classified truncation has a similarly precise boundary. `Trunc_ntype(n,A)` constructs a code in `NType_cat(n)`, decoding that code exposes the stable carrier `Trunc_grpd(n,A)`, and restricted induction reduces on `trunc_intro(a)`. Contractibility of the set truncation of the Circle is a proved property. The carrier is not made judgmentally equal to `Unit`.

Generic groupoidification computes at represented points and at the canonical dependent action over every represented source arrow. Restriction and extension remain whole functors, while their beta and eta laws are paths between whole functors. Thus the universal property controls higher action without replacing every composite mapping expression by one runtime normal form. The compositor of the unit is retained as a directed cell with a next action; it is not globally collapsed to identity.

The Gray experiment makes strictness a property rather than a second functor grammar. `IsStrictFunctor` constrains the compositor already extracted from whole internal action, and `StrictFunctor` pairs an ambient functor with that evidence. One stable evidence-bearing view retains the proof where a selected consumer needs it. The retired blanket rule that sent every such compositor to literal identity was not justified by arbitrary semantic evidence. Curry and uncurry for the selected right closure have whole beta/eta paths, and the walking-square interchanger is a non-identity directed cell projected from whole laxity. These facts do not install a general weak-category normalizer, a profile-local replacement for every historical global strict cut, or a full Crans–Gray tensor.

E.4 Direction And Variance In Normal Forms

Covariant postcomposition and contravariant precomposition have different runtime owners. Their mathematical comparison through opposites is available at proof time, but forcing both into one rewrite direction would erase the variance used by `PathOut` and profunctor action.

Similarly, an identity may appear as an ambient categorical identity, a functor identity, a displayed identity, or a specialized projection. These forms are joined only where a typed consumer requires it. Broad eta-style rewrites are avoided because unification is experimental and because a functor-level normal form may be needed to act on the next cell.

E.5 Structural And Indexed Cut Families

The cartesian and slice interfaces illustrate all three equality modes in one calculus.

- Product beta, pairing distribution, terminal composition, adjunction rectangles, and selected mate cancellation are runtime reductions.
- Generic product action and the triangular product map compare at proof time.
- Arbitrary uniqueness into a terminal object is internal equality derived from Hom contractibility.

For pullback mates, the stable point and whole functors remain runtime-distinct from their explicit unit/counit formulas. Their proof-time unifiers match the semantic bodies only after those bodies have reached the canonical post-`Op` shape. In particular, the readable Sigma action has become `sigma_map_obj`, while non-discriminating category slots are `_`.

Whole cancellation is ordinary composition in `Cat_cat`, but the rule cannot discard its endpoint guards. A wildcard-only candidate typechecked while reconstructing the source object from the codomain after opposite normalization. The active rules repeat the exact raw Hom-category source, middle, and target, preserving the intended a and b .

This example sharpens the owner discipline: a rule may be mathematically obvious and syntactically expressible yet still require exact canonical guards to be a sound runtime interface.

E.6 How A Checked Prose Claim Is Reviewed

For a code-facing claim, the review path is:

1. identify the mathematical interface and its direction;
2. locate the active owner declaration with lexical or type-aware search;
3. identify an independent regression or reviewer example;
4. decide whether the observation is runtime, proof-time, or propositional;
5. add or update the evidence-register entry;
6. cite the evidence identifier beside the prose claim;
7. run the evidence, assembly, source, and browser-render checks.

Changes to rewrite or unification behavior require the stronger repository workflow: an owner-position probe, bounded typecheck, warning comparison when relevant, focused assertions, and full CI before handoff. Book prose does not authorize changing kernel normal forms merely to make an explanation shorter.

E.7 What Has Not Been Proved Metatheoretically

The passing executable checks establish the selected interfaces and regression observations. They do not by themselves prove global confluence, strong normalization, canonicity, consistency of every future extension, or soundness with respect to a complete weak omega-categorical model.

Those are research-level metatheorems. A future semantics chapter should state its fragment, model, and computation theorem explicitly. Until then, the book uses “computes” locally for named checked reductions and uses a formal-status note for every stronger reading.

The current development SOP remains the operational authority for rule design and validation. This appendix explains the mathematical reading needed by a book reader; it is not a replacement for that SOP.

E.8 Universal Factors And Computer Algebra

The Abelian and homological constructions in [Chapter 31](#) separate whole operations, data observations and theorem-level laws. Coherent kernel/cokernel presentations expose actual adjunctions over the original selected operations. Their proof-time usability leaves the whole K/Q heads available for the generic cut calculus. Whole H is then derived from the boundary transformation and Q; it inherits the existing functor identity/composition computations and further Hom action.

The whole connecting transformation has a component rule returning the direct construction. Row/window observations and finite-result records have local projection beta rules; the bounded iterator uses ordinary Nat recursion. Thus this entire interface is no longer accurately described as rule-free. The new rules concern retained data and whole observations, not a second set of generic category laws.

Reconstruction, uniqueness, normality and exactness remain theorem-level paths or supplied universal operations. They are not all runtime reductions, and the checked window theorem does not establish a general normalization calculus for complexes. The final symbolic zero-endpoint attachment is a separate deferred interface; computational consumers retain their arrows without requiring normalization of that theorem's evidence.

Polynomial algorithms provide a different computation layer. Their selected matrices, factors and witnesses can be reified into the formal language. Explicit adoption then makes the corresponding equations available to ordinary constructors. An all-test factor law is separately recorded as trusted provider semantics; it is not inferred from finitely many sample equations. The formal constructors retain the original selected objects and raw witnesses. Equality of serialized data is an implementation consistency check, not the definition of equality in the Freyd quotient category.

The supplied-model layer also interprets actual whole-H points, induced maps and, with explicit normality, every retained connecting window. These interpretations are recorded separately from computed equations. The same native results are reused; a model-observation request neither reselects universal objects nor reruns the whole homology computation.

Appendix F. Implementation Status And Research Directions

This appendix summarizes the boundary of the higher-categorical development edition through the seventh, additive and homological spiral. The generated [evidence register](#) provides the detailed claim-by-claim traceability view; active declarations and their checked consumers remain the implementation authority.

F.1 Status Matrix

Area	Checked nucleus used by the book	Explicit boundary
Equality-local type theory	Equality induction, path action, Sigma/Pi path interfaces, elementary inductives	No claim of a complete standalone HoTT implementation
Directed categories	Iterated homs, identities, composition, functors, transors, opposites, products	No complete weak omega-category metatheory or model theorem
Directed families	Fibres, transport, family morphisms, Sigma totals, Pi sections, displayed hom action, fibrewise products, pull-back totalization, displayed evaluation, and finite canonical sibling/Sigma telescopes	Arbitrary dependency or variance graphs, unrestricted mixed introduction/evaluation, and exchange across genuine dependency remain open
Cut and transfor calculus	Lower-star postcomposition, upper-star precomposition, off-diagonal <code>tapp1</code> , horizontal composition, selected universal beta/eta cuts, whole internal displayed laxity, ordinary post/pre surfaces, and the retained functor compositor	No unrestricted runtime associativity rewrite, completed generic lax classifier, or claim that all higher coherence is judgmental
Equivalence and univalence	<code>TypeEquiv</code> , groupoid univalence, truncated-universe univalence, native recursive omega-equivalence facade and one-way hom action	No full general object-equality/ordinary-isomorphism equivalence for arbitrary categories
Induction	Nat and equality induction, fixed/varying-source <code>PathOut</code> induction, composition benchmark	No general equivalence with homotopy-initial categorical algebras
Directed and groupoidal HITs	Opaque <code>WalkingEnd</code> and <code>Circle</code> signatures, the groupoidal interval, category-indexed <code>Groupoidify(C)</code> , selected dependent eliminators, and constructor computation at their reviewed owners	No general directed/HIT signature compiler, arbitrary pushout or cell-complex schema, or automatic positivity/coherence checker
Truncation and height	Recursive truncation properties and closure. evidence-property, finite <code>IsNCat</code> object truncation, classified <code>NType_cat(n)</code> targets, point-computing <code>Trunc_ntype(n,A)</code> , restricted elimination, and whole map action	No general directed categorical truncation, arbitrary quotient schema, left-exactness theorem, or complete comparison with every hub-and-spoke presentation

WalkingEnd calculation	Code, encode, power, spiral, contextual decoder, normalization cell/path, two inverse laws, carrier equivalence, non-invertibility results, and the whole free-inversion comparison with the Circle	No packaged monoid isomorphism, reverse <code>BNat</code> functor, full hom-category equivalence with <code>BNat</code> , or directed initiality theorem
Groupoidal realization	Path categories and path functors; product-path split/join and coherent transport; Circle/Integer encode-decode and monodromy; WalkingEnd/Circle and WalkingArrow/interval mapping theorems; category-indexed groupoidification; path-realized pseudo-laxity	Source functoriality and the packaged groupoidification adjunction, closure for every former, generic simplex regressions, and a complete computational HoTT metatheory remain open
Profiled Gray direction	<code>IsStrictFunctor</code> properties, exact functor/evidence packages, the strict-object/lax-arrow <code>GrayHom_lax</code> profile, one selected right closure, the walking square, and a nonidentity interchanger with retained next action	No mirror closure, full Crans–Gray bi-closed monoidal structure, tensor functoriality/coherence, or global/profile-local strict-cut migration
Semisimplicial and dependent simplexes	Computing injective face codes and augmented index; join-built ordinal shapes; Yoneda standard semisimplices; homd/Sigma triangle and tetrahedron action; intrinsic flagged codes; generic nonempty faces; one canonical ordinal source in variable dimension with selected checks through dimension four and retained next action	No degeneracies, whole <code>DependentSimplex_cat(C, n)</code> classifier, mapping-category equivalence with <code>Functor_cat(Delta[n], C)</code> , judgmental agreement of all finite presentations, or general Kan, Segal, Rezk, complicial, or oriental theory
Ordinary categorical specialization	Precategories, univalent categories, strict categories, functors, natural transformations, and ordinary Yoneda developed over the native vocabulary	These readable one-categorical theorems are mathematical development, not definitions of native <code>Cat</code>
Adjunctions and equivalences	Triangle cuts and hom-profunctor comparison; one-way lift from ordinary isomorphism to native evidence	No checked native fully-faithful/essentially-surjective characterization or general adjointification package
Monads and cartesian/indexed structure	Whole monad extension with ambient beta/accumulation; selected triangular products and terminality; chosen pull-back base change with point/whole mates; selected dependent products and $\Sigma_u \dashv u^* \dashv \Pi_u$	No free-monad syntax, explicit Kleisli category, automatic weighted-product witness, pushout duality, Beck–Chevalley, Frobenius, derived slice exponentials, or final convention-sensitive LCCC package
Additive and Abelian structure	Abelian Hom groups, selected biproducts and zero objects; contractible kernel/cokernel factors; normality, coimage-image isomorphism, fibre products, pushouts and stability	The concrete witnessed Freyd interface does not decode arbitrary quotient paths or supply a closed ring-wide Abelian capability
Snake and homological computation	Generic six-term snake; coherent K/Q presentations and derived whole H on native zero diagrams; direct and whole connecting interfaces; three exact window interiors and finite iteration; retained native/model computation	Final symbolic zero-endpoint attachment is deferred; coherent presentations/normality stay explicit; no closed model, universal-diagram prover, derived category or spectral-sequence theory is claimed

Yoneda and profunctors	Cat-valued profunctors, endpoint reindexing, representables, shaped cells, fixed-middle tensor, co-Yoneda beta/fusion	No general coend semantics, tensor associativity package, full Cat-valued Yoneda equivalence, or profunctor bicategory
Presheaves and sieves	Cat-valued presheaves, Yoneda and slices, higher sieves, ordinary pointwise-subterminal sieves, pullback membership, and commutative-ring invertibility sieves	No global ordinary-sieve classifier, automatic representation by one open, topology, descent, or sheafification follows from this layer
Sites and descent	Ordinary-sieve Grothendieck topology laws, chaotic model, internally generated least topology, whole sieve extensions, matching and section Hom families, and topology-locality	No inductive cover derivations, coverhood decision procedure, automatic subcanonicity, sheafification reflector, or identification with a separate rigid sheaf facade follows from locality alone
Direct cover sheafification	Cat-valued categorical-HIT completion with whole return/glue/silent data, derived topology-locality, recursor, whole Hom universality, adjunction, and reflective counit	Fixed-site and Cat-valued only; no arbitrary coefficients, commutative-ring lift, left exactness, site base-change theorem, or classical plus-construction comparison
Commutative algebra	Set-carrier rings and structured maps, finite unimodular presentations, polynomial and localization universal-property interfaces, selected unit/zero/idempotent models, and whole iterated/product-localization equivalence	No arbitrary polynomial/localization existence, monomial or fraction representation, categorical product theorem, global ring-package identity, or affine geometry follows from this layer alone
Affine geometry	Yoneda functor of points; ordinary basic-open sieve; pointwise localization representation and multiplicative intersection; big affine slice, coordinate presheaf, and least generated Zariski topology; assumption-explicit reflective structure sheaf, localization locality, and thin affine presentation	No whole natural basic-open equivalence, global localization choice, CommRing-valued sheafification construction, small-site comparison, subcanonicity, stalk-local theorem, qcqs comparison, or representation-independent category of affine schemes
Site-relative schemes	One global reflective ringed object and covering sieve; witness-rich binary generation; whole actual-slice restriction; supplied affine-basis realizations; topology-local ring forcing; dependent binary scheme total; selected actual overlap with derived ring restrictions	Binary and relative to the supplied site; no atlas-first gluing, induced slice topology, arbitrary pullback construction, overlap-affineness theorem, scheme-morphism category, compact-open/classical comparison, or representation-independent scheme theorem
Supplied projective-line boundary	Universal-property Laurent transition maps; literal common-overlap identity package; thin adapter to actual inherited chart restrictions; dependent total of one already-global scheme, its actual overlap, and Laurent coordinates	The global object and Laurent identity paths remain supplied; no atlas-first gluing, projectivity or non-affineness proof, graded ring, homogeneous localization, degree-zero construction, <code>Proj</code> , or general projective space
Opposite, duality, and dagger	Opposite category action and selected opposite-duality comparisons	Dagger, unitary structure, and dagger univalence are mathematical development pending a native involutive interface
Structure identity and saturation	Truncation/evidence-property footholds and ordinary-isomorphism lift	Generic native structure identity and Rezk completion, including their higher universal properties, are research boundaries

Weighted limits and Kan interfaces	Weighted representability, beta/eta comparison, right-adjoint preservation, terminal/conjoint specializations	Standard end formulas, pointwise Kan semantics, existence, and general dependent adjunctions are not globally packaged
Weighted colimits and join	Opposite-dual colimit preservation, terminal/companion specializations, primitive join recursor and three beta observations	General coend semantics and join-as-collage mapping, hom-decomposition, opposite, and dependent-elimination theorems remain open
Formal presentation	Checked categorical owners; a bounded TypeScript outer LF, explicit Core, contextual elaborator, checker/runtime, reviewed text subset, adjunction/structure declaration conveniences, and client-side reviewer	No compiler for the complete book surface, arbitrary displayed coherence, general record/inductive facility, or whole-library transfer; readable notation is not a second kernel
Metatheory and models	Bounded typechecking, subject-reduction checks performed by Lambdapi, focused diagnostics, and the concrete BNat model	No global confluence, normalization, canonicity, decidability, consistency, or semantic-soundness theorem for the full combined calculus
Production artifact	Manifest assembly, provenance/evidence checks, local assets, bounded browser validation, and deterministic PDF export	External mathematical peer review and a non-draft public edition remain future release work

F.2 Near-Term Formal Strengthening

The most direct strengthening of Theorem 8.1 is to package composition and addition compatibility. Its proof can use the checked power recursion, Nat addition associativity, and both carrier inverse laws. The desired result is a monoid-level comparison with an explicit orientation matching `BNat`.

Next comes a reverse functor from `BNat` to `WalkingEnd` and a comparison with the existing model functor. This requires reusable action-to-functor and functor-extensionality infrastructure; it should not be simulated by making the opaque `hom` definitionally `Nat`-valued.

Full initiality is a further layer. It asks for a category of endomorphism algebras, structured maps, and coherent higher transfors, followed by an appropriate contractibility or equivalence theorem.

Generic groupoidification now supplies its target-side mapping equivalence, but not yet its action on a source functor $C \rightarrow D$. The next categorical strengthening is to derive that action from extension of the composite unit, prove its identity and composition laws by whole uniqueness, and only then package the adjunction with the path-category functor. The mapping theorem should not be renamed an adjunction before that source action exists.

The homological continuation now has whole K/Q presentations, derived one-degree H on the native zero-cone category, a direct connecting component and its whole transformation. The required normal tests and source-boundary annihilation are proved, as are all three window interiors. A field-indexed finite iterator retains those exactness results and their actual arrows.

The remaining symbolic zero-endpoint attachment has been deferred separately from these checked constructions. Native bounded computation already retains its endpoint zero data, and the supplied-model interface interprets all retained H points/maps and connecting windows when normality is

provided. More economical structure/record encodings with primitive projections remain a later interface-design option. Neither this option nor the current whole one-degree H should be mistaken for a packaged category of all unbounded complexes or a general homological normalization theorem.

F.3 Foundational Extensions

A reusable directed-HIT schema should generate contextual elimination and constructor computation from typed object, arrow, and higher-cell boundaries. Its validation must include rewrite overlap and subject-reduction behavior, not only a semantic signature.

The active classified truncation reflector now constructs a groupoidal n -truncated target rather than merely certifying an existing classifier. Future work should compare that sorted interface with classical hub-and-spoke presentations, add selected quotient consumers, and investigate left-exactness. Directed categorical truncation remains a different problem: it must specify which lower arrows and compositions are preserved.

The univalence programme should continue to separate carrier equivalence, ordinary categorical isomorphism, and native equality-valued recursive equivalence. A full theorem relating object equality and ordinary isomorphism must be proved at the intended categorical level rather than recovered through retired compatibility aliases.

F.4 Categorical Extensions

The representable/profunctor and identity layers suggest four staged projects:

1. package a fully faithful Yoneda embedding with mapping-category equivalences and higher naturality;
2. construct or model Cat-valued coends/coinserter and relate the opaque tensor to their universal property;
3. assemble associators, unitors, and horizontal cell composition into a coherent profunctor bicategory or suitable omega-categorical analogue.
4. design generic structure identity and Rezk completion interfaces only after the intended native equivalence and higher mapping properties are fixed.

Weighted limits, colimits, adjunctions, duality, and joins now enter the expanded chapter sequence through the triangle reductions, right-adjoint weighted-limit preservation, its opposite-dual colimit theorem, and the join recursor. The checked interfaces are the theorem spine; neighboring Kan, end/coend, dagger, collage, and dependent-elimination theory remains explicitly status-labeled rather than presented as a feature catalogue.

The selected Gray slice adds a concrete higher-dimensional stress test. Its walking-square interchanger is derived from the ordinary internal laxity action, but a full Gray theory still needs the mirror closure, tensor action, and coherent associativity and unit data. Those are structural projects, not extra fields to append to the current interchanger example.

The dependent-simplex construction supplies the next exact strengthening. Its present `DependentSimplexObservation(C,n)` packages objects only. A whole comparison with `Functor( $\Delta[n], C$ )` first requires an internal category whose objects recover that package, whose homs express compatible transformations of every dependent frame, and whose higher action agrees with the existing internal-action tower. Comparison functors, whole beta and eta, and compatibility with face restriction must then be constructed. Degeneracies are a separate extension of the index and native recursion, not a consequence of ambient identities alone.

The selected cartesian and slice structures supply a different continuation. Beck–Chevalley should compare pullback followed by dependent product across a selected pullback square, with an explicit direction and invertibility boundary. Frobenius should relate selected Sigma/base-change action to the chosen product structure. Slice exponentials should be derived from pullbacks and Π_u before the neutral `SliceDependentProducts(C)` total is renamed as a convention-sensitive locally cartesian closed package. None of these comparisons requires replacing the active Σ_u , u^* , or Π_u owners.

F.5 Semantics And Proof-Assistant Engineering

The largest research objective is a semantics and metatheory for a precisely stated fragment: typing, substitution, subject reduction, normalization or a weaker operational theorem, and interpretation in a suitable strict/lax omega-categorical model. The current executable artifact is evidence for specific interfaces, not a substitute for that theorem.

There are also known defects, not merely missing metatheorems. Variance diagnostics for the current whole opposite action and Sigma-Hom encoding can construct an inhabitant of the encoded empty type. These are tracked encoding defects, not admitted mathematical principles. Consequently, “checked” in this development edition records typechecking and specified computations in an experimental calculus; it is not a soundness certificate for arbitrary consequences. Repair of these encodings remains a separate foundational task.

The renewed TypeScript product now elaborates a bounded direct-TypeScript and categorical-text surface into backend-neutral explicit Core, then checks and reduces that Core with a small dependent logical framework. Its contextual categorical layer covers reviewed ordinary, natural, displayed-functorial, and displayed-natural binders. Within the canonical sibling/Sigma normal form it supports finite dependency depth and sibling groups; qualified finite Hom-category recursion and finite rigid indexed-section chains are also executable. An optional deterministic Lambdapi path remains a conformance oracle. It is not a production dependency, and the active Lambdapi development remains the mathematical authority.

The same outer LF has two bounded authoring conveniences. One declares an adjunction from already typed rectangular data, or from a counit and whole hom transpose, while retaining proof-time rather than runtime agreement with the stable observations. The other declares an unparameterized, nonrecursive, single-constructor dependent structure with named projections and projection beta rules. Both expand to ordinary declarations; neither adds a trusted Core form, categorical owner, general record eta, eliminator, recursion, or positivity principle.

This is a real executable bridge, visible in the client-side integrated reviewer, but not completion of the canonical mathematical surface. Arbitrary dependency and variance graphs, coherence outside the qualified grammar, a compiler for the whole book notation, a general record or inductive facility, and systematic transfer of the remaining library are still engineering boundaries. The older TypeScript prototype remains historical feasibility evidence; its stale category-specific layer is neither an authority nor the architecture of the renewed product.

The focused CAS and categorical-program layer offers an additional product path. It computes polynomial/Freyd constructions, retains complete selected results, and lowers typed categorical operations to the native algorithms. The proof-CAS interface replays a result, presents exact equations for adoption, and constructs formal objects and witnesses from them. Selected factor algorithms may supply explicitly trusted universal contracts without requiring certification of the CAS as a prerequisite for use. This is an operational integration boundary, not unrestricted evaluation of arbitrary formal terms by an external algebra system.

Ordinary DevOps makes checks, assembly, and release repeatable. The project's MathOps discipline additionally separates mathematical owners, independent reviewers, generated evidence and health views, authored sources, and deterministic release artifacts. That separation makes drift and provenance auditable. It does not convert a passing build, warning inventory, browser run, or reproducible PDF into a confluence, normalization, consistency, or soundness theorem.

F.6 Reading Claims Across Editions

The edition version in `book/book.json` identifies the source snapshot policy for generated artifacts. A later edition may promote a research boundary only when the evidence register names active owners and independent checks, or when the claim is explicitly reclassified as mathematical development with stated prerequisites. Dated reports preserve why a boundary was chosen; they do not override the current code.

Appendix G. Formal Presentation Of Functorial Type Theory

The mathematical chapters have used rules from the beginning: an identity path eliminates by reflexivity, a functor acts on an arrow, a transfor absorbs a naturality cut, and a representing comparison eliminates a universal map. This appendix puts those rules in one formal architecture. It follows the discipline of Appendix A of the HoTT Book—contexts and judgments first, then rule families, extensions, and metatheory—but changes the order of explanation at one decisive point.

In functorial type theory, category theory is not added later as a model of a previously specified, traditional term calculus. The explicit categorical calculus is already the computational core. Its objects include categories, iterated homs, functors, transfor, and directed families; its reductions express functoriality, naturality, induction, and universal properties. The active Lambdapi v3.2 development authors and checks that calculus and remains the mathematical authority.

The canonical mathematical surface used by this book is deliberately more spacious than any one executable grammar. A renewed TypeScript product now implements a bounded route from readable categorical binders to explicit emdash Core and a small dependent logical framework. That route makes a reviewed fragment directly usable; it does not become a second categorical kernel or define the mathematics retroactively.

The architecture therefore distinguishes these roles before its two executable paths meet in the operational diagram that follows:

Layer or role	Responsibility	Present status
canonical mathematical surface	the notation and rule presentation used by this book	active for prose, comments, and examples; not a parser grammar
scoped contextual elaboration	recursively interprets reviewed categorical variables, binders, neutral applications, and structural forms against typed expectations	active for the bounded direct-TypeScript and text profiles
typed outer-LF declarations	validate selected higher-level declarations and expand them into ordinary LF declarations and rules	active for adjunction assumptions and one bounded dependent-structure form; no new trusted Core node
backend-neutral explicit emdash Core	records the selected logical and categorical owners without committing to one runtime backend	active TypeScript intermediate representation
generic TypeScript dependent LF	checks Core terms, performs conversion and bounded reduction, and runs the reviewed proof-time rules	active for the recorded product boundary
active Lambdapi v3.2 kernel	authors the categorical declarations, computation, and proof-time comparisons used as mathematical authority	active and checked in the cited modules; also the conformance oracle
external semantic models	interpret a stated kernel fragment in mathematical categories or other structures	separate mathematical work; available only in selected examples

```

canonical mathematical surface (broader than implemented text)
-> reviewed direct TypeScript / text expressions
-> scoped contextual elaboration
-> explicit Core terms -----+
                                     |
typed host declarations             |
-> deterministic expansion          |
-> ordinary LF declarations and rules -----+
                                     |
                                     v
generic TypeScript LF checker / conversion / bounded runtime
-> optional deterministic Lambdapi emission / conformance

active authored Lambdapi v3.2 kernel = mathematical authority
external models                      = separate mathematical work

```

The text adapter is not the checker, an authoring macro is not a new term former, the TypeScript checker is not the active mathematical authority, and the implemented text subset is not the whole canonical surface. External interpretation is separate again. Keeping these roles distinct lets us say exactly which claims are checked computation, which are executable presentation, which are mathematical exposition, and which remain research.

G.1 Judgments, Contexts, And Classifiers

A formal presentation begins with judgments made in contexts. We use Γ for a finite ordered list of declarations. The basic external judgments have the schematic forms

$$\Gamma \text{ ctx}, \quad \Gamma \vdash T : \text{TYPE}, \quad \Gamma \vdash t : T, \quad \Gamma \vdash t \equiv u : T.$$

These are metatheoretic assertions about expressions. In particular, $t:T$ is not itself an internal proposition whose inhabitant is a proof that t has type T . Lambdapi checks the judgment while reading a declaration, definition, rule, or assertion.

Contexts are ordered because later entries may depend on earlier ones:

$$x : A, \quad y : B(x), \quad z : C(x, y).$$

Substitution replaces a declared variable by a term of the required type and acts in every later entry and conclusion. Renaming, weakening, exchange when dependencies permit it, and substitution are structural operations of the ambient dependent framework. They are not constructors of an internal `Context` classifier in the active emdash kernel.

External Types And Decoded Classifiers

The kernel uses two related universes. `TYPE` is `Lambdapi`'s ambient type level. `Grpd : TYPE` is the small groupoidal or type-like classifier universe used by `emdash`, and

$$\tau : \text{Grpd} \longrightarrow \text{TYPE}$$

decodes a classifier to the ambient type of its elements. Thus the book's readable judgment $a : A$ normally abbreviates the literal judgment `a : τ A` for `A : Grpd`.

Categories live at the ambient level:

$$\text{Cat} : \text{TYPE}.$$

Their object and arrow collections are internal classifiers:

$$\begin{aligned} \text{Obj}(C) &: \text{Grpd}, \\ \text{Hom}_C(x, y) &: \text{Cat}, \\ \text{Hom}(C, x, y) &:= \text{Obj}(\text{Hom}_C(x, y)) : \text{Grpd}. \end{aligned}$$

The second line is the source of higher structure. If $f, g : x \rightarrow y$, then a 2-cell $\alpha : f \rightarrow g$ is an object of $\text{Hom}_{\text{Hom}_C(x, y)}(f, g)$; repeating the same construction exposes higher cells without changing the grammar at each dimension.

The main internal classifiers used in the book are:

Mathematical classifier	Literal owner	Decoded inhabitants
objects of C	<code>Obj C</code>	<code>τ (Obj C)</code>
arrows $x \rightarrow_C y$	<code>Hom C x y</code>	<code>τ (Hom C x y)</code>
functors $A \rightarrow B$	<code>Functor A B</code>	<code>τ (Functor A B)</code>
transforms $F \Rightarrow G$	<code>Transf F G</code>	<code>τ (Transf F G)</code>
Cat-valued families over K	<code>Catd K</code>	<code>τ (Catd K)</code>
displayed functors $E \rightarrow D$	<code>Functord E D</code>	<code>τ (Functord E D)</code>
displayed transforms $FF \Rightarrow GG$	<code>Transfd FF GG</code>	<code>τ (Transfd FF GG)</code>

This distinction prevents two common category mistakes. First, `C : Cat` is not an object of some silently assumed set of all categories; it is an ambient kernel judgment. Second, an arrow classifier is not merely a set of morphisms: it is the object classifier of another category and can therefore be iterated.

Six Ways To Compare Expressions

Several notations that resemble equality have different force.

Notation	Layer	Meaning
----------	-------	---------

$t \rightsquigarrow u$ in this book; $\mathfrak{t} \rightsquigarrow \mathfrak{u}$ in source	runtime	a selected oriented rewrite makes u the computational form
$t \equiv u$	external conversion	the checker regards the terms as definitionally convertible using active computation
<code>unif_rule $t \equiv u \rightsquigarrow [\dots]$</code>	proof-time elaboration	a unification problem is replaced by narrower subproblems; neither side is selected as a runtime normal form
$p : x =_A y$	internal mathematics	<code>p</code> inhabits an equality classifier and can be eliminated by <code>ind_eqr</code>
$A \simeq B$	internal mathematical structure	specified maps and inverse/coherence evidence, such as <code>TypeEquiv</code> or a categorical comparison
$t = u$ in status-labeled free-form prose	mathematical development	a theorem to be proved in the named future interface, not an undisclosed kernel conversion

A runtime rule does not by itself assert equality reflection. An internal path does not make its endpoints definitionally identical. A proof-time unification rule is not a path constructor, and an equivalence is not an unlabeled use of conversion. Chapters 1 and 9 rely on precisely these distinctions.

Formal status — checked. Evidence `FORMAL-KERNEL-PRESENTATION` covers the active categorical classifiers, their application operations, and representative executable checks. The displayed turnstile notation is the book's metanotation; it is not a new internal judgment former.

G.2 The Mathematical Categorical Presentation

We now give a compact signature in the notation of the book. It is the human-readable presentation of the kernel, not an untyped concrete grammar. Implicit arguments are suppressed only when their recovery is forced by the displayed source and target.

Categories And Iterated Homs

The core categorical judgments are

$$\frac{}{C : \mathbf{Cat}}, \quad \frac{C : \mathbf{Cat}}{x : \mathbf{Obj}(C)}, \quad \frac{x, y : \mathbf{Obj}(C)}{f : \mathbf{Hom}_C(x, y)}.$$

Every object has an identity, and composable arrows have a composite:

$$\mathrm{id}_x : x \rightarrow_C x, \quad \frac{f : x \rightarrow_C y \quad g : y \rightarrow_C z}{g \circ f : x \rightarrow_C z}.$$

The convention is always “first f , then g .” Identity and associativity are available at the intended comparison layers. The implementation does not install unrestricted reassociation as a general runtime rewrite.

Opposite categories reverse the hom endpoints:

$$\mathrm{Hom}_{C^{\mathrm{op}}}(x, y) \rightsquigarrow \mathrm{Hom}_C(y, x).$$

The path category embeds the equality-local fragment:

$$\mathrm{Obj}(\mathrm{Path}(A)) \rightsquigarrow A, \quad \mathrm{Hom}_{\mathrm{Path}(A)}(x, y) \rightsquigarrow \mathrm{Path}(x =_A y).$$

This is a groupoidal specialization. It does not identify arbitrary directed arrows with equality paths.

Functors And Their Iterable Action

For categories A, B , the category $A \vdash B$ has functors as objects:

$$\frac{A, B : \mathbf{Cat}}{A \vdash B : \mathbf{Cat}}, \quad \frac{}{F : \mathrm{Obj}(A \vdash B)}.$$

We write the second judgment as $F : A \rightarrow B$. A functor has object action and, for every pair $x, y : A$, a functor on the whole hom-category:

$$\begin{aligned} F[x] &: \mathrm{Obj}(B), \\ F_{x,y} &: \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(F[x], F[y]), \\ F[f] &:= F_{x,y}[f]. \end{aligned}$$

The hom action, not only its value at one arrow, is primary. It can act again on a 2-cell between arrows, and its own hom action continues the same pattern. At the first capped level, the selected functoriality cuts are

$$F[\mathrm{id}_x] \rightsquigarrow \mathrm{id}_{F[x]}, \quad F[g] \circ F[f] \rightsquigarrow F[g \circ f].$$

Transfors And Family Action

For parallel functors $F, G : A \rightarrow B$, the category $F \Rightarrow G$ is their first hom in the functor category. A transfor $\eta : F \Rightarrow G$ has a point component

$$\eta_x : F[x] \longrightarrow G[x]$$

and, more fundamentally, an off-diagonal hom functor

$$\eta_{x,y} : \mathrm{Hom}_A(x, y) \longrightarrow \mathrm{Hom}_B(F[x], G[y]).$$

We write $\eta[f]$ for its value at $f : x \rightarrow y$. Its two adjacent naturality cuts compute:

$$\begin{aligned} G[g] \circ \eta[f] &\rightsquigarrow \eta[g \circ f], \\ \eta[f] \circ F[h] &\rightsquigarrow \eta[f \circ h]. \end{aligned}$$

The diagonal component is the identity-arrow instance of this family action. Thus naturality is not a proposition pasted onto a bare family of arrows. It is part of an operation whose higher action remains available.

Directed Families, Totals, And Sections

A directed Cat-valued family over K is written

$$E : K \longrightarrow \mathbf{Cat}.$$

For $k : K$ it has a fibre $E[k]$, and for $p : k \rightarrow_K k'$ it has transport

$$E[p] : E[k] \longrightarrow E[k'].$$

A displayed functor $FF : E \rightarrow D$ contains fibre functors and off-diagonal comparison cells over base arrows. In book notation its classifier is

$$k :^n K ; E[k] \vdash D[k].$$

Displayed transfors arise by taking the next hom in this category. The `Catd`, `Functord`, and `Transfd` facades keep these levels visible instead of flattening them into pointwise functions.

Two categorical dependent formers organize families:

$$\sum_{k :^n K} E[k] \quad \text{and} \quad \prod_{k :^n K} E[k].$$

The Sigma total has objects (k, u) with $u : E[k]$. An arrow consists of a base arrow $p : k \rightarrow k'$ and a fibre arrow

$$E[p](u) \longrightarrow u' \quad \text{in } E[k'].$$

The Pi category has coherent sections as objects. Its evaluation at k is a functor to $E[k]$, not merely a carrier-level application function.

Chosen Arrows And Natural Families

The two represented-hom actions expose the first Došen-style cut discipline. For $u : x \rightarrow_A y$,

$$u_*(g) = u \circ g \quad \text{and} \quad u^*(h) = h \circ u.$$

Lower star is covariant postcomposition; upper star is contravariant precomposition. With a functor H , the acting arrow is $H[u]$, so $(H[u])^*(h) = h \circ H[u]$. By contrast, $\eta[f]$ uses the whole natural family η , not one selected arrow. Structural and universal eliminators continue this progression.

The product/projection benchmark of Chapter 9 is a theorem in an arbitrary category K equipped with products. Its Cat-specialized executable probe is evidence about the current owner calculus, not a restriction of the mathematical statement to the category of categories.

Signature-To-Owner Map

The following table records the correspondence without treating readable notation as a second implementation.

Readable operation	Active owner
$x \rightarrow_C y$	Hom_cat C x y and Hom C x y
$A \vdash B$	Functor_cat A B
$F[x], F[f]$	fapp0 , fapp1_func , fapp1_fapp0
$F \Rightarrow G$	Transf_cat F G
$\eta_x, \eta[f]$	tapp0_fapp0 , tapp1_func , tapp1_fapp0
$E : K \rightarrow \mathbf{Cat}$ and $E[k]$	Catd K and Fibre_cat E k
displayed functors and transfors	Functord and Transfd
$\sum_k E[k], \prod_k E[k]$	Sigma_cat E and Pi_cat E
u_*, u^*	hom_postcomp_* and hom_precomp_along_*

Formal status — checked. Evidence
 CAT-ITERATED-HOMS ,
 CAT-FUNCTOR-CALCULUS ,
 TRANSF-POINT-OFFDIAGONAL ,
 TRANSF-STRICT-NATURALITY ,
 and
 CAT-DIRECTED-FAMILIES
 support the displayed nucleus. The typography in this section is canonical mathematical surface notation; the next section shows literal source.

G.3 The Checked Lambdapi Presentation

The active source is a Lambdapi signature. This section gives representative literal excerpts, enough to explain how the mathematical presentation is checked without reproducing the kernel.

Declarations And Definitions

At the universe boundary the source says:

```

constant symbol Grpd : TYPE;
injective symbol  $\tau$  : Grpd  $\rightarrow$  TYPE;

constant symbol Cat : TYPE;
symbol Obj : Cat  $\rightarrow$  Grpd;
injective symbol Hom_cat :
   $\Pi$  (A : Cat) (X_A Y_A :  $\tau$  (Obj A)), Cat;
injective symbol Hom (A : Cat) (X_A Y_A :  $\tau$  (Obj A)) : Grpd
= Obj (Hom_cat A X_A Y_A);

```

These lines exhibit three declaration policies.

- A `constant symbol` cannot receive a definition or rewrite rules. `Cat`, `Grpd`, and the `WalkingEnd` constructors use this literal policy.
- A plain `symbol` may be an undefined operation that later receives rules, or it may have a transparent body after `=`.
- An `injective symbol` gives the unifier a rigid constructor-like head. The modifier is a trusted declaration choice and is used only at selected classifier and stable-owner boundaries.

Lambdapi also supports an `opaque` modifier for a defined symbol whose body must not reduce. In the book, “opaque `WalkingEnd`” describes the mathematical effect of its constant declarations; it does not claim that those lines use the literal `opaque symbol` spelling.

Implicit parameters appear in square brackets and explicit parameters in parentheses. Prefix `@` exposes normally implicit parameters at a use site. In rule patterns, a dollar-prefixed name such as `$A` is a pattern variable, while `_` asks typing and unification to recover a slot that is not a genuine discriminator.

Rewrite Owners

Functor application is declared at a full hom level and a capped arrow level:


```

symbol fapp1_func :  $\Pi$  [A B : Cat],  $\Pi$  (F_AB :  $\tau$  (Functor A B)),
   $\Pi$  [X_A Y_A :  $\tau$  (Obj A)],
   $\tau$  (Functor
    (Hom_cat A X_A Y_A)
    (Hom_cat B (fapp0 F_AB X_A) (fapp0 F_AB Y_A)));

symbol fapp1_fapp0 :  $\Pi$  [A B : Cat],  $\Pi$  (F_AB :  $\tau$  (Functor A B)),
   $\Pi$  [X_A Y_A :  $\tau$  (Obj A)],
   $\Pi$  (f :  $\tau$  (Hom A X_A Y_A)),
   $\tau$  (Hom B (fapp0 F_AB X_A) (fapp0 F_AB Y_A));

rule fapp0 (fapp1_func $F_AB) $f
   $\hookrightarrow$  fapp1_fapp0 $F_AB $f;

```

The last line is runtime computation: observing the full hom-action at one arrow exposes the capped action. The generic identity and composition rules then contract $F[\text{id}]$ and $F[g] \circ F[f]$. Concrete functor constructors inherit those rules; they do not each receive private copies of ordinary functoriality.

The same ownership policy governs transfors. `tapp0_fapp0` observes a point component, while `tapp1_func` and `tapp1_fapp0` own off-diagonal action. The two strict naturality rewrites are attached to that generic action. A constructor-specific rule is justified only when it expresses extra constructor computation, not the fact that something already typed as a transfor is natural.

Proof-Time Unification

Some stable category presentations should elaborate together without one being erased at runtime. For rigid hom-category heads the source includes:

```

unif_rule Obj (Hom_cat $A $X $Y)  $\equiv$  Obj (Hom_cat $A' $X' $Y')
   $\hookrightarrow$  [ $A  $\equiv$  $A'; $X  $\equiv$  $X'; $Y  $\equiv$  $Y' ];

```

This rule decomposes one proof-time unification problem into three. It does not rewrite an object classifier during execution, prove an internal path, or assert that `Obj` is globally injective for every category construction.

Similar narrow comparisons relate the ordinary functor-category presentation to `Catd_cat`, and the ordinary transfor presentation to `Functord_cat`.

Associativity illustrates the boundary particularly well. The two bracketings of ordinary composition are compared at proof time, and `comp_assoc` packages a propositional witness. There is no global runtime rule that continually reassociates every composite. Represented hom actions, `tapp1`, and universal comparisons instead own the specific cuts they can normalize without losing higher action.

Assertions And Negative Assertions

Executable diagnostics are source commands, not theorem prose. A small example is:

```
assert ⊢ Nat_grpd : Grpd;  
assert ⊢ zero : τ Nat_grpd;  
assertnot ⊢ @eq_refl Nat_grpd zero ≡ tt;
```

The first two ask `Lambdapi` to accept a type and an inhabitant. The last checks that two terms are not definitionally convertible. A typed reflexivity term is used when a proof-time unification rule must be exercised; a bare conversion assertion does not test that same mechanism.

The diagnostic suite is intentionally separate from implementation owners. Permanent examples and assertions provide regression evidence, while the evidence register connects book claims to both declarations and reviewers.

Modules And Ownership

The source graph is larger than a useful reading list. The following map groups adjacent modules by mathematical responsibility; the evidence register supplies the exact owner and reviewer for each cited claim.

Module family	Formal role
emdash3_2.lp	categorical nucleus: classifiers, iterated homs, functors, transfors, directed families, cuts, and universal-construction interfaces
emdash3_2_monads.lp , emdash3_2_triangular_binary_products.lp , emdash3_2_terminal_objects.lp , emdash3_2_cartesian_categories.lp	selected monad extension and finite cartesian introduction/elimination computation, with thin packaging separated from runtime owners
emdash3_2_pullbacks.lp , emdash3_2_slice_dependent_products.lp	exact whole slice families, $\Sigma_u \dashv u^* \dashv \Pi_u$, full rectangular cuts, point/whole mates, and derived pullback observations without a cone record
emdash3_2_presheaves.lp , emdash3_2_sieves.lp , emdash3_2_sites.lp	presheaves, higher and ordinary sieves, pullback, and the direct Grothendieck-topology laws
emdash3_2_generated_topologies.lp , emdash3_2_sieve_extensions.lp , emdash3_2_site_basis.lp , emdash3_2_ringed_sites.lp	least generated topology, whole matching/section families, basis comparison, and ringed-site presentations
emdash3_2_direct_cover_*.lp	return/glue/silent cover completion, recursion, topology-locality, whole Hom universality, and the resulting Cat-valued reflector

Module family	Formal role
---------------	-------------

<code>emdash3_2_commutative_algebra.lp</code> through the polynomial and localization modules	set-carrier rings and structured maps, finite unit-ideal data, free extension, universal localization, and whole localization comparisons without polynomial or fraction syntax
the commutative-algebra presheaf, affine-points, affine-Zariski, ringed-site, and affine-scheme modules	the invertibility sieve $D(f)$, localization representation, generated big Zariski topology, coordinate presheaf, and assumption-explicit affine presentations
the ringed-space cover, affine-chart, site-relative-scheme, and chart-overlap modules	one supplied global ringed object, constructively generated covers, whole actual-slice restrictions, affine realizations, topology-local rings, and inherited overlaps
<code>emdash3_2_commutative_algebra_laurent.lp</code> , <code>emdash3_2_commutative_algebra_scheme_laurent_overlaps.lp</code> , <code>emdash3_2_commutative_algebra_projective_line.lp</code>	universal-property coordinate inversion on one literal overlap and the supplied projective-line boundary; no graded <code>Proj</code> construction
<code>emdash3_2_eq1*.lp</code> , <code>emdash3_2_nat_arithmetic.lp</code> , <code>emdash3_2_walking_end_hit.lp</code>	equality-valued higher action, reusable arithmetic, and the WalkingEnd encode-decode development
the groupoidal-closure, Integer, Circle, truncation, and connectedness modules	path-former comparisons, successor-localized integers, Circle encode-decode, classified truncation, and the selected connectedness consumer
the groupoidal-interval, walking-comparison, and groupoidification modules	two finite free-inversion tests, category-indexed formation and whole unit, target extension/restriction, whole mapping equivalence, compositor, and Interval recovery
the whole-laxity and Gray profile/right-closure modules	displayed and ordinary whole laxity surfaces, <code>IsStrictFunctor</code> properties, exact functor/evidence packages, the shared Gray hom profile, one selected right closure, and the derived walking interchanger
<code>emdash3_2_checks.lp</code> and <code>examples/</code>	executable diagnostics and independent reviewer-facing witnesses rather than mathematical owners

Imports use `require`; `open` brings imported public names into scope. The file split records dependency and evidence ownership. A conceptual chapter may use several owners, and one source family may support several chapters; neither direction is forced to mirror the table of contents.

Three source policies are essential for reading rules correctly.

1. Match a computation at its semantic owner and retain stable heads needed by later higher action.
2. Keep inferred rule slots anonymous unless a slot is a measured type, subject-reduction, or decision-tree guard.
3. Use runtime rewrites only for intended normal forms; use narrowly typed proof-time comparisons when neither side should compute to the other.

Formal status — checked. Evidence `FORMAL-KERNEL-PRESENTATION` records the representative declaration, action, rule, module, and diagnostic surface described here. Successful source checking warrants these interfaces; it does not establish the global metatheorems listed in G.7.

G.4 Formation, Introduction, Elimination, And Computation

The familiar rule schema remains useful, provided we add a sixth question suited to directed mathematics.

Rule aspect	Question
formation	when is the classifier or categorical object well formed?
introduction	what data construct an inhabitant or structured object?
elimination	how may an inhabitant be observed or used?
computation	what does elimination do to introduced data?
uniqueness or universality	is an arbitrary inhabitant recovered, propositionally compared, or characterized by a mapping property?
action and coherence	how does the construction act on arrows and higher cells as its parameters vary?

The last row is not optional decoration. A pointwise formula may answer the first five questions at objects while failing to define a functor, transfor, or displayed family.

Equality Induction

For $A : \mathbf{Grpd}$ and $x, y : A$, equality formation gives $x =_A y$. Reflexivity introduces an inhabitant $\text{refl}_x : x = x$. Right-based elimination fixes y , takes

$$P : \prod_{x:A} (x = y) \longrightarrow \mathbf{Grpd}, \quad u : P(y, \text{refl}_y),$$

and returns

$$\text{ind_eqr}(P, u, p) : P(x, p) \quad \text{for } p : x = y.$$

Its literal-reflexivity beta computes:

$$\text{ind_eqr}(P, u, \text{refl}_y) \rightsquigarrow u.$$

Path action, dependent path action, symmetry, and transitivity are derived uses. No equality reflection, uniqueness of identity proofs, or global path-eta rule is added. At the categorical layer, `Path_cat` and `path_map_func` package equality and function action so that higher path action can be iterated.

Formal status — checked. Evidence `TT-EQUALITY-INDUCTION`. Formation, reflexivity, right-based elimination, beta, `ap`, and `apd` are active. A stronger global uniqueness principle is not silently inferred from the beta rule.

Categories, Functors, And Transfors

`Cat` , `Obj` , and `Hom_cat` give formation for the categorical tower. Identity and composition are introduction operations for arrows, while iteration of `Hom_cat` eliminates an arrow into its next-cell context. The selected unit and associativity comparisons say how these introductions compose; there is no eliminator claiming that every category is freely generated by its displayed arrows.

For a functor, formation is `Functor A B` . Its elimination operations are `fapp0` and `fapp1_func` . The projection beta from full hom action to `fapp1_fapp0` and the identity/composition cuts are its generic computations. The active theory obtains inhabitants from named functor constructors and categorical operations; the book does not posit one record-style constructor whose fields may be supplied incoherently.

For a transfor, formation is `Transf F G` . Point and off-diagonal application are eliminations. Identity-boundary, composition, and the two naturality cuts are computations. The full off-diagonal functor is the action clause: it says what happens not just to an arrow f but to a higher cell between possible values of f .

This gives a useful criterion:

A point-component formula does not define a transfor until the off-diagonal arrow action and its next-hom behavior are supplied or explicitly deferred.

Sigma Totals And Pi Sections

For $E : K \rightarrow \mathbf{Cat}$, categorical Sigma formation gives $\Sigma_K E : \mathbf{Cat}$. An object is introduced as (k, u) with $u : E[k]$. The first projection and the fibre component eliminate it. A total arrow is introduced by a pair

$$(p : k \rightarrow k', \alpha : E[p](u) \rightarrow_{E[k']} u').$$

Projection and composition rules compute on this structure. The `sigma_intro_transf` packages the inclusions $E[k] \rightarrow \Sigma_K E$ naturally in k , so introduction also has a directed action rather than only a pair constructor.

Pi formation gives the section category $\Pi_K E$. Evaluation is packaged by `pi_eval_transf` , whose component at k is the functor

$$\text{ev}_k : \Pi_K E \longrightarrow E[k].$$

The evaluation projection computes through `piapp0_func` . Constant sections and pullback of sections supply important introductions, but the current section interface does not assert a general categorical Pi-eta or the separately proposed right-Kan direct image `Pi_along_func` . Selected slice dependent products now form a different whole family $\Pi_u : C/X \rightarrow C/Y$ right adjoint to pullback base change. Beck–Chevalley and comparison with the section/right-Kan presentations remain later layers.

At the groupoid layer, encoded Sigma and Pi classifiers separately provide dependent pairs, projections, pointwise path observation, and the selected `happly` / `funext` equivalence. The categorical Sigma/Pi operations above must not be flattened into those carrier-level formers: their objects vary in categories and their arrow action is part of the interface.

Formal status — checked nucleus. Evidence `CAT-SIGMA-PI` and `TT-SIGMA-PI-PATHS`. The active rules cover the cited constructors, projections, evaluation, and action. Evidence `SLICE-DEPENDENT-PRODUCTS` separately records the selected slice adjunction $u^* \dashv \Pi_u$; it does not identify slice dependent products with global section categories.

The WalkingEnd Rule Package

The walking endomorphism makes every row of the schema visible:

Aspect	Selected WalkingEnd datum
formation	<code>WalkingEnd_cat : Cat</code>
object introduction	<code>walking_base : Obj(WalkingEnd)</code>
arrow introduction	<code>walking_loop : Hom(WalkingEnd, base, base)</code>
height datum	<code>walking_end_is_one_cat : IsNCat(1, WalkingEnd)</code>
contextual algebra	$R, D : W \rightarrow \mathbf{Cat}, u : R[*] \rightarrow D[*]$, and $\sigma : D[\ell] \circ u \Rightarrow u \circ R[\ell]$
elimination	<code>walking_end_ind_funcd R D u sigma : Functord R D</code>
base computation	the fibre component at $*$ reduces to u
loop computation	the displayed action at ℓ reduces to the supplied component of σ
uniqueness	no general initiality or contractibility theorem is currently packaged
action	the result is a displayed functor, so base-arrow and higher action are retained

The section eliminator specializes R to the terminal family. The ordinary recursor specializes both families to constants. They are derived views of the contextual eliminator, not three independent postulates. The literal constructor betas are attached to stable generic observers, with two narrow projection joins for the concrete ordinary-recursion consumers.

Formal status — checked. Evidence `WE-SIGNATURE`, `WE-CONTEXTUAL-ELIMINATOR`, and `DHIT-DERIVED-ELIMINATORS`. The absence of a uniqueness theorem is part of the formal statement, not a prose omission.

Selected Groupoidal HITs And Free Inversion

The groupoidal signatures used by the fourth spiral share the ordinary equality eliminator but select different constructor boundaries.

Construction	Formation and introduction	Selected elimination and computation	Whole boundary
Circle	<code>Circle_grpd</code> , <code>circle_base</code> , <code>circle_loop</code>	unrestricted dependent <code>circle_ind</code> ; point beta and dependent <code>PathOver</code> loop beta compute	ordinary constant-family <code>ap</code> beta is propositional; the based loop carrier is equivalent to Integer
Interval	<code>Interval_grpd</code> , <code>interval_i0</code> , <code>interval_i1</code> , <code>interval_seg</code>	dependent <code>interval_ind</code> ; both endpoint betas and depen- dent segment beta compute	ordinary segment <code>ap</code> beta is propositional; WalkingArrow supplies the free-inversion comparison
classified truncation	<code>Trunc_ntype(n,A)</code> in <code>NType_cat(n)</code> , with point <code>trunc_intro</code>	elimination only into classi- fied n -truncated fibres; point beta computes	decoding exposes <code>Trunc_grpd(n,A)</code> and retained truncation evi- dence without identifying the result with an arbitrary equivalent carrier
groupoidification	<code>Groupoidify(C)</code> with one whole unit $\eta_C : C \rightarrow$ <code>Path(Groupoidify(C))</code>	recursion computes on rep- resented points and on de- pendent action over repre- sented arrows	extension and restriction are whole functors with path- valued beta/eta and retained higher action

The Circle and Interval path-constructor computations are attached to dependent action:

$$\text{apd}(\text{circle_ind}(D, b, \ell), \text{loop}) \rightsquigarrow \ell,$$

and analogously for `interval_ind` at `seg` . Passing to a constant family produces the familiar homogeneous path only after the general bridge from constant-family `PathOver` to `ap` . The resulting equation is internal equality, not a second rewrite. This keeps one higher-constructor owner while still recovering the usual recursion theorem.

For a category C and groupoid G , the free-inversion boundary is the whole mapping equivalence

$$\text{Hom}_{\text{Grpd}}(\text{Groupoidify}(C), G) \simeq_{\omega} \text{Functor}(C, \text{Path}(G)).$$

Restriction is path action followed by precomposition with η_C ; extension is the categorical-HIT recursor varying in the entire source representation. Their beta and eta are paths between whole functors, so their first and next hom actions remain available. This is the target-side universal property for every fixed C . The present package does not yet construct the action of `Groupoidify` on an arbitrary source functor or assemble the resulting adjunction.

Formal status — checked selected signatures. Evidence `CIRCLE-HIT-COMPUTATION` , `WALKING-INTERVAL-GROUPOIDIFICATION` , and `GENERIC-GROUPOIDIFICATION-MAPPING` . These are computationally reviewed HIT slices with whole action; they do not constitute a general HIT declaration compiler or a complete computational HoTT metatheory.

Whole Laxity And The Profiled Gray Closure

The strict naturality cuts of the historical prototype do not exhaust the internal action. Before point-wise projection, the displayed hom calculus owns a whole laxity transformation. Ordinary post/left and pre/right comparisons are transparent specializations of that displayed owner, and the functor compositor is its identity-transfor specialization:

$$\phi_{g,f}^F : F[g] \circ F[f] \Longrightarrow F[g \circ f].$$

Because f still ranges over a whole hom category, one further hom action can observe how ϕ varies. A path-valued target makes the comparison invertible. A strict-functor property instead stores an endpoint path and an equality identifying the selected compositor with its equality-induced arrow. This is evidence about one action, not a duplicate functor theory or a blanket compositor-to-identity rewrite.

The category $\mathbf{GrayHom}_{\text{lax}}(A, B)$ uses proof-carrying strict-functor packages as objects: each package is an ambient functor paired with `IsStrictFunctor` evidence. It reuses the ambient transfor and higher-hom tower between their stable views. One selected right closure is checked:

$$\mathbf{GrayHom}_{\text{lax}}(A \otimes_R B, C) \simeq_{\omega} \mathbf{GrayHom}_{\text{lax}}(A, \mathbf{GrayHom}_{\text{lax}}(B, C)).$$

Coevaluation at the walking-arrow shape exposes four vertices and two coordinate routes. Projecting the already-existing whole post/left laxity action supplies their oriented interchanger and retains its next action. No independent square axiom is introduced. The checked slice does not supply the mirror closure, tensor functoriality and coherence, or a full Crans–Gray biclosed monoidal structure.

Formal status — checked selected profile. Evidence `FUNCTORD-WHOLE-LAXITY`, `GRAY-COMPUTATIONAL-PROFILE`, and `GRAY-RIGHT-CLOSURE`. The result is a computational stress test for the foundations, not a reclassification of every existing functor as globally lax or strict.

Adjunction And Weighted Representability

An adjunction illustrates a rule package whose principal eliminations are observations rather than a public record constructor. For named functors $F : R \rightarrow L$ and $G : L \rightarrow R$, the classifier

$$\mathbf{Adjunction}(F, G)$$

forms the proposition-like structure accepted by the kernel. From $J : \mathbf{Adjunction}(F, G)$ one eliminates the selected unit and counit:

$$\eta : \text{id}_R \Rightarrow GF, \quad \varepsilon : FG \Rightarrow \text{id}_L.$$

The two triangle cuts compute at off-diagonal components. Their role is exactly beta reduction: an introduction by a unit followed by elimination by a counit contracts to the underlying map, and dually.

A weighted limit is presented one level more abstractly. Given $F : J \rightarrow B$, $W : J' \rightsquigarrow J$, and $L : J' \rightarrow B$, formation gives the classifier

$$\text{IsWeightedLimit}_{\text{cov}}(F, W, L).$$

An inhabitant is a computational comparison between the weighted-cone profunctor and the representable hom profunctor. Reindexing and applying that certificate eliminates it into inverse operations

push and pull.

Their composites reduce by the generic profunctor-comparison beta and eta rules. The action clause is reindexing along every probe functor $M : I \rightarrow B$; the universal property is not restricted to one set of global elements. Adjunction mates then transport the whole comparison, which is why right-adjoint preservation is a computation on certificates rather than a fresh pointwise proof.

Existence for every diagram and uniqueness of representing objects are separate theorems. The active classifier says what data certify a chosen L ; it does not postulate a global limit operator or a native univalent uniqueness package.

Formal status — checked. Evidence `ADJ-TRIANGLE-CUTS` and `WEIGHTED-LIMIT-REPRESENTABILITY`.

The formation/elimination/computation interface is active. Semantic end formulas, general existence, and univalent uniqueness remain separately status-labeled mathematics.

Monads, Cartesian Structure, And Slice Adjunctions

The newer structural interfaces reuse the same rule schema rather than adding a second logical framework.

Structure	Formation/selected data	Elimination or observation	Principal computation
monad	<code>Monad(A, T)</code> indexed by an endofunctor	whole unit, multiplication, and extension; point extension f^*	extension beta and accumulation; multiplication component to extension of identity
binary products	<code>BinaryProducts(C, P)</code> indexed by $P : C \times C \rightarrow C$	whole projections, represented-family pairing, K_1^a / K_2^a	projection beta, pairing distribution, eta, selected normalizations
terminal object	<code>TerminalObject(C, t)</code> indexed by t	whole $! : \text{id} \Rightarrow \text{Const}_t$ and Hom contraction	$!_B \circ h \rightsquigarrow !_A$; arbitrary uniqueness is equality
pullbacks	<code>PullbackStructure(C)</code>	exact whole base-change family u^* and $\Sigma_u \dashv u^*$	both adjunction rectangles, point/whole mate cancellation, derived object/projections/square

slice dependent products	<code>DependentProductStructure(C, PB)</code>	exact whole family Π_u and $u^* \dashv \Pi_u$	both second-adjunction rectangles and component triangles
--------------------------	---	---	---

The product and terminal structures are selected: their classifiers do not inhabit themselves for every category. The thin cartesian package only pairs the two existing capabilities.

For pullbacks, a cone is already an object of $\text{Hom}_{C/Y}(\sum_u a, g)$. The pullback object is the selected slice object $u^*(g)$, and its counit yields the second projection and directed square. Hence no record constructor accepts two legs plus a manually supplied commutativity equation.

At the whole mate level, proof-time unifiers compare stable functors with their canonical semantic bodies after `slice_sigma_obj` has normalized to `sigma_map_obj`. Runtime cancellation is composition in `Cat_cat`, but the active rules repeat exact raw Hom-category endpoints; wildcard-only endpoints could reconstruct the wrong source after opposite normalization.

Dependent products add the second adjunction without identifying three different Pi notions:

<code>Pi_cat(E)</code>	category of global sections
<code>Pi_along_func(F)</code>	proposed general right-Kan direct image
<code>Pi_u</code>	selected right adjoint $C/X \rightarrow C/Y$.

The neutral `SliceDependentProducts(C)` total packages the pullback structure and its indexed dependent-product structure. It does not yet assert the final finite-limit convention, derived slice exponentials, Beck–Chevalley, or Frobenius coherence of a convention-sensitive LCCC package.

Formal status — checked selected structures. The five cited evidence rows connect each classifier to its whole operations and independent reviewer. They establish the displayed local cut computations, not global existence, a free syntax, or coherence layers explicitly listed as future.

Whole Homological Operations And Their Record Views

[Chapter 31](#) uses the same declaration/observation discipline for homology. The kernel and cokernel presentations supply actual functors and adjunctions over retained universal-operation families. Whole H is a definition from those owners, while its operational records are views at the actual selected objects.

Interface	Principal owner	Computational boundary
Coherent kernels and cokernels	<code>KernelPresentation</code> , <code>CokernelPresentation</code>	Whole adjunction cuts and proof-time agreement with original selections; coherent structure is supplied
Whole one-degree H	<code>homology_family_func</code> , <code>zero_arrow_cone_homology_func</code>	Derived from the whole boundary and Q; generic functor and further Hom action remain available
Connecting family	<code>homology_window_connecting_transf</code>	Component beta returns the direct construction at the same H endpoints

Finite exact tail	<code>homology_row_field_span_exact_tail</code>	Nat iteration over coherent rows retains existing arrows and interior evidence; final symbolic endpoint attachment is deferred
Supplied Freyd model	<code>FreydHomologyModel</code>	Actual H observations and normality-indexed connecting observations; no automatic closed-model construction

The input zero diagrams and row maps use actual internal functors and transformations. They are not a new record grammar of manually supplied commuting squares. The whole connecting transformation is declared with its source and target functors, then related to the direct component by projection computation. The surrounding universal-record reconstructions and exactness laws remain explicit proofs; the presence of whole operations does not turn every such proof into a runtime rewrite.

The proof-CAS layer interprets selected observations through explicit bindings and adoption. Computed equations, all-test provider semantics and whole-model interpretations retain separate classifications. This is a reviewed interface to the native algebra engine, not unrestricted execution of arbitrary formal terms or a claim of whole-library TypeScript transfer.

Formal status — checked. The cited owners and reviewers establish these conditional interfaces and their selected computations. Their coherent-model premises and the deferred endpoint theorem remain explicit.

G.5 Elaboration And Canonical Surface Syntax

Readable notation is indispensable, but it need not be the foundational layer. The renewed TypeScript path compiles a reviewed subset into the explicit owners described above:

Stage	Implemented bounded profile	Retained boundary
parse	<code>located</code> <code>^f</code> , <code>^n</code> , <code>^fd</code> , and <code>^nd</code> binders, neutral application, selected constructors, and grouped displayed contexts	not the complete book or <code>Lambdapi</code> grammar
elaborate	typed expected classifiers route recursively through the existing contextual categorical program	no arbitrary pointwise-to-coherent synthesis
select owner	reviewed operation families lower to internal categorical and structural owners	no whole-library owner-acquisition claim
check and reduce	the generic TypeScript LF checks explicit Core, compares terms, and executes the bounded runtime	no global metatheory
conform	optional deterministic <code>Lambdapi</code> emission and a bounded oracle compare selected results with the active kernel	no production <code>Lambdapi</code> dependency

Elaboration may recover information; it may not invent mathematics. If a pointwise family lacks arrow action, the elaborator must report missing coherence rather than synthesize an arbitrary transform. If lower-star and upper-star action are both type-correct, the written variance or expected type must disambiguate them.

Surface Forms And Explicit Targets

The canonical notation includes:

Surface	Explicit target
$a \multimap^C b$	<code>Hom_cat C a b</code>
$A \vdash B$	<code>Functor_cat A B</code>
$F \Rightarrow G$	<code>Transf_cat F G</code>
$E[k]$	<code>Fibre_cat E k</code>
$A[k^\wedge -] \vdash_{[k]} B[k]$	<code>Functor_catd A B</code>
$\Pi (k : ^\wedge n K), E[k]$	<code>Pi_cat E</code>
$u_*(g)$	<code>a hom_postcomp_* application</code>
$u^*(h)$	<code>a hom_precomp_along_* application</code>

For example, if $\eta : F \Rightarrow G$ and $f : x \rightarrow_A y$, the readable term $\eta[f] : F[x] \rightarrow_B G[y]$ elaborates toward the fully explicit owner `@tapp1_fapp0 A B F G x y eta f`.

The source notation does not have to expose all seven parameters, but the result must typecheck as that operation or an explicitly documented equivalent owner. Readability changes what the author writes, not what the checker trusts.

One compositional motif, four binder modes. Binder modes say how a variable is allowed to vary. The reviewed executable forms are

```

 $\lambda^f x : A. \dots$ 
 $\lambda^\wedge n k : K. \dots$ 
 $\lambda^{fd} a : E. \dots$ 
 $\lambda^{nd} k : K. \dots$ 

```

The mode belongs to the lambda. The classifier annotation after the variable may be omitted when the bidirectional expected classifier supplies it, but the mode is not inferred from that annotation. The mathematical telescope notation $k : ^n K$ therefore records the same natural/indexed role as `$\lambda^\wedge n k : K. \dots$` without pretending that mathematical declarations and executable binders are literally the same grammar. Ordinary object binding in the outer LF uses its ordinary dependent lambda.

The four modes are easiest to compare around one compositional motif. Let $H : A \rightarrow B$ be an ordinary functor. Let $E, D, Q : K \rightarrow \mathbf{Cat}$ be directed families, $FF : E \rightarrow D$ and $GG : D \rightarrow Q$ displayed functors, and s a coherent section of E . Finally, let $\eta : F_0 \Rightarrow F_1$ and $\theta : F_1 \Rightarrow F_2$ be displayed transors. At successive categorical levels the same idea appears as:

Mode	Representative expression	Mathematical reading
$\wedge f$	$\lambda^{\wedge f} x : A. H\ x$	an ordinary functorial variable inside one category
$\wedge n$	$\lambda^{\wedge n} k : K. (GG\ k)\ ((FF\ k)\ (s\ k))$	a base variable whose result is a coherent section of Q
$\wedge fd$	$\lambda^{\wedge fd} a : E. GG\ (FF\ a)$	an object varying in a displayed family, retaining its hidden base index
$\wedge nd$	$\lambda^{\wedge nd} k : K. \text{composeCells}\ (\theta k)\ (\eta k)$	a coherent family of cells between displayed functors, one hom level higher

These are not four spellings for an ordinary lambda. The $\wedge n$ form must respect transport in the base; the $\wedge fd$ form must retain displayed object and arrow action; the $\wedge nd$ form must construct a transor rather than a bare pointwise family. In each case the expected classifier selects a reviewed internal construction. If that construction is absent, elaboration fails instead of accepting a JavaScript callback with an external naturality promise.

Ordinary nesting already shows why recursive scope matters. Assume $A, B, C : \mathbf{Cat}$ and $E : \mathbf{Functor}\ B\ (\mathbf{Functor_cat}\ A\ C)$. The reviewed expression is:

```
λ^f x : A. λ^f y : B. E y x
```

This term has classifier $\mathbf{Functor}\ A\ (\mathbf{Functor_cat}\ B\ C)$. Neutral application first selects the action of E on y and then its action on x . Recursive abstraction lowers the result through the existing `exchange-functor-abstraction` owner before explicit Core is checked.

No external functoriality equation accompanies the source expression. The selected owner already carries object and arrow action, and the resulting explicit Core is checked by the same generic LF as other terms.

Dependency Levels And Independent Siblings

Displayed contexts make the distinction between dependency and independence visible. A representative mixed telescope is

```
λ^fd (a : A; b : B, c : C; d : D). fibrePair b c
```

It has dependency levels `A; B,C; D`. A semicolon advances to a family over the preceding total context, while a comma groups independent siblings over the same prefix. The middle pair lowers through the transparent fibrewise product, displayed pairing, Sigma projections, and reindexing owners. Thus `b` and `c` may be paired, weakened, contracted, or exchanged fibrewise; no exchange of `a` across a classifier depending on `a` is implied. Object and base-arrow behavior remain internal to those owners rather than being supplied as external coherence evidence.

The implemented normal form is not limited to the displayed four-variable example. It supports any finite sequence of these canonical dependency levels, with finite sibling groups at a level, for the reviewed displayed functorial and displayed-natural constructions. Separately, the category resolver can descend through any finite number of qualified Hom-category levels over its supported roots, and the indexed-section route can compose a finite rigid chain of displayed functors on a section. These depth results do not amount to arbitrary dependency or variance graphs. Exchange across a dependency edge, unrestricted mixed introduction and currying, and coherence synthesis outside the qualified grammar remain open.

Declaration Convenience Without New Mathematics

Some repetition belongs to the surrounding logical framework rather than to categorical terms. Two direct-TypeScript declaration forms remove that repetition before explicit Core is checked.

For an adjunction, `assumeAdjunction` receives already declared functors, unit, and counit. It expands to an ordinary `Adjunction(F,G)` assumption and two proof-time agreements identifying the declared transformations with the kernel's stable unit and counit observations. A second form accepts a counit and a coherent whole hom-profunctor transpose. In both cases the declaration preserves the distinction between proof-time agreement and runtime conversion: independently named maps do not silently become new reduction rules for the categorical kernel.

For a finite dependent package, `declareStructure` expands one unparameterized, nonrecursive, single-constructor structure into an opaque carrier, an injective constructor, named primitive projections, and one ordered, subject-reducing beta rule for each projection. Later field types may depend on earlier fields, which is the essential convenience for mathematical presentations. The form generates no record eta, eliminator, recursion, positivity theorem, general inductive declaration, or browser/text syntax.

Both conveniences are conservative in the practical architectural sense: their output consists of the same ordinary LF declarations and rules that could have been written explicitly. Neither adds a trusted Core node or a `Lambdapi` mathematical owner. The elaborator improves the act of stating a presentation; it cannot turn missing structure or coherence into a theorem.

Located Text And The Browser Reviewer

The text adapter accepts a small, located language rather than a string that is later treated as trusted code. It records source spans, parses the reviewed binders, grouped contexts, neutral whitespace application, and selected term, category, and displayed-family constructors, then delegates typing and

owner selection to the same contextual program used by direct TypeScript. A failure therefore reports its parsing, resolution, or elaboration phase together with the source location. It is not a second action table or checker.

The integrated browser reviewer makes this path inspectable without a server. Its twelve editable examples span the four binder modes, the canonical sibling/Sigma context, qualified recursive Hom categories, and finite rigid section chains. The current natural-binder example is the two-step section

```
λ^n k : K. (GG k) ((FF k) (s k))
```

from the running motif above. For an accepted expression the client displays the explicit backend-neutral Core, inferred and expected classifiers, and the structural owners used in lowering. For a rejected edit it displays the source-located diagnostic. The same page can run the outer-LF/ordinary/ displayed research report, retain the minimal explicit-Core playground, and open the generated book. All of this execution is client-side; Lambdapi is an optional development oracle, not a browser or production dependency.

Historical Prototype And Retained Boundary

The repository history contains an older TypeScript feasibility prototype with generic bidirectional checking, holes, unification, rewriting, and proof-state machinery. Those mechanisms informed the renewed work, but its stale category-specific layer is not an authority for v3.2. The renewed product instead targets backend-neutral explicit Core aligned with active owners and uses Lambdapi only as an optional conformance oracle.

The current path is deliberately bounded: it does not parse every notation in the book, accept arbitrary dependency or variance graphs, synthesize coherence from an unstructured pointwise function, or mechanically transfer the whole Lambdapi library. Its qualified finite-depth results are neither one hard-coded example nor a complete surface language. They are explicit continuation boundaries, not hidden assumptions of the examples that run.

Formal status — research boundary. Evidence `FORMAL-ELABORATION-BOUNDARY`. The direct-TypeScript and categorical-text paths, explicit Core, generic checker/evaluator, bounded adjunction and dependent-structure declarations, client-side reviewer, and optional conformance route are executable for the reviewed profile. A complete compiler for the canonical surface, arbitrary displayed coherence, a general record or inductive facility, and whole-library transfer are not claimed. The active Lambdapi sources remain the mathematical authority.

G.6 Directed Higher-Inductive Signatures

A directed higher-inductive signature must specify more than a list of generators. At minimum it needs:

1. object, arrow, and higher-cell constructors with typed boundaries;
2. the categories and families into which they may be interpreted;

3. recursion, dependent elimination, and any contextual elimination principles;
4. coherence data demanded by varying source and target families;
5. constructor computation at named observers;
6. action on arrows and higher cells of every varying parameter;
7. optional dimension or truncation evidence;
8. a statement of uniqueness or initiality when one has actually been proved.

The `WalkingEnd` signature is the selected worked instance. With $W = \mathbf{WalkingEnd}$, $* : \mathbf{Obj}(W)$, and $\ell : * \rightarrow_W *$, its contextual algebra is

$$\begin{aligned} R, D : W &\longrightarrow \mathbf{Cat}, \\ u : R[*] &\longrightarrow D[*], \\ \sigma : D[\ell] \circ u &\Longrightarrow u \circ R[\ell]. \end{aligned}$$

The eliminator returns

$$\mathbf{ind}^d(R, D, u, \sigma) : R \longrightarrow^d D.$$

At the base, its fibre functor reduces to u . At the literal generator, its displayed laxity component reduces to the supplied component of σ . These are constructor beta rules. Generic functoriality and transfor naturality own the remaining identity, composition, and ordinary naturality cuts.

The coherence cell is directed:

$$D[\ell] \circ u \Longrightarrow u \circ R[\ell].$$

It is neither an equality nor an automatically invertible path. Reversing it would define a different eliminator orientation. Supplying only its pointwise components would also be incomplete, because the transfor must act on arrows of $R[*]$ and on their higher cells.

The one-dimensional witness is separate signature data. It lets a later directed 2-cell between parallel based arrows be converted to equality; it does not make the generating 1-cell invertible. This separation is essential to the normalization proof of Chapter 8.

The ordinary section eliminator and recursor are specializations:

view	R	D
contextual	arbitrary	arbitrary
section	terminal family	arbitrary
recursor	terminal family	constant family.

This relationship is an architectural requirement for a future signature compiler: it should generate one coherent principle and derive weaker views, not postulate unrelated recursors whose computations may disagree.

What is not yet active is equally specific. There is no general language of directed cell boundaries, no compiler that generates contextual eliminators and projection-stable beta rules, no general algebra category, and no theorem that every such presentation is initial. A plausible implementation must also generate focused diagnostics for typing, subject reduction, critical pairs, and both possible projection orders.

Formal status — checked instance and research boundary. Evidence `WE-SIGNATURE` and `WE-CONTEXTUAL-ELIMINATOR` support the complete selected instance. Evidence `DHIT-GENERAL-SCHEMA` records the missing general signature and compiler rather than extrapolating them from one HIT.

G.7 Basic Metatheory And Its Boundary

A successful checker run is strong evidence that the submitted declarations and rules satisfy the tool's current acceptance conditions. It is not, by itself, a proof of every global property one might want from the combined rewrite and unification theory. The current warranted statements are:

Property	What this edition may say
typing of active sources	checked by bounded <code>Lambdapi</code> runs over the active module graph
local subject-reduction obligations of promoted rewrite rules	checked by <code>Lambdapi</code> 's ordinary rule acceptance; not separately formalized as one global project theorem
selected computation	witnessed by promoted rules and focused positive and negative assertions
selected proof-time comparison	exercised by typed uses or typed reflexivity checks, not inferred merely from a conversion assertion
evidence traceability	checked syntactically from book markers to active owners and reviewers
build and render reproducibility	tested by the release tooling for the recorded source snapshot
global confluence	not established for the whole emdash rewrite and unification theory
strong normalization	not established for the whole theory
global canonicity	not established; only selected constructor and normalization computations are tested
decidable conversion or type checking as an emdash metatheorem	not claimed beyond observed behavior of the current <code>Lambdapi</code> toolchain on the active sources
consistency and semantic soundness	require model and metatheory proofs for a precisely stated fragment; they do not follow silently from compilation

Engineering Evidence Is Not A Metatheorem

The repository records warning inventories, focused owner-position probes, critical-pair investigations, strict inferred-slot audits, and bounded full checks. These practices are indispensable. They locate overlap risks, reject ill-typed rules, compare reduction orders, and protect intended normal

forms. They still sample or delegate parts of a global theorem rather than proving one in the object theory.

Warning counts are therefore diagnostics, not a numeric confluence proof. A zero count would not establish normalization, and a nonzero count does not by itself refute a deliberately joined computation. Likewise, deterministic PDF output establishes release reproducibility, not mathematical consistency.

The Role Of Models

`BNat` is a separate concrete one-object category whose endomorphisms are natural numbers under addition. The checked functor from `WalkingEnd` to `BNat` is meaningful model evidence for the selected generators and recursor. The encode–decode theorem then proves much more about the based hom inside the opaque source.

That model is not a soundness interpretation for all of `emdash3_2.lp`. It does not interpret every higher category, displayed family, equality principle, profunctor, or rewrite rule. A global consistency claim would require a stated syntax fragment, an interpretation of every formation and computation rule in that fragment, and a proof that conversion is preserved.

Formal status — checked local model evidence. Evidence `WE-BNAT-MODEL` supports the separate `WalkingEnd`-to-`BNat` interpretation. No global soundness theorem is inferred from it.

Adaptation Of The HoTT Formal Appendix

The four source units of the HoTT formal appendix enter this presentation as follows:

HoTT source unit	Functorial adaptation
A.1, first presentation	G.2 gives the readable categorical signature; G.5 keeps it distinct from a parser
A.2, second presentation	G.1 and G.3 state judgments and literal source; G.4 organizes representative rule families
A.3, homotopy type theory	equality and univalence remain qualified layers, while G.6 presents the selected directed <code>WalkingEnd</code> extension
A.4, basic metatheory	this section retains the taxonomy of properties but replaces inherited conclusions by the conservative status matrix above

The adaptation deliberately does not import the HoTT appendix's normalization or metatheoretic conclusions as results about `emdash`. Different rewrite owners, proof-time unification rules, opaque categorical structure, and directed higher cells require their own theorem.

Formal status — research boundary. Evidence `FORMAL-METATHEORY-BOUNDARY`. The matrix states what current checks establish and gives a concrete specification for future metatheory. Global confluence, strong normalization, canonicity, decidability, consistency, and semantic soundness remain unclaimed until separately proved.

The resulting architecture is intentionally asymmetric. The categorical kernel computes without depending on a traditional front end. The bounded elaborator improves usability without changing the foundation, and future semantic models may justify larger fragments without becoming a second source language. That is the formal sense in which functorial type theory begins from categorical computation.

Bibliography

1. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. [Source repository](#), reviewed at revision 578b85cc8d586b1677ec4335148adeb443057d24 .
2. Saunders Mac Lane. *Categories for the Working Mathematician*, second edition. Graduate Texts in Mathematics 5. Springer, 1998; DOI 10.1007/978-1-4757-4721-8.
3. G. M. Kelly. *Basic Concepts of Enriched Category Theory*. London Mathematical Society Lecture Note Series 64. Cambridge University Press, 1982; reprinted in *Theory and Applications of Categories*, Reprints 10, 2005. [TAC reprint](#).
4. Jean Bénabou. “Introduction to Bicategories.” In *Reports of the Midwest Category Seminar*, Lecture Notes in Mathematics 47, pages 1–77. Springer, 1967. DOI 10.1007/BFb0074299.
5. GPT 5.6 Codex.
6. The Lambdapi contributors. *Lamdapi User Manual*. [Current online manual](#). The repository copies under docs/ are the operational references used while maintaining the accompanying formal development.
7. The emdash contributors. *emdash v3.2 Lambdapi Sources*. Accompanying computational artifact for this development edition: emdash3_2.lp and its one-way extension modules. [Code repository](#).
8. Kosta Došen. *Cut Elimination in Categories*. Trends in Logic 6. Kluwer Academic Publishers, Dordrecht, 1999; DOI 10.1007/978-94-017-1207-1.
9. Max Zeuner. *Univalent Foundations of Constructive Algebraic Geometry*. arXiv:2407.17362v1, 2024. [arXiv record](#).
10. Pierre-Marie Pédro. “Pursuing Shtuck.” Preprint, 2023. [HAL record](#).
11. Amar Hadzihasanovic. *Combinatorics of Higher-Categorical Diagrams*. arXiv:2404.07273v2, 2024. [arXiv record](#).
12. Astra Kolomatskaia and Michael Shulman. *Displayed Type Theory and Semi-Simplicial Types*. arXiv:2311.18781v2, 2024. [arXiv record](#).
13. Hugo Herbelin and Ramkumar Ramachandra. *A Parametricity-Based Formalization of Semi-Simplicial and Semi-Cubical Sets*. arXiv:2401.00512v2, 2025. [arXiv record](#).
14. Hugo Herbelin and Ramkumar Ramachandra. *The Very Dependent Recursive Structure of Iterated Parametricity in Indexed Form*. arXiv:2602.12689v1, 2026. [arXiv record](#).
15. Sebastian Posur. *A Constructive Approach to Freyd Categories*. Reviewed preprint arXiv:1712.03492v1, 2017. [Pinned arXiv record](#).

16. Mohamed Barakat and Daniel Robertz. *homalg: A Meta-Package for Homological Algebra*. Reviewed preprint arXiv:math/0701146v2, 2007; *Journal of Algebra and Its Applications* 7 (2008), 299-317. [Pinned arXiv record](#).
17. Mohamed Barakat and Markus Lange-Hegermann. *An Axiomatic Setup for Algorithmic Homological Algebra and an Alternative Approach to Localization*. *Journal of Algebra and Its Applications* 10 (2011), 269-293; reviewed preprint arXiv:1003.1943v5, 2017. [Pinned arXiv record](#).
18. Sebastian Posur. *Methods of Constructive Category Theory*. arXiv:1908.04132v1, 2019. [Pinned arXiv record](#).
19. Sebastian Posur. *Closing the Category of Finitely Presented Functors under Images Made Constructive*. arXiv:1911.11469v3, 2020. [Pinned arXiv record](#).
20. Sebastian Posur. *On Free Abelian Categories for Theorem Proving*. arXiv:2103.08379v1, 2021. [Pinned arXiv record](#).
21. The CAP contributors. *Categories, Algorithms, and Programming*, including CompilerForCAP. [Source repository](#), reviewed at revision `d21dc5f5f420` .
22. The homalg contributors. *The homalg Project*. [Source repository](#), reviewed at revision `802993da0095` .
23. The emdash contributors. *Functorial Type Theory: Univalent Foundations for Mathematics*. Ongoing book, 2026. [Book DOI 10.5281/zenodo.21544186](#). [Code and source repository](#).

Items 6-7 identify proof infrastructure. Item 23 is this book's self-reference; its published record and ongoing source have independent version histories. Items 12-14 are comparative Chapter 29 references. Items 15-22 give mathematical and algorithmic context for Chapter 31; no source prose or software code is reproduced in that chapter.

Credits And Third-Party Attribution

This development edition is prepared by the emdash contributors. Individual authorship and editorial credits will be made explicit before a public edition is released.

Homotopy Type Theory book

The organization and adapted passages of this book are inspired by:

The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, Institute for Advanced Study, 2013.

The reviewed source is the [HoTT Book repository](#) at revision 578b85cc8d586b1677ec4335148adeb443057d24 (2026-05-12). That work is licensed under the [Creative Commons Attribution-ShareAlike 3.0 Unported License](#).

Any adapted material in this book is modified for a directed, category-theoretic setting. In particular, the circle/integer calculation is not reproduced as the WalkingEnd/Nat calculation: invertible paths are replaced by genuinely directed arrows, integer powers by natural powers, and the hard inverse by a directed normalization cell followed by one-dimensionality. Chapter 26 then returns to the Circle itself and adapts the universal-cover encode–decode architecture to the checked emdash Circle/Integer construction, including its distinct computational and categorical boundaries.

The exact source map and adaptation ledger live in `references/third-party-sources.json`. They were established before the corresponding prose was drafted. The Chapter 8 vertical slice records structural and conceptual adaptations from the pinned source, while Chapters 10–15 adapt all nine sections of `categories.tex`, Chapter 26 records the Circle HIT and universal-cover source map, and Appendix G adapts all four parts of `formal.tex`. The resulting prose is newly written for the directed categorical setting; the ledger records the source labels, adaptation kind, and target under this attribution and ShareAlike notice.

Max Zeuner's constructive algebraic geometry

The local-to-global geometry spiral takes mathematical and expository inspiration from:

Max Zeuner, *Univalent Foundations of Constructive Algebraic Geometry*, arXiv:2407.17362v1, 2024.

The reviewed [arXiv version](#) is licensed under the [Creative Commons Attribution 4.0 International License](#). Chapter 18 adapts the locally ringed lattice's largest compact-open invertibility support into a comparison with the sieve $D_U(s)$ of all invertibility probes. This is a change of organizing viewpoint: the compact open remains the appropriate representative in Zeuner's coherent or qcqs setting when it exists, while the sieve is defined on a general site before representability is known. Chapter 22 structurally adapts the Zariski-lattice, coverage, compact-open, and functor-of-points narrative to this sieve-first organization: a supplied localization represents $D_R(f)$ pointwise, and the big-site

topology is generated from selected finite localization charts. Neither chapter imports Zeuner's qcqs comparison theorem or general scheme theorem as an emdash result. Chapter 23 comparatively adapts the finite affine-cover architecture, but reverses the direction of construction: its global ringed object is supplied first, two charts constructively generate one retained covering sieve, and restrictions and a selected intersection are inherited from that single object. It does not import Zeuner's gluing theorem, compact-open classifier, or equivalence between functorial and locally ringed-lattice qcqs schemes. Chapter 24 carries that finite-cover rhythm and comparison boundary into a supplied projective-line presentation, but its Laurent calculation and explicit `Proj` horizon are an emdash synthesis rather than a construction drawn from Zeuner's thesis. The source sections, targets, adaptation kinds, and mathematical changes are recorded in `references/third-party-sources.json`.

Pierre-Marie Pédrot's computational sheafification

The return/glue/silent presentation in Chapter 20 takes conceptual and structural inspiration from:

Pierre-Marie Pédrot, “Pursuing Shtuck,” preprint, 2023.

The reviewed [HAL version](#) is licensed under the [Creative Commons Attribution 4.0 International License](#). Pédrot presents free sheaves by a return constructor, a branching glue constructor, and an equation that erases a branch whose result is ignored, then explains the last as a silent transition. Emdash adapts that computational picture to actual varying cover questions in categorical semantics: the branches are matching objects of Cat-valued presheaves, and the checked endpoint is a whole Hom-category universal property and reflector. The book does not import the paper's internal type theory, metatheory, universe claims, or dependent-elimination results. Exact source sections and adaptation boundaries are recorded in `references/third-party-sources.json`.

Došen's cut-elimination perspective

The four-level cut calculus in Chapter 9 and the monad/product/adjunction rectangles in Chapters 12 and 30 take conceptual inspiration from Kosta Došen's *Cut Elimination in Categories* (Kluwer, 1999). The cited work is not licensed for textual adaptation here. It is used only as a bibliographic and conceptual reference: the exposition, notation, examples, and emdash correspondence in this book are newly written, and no passage from Došen's text is copied or closely paraphrased.

Hadzahasanovic's higher-categorical diagrams

Chapter 28 uses the Gray-product and oriented-cube discussion in Amar Hadzahasanovic's *Combinatorics of Higher-Categorical Diagrams*, arXiv:2404.07273v2, as comparative mathematical orientation. The source is cited rather than textually adapted: no passage is copied or closely paraphrased. In particular, the chapter distinguishes Hadzahasanovic's combinatorially constructed higher-dimensional products from emdash's checked and deliberately narrower experiment—one profiled right closure and its walking-square interchanger. The exact sections and reference-only boundary are recorded in `references/third-party-sources.json`.

Constructive categories and homological algorithms

Chapter 31 uses [Posur's constructive Freyd-category development](#) as mathematical and algorithmic orientation for presentations, explicit agreement witnesses and the two-weak-pullback kernel construction. The [homalg meta-package paper](#) and [axiomatic algorithmic setup](#) inform the separation between ring algorithms and categorical universal operations. The [CAP](#) and [homalg](#) source reviews also inform the method/prerequisite architecture and the independent computational comparisons. They are references, not runtime dependencies of emdash's native implementation.

The chapter's later research boundary distinguishes the constructions in Posur's [methods](#), [image-completion](#), and [free-Abelian theorem-proving](#) papers. It does not claim that a universal-diagram prover, generalized-morphism compiler, or comparison between these category constructors has been implemented. The exposition is newly written from the emdash construction; no source prose or software code is reproduced. The reference versions and uses are recorded in `references/third-party-sources.json`.

License For The Book

Except where a source or quotation is identified separately, the textual contents of the `book/` directory and the generated `print/public/emdash-book.md` artifact are licensed under the Creative Commons Attribution-ShareAlike 3.0 Unported License, abbreviated `CC BY-SA 3.0`.

You may share and adapt this material under the terms of that license, including attribution and ShareAlike. Third-party material remains subject to its identified license and attribution requirements.

This notice applies only to the book text and its generated Markdown form. It does not assign or change a license for Lambdapi sources, renderer code, reports, or other repository content outside that book artifact.